

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO ASSIGNMENT 03 MÔN HỌC PHÁT TRIỂN
CÁC HỆ THỐNG THÔNG MINH

Giảng viên hướng dẫn: PGS.TS. Trần Đình Quế

Họ và tên: Nguyễn Chí Hùng

Mã sinh viên: B23DCCN359

Lớp: D23CNPM04

Nhóm lớp: 05

Contents

PHẦN I: CƠ SỞ LÝ THUYẾT	8
1. Giới thiệu tổng quan	8
1.1 Bối cảnh và động lực	8
1.2 Định nghĩa Deep Learning	8
1.3 Công thức toán học cơ bản	8
1.4 Tại sao gọi là "Deep"?	9
2. Machine learning truyền thống và deep learning	9
2.1 Quy trình của Machine Learning truyền thống	9
2.2 Quy trình của Deep Learning	10
2.3 So sánh chi tiết	11
2.4 Khi nào nên dùng Deep Learning?	12
3. Mạng nơ-ron nhân tạo (neural networks)	12
3.1 Neuron (Nơ-ron) — Đơn vị cơ bản	12
3.2 Layer (Tầng)	13
3.3 Multi-Layer Perceptron (MLP)	14
3.4 Tại sao cần Nonlinearity?	14
4. Các hàm kích hoạt (activation functions)	15
4.1 ReLU (Rectified Linear Unit)	15
4.2 Sigmoid	16
4.3 Softmax	16
4.4 Tại sao cần Activation Functions?	17
5. Kiến trúc mạng nơ-ron	17
5.1 Kiến trúc ví dụ: $8 \rightarrow 16 \rightarrow 8 \rightarrow 1$	17
5.2 Mô hình toán học đầy đủ	18
5.3 Kích thước ma trận (Matrix Dimensions)	18
5.4 Vai trò của từng tầng	19
6. Forward propagation (lan truyền xuôi)	20
6.1 Định nghĩa	20
6.2 Chi tiết từng bước	20

6.3 Lưu trữ giá trị trung gian.....	20
6.4 Code Forward Propagation.....	20
7. Loss functions (hàm mất mát).....	21
7.1 Tại sao cần Loss Function?	21
7.2 Binary Cross-Entropy Loss	22
7.3 Cross-Entropy Loss (Multi-class)	22
7.4 Các Loss Functions khác cho các bài toán khác	22
7.5 Numerical Stability	24
8. Backpropagation (lan truyền ngược)	24
8.1 Định nghĩa chi tiết	24
8.2 Gradient là gì?	24
8.3 Chain Rule (Quy tắc chuỗi) — Nền tảng toán học.....	25
8.4 Backpropagation cho kiến trúc $8 \rightarrow 16 \rightarrow 8 \rightarrow 1$	26
8.5 Luồng Backward	27
8.6 Tầm quan trọng của việc lưu cache.....	27
8.7 Code Backpropagation	27
9. Định nghĩa gradient descent.....	29
9.1 Định nghĩa	29
9.2 Công thức cập nhật	29
9.3 Cập nhật cho từng tham số	29
9.4 Learning Rate — Ảnh hưởng và lựa chọn.....	30
9.5 Các biến thể của Gradient Descent	30
9.6 Các Optimizer nâng cao	31
9.7 Chu trình học hoàn chỉnh.....	31
10. Training loop.....	31
10.1 Cấu trúc cơ bản	31
10.2 Một epoch là gì?.....	32
10.3 Mini-batch Training.....	33
10.4 Training Loop trong PyTorch	33
11. Các metrics đánh giá trong deep learning	34

11.1 Tổng quan về Metrics.....	34
11.2 Metrics cho Classification (Phân loại)	34
11.3 Metrics cho Regression (Hồi quy).....	38
11.4 Metrics cho Ranking	41
11.5 Metrics cho Segmentation.....	41
11.6 Training vs Validation vs Test.....	42
11.7 Cross-Validation	42
12. Training curves và chẩn đoán	43
12.1 Training Curves.....	43
12.2 Underfitting (Underfitting)	43
12.3 Good Fit.....	43
12.4 Overfitting.....	44
12.5 Generalization.....	44
12.6 Regularization Techniques	44
13. Pytorch và deep learning frameworks	45
13.1 Tại sao dùng Framework?.....	45
13.2 Xây dựng MLP trong PyTorch.....	46
13.3 PyTorch Training Cycle	46
13.4 Training vs Evaluation Mode	47
13.5 Loss Function và Optimizer	47
13.6 Lưu và Load Model.....	47
14. Từ numpy đến pytorch	48
14.1 So sánh Code.....	48
14.2 Conceptual Mapping.....	48
15.1 Hidden Representation là gì?	49
15.2 Ý nghĩa của Learned Representation	50
15.3 Representation Learning	50
15.4 Tại sao Deep Learning gọi là "Deep"?.....	50
16. Tổng kết và mental model	51
16.1 Bốn phương trình quan trọng nhất.....	51

16.2 Mental Model hoàn chỉnh.....	51
16.3 Từ bài toán đơn giản đến Modern Deep Learning	52
16.4 Từ NumPy đến Framework	52
17. Các khái niệm nâng cao	53
17.1 Softmax và Logits	53
17.2 Dataset và DataLoader trong PyTorch	53
17.3 Từ Vectors đến Images.....	53
18. Trả lời các câu hỏi ôn tập.....	54
PHẦN II: THỰC HÀNH	61
BÀI 1: DỰ ĐOÁN TIỂU ĐƯỜNG. Dataset: Diabetes Dataset	61
1. IMPORT & SETUP	61
2. LOAD DATA & INITIAL UNDERSTANDING.....	64
3. DATA QUALITY - MISSING VALUES & '?' MARKERS	68
4. DATA CLEANING.....	72
5. EDA - NUMERICAL & CATEGORICAL	76
6. CATEGORICAL ANALYSIS & HANDLING	81
7. IMPUTATION & ENCODING.....	84
8. TRAIN / VALIDATION / TEST SPLIT + NORMALIZATION	87
9. BASELINE MACHINE LEARNING MODELS.....	91
10. VISUALIZE ML RESULTS.....	95
11. DEEP LEARNING FROM SCRATCH - 5 LAYER MLP	99
12. TRAINING LOOP WITH MINI-BATCH + ADAM & SGD.....	104
13. TRAIN BASELINE DEEP LEARNING MODEL	108
14. VISUALIZE BASELINE TRAINING	111
15. HYPERPARAMETER EXPERIMENT 1 - LEARNING RATE .	114
16. HYPERPARAMETER EXPERIMENT 2 - BATCH SIZE (OPTIMIZED).....	119
17. HYPERPARAMETER EXPERIMENT 3 - ARCHITECTURE (OPTIMIZED).....	124
18. HYPERPARAMETER EXPERIMENT 4 - OPTIMIZER (OPTIMIZED).....	129

19. BEST MODEL SELECTION & FINAL EVALUATION	134
20. RETRAIN BEST CONFIG WITH MORE EPOCHS.....	137
21. VISUALIZE BEST DL MODEL	140
22. FINAL COMPARISON - ML VS DL.....	143
23. TOP-K FEATURE IMPORTANCE VIA PERMUTATION.....	147
24. DECISION THRESHOLD TUNING.....	150
25. FINAL SUMMARY	154
26. Lưu model	157
BÀI 2: Dự đoán giá nhà. Dataset: Vietnam Real Estate Datasets - Catalyst.....	159
1. Load dataset.....	160
2. Data inspection and understanding.....	162
3. Data cleaning	165
4. Outlier analysis bằng IQR.....	167
5. Visualization.....	169
6. Location analysis	173
7. Feature engineering và preprocessing.....	176
8. Train / validation / test split.....	178
9. ML baselines	181
10. Deep Learning from scratch bằng NumPy	183
11. Scaling dữ liệu cho Deep Learning	187
12. Baseline Deep Learning	189
13. Hyperparameter experiments	192
14. So sánh learning curves	197
15. Chọn best Deep Learning model.....	199
16. Final comparison ML vs DL	201
17. Best DL — Actual vs Predicted	203
18. Residual analysis	205
19. Ảnh hưởng của learning rate / optimizer / batch size / depth.....	207
20. K-Fold Cross Validation cho DL (optional)	210

21. Save model bằng NumPy	213
22. Final leaderboard	215
BÀI 3: PHÂN TÍCH COMMENTS KHÁCH HÀNG. Dataset: Women's E-Commerce Clothing Reviews	217
1. Dataset and Problem Definition.....	217
2. Data Inspection and Understanding.....	218
3. Data Cleaning	222
4. Feature Engineering.....	224
5. Train / Validation / Test Split.....	225
6. Build Feature Matrix	226
7. Baseline Machine Learning.....	228
8. Deep Learning from Scratch — NumPy Only	230
9. Baseline NumPy Deep Learning	235
10. Hyperparameter Experiments	237
11. Chọn Best Deep Learning Model.....	240
12. So sánh ML vs Deep Learning	242
13. Confusion Matrix — Best DL	245
14. Overfitting / Underfitting Analysis.....	247
15. Feature Importance.....	249
16. Save Model / Preprocessing.....	251
17. Final Conclusion	252

PHẦN I: CƠ SỞ LÝ THUYẾT

1. Giới thiệu tổng quan

1.1 Bối cảnh và động lực

Trong lịch sử phát triển của Machine Learning, con người luôn phải đối mặt với một thách thức lớn: làm thế nào để biểu diễn dữ liệu một cách hiệu quả để máy tính có thể học được? Trong nhiều thập kỷ, các nhà nghiên cứu và kỹ sư phải dành phần lớn thời gian cho **feature engineering** (kỹ nghệ đặc trưng) — quá trình thủ công thiết kế các đặc trưng có ý nghĩa từ dữ liệu thô. Ví dụ, để nhận dạng một khuôn mặt trong ảnh, họ phải tự tay viết các thuật toán phát hiện cạnh, phát hiện mắt, mũi, miệng, v.v. Điều này không chỉ tốn thời gian mà còn đòi hỏi kiến thức chuyên môn sâu sắc về lĩnh vực.

Deep Learning ra đời như một giải pháp mang tính cách mạng cho vấn đề này. Thay vì con người phải tự thiết kế các đặc trưng, Deep Learning cho phép mô hình **tự động học các biểu diễn** (learned representations) từ dữ liệu thô. Đây là sự chuyển dịch triết lý căn bản: từ "con người dạy máy" sang "máy tự học cách biểu diễn".

1.2 Định nghĩa Deep Learning

Deep Learning (Học sâu) là một nhánh của Machine Learning sử dụng các **mạng nơ-ron nhân tạo nhiều tầng** (multi-layer artificial neural networks) để học các biểu diễn dữ liệu ở nhiều mức độ trừu tượng khác nhau. Thuật ngữ "deep" (sâu) ở đây đề cập đến **số lượng tầng** (layers) trong mạng — một mạng càng có nhiều tầng thì càng "sâu".

Ý tưởng trung tâm:

Deep Learning = Composition of learned functions

(Học sâu = Sự kết hợp của các hàm được học)

Thay vì chỉ học một ánh xạ trực tiếp từ input đến output, Deep Learning xây dựng **nhiều phép biến đổi liên tiếp** và học các tham số của những phép biến đổi này.

1.3 Công thức toán học cơ bản

Mô hình truyền thống:

$$\hat{y} = f(X)$$

Trong đó f là một hàm duy nhất (có thể là logistic regression, SVM, decision tree, v.v.).

Mô hình Deep Learning:

$$X \rightarrow f_1(X) \rightarrow f_2(f_1(X)) \rightarrow f_3(f_2(f_1(X)))$$

Do đó:

$$\hat{y} = f_3(f_2(f_1(X)))$$

Hoặc sử dụng ký hiệu hàm hợp (function composition):

$$\hat{y} = (f_3 \circ f_2 \circ f_1)(X)$$

Giải thích ký hiệu: $f_3 \circ f_2 \circ f_1$ có nghĩa là "áp dụng f_1 trước, sau đó f_2 , sau đó f_3 ". Đây là cách viết chuẩn trong toán học để biểu diễn composition of functions.

1.4 Tại sao gọi là "Deep"?

Thuật ngữ "deep" đề cập đến **số lượng tầng** trong mạng nơ-ron. Một mạng có nhiều tầng ẩn (hidden layers) được gọi là mạng "deep". Mỗi tầng thực hiện một phép biến đổi, và các phép biến đổi này được xếp chồng lên nhau.

Phân biệt width và depth:

- **Width (độ rộng):** Số lượng neuron trong một tầng
- **Depth (độ sâu):** Số lượng tầng tuần tự

Ví dụ:

- Mạng $8 \rightarrow 100 \rightarrow 1$: tương đối **rộng** nhưng **nh nông** (shallow)
- Mạng $8 \rightarrow 16 \rightarrow 16 \rightarrow 1$: **sâu** hơn (deeper)

Lưu ý quan trọng: Nhiều tầng hơn không tự động tốt hơn. Kiến trúc phải phù hợp với bài toán. Một mạng quá sâu với dữ liệu ít sẽ dẫn đến overfitting.

2. Machine learning truyền thống và deep learning

2.1 Quy trình của Machine Learning truyền thống

Data $\rightarrow$ Feature Engineering $\rightarrow$ ML Model $\rightarrow$ Prediction

Con người thiết kế các đặc trưng (features) dựa trên kiến thức chuyên môn. Ví dụ, trong bài toán dự đoán tiểu đường (Pima Indians Diabetes Dataset), con người quyết định rằng các đặc trưng sau là quan trọng:

Index	Feature	Meaning
0	Pregnancies	Number of pregnancies
1	Glucose	Plasma glucose concentration
2	BloodPressure	Blood pressure
3	SkinThickness	Skin fold thickness
4	Insulin	Serum insulin
5	BMI	Body mass index
6	DiabetesPedigreeFunction	Diabetes pedigree score
7	Age	Age

Đặc điểm của ML truyền thống:

- Con người quyết định những biến đổi nào hữu ích
- Cần kiến thức miền (domain knowledge) sâu sắc
- Features có ý nghĩa rõ ràng, dễ hiểu
- **Interpretability** (khả năng giải thích) cao
- Cần ít dữ liệu hơn
- Chi phí tính toán thấp hơn

2.2 Quy trình của Deep Learning

Data → Learned Representations → Prediction

Mô hình tự học các biểu diễn trung gian (intermediate representations) từ dữ liệu.

Đặc điểm của Deep Learning:

- Mô hình tự học features
- Không cần feature engineering thủ công
- Features học được thường **không có ý nghĩa trực tiếp** với con người
- **Interpretability** thấp hơn
- Cần nhiều dữ liệu hơn
- Chi phí tính toán cao hơn (thường cần GPU)

2.3 So sánh chi tiết

Tiêu chí	Machine Learning truyền thống	Deep Learning
Representation	Prepared (chuẩn bị thủ công)	Learned (học tự động)
Model capacity	Thấp hơn	Cao hơn
Nonlinearity	Không (hoặc hạn chế)	Có (nhờ activation functions)
Parameters	Ít hơn	Nhiều hơn
Interpretability	Cao hơn	Thấp hơn
Training cost	Thấp hơn	Cao hơn
Data requirement	Ít hơn	Nhiều hơn
Feature engineering	Cần thiết	Không cần

Tiêu chí	Machine Learning truyền thống	Deep Learning
Hardware	CPU là đủ	Thường cần GPU
Thời gian huấn luyện	Nhanh hơn	Chậm hơn
Khả năng mở rộng	Hạn chế	Tốt hơn với dữ liệu lớn

2.4 Khi nào nên dùng Deep Learning?

Deep Learning không tự động tốt hơn Machine Learning truyền thống. Cần xem xét các câu hỏi sau:

1. **Dataset có đủ lớn không?** Deep Learning cần nhiều dữ liệu để học representations hiệu quả.
2. **Learned representation có hữu ích không?** Nếu dữ liệu đã có features tốt, ML truyền thống có thể đủ.
3. **Validation performance có cải thiện không?** So sánh với baseline.
4. **Test performance có cải thiện không?** Đánh giá trên dữ liệu chưa từng thấy.
5. **Bao nhiêu tham số được thêm vào?** Trade-off giữa performance và complexity.
6. **Chi phí tính toán là bao nhiêu?** Thời gian, phần cứng, năng lượng.
7. **Sự cải thiện có ý nghĩa thực tiễn không?** Đôi khi cải thiện 1-2% không đáng kể so với chi phí.

3. Mạng nơ-ron nhân tạo (neural networks)

3.1 Neuron (Nơ-ron) — Đơn vị cơ bản

Một neuron đơn lẻ là đơn vị tính toán cơ bản của mạng nơ-ron. Nó nhận đầu vào $x_1, x_2, \dots, x_d$ và tính toán:

$$z = w_1x_1 + w_2x_2 + \dots + w_dx_d + b = w^Tx + b$$

Sau đó áp dụng hàm kích hoạt (activation function):

$$h = \phi(z)$$

Trong đó:

- $w = [w_1, w_2, \dots, w_d]^T$: **weights** (trọng số) — thể hiện tầm quan trọng của từng input
- b : **bias** (độ lệch) — cho phép dịch chuyển hàm kích hoạt
- ϕ : **activation function** — giới thiệu tính phi tuyến
- h : **output** của neuron

Ý nghĩa trực quan: Neuron học cách kết hợp các đặc trưng đầu vào. Trọng số w_i càng lớn thì đặc trưng x_i càng quan trọng. Bias b cho phép neuron "kích hoạt" ngay cả khi tất cả input bằng 0.

Ví dụ: Một neuron trong bài toán dự đoán tiểu đường có thể học: "Nếu glucose cao VÀ BMI cao THÌ nguy cơ tiểu đường cao". Trọng số của glucose và BMI sẽ lớn, trong khi trọng số của các đặc trưng khác nhỏ hơn.

3.2 Layer (Tầng)

Một tầng chứa nhiều neuron. Với đầu vào $x \in \mathbb{R}^d$:

$$z = Wx + b$$

Trong đó:

- $W \in \mathbb{R}^{k \times d}$: ma trận trọng số (mỗi hàng là weights của một neuron)
- $b \in \mathbb{R}^k$: vector bias
- $z \in \mathbb{R}^k$: output tuyến tính

Sau đó áp dụng activation:

$$h = \phi(z)$$

Conceptually:

$$x \in \mathbb{R}^d \rightarrow h \in \mathbb{R}^k$$

Mỗi chiều output tương ứng với một **phép biến đổi được học** của input. Nếu $k > d$, tầng đang **mở rộng** chiều (expansion). Nếu $k < d$, tầng đang **nén** chiều (compression).

3.3 Multi-Layer Perceptron (MLP)

MLP là mạng nơ-ron có nhiều tầng fully connected (mỗi neuron ở tầng sau kết nối với tất cả neuron ở tầng trước). Ví dụ:

$$x \rightarrow W_1x + b_1 \rightarrow \text{ReLU} \rightarrow W_2h + b_2 \rightarrow \hat{y}$$

Kiến trúc ví dụ:

$$d \rightarrow 64 \rightarrow 32 \rightarrow C$$

Forward propagation trong MLP 2 tầng ẩn:

$$\begin{aligned}h_1 &= \text{ReLU}(W_1x + b_1) \\h_2 &= \text{ReLU}(W_2h_1 + b_2) \\z &= W_3h_2 + b_3\end{aligned}$$

Giải thích: h_1 là learned representation thứ nhất (64 chiều), h_2 là learned representation thứ hai (32 chiều), z là logits (C chiều cho C classes).

3.4 Tại sao cần Nonlinearity?

Vấn đề: Nếu tất cả các tầng đều tuyến tính:

$$\begin{aligned}h_1 &= W_1x + b_1 \\h_2 &= W_2h_1 + b_2\end{aligned}$$

Thì composition vẫn là một phép biến đổi tuyến tính:

$$h_2 = W_2(W_1x + b_1) + b_2 = W_2W_1x + W_2b_1 + b_2$$

Đặt $W' = W_2W_1$ và $b' = W_2b_1 + b_2$, ta có:

$$h_2 = W'x + b'$$

Đây vẫn là một phép biến đổi tuyến tính! Dù có xếp chồng bao nhiêu tầng tuyến tính, kết quả cuối cùng vẫn chỉ là một phép biến đổi tuyến tính duy nhất.

Kết luận: Xếp chồng các tầng tuyến tính **không cung cấp sức mạnh biểu diễn** (expressive power) như mong đợi từ mạng sâu.

Depth + Nonlinearity → Expressive Model

Độ sâu chỉ có ý nghĩa khi đi kèm với phi tuyến tính. Chính các activation functions phi tuyến cho phép mạng học được các hàm phức tạp.

4. Các hàm kích hoạt (activation functions)

4.1 ReLU (Rectified Linear Unit)

Công thức:

$$\text{ReLU}(x) = \max(0, x)$$

Ví dụ:

- $\text{ReLU}(-3) = 0$
- $\text{ReLU}(0) = 0$
- $\text{ReLU}(4) = 4$

Đạo hàm:

$$\text{ReLU}'(x) = \begin{cases} 1 & \text{nếu } x > 0 \\ 0 & \text{nếu } x \leq 0 \end{cases}$$

Trong thực tế, đạo hàm tại 0 thường được đặt là 0.

Ý nghĩa:

- Giới thiệu tính phi tuyến (nonlinearity)
- Giá trị âm bị đưa về 0, giá trị dương được giữ nguyên
- Tính toán đơn giản, hiệu quả
- Giảm vấn đề **vanishing gradient** so với sigmoid/tanh
- Tạo ra **sparse representations** (nhiều neuron output bằng 0)

Trong PyTorch:

python

nn.ReLU()

Hạn chế: "Dying ReLU" — nếu một neuron chỉ nhận giá trị âm, nó sẽ luôn output 0 và không bao giờ được cập nhật (vì đạo hàm bằng 0).

4.2 Sigmoid

Công thức:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Tính chất:

- Output nằm trong khoảng (0, 1)
- Được sử dụng cho **binary classification** ở tầng output
- Output có thể được hiểu như xác suất

Đạo hàm:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Nhược điểm:

- **Vanishing gradient:** khi z rất lớn hoặc rất nhỏ, đạo hàm gần 0
- Output không centered at 0
- Tính toán phức tạp hơn ReLU (có hàm exp)

Ứng dụng: Tầng output cho binary classification.

4.3 Softmax

Công thức:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}$$

Tính chất:

- Chuyển đổi vector logits thành **phân phối xác suất**
- Tổng các output bằng 1: $\sum_i p_i = 1$
- Dùng cho **multi-class classification**

Predicted class:

$$\hat{y} = \arg \max_i z_i$$

Lưu ý quan trọng: Khi sử dụng CrossEntropyLoss trong PyTorch, model nên trả về **logits** (output thô) thay vì áp dụng softmax bên trong model. PyTorch sẽ tự động áp dụng softmax bên trong loss function để đảm bảo numerical stability.

4.4 Tại sao cần Activation Functions?

Important: Without nonlinear activation functions, stacking several linear layers would still produce one overall linear transformation. Therefore, nonlinear activations are essential for learning complex functions.

Không có activation functions phi tuyến, việc xếp chồng nhiều tầng tuyến tính vẫn chỉ tạo ra một phép biến đổi tuyến tính tổng thể. Do đó, activation functions phi tuyến là **thiết yếu** để học các hàm phức tạp.

Trực quan: Activation functions giống như "công tắc" — chúng quyết định neuron nào được kích hoạt và ở mức độ nào. Nhờ đó, mạng có thể học được các ranh giới quyết định phi tuyến phức tạp.

5. Kiến trúc mạng nơ-ron

5.1 Kiến trúc ví dụ: $8 \rightarrow 16 \rightarrow 8 \rightarrow 1$

Đây là kiến trúc được sử dụng trong bài toán dự đoán tiểu đường:

- **Input layer:** 8 features
- **Layer 1:** $8 \rightarrow 16$, ReLU
- **Layer 2:** $16 \rightarrow 8$, ReLU
- **Layer 3 (Output):** $8 \rightarrow 1$, Sigmoid

Cấu trúc khái niệm:

$$X \xrightarrow{f_1} H_1 \xrightarrow{f_2} H_2 \xrightarrow{f_3} \hat{y}$$

Trong đó:

- X : raw features (8 chiều) — thông tin bệnh nhân thô
- H_1 : learned representation 1 (16 chiều) — biểu diễn học được thứ nhất

- H_2 : learned representation 2 (8 chiều) — biểu diễn học được thứ hai
- $\hat{y}$: prediction (1 chiều) — xác suất mắc tiểu đường

Hình ảnh minh họa:

text

Input	Layer 1	Layer 2	Layer 3
8 features	$8 \rightarrow 16$	$16 \rightarrow 8$	$8 \rightarrow 1$
	ReLU	ReLU	Sigmoid

Patient data Representation 1 Representation 2 Probability

5.2 Mô hình toán học đầy đủ

Layer 1:

$$Z_1 = XW_1 + b_1$$

$$H_1 = \text{ReLU}(Z_1)$$

Layer 2:

$$Z_2 = H_1W_2 + b_2$$

$$H_2 = \text{ReLU}(Z_2)$$

Layer 3:

$$Z_3 = H_2W_3 + b_3$$

$$\hat{y} = \sigma(Z_3)$$

Kết hợp:

$$\hat{y} = \sigma(\text{ReLU}(\text{ReLU}(XW_1 + b_1)W_2 + b_2)W_3 + b_3)$$

Đây là mô hình Deep Learning hoàn chỉnh của chúng ta.

5.3 Kích thước ma trận (Matrix Dimensions)

Với N mẫu (patients):

Layer 1:

- $X \in \mathbb{R}^{N \times 8}$

- $W_1 \in \mathbb{R}^{8 \times 16}$
- $b_1 \in \mathbb{R}^{1 \times 16}$
- $Z_1 = XW_1 + b_1 \in \mathbb{R}^{N \times 16}$
- $H_1 \in \mathbb{R}^{N \times 16}$

Layer 2:

- $W_2 \in \mathbb{R}^{16 \times 8}$
- $b_2 \in \mathbb{R}^{1 \times 8}$
- $Z_2 = H_1W_2 + b_2 \in \mathbb{R}^{N \times 8}$
- $H_2 \in \mathbb{R}^{N \times 8}$

Layer 3:

- $W_3 \in \mathbb{R}^{8 \times 1}$
- $b_3 \in \mathbb{R}^{1 \times 1}$
- $Z_3 = H_2W_3 + b_3 \in \mathbb{R}^{N \times 1}$
- $\hat{y} \in \mathbb{R}^{N \times 1}$

Quy tắc: $(N \times d) \times (d \times k) = N \times k$

5.4 Vai trò của từng tầng

Stage	Operation	Role
Input	Raw features	Mô tả bệnh nhân
Layer 1	$8 \rightarrow 16$	Học biểu diễn đầu tiên
Layer 2	$16 \rightarrow 8$	Kết hợp các biểu diễn
Layer 3	$8 \rightarrow 1$	Tạo dự đoán

Giải thích:

- **Layer 1:** Học representation từ data — tìm các pattern đơn giản trong features gốc

- **Layer 2:** Học representation từ representation — kết hợp các pattern đơn giản thành pattern phức tạp hơn
- **Layer 3:** Dự đoán từ learned representation — ánh xạ representation cuối cùng đến output

Đây là một trong những ý tưởng cơ bản đằng sau Deep Learning: **học theo cấp độ trừu tượng tăng dần.**

6. Forward propagation (lan truyền xuôi)

6.1 Định nghĩa

Forward propagation là quá trình tính toán dự đoán từ dữ liệu đầu vào. Computation diễn ra từ trái sang phải:

$$X \rightarrow Z_1 \rightarrow H_1 \rightarrow Z_2 \rightarrow H_2 \rightarrow Z_3 \rightarrow \hat{y}$$

6.2 Chi tiết từng bước

$$\begin{aligned} Z_1 &= XW_1 + b_1 \\ H_1 &= \text{ReLU}(Z_1) \\ Z_2 &= H_1W_2 + b_2 \\ H_2 &= \text{ReLU}(Z_2) \\ Z_3 &= H_2W_3 + b_3 \\ \hat{y} &= \sigma(Z_3) \end{aligned}$$

6.3 Lưu trữ giá trị trung gian

Trong quá trình forward propagation, cần lưu trữ các giá trị trung gian:

$$Z_1, H_1, Z_2, H_2, Z_3, \hat{y}$$

Lý do: Các giá trị này cần thiết cho backpropagation. Ví dụ, đạo hàm của ReLU cần biết $Z_1 > 0$ hay không.

Forward pass tạo ra thông tin mà backward pass sẽ sử dụng. Đây chính là ý tưởng cơ bản đằng sau **computational graphs** được sử dụng bởi các Deep Learning frameworks hiện đại.

6.4 Code Forward Propagation

python

```
def forward(X):
```

```

# Layer 1
z1 = X @ W1 + b1
h1 = relu(z1)

# Layer 2
z2 = h1 @ W2 + b2
h2 = relu(z2)

# Layer 3
z3 = h2 @ W3 + b3
y_hat = sigmoid(z3)

cache = {
    "X": X, "z1": z1, "h1": h1,
    "z2": z2, "h2": h2,
    "z3": z3, "y_hat": y_hat
}

return y_hat, cache

```

7. Loss functions (hàm mất mát)

7.1 Tại sao cần Loss Function?

Ban đầu, weights là ngẫu nhiên. Do đó mạng sẽ tạo ra dự đoán kém. Ví dụ:

- $y = 1, \hat{y} = 0.23 \rightarrow$ Model sai

Cần một cách để **đo lường mức độ sai**. Đây là vai trò của loss function.

Loss function (hàm mất mát) là một hàm toán học đo lường sự khác biệt giữa dự đoán của mô hình và giá trị thực tế. Giá trị loss càng nhỏ, mô hình càng chính xác.

7.2 Binary Cross-Entropy Loss

Công thức:

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Phân tích:

- Khi $y = 1$: $L = -\log(\hat{y})$
 - Nếu $\hat{y} = 0.99$: $L = -\log(0.99) \approx 0.01$ (loss nhỏ)
 - Nếu $\hat{y} = 0.01$: $L = -\log(0.01) \approx 4.6$ (loss rất lớn)
- Khi $y = 0$: $L = -\log(1 - \hat{y})$
 - Nếu $\hat{y} = 0.01$: $L = -\log(0.99) \approx 0.01$ (loss nhỏ)
 - Nếu $\hat{y} = 0.99$: $L = -\log(0.01) \approx 4.6$ (loss rất lớn)

Ý nghĩa: Loss cho mạng biết "dự đoán hiện tại của tôi tệ đến mức nào". Mục tiêu của quá trình học là **giảm loss**.

7.3 Cross-Entropy Loss (Multi-class)

Công thức:

$$\mathcal{L} = \text{CrossEntropyLoss}(z, y)$$

Trong đó z là logits từ model và y là target class.

Trong PyTorch:

python

```
criterion = nn.CrossEntropyLoss()
```

Lưu ý: Khi dùng CrossEntropyLoss, model nên trả về logits (không áp dụng softmax bên trong model).

7.4 Các Loss Functions khác cho các bài toán khác

7.4.1 Mean Squared Error (MSE) — cho Regression

Công thức:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Đặc điểm:

- Phù hợp cho bài toán hồi quy (dự đoán giá trị liên tục)
- Phạt nặng các sai số lớn (do bình phương)
- Đạo hàm: $\frac{\partial \text{MSE}}{\partial \hat{y}_i} = \frac{2}{N} (\hat{y}_i - y_i)$

Ứng dụng: Dự đoán giá nhà, nhiệt độ, doanh thu, v.v.

7.4.2 Mean Absolute Error (MAE) — cho Regression

Công thức:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

Đặc điểm:

- Ít bị ảnh hưởng bởi outliers hơn MSE
- Đạo hàm không liên tục tại 0
- Robust hơn với dữ liệu có nhiễu

7.4.3 Huber Loss — cho Regression

Công thức:

$$L_{\delta}(y, \hat{y}) = \begin{cases} \frac{1}{2} (y - \hat{y})^2 & \text{neú } |y - \hat{y}| \leq \delta \\ \delta |y - \hat{y}| - \frac{1}{2} \delta^2 & \text{neú } |y - \hat{y}| > \delta \end{cases}$$

Đặc điểm:

- Kết hợp ưu điểm của MSE và MAE
- Ít nhạy cảm với outliers hơn MSE
- Khả vi tại 0

7.4.4 Hinge Loss — cho SVM

Công thức:

$$L = \max(0, 1 - y \cdot \hat{y})$$

Ứng dụng: Support Vector Machines, margin-based classification.

7.4.5 Kullback-Leibler Divergence — cho Distribution Matching

Công thức:

$$D_{KL}(P \parallel Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

Ứng dụng: Variational Autoencoders, knowledge distillation.

7.5 Numerical Stability

Trong implementation, cần thêm một giá trị nhỏ ϵ để tránh $\log(0)$:

```
python
```

```
eps = 1e-8
```

```
y_hat = np.clip(y_hat, eps, 1 - eps)
```

Lý do: $\log(0)$ là undefined (tiến tới $-\infty$), gây lỗi NaN trong tính toán.

8. Backpropagation (lan truyền ngược)

8.1 Định nghĩa chi tiết

Backpropagation (lan truyền ngược) là thuật toán tính toán **gradients** (đạo hàm) của loss function đối với các tham số của mạng. Nó cho biết mỗi tham số đã đóng góp như thế nào vào lỗi.

Tại sao gọi là "backpropagation"? Vì gradients được tính từ tầng cuối cùng (output) lan truyền ngược về các tầng trước đó.

Mục đích: Cung cấp thông tin để cập nhật weights theo hướng giảm loss.

8.2 Gradient là gì?

Gradient là một vector chứa các đạo hàm riêng (partial derivatives) của một hàm số đối với từng biến số của nó.

Ví dụ: Nếu $L = f(w_1, w_2, b)$, thì gradient của L là:

$$\nabla L = \left[\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \frac{\partial L}{\partial b} \right]$$

Ý nghĩa trực quan: Gradient chỉ ra hướng và mức độ thay đổi của loss khi thay đổi từng tham số. Nếu $\frac{\partial L}{\partial w_1} > 0$, tăng w_1 sẽ làm tăng loss. Ngược lại, nếu $\frac{\partial L}{\partial w_1} < 0$, tăng w_1 sẽ làm giảm loss.

Trong Deep Learning: Chúng ta muốn tìm bộ tham số θ^* sao cho loss nhỏ nhất:

$$\theta^* = \arg \min_{\theta} L(\theta)$$

Gradient descent sử dụng gradient để cập nhật tham số theo hướng ngược lại với gradient.

8.3 Chain Rule (Quy tắc chuỗi) — Nền tảng toán học

Chain rule là công cụ toán học cơ bản để tính đạo hàm của hàm hợp (composite function).

Phát biểu: Nếu $y = f(u)$ và $u = g(x)$, thì:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

Mở rộng cho nhiều biến:

Nếu $L = L(H_2)$, $H_2 = H_2(H_1)$, và $H_1 = H_1(W_1)$, thì:

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial H_2} \cdot \frac{\partial H_2}{\partial H_1} \cdot \frac{\partial H_1}{\partial W_1}$$

Tại sao chain rule quan trọng trong Deep Learning? Vì mạng nơ-ron là một **composition of functions** (hàm hợp). Để tính gradient của loss đối với một weight ở tầng đầu, cần nhân các gradient trung gian qua nhiều tầng.

Backpropagation không phải là một phép toán Deep Learning bí ẩn. Nó là **ứng dụng có hệ thống của quy tắc chuỗi** (chain rule) cho một composition of functions.

8.4 Backpropagation cho kiến trúc $8 \rightarrow 16 \rightarrow 8 \rightarrow 1$

8.4.1 Layer 3 (Output Layer)

Với sigmoid + binary cross-entropy, gradient rất thuận tiện:

$$\frac{\partial L}{\partial Z_3} = \hat{y} - y$$

Tại sao? Vì đạo hàm của sigmoid kết hợp với binary cross-entropy triệt tiêu nhau một cách đẹp đẽ:

$$\frac{\partial L}{\partial Z_3} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial Z_3} = \left(-\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} \right) \cdot \hat{y}(1-\hat{y}) = \hat{y} - y$$

Do đó:

$$dZ_3 = \hat{y} - y$$

$$\boxed{dW_3 = H_2^T dZ_3}$$

$$\boxed{db_3 = \sum dZ_3}$$

Giải thích kích thước:

- $dZ_3 \in \mathbb{R}^{N \times 1}$
- $H_2 \in \mathbb{R}^{N \times 8}$
- $dW_3 = H_2^T dZ_3 \in \mathbb{R}^{8 \times 1}$

8.4.2 Layer 2

Gradient lan truyền ngược:

$$dH_2 = dZ_3 W_3^T$$

Vì $H_2 = \text{ReLU}(Z_2)$:

$$dZ_2 = dH_2 \odot \text{ReLU}'(Z_2)$$

$$dW_2 = H_1^T dZ_2$$

$$db_2 = \sum dZ_2$$

Trong đó $\odot$ là element-wise multiplication (Hadamard product).

8.4.3 Layer 1

$$\begin{aligned}dH_1 &= dZ_2 W_2^T \\dZ_1 &= dH_1 \odot \text{ReLU}'(Z_1) \\dW_1 &= X^T dZ_1 \\db_1 &= \sum dZ_1\end{aligned}$$

8.5 Luồng Backward

Forward direction:

$$X \rightarrow H_1 \rightarrow H_2 \rightarrow \hat{y} \rightarrow L$$

Backward direction:

$$L \rightarrow W_3 \rightarrow W_2 \rightarrow W_1$$

text

Forward: $X \longrightarrow H_1 \longrightarrow H_2 \longrightarrow \hat{y} \longrightarrow L$

Backward: $L \longrightarrow W_3 \longrightarrow W_2 \longrightarrow W_1$

8.6 Tầm quan trọng của việc lưu cache

Trong forward pass, cần lưu:

- $X, Z_1, H_1, Z_2, H_2, Z_3, \hat{y}$

Lý do:

- Z_1, Z_2 cần cho ReLU derivative
- H_1, H_2 cần cho weight gradients
- $\hat{y}$ cần cho output gradient

8.7 Code Backpropagation

python

```
def backward(y, cache):
```

```
    X = cache["X"]
```

```
    z1 = cache["z1"]
```

```

h1 = cache["h1"]
z2 = cache["z2"]
h2 = cache["h2"]
yhat = cache["y_hat"]
n = len(X)

# Layer 3
dz3 = (yhat - y) / n
dW3 = h2.T @ dz3
db3 = np.sum(dz3, axis=0, keepdims=True)

# Layer 2
dh2 = dz3 @ W3.T
dz2 = dh2 * relu_derivative(z2)
dW2 = h1.T @ dz2
db2 = np.sum(dz2, axis=0, keepdims=True)

# Layer 1
dh1 = dz2 @ W2.T
dz1 = dh1 * relu_derivative(z1)
dW1 = X.T @ dz1
db1 = np.sum(dz1, axis=0, keepdims=True)

gradients = {
    "dW1": dW1, "db1": db1,
    "dW2": dW2, "db2": db2,
    "dW3": dW3, "db3": db3

```

}

return gradients

9. Định nghĩa gradient descent

9.1 Định nghĩa

Gradient Descent là thuật toán tối ưu hóa (optimization algorithm) được sử dụng để cập nhật các tham số của mô hình theo hướng giảm loss.

Ý tưởng cơ bản: Di chuyển ngược hướng với gradient để tìm điểm cực tiểu của hàm loss.

9.2 Công thức cập nhật

Sau khi biết gradient, cập nhật weights:

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

Trong đó η là **learning rate** (tốc độ học).

Giải thích:

- $\frac{\partial L}{\partial W}$: gradient — hướng tăng nhanh nhất của loss
- Dấu trừ (-): di chuyển ngược hướng gradient để giảm loss
- η : kiểm soát kích thước bước đi

9.3 Cập nhật cho từng tham số

$$\begin{aligned} W_1 &\leftarrow W_1 - \eta \frac{\partial L}{\partial W_1} \\ W_2 &\leftarrow W_2 - \eta \frac{\partial L}{\partial W_2} \\ W_3 &\leftarrow W_3 - \eta \frac{\partial L}{\partial W_3} \end{aligned}$$

Tương tự cho biases:

$$b_1 \leftarrow b_1 - \eta \frac{\partial L}{\partial b_1}$$

$$b_2 \leftarrow b_2 - \eta \frac{\partial L}{\partial b_2}$$

$$b_3 \leftarrow b_3 - \eta \frac{\partial L}{\partial b_3}$$

9.4 Learning Rate — Ảnh hưởng và lựa chọn

Learning rate quá nhỏ:

- Hội tụ chậm
- Có thể mất nhiều thời gian
- Có thể bị kẹt ở local minimum

Learning rate quá lớn:

- Có thể overshoot
- Loss có thể dao động hoặc tăng
- Có thể không hội tụ

Giá trị thường dùng: 0.001, 0.01, 0.1

Chiến lược: Bắt đầu với 0.01, sau đó điều chỉnh dựa trên training curves.

9.5 Các biến thể của Gradient Descent

9.5.1 Batch Gradient Descent

Sử dụng toàn bộ training set để tính gradient trong mỗi bước.

Ưu điểm: Ổn định, hội tụ mượt.

Nhược điểm: Chậm với dữ liệu lớn, tốn bộ nhớ.

9.5.2 Stochastic Gradient Descent (SGD)

Sử dụng một mẫu duy nhất để tính gradient trong mỗi bước.

Ưu điểm: Nhanh, cập nhật thường xuyên.

Nhược điểm: Dao động mạnh, khó hội tụ chính xác.

9.5.3 Mini-batch Gradient Descent

Sử dụng một batch nhỏ (ví dụ: 32, 64, 128 mẫu) để tính gradient.

Ưu điểm: Cân bằng giữa ổn định và tốc độ, tương thích với GPU.

Nhược điểm: Cần chọn batch size phù hợp.

9.6 Các Optimizer nâng cao

9.6.1 Momentum

Thêm "đà" vào quá trình cập nhật:

$$\begin{aligned}v_t &= \beta v_{t-1} + (1 - \beta) \nabla L \\ W &\leftarrow W - \eta v_t\end{aligned}$$

Ưu điểm: Giảm dao động, tăng tốc hội tụ.

9.6.2 Adam (Adaptive Moment Estimation)

Kết hợp Momentum và RMSProp:

$$\begin{aligned}m_t &= \beta_1 m_{t-1} + (1 - \beta_1) \nabla L \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) (\nabla L)^2 \\ W &\leftarrow W - \eta \frac{m_t}{\sqrt{v_t} + \epsilon}\end{aligned}$$

Ưu điểm: Tự động điều chỉnh learning rate cho từng tham số, thường hội tụ nhanh.

Trong PyTorch:

python

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

9.7 Chu trình học hoàn chỉnh

text

Forward $\rightarrow$ Loss $\rightarrow$ Backward $\rightarrow$ Update $\rightarrow$ (repeat)

Một training step bao gồm:

1. **Forward:** Tính dự đoán
2. **Loss:** Đo lường sai số
3. **Backward:** Tính gradients
4. **Update:** Cập nhật parameters

10. Training loop

10.1 Cấu trúc cơ bản

python

```

learning_rate = 0.01
epochs = 1000

for epoch in range(epochs):
    # Forward
    y_hat, cache = forward(X_train)

    # Loss
    loss = binary_cross_entropy(y_train, y_hat)

    # Backward
    gradients = backward(y_train, cache)

    # Gradient descent
    W1 -= learning_rate * gradients["dW1"]
    b1 -= learning_rate * gradients["db1"]
    W2 -= learning_rate * gradients["dW2"]
    b2 -= learning_rate * gradients["db2"]
    W3 -= learning_rate * gradients["dW3"]
    b3 -= learning_rate * gradients["db3"]

    if epoch % 100 == 0:
        print(f'Epoch {epoch:4d} | Loss = {loss:.4f}')

```

10.2 Một epoch là gì?

Epoch là một lần duyệt qua **toàn bộ** training dataset.

Trong một epoch:

1. Forward: $X \rightarrow H_1 \rightarrow H_2 \rightarrow \hat{y}$

2. Loss: $\hat{y} \rightarrow L$
3. Backward: $L \rightarrow \nabla W_3, \nabla W_2, \nabla W_1$
4. Update: $W \rightarrow W - \eta \nabla W$

Số epochs: Thường từ 10 đến 1000, tùy bài toán. Theo dõi training curves để chọn số epoch phù hợp.

10.3 Mini-batch Training

Thay vì xử lý tất cả N observations cùng lúc, có thể chia thành các mini-batches:

$$X \rightarrow X_1, X_2, \dots, X_B$$

Với mỗi batch:

$$\mathcal{L}_b = \mathcal{L}(X_b, y_b)$$

Ưu điểm:

- Sử dụng bộ nhớ hiệu quả
- Tính toán hiệu quả
- Cập nhật parameters thường xuyên hơn
- Tương thích với GPU computation

Trong PyTorch:

```
python
```

```
from torch.utils.data import TensorDataset, DataLoader
```

```
dataset = TensorDataset(X_tensor, y_tensor)
```

```
loader = DataLoader(dataset, batch_size=32, shuffle=True)
```

10.4 Training Loop trong PyTorch

```
python
```

```
for epoch in range(num_epochs):
```

```
    model.train()
```

```
    for X_batch, y_batch in train_loader:
```

```
optimizer.zero_grad()
outputs = model(X_batch)
loss = criterion(outputs, y_batch)
loss.backward()
optimizer.step()
```

Giải thích:

1. `optimizer.zero_grad()`: Xóa gradients cũ
2. `outputs = model(X_batch)`: Forward pass
3. `loss = criterion(outputs, y_batch)`: Tính loss
4. `loss.backward()`: Backward pass (tính gradients)
5. `optimizer.step()`: Cập nhật parameters

11. Các metrics đánh giá trong deep learning

11.1 Tổng quan về Metrics

Metrics (chỉ số đo lường) là các công cụ để đánh giá hiệu suất của mô hình. Việc chọn metric phù hợp phụ thuộc vào:

- Loại bài toán (classification, regression, ranking, v.v.)
- Mục tiêu cụ thể (cân bằng precision/recall, tối ưu accuracy, v.v.)
- Đặc điểm dữ liệu (cân bằng hay mất cân bằng)

11.2 Metrics cho Classification (Phân loại)

11.2.1 Accuracy (Độ chính xác)

Công thức:

$$\text{Accuracy} = \frac{\text{Số dự đoán đúng}}{\text{Tổng số dự đoán}} = \frac{TP + TN}{TP + TN + FP + FN}$$

Ưu điểm:

- Đơn giản, dễ hiểu
- Phù hợp khi các class cân bằng

Nhược điểm:

- Không phù hợp khi dữ liệu mất cân bằng (imbalanced)
- Ví dụ: Nếu 95% là class 0, model luôn dự đoán 0 sẽ có accuracy 95% nhưng vô dụng

Khi nào dùng: Khi các class có tần suất tương đương và chi phí của các loại lỗi là như nhau.

11.2.2 Confusion Matrix (Ma trận nhầm lẫn)

	Predicted Positive	Predicted Negative
Actual Positive	TP (True Positive)	FN (False Negative)
Actual Negative	FP (False Positive)	TN (True Negative)

Trong đó:

- **TP:** Dự đoán positive, thực tế positive → Đúng
- **TN:** Dự đoán negative, thực tế negative → Đúng
- **FP:** Dự đoán positive, thực tế negative → Sai (Type I error)
- **FN:** Dự đoán negative, thực tế positive → Sai (Type II error)

Ví dụ: Trong bài toán phát hiện ung thư:

- **TP:** Bệnh nhân có ung thư, model dự đoán có ung thư → Đúng
- **TN:** Bệnh nhân không ung thư, model dự đoán không ung thư → Đúng
- **FP:** Bệnh nhân không ung thư, model dự đoán có ung thư → Sai (dương tính giả)
- **FN:** Bệnh nhân có ung thư, model dự đoán không ung thư → Sai (âm tính giả) — **nguy hiểm nhất**

11.2.3 Precision (Độ chính xác dương tính)

Công thức:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Ý nghĩa: Trong số các mẫu được dự đoán là positive, bao nhiêu phần trăm thực sự là positive?

Khi nào quan trọng: Khi false positive có chi phí cao.

- Ví dụ: Spam detection — không muốn email quan trọng bị đánh dấu là spam.
- Ví dụ: Hệ thống gợi ý — không muốn gợi ý sản phẩm kém chất lượng.

11.2.4 Recall (Độ nhạy)

Công thức:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Ý nghĩa: Trong số các mẫu thực sự là positive, bao nhiêu phần trăm được dự đoán đúng?

Khi nào quan trọng: Khi false negative có chi phí cao.

- Ví dụ: Phát hiện ung thư — không muốn bỏ sót bệnh nhân có bệnh.
- Ví dụ: Phát hiện gian lận — không muốn bỏ sót giao dịch gian lận.

11.2.5 F1-Score

Công thức:

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Ý nghĩa: Trung bình điều hòa (harmonic mean) của Precision và Recall. Cân bằng giữa hai metric.

Khi nào dùng: Khi cần cân bằng giữa Precision và Recall, đặc biệt khi dữ liệu mất cân bằng.

Tại sao dùng harmonic mean thay vì arithmetic mean? Harmonic mean phạt nặng các giá trị thấp. Nếu Precision = 1.0 và Recall = 0.1, arithmetic mean =

0.55 nhưng harmonic mean = 0.18. Điều này phản ánh đúng hơn hiệu suất thực tế.

11.2.6 Precision-Recall Trade-off

- **Tăng threshold** (ví dụ: từ 0.5 lên 0.7):
 - Precision tăng (ít false positive hơn)
 - Recall giảm (nhiều false negative hơn)
- **Giảm threshold** (ví dụ: từ 0.5 xuống 0.3):
 - Precision giảm (nhiều false positive hơn)
 - Recall tăng (ít false negative hơn)

Đường cong Precision-Recall: Plot của Precision vs Recall tại các threshold khác nhau. AUC-PR càng cao càng tốt.

11.2.7 Specificity (True Negative Rate)

Công thức:

$$\text{Specificity} = \frac{TN}{TN + FP}$$

Ý nghĩa: Trong số các mẫu thực sự negative, bao nhiêu phần trăm được dự đoán đúng?

11.2.8 False Positive Rate (FPR)

Công thức:

$$\text{FPR} = \frac{FP}{FP + TN} = 1 - \text{Specificity}$$

11.2.9 ROC Curve và AUC-ROC

ROC Curve (Receiver Operating Characteristic):

- Plot của True Positive Rate (Recall) vs False Positive Rate tại các threshold khác nhau.
- Đường chéo từ (0,0) đến (1,1) đại diện cho random guessing.

AUC-ROC (Area Under the ROC Curve):

- Diện tích dưới đường cong ROC.

- Giá trị từ 0 đến 1.
 - 0.5: Random guessing
 - 1.0: Perfect classification
 - < 0.5 : Worse than random (có thể đảo ngược dự đoán)

Ưu điểm: Không phụ thuộc vào threshold, đánh giá khả năng phân biệt tổng thể của model.

11.2.10 AUC-PR (Area Under Precision-Recall Curve)

Ưu điểm so với AUC-ROC: Phù hợp hơn cho dữ liệu mất cân bằng (imbalanced data), vì tập trung vào class positive.

11.2.11 Cohen's Kappa

Công thức:

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

Trong đó p_o là observed accuracy và p_e là expected accuracy (do random).

Ý nghĩa: Đo lường mức độ đồng thuận giữa dự đoán và thực tế, có tính đến yếu tố ngẫu nhiên.

11.2.12 Matthews Correlation Coefficient (MCC)

Công thức:

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

Ý nghĩa: Hệ số tương quan giữa dự đoán và thực tế. Giá trị từ -1 đến 1.

- 1: Perfect prediction
- 0: Random prediction
- -1: Complete disagreement

Ưu điểm: Cân bằng, phù hợp cho imbalanced data.

11.3 Metrics cho Regression (Hồi quy)

11.3.1 Mean Squared Error (MSE)

Công thức:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Đặc điểm:

- Phạt nặng các sai số lớn (do bình phương)
- Đơn vị là bình phương của đơn vị output
- Nhạy cảm với outliers

11.3.2 Root Mean Squared Error (RMSE)

Công thức:

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

Đặc điểm:

- Cùng đơn vị với output
- Dễ diễn giải hơn MSE
- Vẫn nhạy cảm với outliers

11.3.3 Mean Absolute Error (MAE)

Công thức:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

Đặc điểm:

- Ít nhạy cảm với outliers hơn MSE
- Cùng đơn vị với output
- Robust hơn

11.3.4 Mean Absolute Percentage Error (MAPE)

Công thức:

$$\text{MAPE} = \frac{100\%}{N} \sum_{i=1}^N \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

Đặc điểm:

- Đo lường sai số tương đối (%)
- Không phù hợp khi $y_i = 0$
- Không đối xứng (penalize over-forecast và under-forecast khác nhau)

11.3.5 R-squared (R^2)

Công thức:

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

Ý nghĩa: Tỷ lệ phương sai của output được giải thích bởi model.

- $R^2 = 1$: Perfect fit
- $R^2 = 0$: Model không tốt hơn việc dự đoán giá trị trung bình
- $R^2 < 0$: Model tệ hơn việc dự đoán giá trị trung bình

11.3.6 Adjusted R-squared

Công thức:

$$\bar{R}^2 = 1 - \frac{(1 - R^2)(N - 1)}{N - p - 1}$$

Trong đó p là số lượng features.

Ý nghĩa: Điều chỉnh R^2 cho số lượng features, tránh overfitting.

11.3.7 Huber Loss

Công thức:

$$L_{\delta}(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{ne\`u } |y - \hat{y}| \leq \delta \\ \delta |y - \hat{y}| - \frac{1}{2}\delta^2 & \text{ne\`u } |y - \hat{y}| > \delta \end{cases}$$

Đặc điểm: Kết hợp ưu điểm của MSE và MAE, ít nhạy cảm với outliers.

11.4 Metrics cho Ranking

11.4.1 Mean Reciprocal Rank (MRR)

Công thức:

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$$

Ứng dụng: Đánh giá hệ thống tìm kiếm, gợi ý.

11.4.2 Normalized Discounted Cumulative Gain (NDCG)

Công thức:

$$\text{DCG}_p = \sum_{i=1}^p \frac{2^{\text{rel}_i} - 1}{\log_2(i + 1)}$$

$$\text{NDCG}_p = \frac{\text{DCG}_p}{\text{IDCG}_p}$$

Ứng dụng: Đánh giá chất lượng xếp hạng, tìm kiếm.

11.5 Metrics cho Segmentation

11.5.1 Intersection over Union (IoU)

Công thức:

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

Ứng dụng: Đánh giá chất lượng segmentation trong ảnh.

11.5.2 Dice Coefficient

Công thức:

$$\text{Dice} = \frac{2 \cdot |A \cap B|}{|A| + |B|}$$

Ứng dụng: Tương tự IoU, thường dùng trong y tế.

11.6 Training vs Validation vs Test

Set	Mục đích
Training	Học parameters
Validation	Tune hyperparameters, chọn model
Test	Đánh giá generalization cuối cùng

Quan trọng:

Training $\neq$ Validation $\neq$ Test

Test set cung cấp ước lượng không thiên lệch (unbiased estimate) về generalization.

Tỷ lệ thường dùng: 60-80% training, 10-20% validation, 10-20% test.

11.7 Cross-Validation

K-fold Cross-Validation:

- Chia dữ liệu thành K folds.
- Train trên K-1 folds, validate trên 1 fold.
- Lặp lại K lần, mỗi fold làm validation một lần.
- Kết quả cuối cùng là trung bình của K lần.

Ưu điểm: Sử dụng hiệu quả dữ liệu, giảm variance trong đánh giá.

Nhược điểm: Tốn thời gian tính toán hơn.

12. Training curves và chẩn đoán

12.1 Training Curves

Theo dõi:

- Training loss
- Validation loss
- Training accuracy
- Validation accuracy

Cách đọc training curves:

- Nếu cả hai loss đều giảm: Model đang học tốt.
- Nếu training loss giảm nhưng validation loss tăng: Overfitting.
- Nếu cả hai loss đều cao: Underfitting.

12.2 Underfitting (Underfitting)

Triệu chứng:

- Training performance kém
- Validation performance cũng kém

Nguyên nhân:

- Model capacity không đủ (quá ít layers/neurons)
- Optimization không đủ tốt (learning rate sai, chưa train đủ lâu)
- Features không đủ thông tin

Giải pháp:

- Tăng model capacity (nhiều layers/neurons hơn)
- Train lâu hơn
- Cải thiện features
- Điều chỉnh learning rate

12.3 Good Fit

Triệu chứng:

- Training performance tốt

- Validation performance tốt

12.4 Overfitting

Triệu chứng:

- Training performance rất cao
- Validation performance ngừng cải thiện
- Training và validation curves phân kỳ

Nguyên nhân:

- Model quá phức tạp so với dữ liệu
- Dữ liệu training quá ít
- Thiếu regularization

Giải pháp:

- Thêm dữ liệu
- Regularization (L1, L2, Dropout)
- Early stopping
- Giảm model capacity
- Data augmentation

12.5 Generalization

Generalization > Training Accuracy Alone

Một training accuracy cao không có nghĩa là model generalize tốt. Mục tiêu cuối cùng là **test performance** — khả năng dự đoán đúng trên dữ liệu chưa từng thấy.

12.6 Regularization Techniques

12.6.1 L1 Regularization (Lasso)

Thêm penalty vào loss:

$$L' = L + \lambda \sum |w_i|$$

Hiệu ứng: Tạo ra sparse weights (nhiều weight bằng 0).

12.6.2 L2 Regularization (Ridge)

Thêm penalty vào loss:

$$L' = L + \lambda \sum w_i^2$$

Hiệu ứng: Giữ weights nhỏ, giảm overfitting.

12.6.3 Dropout

Trong quá trình training, ngẫu nhiên "tắt" một số neuron với xác suất p .

Hiệu ứng: Buộc mạng học redundant representations, giảm overfitting.

12.6.4 Early Stopping

Dừng training khi validation loss bắt đầu tăng.

Hiệu ứng: Tránh overfitting mà không cần thay đổi kiến trúc.

12.6.5 Data Augmentation

Tạo thêm dữ liệu training bằng cách biến đổi dữ liệu gốc (xoay ảnh, thêm nhiễu, v.v.).

Hiệu ứng: Tăng kích thước dataset, giảm overfitting.

13. Pytorch và deep learning frameworks

13.1 Tại sao dùng Framework?

Modern frameworks tự động hóa:

1. Parameter storage
2. Matrix multiplication
3. Activation functions
4. Loss calculation
5. Gradient calculation
6. Backpropagation
7. Parameter updates

Automatic Differentiation: Tính toán $\frac{\partial L}{\partial w}$ tự động, không cần viết code backpropagation thủ công.

13.2 Xây dựng MLP trong PyTorch

```
python
import torch
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, input_dim, num_classes):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(input_dim, 64),
            nn.ReLU(),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, num_classes)
        )

    def forward(self, x):
        return self.network(x)
```

13.3 PyTorch Training Cycle

```
python
optimizer.zero_grad()
outputs = model(X_batch)
loss = criterion(outputs, y_batch)
loss.backward()
optimizer.step()
```

Giải thích:

1. `optimizer.zero_grad()`: Xóa gradients cũ

2. `outputs = model(X_batch)`: Forward pass
3. `loss = criterion(outputs, y_batch)`: Tính loss
4. `loss.backward()`: Backward pass (tính gradients)
5. `optimizer.step()`: Cập nhật parameters

13.4 Training vs Evaluation Mode

Training:

python

```
model.train()
```

Evaluation:

python

```
model.eval()
```

```
with torch.no_grad():
```

```
    outputs = model(X_test)
```

Phân biệt:

- `model.train()` và `model.eval()`: Kiểm soát operating mode của model (ảnh hưởng đến Dropout, BatchNorm)
- `torch.no_grad()`: Tắt gradient computation

13.5 Loss Function và Optimizer

python

```
criterion = nn.CrossEntropyLoss()
```

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

Vai trò:

- **Loss Function:** Đo lường prediction error
- **Optimizer:** Cập nhật model parameters

13.6 Lưu và Load Model

Lưu:

python

```
torch.save(model.state_dict(), "mlp_model.pth")
```

Load:

python

```
model = MLP(input_dim=input_dim, num_classes=num_classes)
```

```
model.load_state_dict(torch.load("mlp_model.pth"))
```

```
model.eval()
```

Lưu ý: state_dict() lưu learned parameter state. Model architecture cần được định nghĩa lại.

14. Từ numpy đến pytorch

14.1 So sánh Code

NumPy	PyTorch
$z1 = X @ W1 + b1$	<code>nn.Linear(8, 16)</code>
<code>h1 = relu(z1)</code>	<code>nn.ReLU()</code>
<code>loss = binary_cross_entropy(...)</code>	<code>nn.BCELoss()</code>
<code>gradients = backward(...)</code>	<code>loss.backward()</code>
$W -= lr * dW$	<code>optimizer.step()</code>

14.2 Conceptual Mapping

Python	Mathematics
$z1 = X @ W1 + b1$	$Z_1 = XW_1 + b_1$
<code>h1 = relu(z1)</code>	$H_1 = \text{ReLU}(Z_1)$

Python	Mathematics
<code>z2 = h1 @ W2 + b2</code>	$Z_2 = H_1 W_2 + b_2$
<code>h2 = relu(z2)</code>	$H_2 = \text{ReLU}(Z_2)$
<code>z3 = h2 @ W3 + b3</code>	$Z_3 = H_2 W_3 + b_3$
<code>y_hat = sigmoid(z3)</code>	$\hat{y} = \sigma(Z_3)$
<code>loss = binary_cross_entropy(...)</code>	$L = -\frac{1}{N} \sum [\dots]$
<code>gradients = backward(...)</code>	$\nabla_{\theta} L$
<code>W = W - lr * dW</code>	$W \leftarrow W - \eta \nabla_W L$

15. Learned representations

15.1 Hidden Representation là gì?

Xét:

$$h_1 = \text{ReLU}(W_1 x + b_1)$$

Vector h_1 là một learned representation.

$$x \in \mathbb{R}^d \rightarrow h_1 \in \mathbb{R}^{64}$$

Khác biệt với Assignment 02: 64 features không được định nghĩa thủ công bởi programmer. Chúng được xác định bởi learned parameters.

A hidden layer can be viewed as a learned transformation of the original representation.

15.2 Ý nghĩa của Learned Representation

Traditional ML:

$$x = [x_1, x_2, \dots, x_d]$$

Features có ý nghĩa rõ ràng: age, glucose, blood pressure, BMI, etc.

Neural Network:

$$h = [h_1, h_2, \dots, h_k]$$

Hidden features thường không có ý nghĩa trực tiếp với con người.

15.3 Representation Learning

Traditional ML:

$$x \rightarrow \text{ML Algorithm} \rightarrow \hat{y}$$

Deep Learning:

$$x \rightarrow h_1 \rightarrow h_2 \rightarrow \dots \rightarrow \hat{y}$$

Mỗi hidden layer áp dụng một learned transformation.

Deep Network = Composition of Learned Transformations

15.4 Tại sao Deep Learning gọi là "Deep"?

Logistic Regression:

$$\hat{y} = \sigma(XW + b)$$

Deep Learning Model:

$$\hat{y} = f_3(f_2(f_1(X)))$$

Khác biệt: Learned intermediate representations.

$$X \rightarrow H_1 \rightarrow H_2 \rightarrow \hat{y}$$

Deep Learning = Learning a hierarchy of representations

- Representation đầu tiên có thể nắm bắt các pattern tương đối đơn giản.
- Representation thứ hai kết hợp các pattern này.
- Tầng cuối cùng ánh xạ learned representation đến task output.

16. Tổng kết và mental model

16.1 Bốn phương trình quan trọng nhất

1. Representation:

$$H_1 = \text{ReLU}(XW_1 + b_1)$$

2. Deeper Representation:

$$H_2 = \text{ReLU}(H_1W_2 + b_2)$$

3. Prediction:

$$\hat{y} = \sigma(H_2W_3 + b_3)$$

4. Learning:

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

Kết hợp:

Deep Learning = Function Composition + Representation Learning + Optimization

16.2 Mental Model hoàn chỉnh

$X \rightarrow f_1 \rightarrow f_2 \rightarrow f_3 \rightarrow \hat{y} \rightarrow L \rightarrow \nabla L \rightarrow \text{Update}$
--

Sau đó lặp lại.

16.3 Từ bài toán đơn giản đến Modern Deep Learning

Mô hình tiểu đường nhỏ này là intentionally simple. Real Deep Learning systems có thể chứa:

- Hàng trăm hoặc hàng nghìn layers
- Hàng triệu hoặc hàng tỷ parameters
- Convolutional layers
- Recurrent layers
- Attention mechanisms
- Transformers
- Residual connections
- Normalization layers
- GPUs và distributed computation

Tuy nhiên, fundamental pattern vẫn là:

Compose functions → calculate loss → differentiate → update parameters

Độ phức tạp của modern systems đến chủ yếu từ architecture, scale, data, và optimization methods — không phải từ việc từ bỏ các nguyên tắc cơ bản này.

16.4 Từ NumPy đến Framework

Sau khi hiểu tutorial này, sinh viên đã sẵn sàng học framework. Framework cung cấp abstractions cho:

1. Tensors
2. Layers
3. Models
4. Automatic differentiation
5. Optimizers
6. GPU computation
7. Dataset management
8. Training utilities

Nhưng ý tưởng cơ bản vẫn là:

$$\begin{aligned}\tilde{y} &= f(X; \theta) \\ \theta &\leftarrow \theta - \eta \nabla_{\theta} L\end{aligned}$$

17. Các khái niệm nâng cao

17.1 Softmax và Logits

Logits: Output thô của mạng trước khi áp dụng softmax.

$$z = [z_1, \dots, z_C]$$

Softmax:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}$$

Predicted class:

$$\hat{y} = \arg \max_i z_i$$

17.2 Dataset và DataLoader trong PyTorch

python

```
from torch.utils.data import TensorDataset, DataLoader
```

```
dataset = TensorDataset(X_tensor, y_tensor)
```

```
loader = DataLoader(dataset, batch_size=32, shuffle=True)
```

Dataset: Lưu trữ hoặc cung cấp individual samples.

DataLoader: Tổ chức samples thành mini-batches.

17.3 Từ Vectors đến Images

Hiện tại:

$$x \in \mathbb{R}^d$$

là một feature vector.

Image:

$$X \in \mathbb{R}^{C \times H \times W}$$

Ví dụ: $3 \times 224 \times 224$

Vấn đề: Fully connected MLP có thể xử lý flattened vector, nhưng điều này có thể bỏ qua spatial structure.

Câu hỏi tiếp theo: Làm thế nào để mạng nơ-ron học spatial representations?

Trả lời: Convolutional Neural Networks (CNNs).

Vector $\rightarrow$ **MLP** trở thành **Image Tensor** $\rightarrow$ **CNN**

Representation bây giờ bảo toàn spatial organization.

Deep Learning $\rightarrow$ **Spatial Representation Learning**

18. Trả lời các câu hỏi ôn tập

Câu 1: What is a parameter?

Parameter (tham số) là các biến số được học từ dữ liệu trong quá trình training. Trong mạng nơ-ron, parameters bao gồm:

- **Weights (W):** Ma trận trọng số kết nối giữa các tầng. Ví dụ: $W_1 \in \mathbb{R}^{8 \times 16}$, $W_2 \in \mathbb{R}^{16 \times 8}$, $W_3 \in \mathbb{R}^{8 \times 1}$.
- **Biases (b):** Vector độ lệch cho mỗi tầng. Ví dụ: $b_1 \in \mathbb{R}^{1 \times 16}$, $b_2 \in \mathbb{R}^{1 \times 8}$, $b_3 \in \mathbb{R}^{1 \times 1}$.

Tổng số parameters trong kiến trúc $8 \rightarrow 16 \rightarrow 8 \rightarrow 1$:

- $W_1: 8 \times 16 = 128$
- $b_1: 16$
- $W_2: 16 \times 8 = 128$
- $b_2: 8$
- $W_3: 8 \times 1 = 8$
- $b_3: 1$
- **Tổng:** $128 + 16 + 128 + 8 + 8 + 1 = 289$ parameters.

Parameters được cập nhật thông qua gradient descent:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L$$

Câu 2: What is a hyperparameter?

Hyperparameter (siêu tham số) là các tham số **không được học từ dữ liệu** mà do con người thiết lập trước khi training. Chúng kiểm soát quá trình học và kiến trúc mô hình.

Ví dụ hyperparameters:

- **Learning rate (η):** 0.01, 0.001, 0.1
- **Số epochs:** 100, 1000
- **Batch size:** 16, 32, 64, 128
- **Số layers:** 3
- **Số neurons mỗi layer:** 16, 8
- **Activation function:** ReLU, Sigmoid
- **Optimizer:** SGD, Adam
- **Regularization strength (λ):** 0.001, 0.01

Phân biệt:

- **Parameters:** Được học từ dữ liệu (weights, biases)
- **Hyperparameters:** Được thiết lập bởi con người (learning rate, architecture)

Câu 3: What is the role of W ?

W (**weights**) là ma trận trọng số thể hiện **tầm quan trọng** của từng kết nối giữa các neuron. Trong phép biến đổi tuyến tính:

$$Z = XW + b$$

- W quyết định mức độ ảnh hưởng của từng input feature đến từng output neuron.
- Trọng số càng lớn, kết nối càng quan trọng.
- Trọng số được học từ dữ liệu thông qua backpropagation và gradient descent.

Ví dụ: Trong bài toán tiểu đường, nếu $W_{glucose}$ lớn, điều đó có nghĩa là nồng độ glucose có ảnh hưởng mạnh đến dự đoán.

Câu 4: What is the purpose of b ?

b (**bias**) là vector độ lệch cho phép **địch chuyển** hàm kích hoạt. Trong phép biến đổi:

$$Z = XW + b$$

- Nếu không có b , đường quyết định luôn đi qua gốc tọa độ (0,0).
- b cho phép mô hình học các đường quyết định không đi qua gốc.
- b cũng giúp neuron "kích hoạt" ngay cả khi tất cả input bằng 0.

Ví dụ: Trong bài toán tiểu đường, bias có thể đại diện cho nguy cơ cơ bản (baseline risk) của bệnh nhân.

Câu 5: Why do we need nonlinear activation functions?

Lý do: Nếu không có nonlinear activation functions, việc xếp chồng nhiều tầng tuyến tính vẫn chỉ tạo ra một phép biến đổi tuyến tính tổng thể.

Chứng minh:

$$h_1 = W_1x + b_1$$

$$h_2 = W_2h_1 + b_2 = W_2(W_1x + b_1) + b_2 = W_2W_1x + W_2b_1 + b_2$$

Đặt $W' = W_2W_1$ và $b' = W_2b_1 + b_2$, ta có $h_2 = W'x + b'$.

Kết luận: Dù có bao nhiêu tầng tuyến tính, mạng vẫn chỉ là một phép biến đổi tuyến tính. Nonlinear activation functions là **thiết yếu** để học các hàm phức tạp.

Câu 6: What does ReLU do?

ReLU (Rectified Linear Unit) thực hiện phép biến đổi:

$$\text{ReLU}(x) = \max(0, x)$$

Chức năng:

- Giá trị âm $\rightarrow 0$
- Giá trị dương $\rightarrow$ giữ nguyên

Đạo hàm:

$$\text{ReLU}'(x) = \begin{cases} 1 & \text{nếu } x > 0 \\ 0 & \text{nếu } x \leq 0 \end{cases}$$

Vai trò:

- Giới thiệu tính phi tuyến
- Tính toán đơn giản, hiệu quả
- Giảm vanishing gradient
- Tạo sparse representations

Câu 7: Why do we use sigmoid in the final layer?

Sigmoid được sử dụng ở tầng cuối cho **binary classification** vì:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1)$$

- Output nằm trong khoảng (0, 1), có thể interpret như **xác suất**.
- Ví dụ: $\hat{y} = 0.91$ có nghĩa là model tin 91% bệnh nhân mắc tiểu đường.
- Quyết định phân loại: $\hat{y} \geq 0.5 \Rightarrow \text{Diabetes}$, $\hat{y} < 0.5 \Rightarrow \text{No Diabetes}$.

Câu 8: Why do we need a loss function?

Loss function đo lường **mức độ sai** của dự đoán so với giá trị thực tế. Nó cần thiết vì:

1. **Định lượng sai số:** Chuyển đổi "model sai" thành một con số cụ thể.
2. **Cung cấp tín hiệu học:** Loss là cơ sở để tính gradients.
3. **Mục tiêu tối ưu:** Quá trình học là tìm parameters để minimize loss.

Ví dụ: Với $y = 1$ và $\hat{y} = 0.01$, binary cross-entropy loss rất lớn, cho mạng biết dự đoán rất tệ.

Câu 9: What does binary cross-entropy measure?

Binary Cross-Entropy đo lường **sự khác biệt** giữa phân phối xác suất dự đoán và phân phối thực tế cho bài toán binary classification.

Công thức:

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Ý nghĩa:

- Khi $y = 1$: $L = -\log(\hat{y})$ — loss nhỏ khi $\hat{y}$ gần 1
- Khi $y = 0$: $L = -\log(1 - \hat{y})$ — loss nhỏ khi $\hat{y}$ gần 0
- Loss càng nhỏ, dự đoán càng chính xác.

Câu 10: What is a gradient?

Gradient là vector chứa các **đạo hàm riêng** (partial derivatives) của một hàm số đối với từng biến số.

Ví dụ: Nếu $L = f(w_1, w_2, b)$, gradient là:

$$\nabla L = \left[\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \frac{\partial L}{\partial b} \right]$$

Ý nghĩa:

- Gradient chỉ ra hướng và mức độ thay đổi của loss khi thay đổi từng tham số.
- $\frac{\partial L}{\partial w_1} > 0$: tăng w_1 làm tăng loss.
- $\frac{\partial L}{\partial w_1} < 0$: tăng w_1 làm giảm loss.

Trong Deep Learning: Gradient được sử dụng để cập nhật parameters theo hướng giảm loss.

Câu 11: What does backpropagation calculate?

Backpropagation tính toán **gradients** của **loss function** đối với tất cả **parameters** của mạng.

Cụ thể:

$$\frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial b_1}, \frac{\partial L}{\partial W_2}, \frac{\partial L}{\partial b_2}, \frac{\partial L}{\partial W_3}, \frac{\partial L}{\partial b_3}$$

Cách hoạt động:

- Sử dụng **chain rule** để lan truyền gradients từ tầng output về các tầng trước.
- Lưu trữ intermediate values từ forward pass để sử dụng trong backward pass.

Câu 12: What does gradient descent do?

Gradient Descent sử dụng gradients để **cập nhật parameters** theo hướng giảm loss.

Công thức:

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

Quá trình:

1. Tính gradient của loss đối với parameters.
2. Di chuyển parameters ngược hướng gradient (để giảm loss).
3. Lặp lại cho đến khi hội tụ.

Câu 13: Why are the weights updated repeatedly?

Weights được cập nhật **nhiều lần** vì:

1. **Loss landscape phức tạp:** Không thể tìm được minimum trong một bước.
2. **Gradient descent là iterative:** Mỗi bước chỉ di chuyển một khoảng nhỏ (η).
3. **Cần nhiều epochs:** Để hội tụ đến điểm tối ưu.
4. **Mini-batch training:** Mỗi batch cung cấp một ước lượng gradient khác nhau.

Ví dụ: Với 1000 epochs, weights được cập nhật 1000 lần (hoặc nhiều hơn nếu dùng mini-batches).

Câu 14: Why do we separate training and testing data?

Lý do:

1. **Đánh giá generalization:** Test set đo lường khả năng dự đoán trên dữ liệu chưa từng thấy.
2. **Tránh overfitting:** Nếu đánh giá trên training set, model có thể "học thuộc" dữ liệu mà không generalize.
3. **Unbiased estimate:** Test set cung cấp ước lượng không thiên lệch về hiệu suất thực tế.

Tỷ lệ thường dùng: 80% training, 20% test (hoặc 60/20/20 với validation set).

Câu 15: Why do we normalize the features?

Normalization (chuẩn hóa) đưa các features về cùng scale, thường là mean = 0 và std = 1.

Công thức:

$$X_{normalized} = \frac{X - \mu}{\sigma}$$

Lý do:

1. **Tránh features có scale lớn chi phối:** Ví dụ, glucose (0-200) vs age (20-80).
2. **Tăng tốc hội tụ:** Gradient descent hội tụ nhanh hơn khi features cùng scale.
3. **Cải thiện numerical stability:** Tránh overflow/underflow.
4. **Đảm bảo công bằng:** Mỗi feature đóng góp tương đương vào khoảng cách.

Lưu ý: Sử dụng mean và std từ **training set** cho cả training và test set để tránh data leakage.

Câu 16: What is the difference between a probability and a class prediction?

Probability	Class Prediction
Output của sigmoid: $\hat{y} \in (0, 1)$	Output nhị phân: 0 hoặc 1

Probability	Class Prediction
Ví dụ: 0.91	Ví dụ: 1
Thể hiện mức độ tự tin	Thể hiện quyết định cuối cùng
Liên tục	Rời rạc
Được tính từ forward pass	Được tính bằng threshold

Chuyển đổi:

$$\text{class} = \begin{cases} 1 & \text{nếu } \hat{y} \geq 0.5 \\ 0 & \text{nếu } \hat{y} < 0.5 \end{cases}$$

Threshold có thể điều chỉnh (0.3, 0.7) để trade-off precision và recall.

PHẦN II: THỰC HÀNH

BÀI 1: DỰ ĐOÁN TIỂU ĐƯỜNG. Dataset: Diabetes Dataset

1. IMPORT & SETUP

1. Mục đích của cell

Thiết lập môi trường và các thư viện cần thiết trước khi bắt đầu toàn bộ quy trình. Đây là cell nền tảng: nếu các thư viện, seed hoặc cấu hình đồ họa chưa sẵn sàng thì những cell phía sau có thể cho kết quả không nhất quán hoặc không thể thực thi.

2. Ảnh code thực tế

```
# =====  
# CELL 1: IMPORT & SETUP  
# =====  
  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
import warnings  
import time  
import os  
  
warnings.filterwarnings('ignore')  
sns.set_style("whitegrid")  
plt.rcParams['figure.figsize'] = (12, 6)  
plt.rcParams['font.size'] = 11  
  
# For ML baseline comparison  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler, LabelEncoder  
from sklearn.linear_model import LogisticRegression  
from sklearn.ensemble import RandomForestClassifier  
from sklearn.metrics import (  
    accuracy_score, precision_score, recall_score, f1_score,  
    confusion_matrix, roc_auc_score, roc_curve,  
    classification_report  
)  
  
try:  
    from xgboost import XGBClassifier  
    HAS_XGB = True  
except ImportError:  
    HAS_XGB = False  
    print("XGBoost not available, will use GradientBoosting instead")  
    from sklearn.ensemble import GradientBoostingClassifier  
  
np.random.seed(42)  
print("Setup complete.")  
print(f"NumPy version: {np.__version__}")  
print(f"Pandas version: {pd.__version__}")
```

3. Output thực tế lấy từ notebook

Output 1

OUTPUT THỰC TẾ — OUTPUT 1

```
XGBoost not available, will use GradientBoosting instead  
Setup complete.  
NumPy version: 2.5.2  
Pandas version: 3.0.5
```

Hình 1.1. Output thực tế của cell “IMPORT & SETUP”.

4. Giải thích chi tiết code

- NumPy được import dưới tên np và là thư viện cốt lõi cho phần Deep Learning tự cài đặt. Toàn bộ phép nhân ma trận, hàm kích hoạt, tính gradient, cập nhật trọng số và các phép toán vector hóa của MLP đều dựa trên NumPy.

- Pandas được dùng để đọc CSV, kiểm tra kiểu dữ liệu, xử lý missing value, mã hóa biến phân loại và tạo các bảng kết quả. Matplotlib và Seaborn được dùng cho histogram, heatmap, biểu đồ metric, ROC và confusion matrix.
- Nhóm import từ scikit-learn phục vụ hai mục đích: xây dựng các mô hình Machine Learning baseline và tính các metric đánh giá. Notebook không dùng scikit-learn để xây dựng MLP; MLP được viết thủ công bằng NumPy.
- Khối try/except kiểm tra XGBoost. Nếu import được XGBClassifier thì HAS_XGB=True; nếu không, notebook in thông báo và chuyển sang GradientBoostingClassifier. Cách viết này giúp notebook có phương án thay thế thay vì dùng toàn bộ khi thiếu XGBoost.
- np.random.seed(42) cố định nguồn ngẫu nhiên ở cấp NumPy, hỗ trợ khả năng tái lập các phép xáo trộn và khởi tạo. Các cấu hình figure size và font giúp các biểu đồ sau có kích thước nhất quán.

5. Phân tích output và ý nghĩa

Output xác nhận môi trường đã chạy đến cuối cell. Theo kết quả thực tế của notebook, XGBoost không khả dụng nên nhánh Gradient Boosting được sử dụng. Notebook cũng in phiên bản NumPy 2.5.2 và Pandas 3.0.5. Như vậy cell vừa kiểm tra dependency vừa xác nhận phiên bản thư viện đang thực sự được dùng.

2. LOAD DATA & INITIAL UNDERSTANDING

1. Mục đích của cell

Đọc dataset và tạo một ảnh chụp ban đầu về quy mô, cấu trúc, kiểu dữ liệu và phân bố target. Đây là bước quan trọng để phát hiện sớm các vấn đề trước khi làm sạch.

2. Ảnh code thực tế

```
# =====  
# CELL 2: LOAD DATA & INITIAL UNDERSTANDING  
# =====  
  
DATA_PATH = "data/final_messy_diabetes_dataset.csv"  
  
df = pd.read_csv(DATA_PATH)  
  
print(f"Dataset shape: {df.shape}")  
print(f"Number of rows: {df.shape[0]:,}")  
print(f"Number of columns: {df.shape[1]}")  
  
print("\n=== First 5 rows ===")  
display(df.head())  
  
print("\n=== Column dtypes ===")  
print(df.dtypes)  
  
print("\n=== Target distribution ===")  
print(df['diagnosed_diabetes'].value_counts())  
  
print(f"Positive rate: {df['diagnosed_diabetes'].mean():.4f}")
```

3. Output thực tế lấy từ notebook

Output 1

OUTPUT THỰC TẾ — OUTPUT 1

```
Dataset shape: (100000, 34)  
Number of rows: 100,000  
Number of columns: 34  
  
=== First 5 rows ===
```

Hình 2.1. Output thực tế của cell “LOAD DATA & INITIAL UNDERSTANDING”.

Output 2

OUTPUT THỰC TẾ – OUTPUT 2

```

    age gender ethnicity education_level income_level employment_status \
0  58.0   Male    Asian    Highschool  Lower-Middle      Employed
1  48.0  Female   White    Highschool    Middle      Employed
2  60.0   Male  Hispanic    Highschool    Middle  Unemployed
3  74.0  Female   Black    Highschool     Low      Retired
4  46.0   Male   White     Graduate    Middle      Retired

    smoking_status alcohol_consumption_per_week \
0      Never                      0
1    Former                      1
2      Never                      1
3      Never                      0
4      Never                      1

    physical_activity_minutes_per_week diet_score ... glucose_fasting \
0                      215      5.7 ...      136
1                      143      6.7 ...      93
2                      57      6.4 ...     118
3                      49      3.4 ...     139
4                      109      7.2 ...     137

    glucose_postprandial insulin_level hba1c diabetes_risk_score \
0                      236      6.36  8.18      29.6
1                      150      2.00  5.63      23.0
2                      195      5.07  7.51      44.7
3                      253      5.28  9.03      38.2
4                      184     12.74  7.20      23.5

    diabetes_stage diagnosed_diabetes patient_tracking_id data_entry_notes \
0      Type 2              1              NaN      IGNORE
1    No Diabetes              0              NaN      IGNORE
2      Type 2              1              NaN      IGNORE
3      Type 2              1              NaN      IGNORE
4      Type 2              1              NaN      IGNORE

    experimental_peptide_level
0              NaN
1              NaN
2              NaN
3              NaN
4              NaN

[5 rows x 34 columns]
```

Hình 2.2. Output thực tế của cell “LOAD DATA & INITIAL UNDERSTANDING”.

Output 3

OUTPUT THỰC TẾ — OUTPUT 3

```
=== Column dtypes ===
age                                float64
gender                            str
ethnicity                         str
education_level                   str
income_level                      str
employment_status                 str
smoking_status                    str
alcohol_consumption_per_week     int64
physical_activity_minutes_per_week int64
diet_score                        float64
sleep_hours_per_day               float64
screen_time_hours_per_day         float64
family_history_diabetes            int64
hypertension_history              int64
cardiovascular_history            int64
bmi                               float64
waist_to_hip_ratio                float64
systolic_bp                       int64
diastolic_bp                      int64
heart_rate                        int64
cholesterol_total                 str
hdl_cholesterol                  int64
ldl_cholesterol                  int64
triglycerides                     int64
glucose_fasting                   str
glucose_postprandial              int64
insulin_level                     float64
hba1c                             float64
diabetes_risk_score               float64
diabetes_stage                    str
diagnosed_diabetes                int64
patient_tracking_id               float64
data_entry_notes                  str
experimental_peptide_level        float64
dtype: object

=== Target distribution ===
diagnosed_diabetes
1    59998
0    40002
Name: count, dtype: int64
Positive rate: 0.6000
```

Hình 2.3. Output thực tế của cell “LOAD DATA & INITIAL UNDERSTANDING”.

4. Giải thích chi tiết code

- DATA_PATH trỏ đến file data/final_messy_diabetes_dataset.csv và pd.read_csv nạp toàn bộ dữ liệu vào DataFrame df.
- df.shape được dùng để lấy số dòng và số cột; hai giá trị này tiếp tục được in riêng để người đọc dễ kiểm tra quy mô. display(df.head()) cho phép quan sát trực tiếp vài bản ghi đầu, nhờ đó nhận biết tên cột và dạng giá trị thực tế.
- df.dtypes kiểm tra kiểu dữ liệu của từng cột. Đây là kiểm tra đặc biệt quan trọng với dataset 'messy' vì một biến có ý nghĩa số học có thể đang bị lưu dưới dạng object/string.
- value_counts() trên diagnosed_diabetes cho biết số lượng mẫu ở từng lớp. Mean của biến nhị phân 0/1 chính là tỷ lệ mẫu dương, nên positive rate được in ra để đánh giá cân bằng lớp.

5. Phân tích output và ý nghĩa

Dataset thực tế có 100.000 dòng và 34 cột. Target diagnosed_diabetes có 59.998 mẫu lớp 1 và 40.002 mẫu lớp 0, tương ứng positive rate 0.6000. Output đầu vào cũng cho thấy tồn tại các cột như patient_tracking_id và data_entry_notes; các cột này cần được xem xét vì ID không mang ý nghĩa dự báo và text ghi chú có thể không phù hợp với pipeline NumPy đang xây dựng. Một số trường số còn ở dạng chuỗi, là dấu hiệu cần chuyển kiểu ở bước cleaning.

3. DATA QUALITY - MISSING VALUES & '?' MARKERS

1. Mục đích của cell

Kiểm tra hai dạng thiếu dữ liệu: ký hiệu '?' và NaN; đồng thời trực quan hóa mức độ missing và phân bố target. Cell này tạo căn cứ thực nghiệm cho quyết định làm sạch ở cell kế tiếp.

2. Ảnh code thực tế

```
# =====
# CELL 3: DATA QUALITY - MISSING VALUES & '?' MARKERS
# =====

# Count '?' in object columns
print("=== '?' counts per object column ===")
for col in df.columns:
    if df[col].dtype == 'object':
        n_q = (df[col] == '?').sum()
        if n_q > 0:
            print(f" {col}: {n_q}")

# Missing values
missing = df.isnull().sum()
missing_pct = (missing / len(df)) * 100

miss_df = pd.DataFrame({
    'Missing': missing,
    'Percent': missing_pct
}).sort_values('Missing', ascending=False)

print("\n=== Missing values (top) ===")
print(miss_df[miss_df['Missing'] > 0].head(20))

# Visualize missing
fig, ax = plt.subplots(figsize=(12, 6))
miss_plot = miss_df[miss_df['Missing'] > 0]

if len(miss_plot) > 0:
    ax.barh(miss_plot.index, miss_plot['Percent'], color='salmon')
    ax.set_xlabel('Missing (%)')
    ax.set_title('Missing Values by Column')
    plt.tight_layout()
    plt.show()

# Target distribution plot
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

df['diagnosed_diabetes'].value_counts().plot(
    kind='bar',
    ax=axes[0],
    color=['steelblue', 'coral']
)
axes[0].set_title('Target Distribution')
axes[0].set_xticklabels(
    ['No Diabetes (0)', 'Diabetes (1)'],
    rotation=0
)

df['diagnosed_diabetes'].value_counts().plot(
    kind='pie',
    ax=axes[1],
    autopct='%1.1f%%',
    colors=['steelblue', 'coral']
)
axes[1].set_title('Target Proportion')
axes[1].set_ylabel('')

plt.tight_layout()
plt.show()
```

3. Output thực tế lấy từ notebook

Output 1

OUTPUT THỰC TẾ – OUTPUT 1

=== '?' counts per object column ===

=== Missing values (top) ===

	Missing	Percent
patient_tracking_id	95000	95.0
experimental_peptide_level	50000	50.0
data_entry_notes	10000	10.0
age	5000	5.0
bmi	5000	5.0

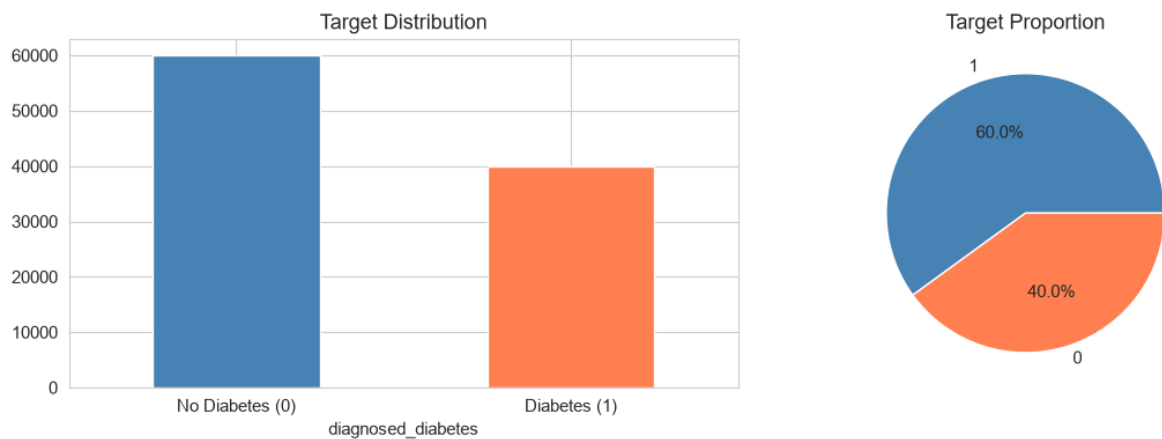
Hình 3.1. Output thực tế của cell “DATA QUALITY - MISSING VALUES & '?' MARKERS”.

Output 2



Hình 3.2. Output thực tế của cell “DATA QUALITY - MISSING VALUES & '?' MARKERS”.

Output 3



Hình 3.3. Output thực tế của cell “DATA QUALITY - MISSING VALUES & '?' MARKERS”.

4. Giải thích chi tiết code

- Vòng lặp chỉ xét các cột object rồi đếm số phần tử bằng '?'. Việc giới hạn ở object giúp kiểm tra đúng loại biểu diễn thường gặp của missing dạng ký tự.
- `df.isnull().sum()` tạo số lượng NaN theo cột. Chia cho `len(df)` rồi nhân 100 tạo missing percentage. Hai đại lượng được ghép vào `miss_df` và sắp xếp giảm dần để ưu tiên các cột có vấn đề lớn.
- Phần barh chỉ vẽ các cột có missing > 0. Như vậy biểu đồ tập trung vào các biến thực sự cần xử lý, thay vì làm rối hình bằng các cột không thiếu.
- Phần cuối dùng `value_counts().plot` để tạo bar chart và pie chart cho target. Hai cách nhìn bổ sung cho nhau: bar chart thể hiện số lượng, pie chart thể hiện tỷ trọng.

5. Phân tích output và ý nghĩa

Output thực tế cho thấy missing tập trung đáng kể ở `patient_tracking_id` khoảng 95%, `experimental_peptide_level` khoảng 50%, `data_entry_notes` khoảng 10%, `age` khoảng 5% và `bmi` khoảng 5%. Đồng thời notebook không ghi nhận số lượng '?' khác 0 trong các object columns ở output kiểm tra này. Vì vậy xử lý tiếp theo chủ yếu dựa trên NaN, đồng thời loại các trường ID/text không hữu ích.

4. DATA CLEANING

1. Mục đích của cell

Làm sạch dữ liệu

2. Ảnh code thực tế

```
# =====
# 4. DATA CLEANING
# =====

df_clean = df.copy()

# 3.1 Replace '?' with NaN
for col in df_clean.columns:
    if df_clean[col].dtype == 'object':
        df_clean[col] = df_clean[col].replace('?', np.nan)

# 3.2 Drop ID and text columns (kể cả các biến thể tên)
DROP_COLS = ['patient_tracking_id', 'data_entry_notes']
existing_drop = [c for c in DROP_COLS if c in df_clean.columns]
df_clean = df_clean.drop(columns=existing_drop)
print(f"Dropped columns: {existing_drop}")

# 3.3 DROP target leakage column: diabetes_stage
if 'diabetes_stage' in df_clean.columns:
    print("\n=== diabetes_stage vs target ===")
    print(pd.crosstab(df_clean['diabetes_stage'], df_clean['diagnosed_diabetes']))
    df_clean = df_clean.drop(columns=['diabetes_stage'])
    print("Dropped 'diabetes_stage' (target leakage)")

# 3.4 Convert numeric columns
numeric_cols = []
for col in df_clean.columns:
    if col == 'diagnosed_diabetes':
        continue
    converted = pd.to_numeric(df_clean[col], errors='coerce')
    if converted.notna().mean() > 0.5:
        df_clean[col] = converted
        numeric_cols.append(col)

print(f"\nNumeric columns ({len(numeric_cols)}):")
print(numeric_cols)

# 3.5 Categorical columns – CHỈ giữ những cột object còn lại
cat_cols = [c for c in df_clean.columns
             if c not in numeric_cols and c != 'diagnosed_diabetes']

# ⚠ IN RA CARDINALITY ĐỂ PHÁT HIỆN CỘT NGUY HIỂM
print("\n=== Categorical column cardinality ===")
card_info = []
for col in cat_cols:
    n_unique = df_clean[col].nunique(dropna=True)
    card_info.append({'Column': col, 'Unique': n_unique, 'Dtype': str(df_clean[col].dtype)})
card_df = pd.DataFrame(card_info).sort_values('Unique', ascending=False)
print(card_df.to_string(index=False))

# ⚠ LOẠI BỎ các cột có cardinality quá cao (>50) – đây là thủ phạm gây MemoryError
HIGH_CARD_THRESHOLD = 50
high_card_cols = [c for c in cat_cols if df_clean[c].nunique(dropna=True) > HIGH_CARD_THRESHOLD]
print(f"\n⚠ High-cardinality columns (>{HIGH_CARD_THRESHOLD} unique) sẽ bị DROP:")
print(high_card_cols)

df_clean = df_clean.drop(columns=high_card_cols)
cat_cols = [c for c in cat_cols if c not in high_card_cols]

# Kiểm tra lại
print(f"\nRemaining categorical columns ({len(cat_cols)}):")
for c in cat_cols:
    print(f" {c}: {df_clean[c].nunique(dropna=True)} unique")

print(f"\nAfter cleaning shape: {df_clean.shape}")
```

3. Output thực tế lấy từ notebook

Output 1

OUTPUT THỰC TẾ — OUTPUT 1

```
Dropped columns: ['patient_tracking_id', 'data_entry_notes']

=== diabetes_stage vs target ===
diagnosed_diabetes    0    1
diabetes_stage
Gestational           120   158
No Diabetes           7981    0
Pre-Diabetes          31845   0
Type 1                 56    66
Type 2                 0   59774
Dropped 'diabetes_stage' (target leakage)

Numeric columns (23):
['age', 'alcohol_consumption_per_week', 'physical_activity_minutes_per_week', 'diet_score', 'sleep_hours_per_day', 'screen_time_hours_per_day', 'medication_adherence', 'smoking_status', 'exercise_frequency', 'stress_level', 'social_support', 'family_history', 'comorbidities', 'health_insurance', 'access_to_healthcare', 'health_literacy', 'diet_quality', 'physical_activity_frequency', 'sleep_quality', 'mental_health_status', 'chronic_conditions', 'genetic_risk_factors', 'environmental_exposures']

=== Categorical column cardinality ===
      Column  Unique  Dtype
experimental_peptide_level  50000  float64
      income_level           5    str
      ethnicity              5    str
employment_status           4    str
education_level             4    str
gender                      3    str
smoking_status              3    str

△ High-cardinality columns (>50 unique) sẽ bị DROP:
['experimental_peptide_level']

Remaining categorical columns (6):
gender: 3 unique
ethnicity: 5 unique
education_level: 4 unique
income_level: 5 unique
employment_status: 4 unique
smoking_status: 3 unique

After cleaning shape: (100000, 30)
```

Hình 4.1. Output thực tế của cell “Code Cell 5”.

4. Giải thích chi tiết code

Đoạn code này thực hiện **làm sạch và chuẩn bị dữ liệu trước khi xây dựng mô hình Machine Learning**. Đầu tiên, code tạo bản sao `df_clean` từ dữ liệu gốc và thay các ký hiệu '?' trong các cột dạng object thành NaN để có thể xử lý missing value. Tiếp theo, code loại bỏ các cột không cần thiết như `patient_tracking_id` và `data_entry_notes`, đồng thời loại bỏ `diabetes_stage` vì đây là **target leakage**, có thể làm mô hình biết trước thông tin về kết quả cần dự đoán. Sau đó, code kiểm tra từng cột và chuyển những cột có ít nhất 50% giá trị chuyển đổi được sang dạng numeric, đồng thời lưu tên các cột numeric vào `numeric_cols`; các cột còn lại được xem là categorical và lưu vào `cat_cols`. Code tiếp tục tính **cardinality** (số lượng giá trị khác nhau) của từng biến categorical để phát hiện các cột có quá nhiều category. Những cột có trên **50 giá trị khác nhau** được xem là high-cardinality và bị loại bỏ nhằm tránh tạo quá nhiều dummy variables khi one-hot encoding, gây tốn bộ nhớ. Cuối cùng, code cập nhật lại danh sách categorical columns và in ra số lượng category của từng biến cũng như kích thước dataset sau khi cleaning.

5. Phân tích output và ý nghĩa

Output cho thấy dữ liệu đã được làm sạch thành công: hai cột `patient_tracking_id` và `data_entry_notes` đã được loại bỏ vì không cần thiết,

đồng thời diabetes_stage cũng bị loại bỏ do có **target leakage** với diagnosed_diabetes (bảng đối chiếu cho thấy diabetes_stage có mối liên hệ rất rõ với target). Sau cleaning, dữ liệu còn **23 biến numeric** và ban đầu có 7 biến categorical, nhưng experimental_peptide_level có tới **50.000 giá trị khác nhau**, vượt ngưỡng 50 nên được loại bỏ để tránh high cardinality và làm tăng số lượng feature khi encoding. Còn lại **6 biến categorical** gồm gender (3), ethnicity (5), education_level (4), income_level (5), employment_status (4) và smoking_status (3) unique values; đây là các biến phù hợp để thực hiện one-hot encoding. Cuối cùng, dataset sau cleaning có kích thước **(100.000, 30)**, tức là vẫn giữ 100.000 bản ghi và còn 30 cột trước khi thực hiện bước imputation và encoding.

5. EDA - NUMERICAL & CATEGORICAL

1. Mục đích của cell

Khám phá phân bố của biến số và mối liên hệ tương quan giữa các biến số với nhau cũng như với target. EDA ở đây không chỉ để minh họa mà còn giúp kiểm tra tính hợp lý của dữ liệu trước modeling.

2. Ảnh code thực tế

```
# =====
# CELL 5: EDA - NUMERICAL & CATEGORICAL
# =====

# 5.1 Numeric distributions
n_num = len(numeric_cols)
n_cols_plot = 4
n_rows_plot = (n_num + n_cols_plot - 1) // n_cols_plot

fig, axes = plt.subplots(
    n_rows_plot,
    n_cols_plot,
    figsize=(18, 3 * n_rows_plot)
)

axes = np.atleast_1d(axes).flatten()

for i, col in enumerate(numeric_cols):
    axes[i].hist(
        df_clean[col].dropna(),
        bins=40,
        color='steelblue',
        edgecolor='black',
        alpha=0.7
    )
    axes[i].set_title(
        f'{col}\nmissing: {df_clean[col].isnull().sum()}'
    )
    axes[i].set_xlabel('')

for j in range(len(numeric_cols), len(axes)):
    axes[j].axis('off')

plt.tight_layout()
plt.show()

# 5.2 Correlation heatmap
plt.figure(figsize=(14, 10))

corr_cols = numeric_cols + ['diagnosed_diabetes']
corr = df_clean[corr_cols].corr()

mask = np.triu(
    np.ones_like(corr, dtype=bool)
)

sns.heatmap(
    corr,
    mask=mask,
    annot=True,
    fmt='.2f',
    cmap='coolwarm',
    center=0,
    square=True,
    cbar_kws={'shrink': 0.7}
)
```

```

plt.title('Correlation Matrix (Numeric Features)')
plt.tight_layout()
plt.show()

# 5.3 Target correlation
target_corr = (
    corr['diagnosed_diabetes']
    .drop('diagnosed_diabetes')
    .sort_values()
)

plt.figure(figsize=(10, 6))

colors = [
    'red' if x < 0 else 'green'
    for x in target_corr.values
]

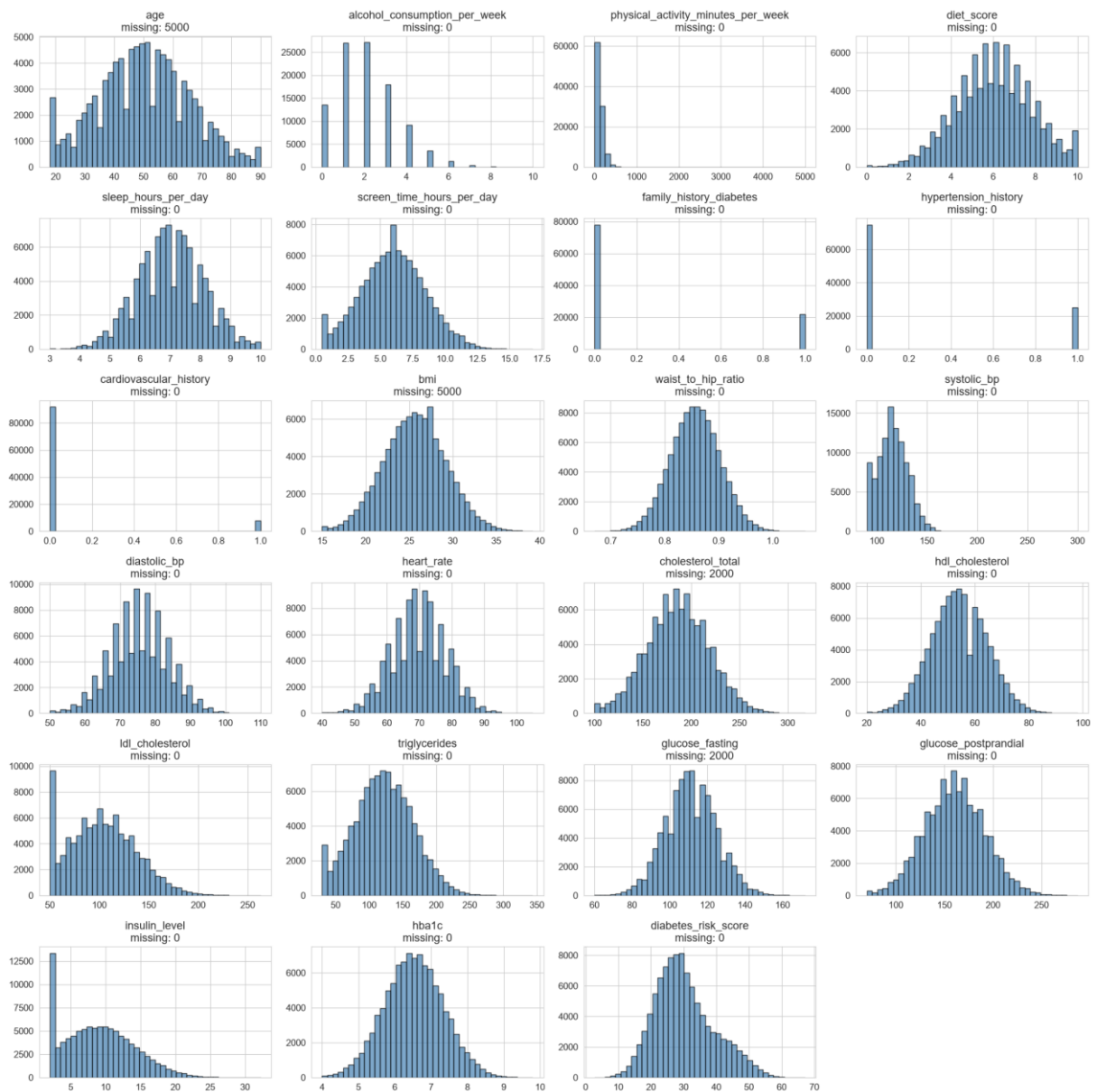
target_corr.plot(
    kind='barh',
    color=colors
)

plt.title(
    'Correlation with Target (diagnosed_diabetes)'
)
plt.axvline(0, color='black', lw=0.8)
plt.tight_layout()
plt.show()

```

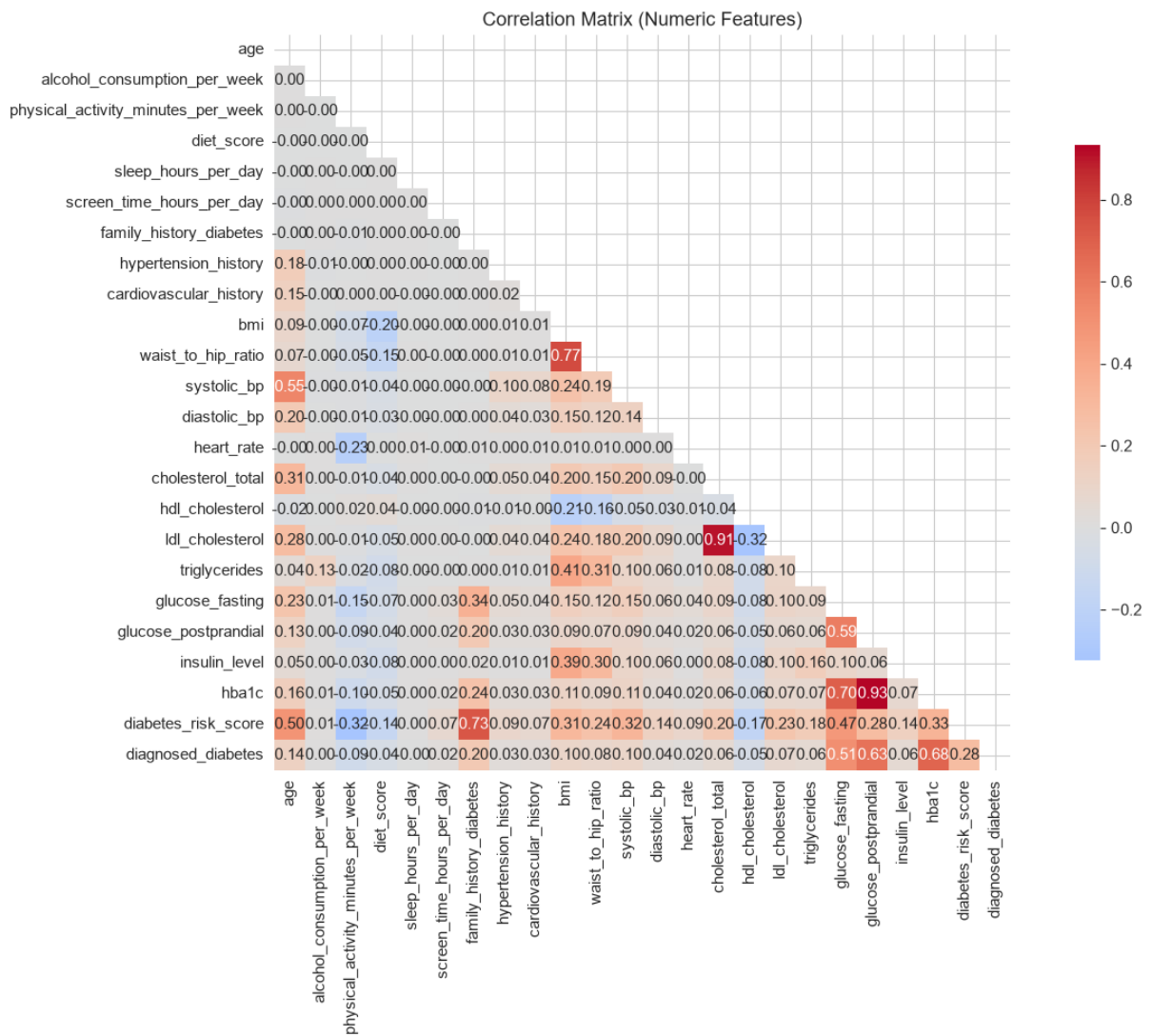
3. Output thực tế lấy từ notebook

Output 1



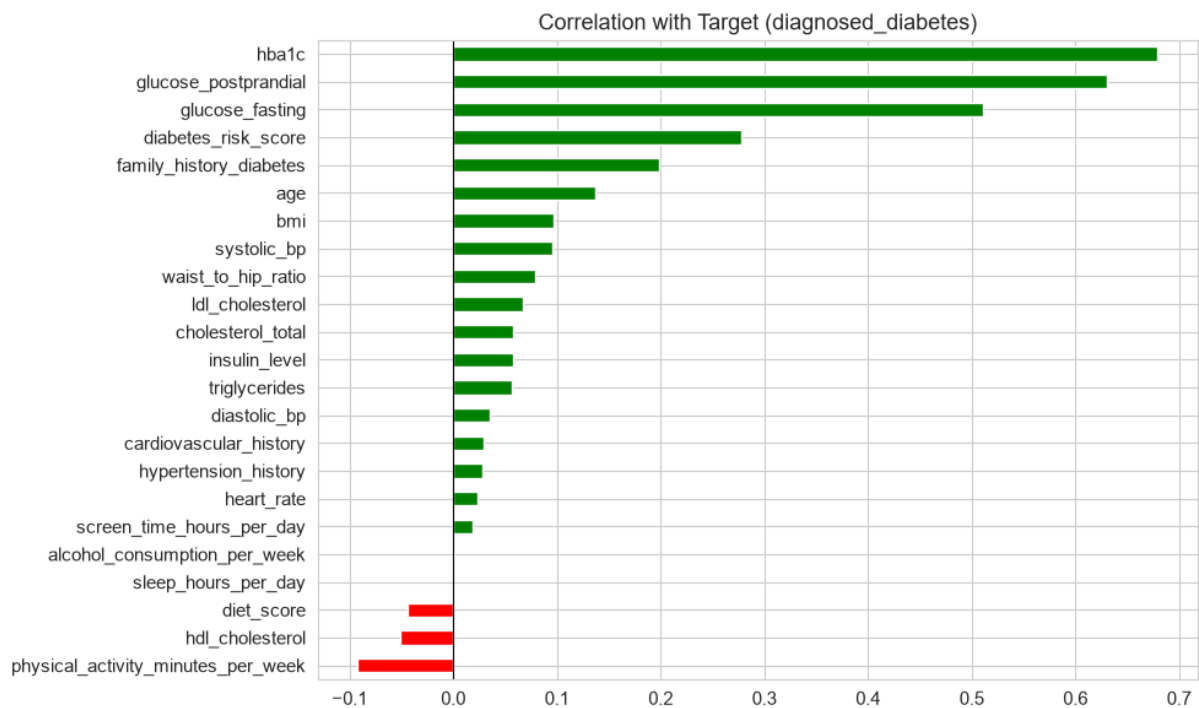
Hình 5.1. Output thực tế của cell “EDA - NUMERICAL & CATEGORICAL”.

Output 2



Hình 5.2. Output thực tế của cell “EDA - NUMERICAL & CATEGORICAL”.

Output 3



Hình 5.3. Output thực tế của cell “EDA - NUMERICAL & CATEGORICAL”.

4. Giải thích chi tiết code

- Cell tính số hàng/cột biểu đồ dựa trên số numeric columns, sau đó tạo histogram cho từng biến. `dropna()` giúp histogram không bị ảnh hưởng bởi các NaN chưa impute.
- Tiêu đề mỗi histogram chứa tên feature và số lượng missing của feature đó. Nhờ vậy người đọc có thể đồng thời quan sát hình dạng phân bố và tình trạng thiếu.
- `corr = df_clean[corr_cols].corr()` tính Pearson correlation trên các numeric columns cùng target. Heatmap dùng mask tam giác trên để tránh hiển thị hai lần cùng một cặp tương quan.
- `target_corr` lấy riêng cột `diagnosed_diabetes`, bỏ chính target rồi `sort_values`. Biểu đồ thanh ngang giúp xếp hạng trực quan mức độ tương quan tuyến tính của từng feature với target.

5. Phân tích output và ý nghĩa

Output thực tế gồm ba nhóm trực quan: histogram của các numeric features, heatmap correlation matrix và biểu đồ target correlation. Các hình cho phép nhận diện feature có phân bố lệch, độ phân tán khác nhau, missing và mức độ liên hệ tuyến tính với target. Cần lưu ý correlation chỉ phản ánh quan hệ tuyến tính; feature có correlation thấp không nhất thiết vô dụng đối với MLP.

6. CATEGORICAL ANALYSIS & HANDLING

1. Mục đích của cell

Phân tích các biến categorical còn lại và xem tỷ lệ diabetes thay đổi như thế nào giữa các nhóm. Đây là bước kiểm tra trước khi one-hot encoding.

2. Ảnh code thực tế

```
# =====
# CELL 6: CATEGORICAL ANALYSIS & HANDLING
# =====

for col in cat_cols:
    print(f"\n== {col} ==")
    print(
        df_clean[col]
        .value_counts(dropna=False)
        .head(10)
    )

# Visualize categorical vs target
if len(cat_cols) > 0:
    n_show = min(6, len(cat_cols))
    n_cat_cols = 3
    n_cat_rows = (n_show + n_cat_cols - 1) // n_cat_cols

    fig, axes = plt.subplots(
        n_cat_rows,
        n_cat_cols,
        figsize=(18, 5 * n_cat_rows)
    )

    axes = np.atleast_1d(axes).flatten()

    for i, col in enumerate(cat_cols[:n_show]):
        ct = pd.crosstab(
            df_clean[col],
            df_clean['diagnosed_diabetes'],
            normalize='index'
        )

        ct.plot(
            kind='bar',
            ax=axes[i],
            stacked=True,
            color=['steelblue', 'coral']
        )

        axes[i].set_title(
            f'{col} vs Target (proportion)'
        )
        axes[i].legend(
            title='diabetes',
            loc='best'
        )
        axes[i].tick_params(
            axis='x',
            rotation=30
        )

    for j in range(n_show, len(axes)):
        axes[j].axis('off')

    plt.tight_layout()
    plt.show()
else:
    print("No categorical columns found.")
```

3. Output thực tế lấy từ notebook

Output 1

OUTPUT THỰC TẾ — OUTPUT 1

```
=== gender ===
gender
Female    50216
Male      47771
Other      2013
Name: count, dtype: int64

=== ethnicity ===
ethnicity
White      44997
Hispanic   20103
Black      17986
Asian      11865
Other       5049
Name: count, dtype: int64

=== education_level ===
education_level
Highschool    44891
Graduate      35037
Postgraduate  14972
No formal     5100
Name: count, dtype: int64

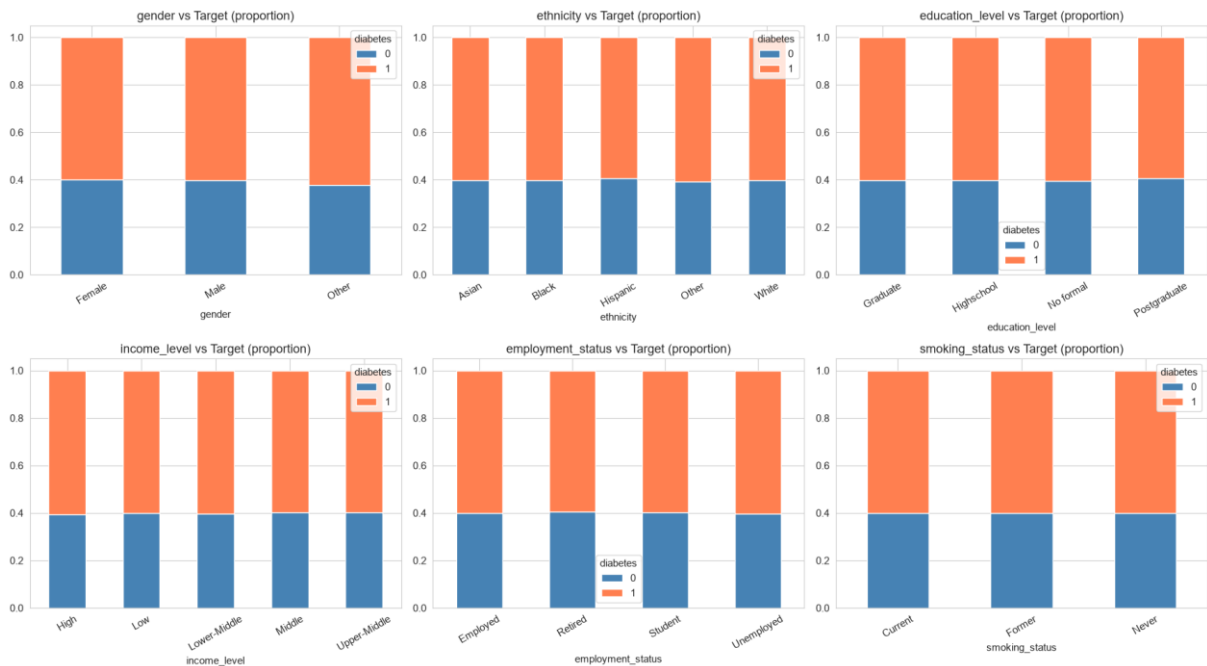
=== income_level ===
income_level
Middle      35152
Lower-Middle 25150
Upper-Middle 19866
Low         14830
High        5002
Name: count, dtype: int64

=== employment_status ===
employment_status
Employed    60175
Retired     21761
Unemployed  11918
Student      6146
Name: count, dtype: int64

=== smoking_status ===
smoking_status
Never       59813
Current     20176
Former      20011
Name: count, dtype: int64
```

Hình 6.1. Output thực tế của cell “CATEGORICAL ANALYSIS & HANDLING”.

Output 2



Hình 6.2. Output thực tế của cell “CATEGORICAL ANALYSIS & HANDLING”.

4. Giải thích chi tiết code

- `value_counts(dropna=False).head(10)` được dùng cho từng categorical column. `dropna=False` giữ cả nhóm missing để không bỏ qua một nguồn thông tin quan trọng khi kiểm tra dữ liệu.
- `pd.crosstab(..., normalize='index')` tạo bảng tỷ lệ theo từng nhóm categorical. Chuẩn hóa theo hàng nghĩa là trong mỗi nhóm, tổng tỷ lệ target bằng 1.
- Stacked bar chart hiển thị tỷ lệ class 0 và class 1 trong từng category. Đây phù hợp hơn so với chỉ dùng số lượng tuyệt đối vì các category có thể có kích thước khác nhau.
- Code chỉ vẽ tối đa 6 categorical columns để tránh một figure quá lớn. Nếu không có categorical columns thì in thông báo thay thế.

5. Phân tích output và ý nghĩa

Output thực tế cho thấy các categorical chính có số mức nhỏ và có thể mã hóa one-hot. Ví dụ gender có Female 50.216, Male 47.771 và Other 2.013; smoking có Never 59.813, Current 20.176 và Former 20.011. Các biểu đồ stacked thể hiện sự khác nhau về tỷ lệ target giữa các nhóm và là cơ sở trực quan để tiếp tục

7. IMPUTATION & ENCODING

1. Mục đích của cell

Cell thực hiện một bước trong pipeline của notebook.

2. Ảnh code thực tế

```
# =====
# 7. IMPUTATION & ENCODING
# =====
df_proc = df_clean.copy()

# 6.1 Separate features and target
y_all = df_proc['diagnosed_diabetes'].values.astype(np.float32)
X_df = df_proc.drop(columns=['diagnosed_diabetes'])

# 6.2 Fill categorical NaN with mode
for col in cat_cols:
    if X_df[col].isnull().any():
        mode_val = X_df[col].mode()[0]
        X_df[col] = X_df[col].fillna(mode_val)

# 6.3 Sanity check TRƯỚC khi one-hot
print("=== Sanity check before one-hot ===")
total_dummies = 0
for col in cat_cols:
    n = X_df[col].nunique(dropna=True)
    total_dummies += n
    print(f" {col}: {n} unique -> {n} dummies")
print(f"Total dummy columns expected: {total_dummies}")
print(f"Estimated memory for one-hot: {len(X_df) * (len(numeric_cols) + total_dummies) * 4 / 1e9:.2f} GB")

# ⚠ Nếu vẫn > 2 GB, cân nhắc dừng Lại
MAX_MEM_GB = 4.0
est_mem = len(X_df) * (len(numeric_cols) + total_dummies) * 4 / 1e9
if est_mem > MAX_MEM_GB:
    raise MemoryError(f"Estimated memory {est_mem:.2f} GB > {MAX_MEM_GB} GB. Cần giảm cardinality!")

# 6.4 One-hot encode (dùng sparse=False nhưng cardinality đã được kiểm soát)
X_encoded = pd.get_dummies(
    X_df,
    columns=cat_cols,
    drop_first=False,
    dtype=np.float32
)
print(f"\nShape after one-hot: {X_encoded.shape}")
print(f"Actual memory: {X_encoded.memory_usage(deep=True).sum() / 1e9:.3f} GB")

# 6.5 Numeric missing
num_missing = X_encoded[numeric_cols].isnull().sum()
print("\nNumeric missing before imputation:")
print(num_missing[num_missing > 0])

feature_names = list(X_encoded.columns)
print(f"\nTotal features: {len(feature_names)}")
```

3. Output thực tế lấy từ notebook

Output 1

OUTPUT THỰC TẾ — OUTPUT 1

```
=== Sanity check before one-hot ===
gender: 3 unique -> 3 dummies
ethnicity: 5 unique -> 5 dummies
education_level: 4 unique -> 4 dummies
income_level: 5 unique -> 5 dummies
employment_status: 4 unique -> 4 dummies
smoking_status: 3 unique -> 3 dummies
Total dummy columns expected: 24
Estimated memory for one-hot: 0.02 GB

Shape after one-hot: (100000, 47)
Actual memory: 0.028 GB

Numeric missing before imputation:
age          5000
bmi          5000
cholesterol_total  2000
glucose_fasting  2000
dtype: int64

Total features: 47
```

Hình 7.1. Output thực tế của cell “Code Cell 8”.

4. Giải thích chi tiết code

Đoạn code này là bước Imputation & Encoding trong quy trình tiền xử lý dữ liệu, cụ thể là:

- Tách biến mục tiêu y và biến đầu vào X.
- Xử lý giá trị thiếu ở biến categorical bằng mode.
- Kiểm tra trước xem one-hot encoding có tạo ra quá nhiều cột hay không.
- One-hot encode các biến categorical.
- Kiểm tra missing value ở các biến numeric.
- Lưu tên toàn bộ feature sau encoding.

5. Phân tích output và ý nghĩa

Output này cho thấy bước **imputation** và **one-hot encoding** đang hoạt động khá tốt về mặt bộ nhớ và số lượng feature. Cụ thể, 6 biến categorical gồm gender, ethnicity, education_level, income_level, employment_status và smoking_status có tổng cộng **24 giá trị khác nhau**, nên sau one-hot encoding dự kiến tạo ra 24 dummy variables. Với **100.000 quan sát**, bộ dữ liệu chỉ cần khoảng **0,02 GB bộ nhớ theo ước tính** và thực tế sau encoding sử dụng khoảng **0,028 GB**, tức khoảng 28 MB, cho thấy **không có vấn đề về memory hay cardinality explosion**. Sau khi encoding, dữ liệu có kích thước **(100000, 47)**, nghĩa là vẫn giữ 100.000 samples và tổng cộng 47 features, trong đó 24 là dummy variables và khoảng 23 là các features numeric khác. Tuy nhiên, output cũng cho thấy **numeric missing values vẫn chưa được xử lý**: age và bmi mỗi biến thiếu 5.000 giá trị (5%), còn cholesterol_total và glucose_fasting mỗi biến thiếu 2.000 giá trị (2%). Vì vậy, bước tiếp theo cần thực hiện **numeric imputation**, chẳng hạn thay thế

các giá trị thiếu bằng median, trước khi đưa dữ liệu vào mô hình machine learning.

8. TRAIN / VALIDATION / TEST SPLIT + NORMALIZATION

1. Mục đích của cell

Tạo ba tập train, validation và test, sau đó xử lý missing và chuẩn hóa theo thống kê của train. Đây là một trong những cell quan trọng nhất về mặt chống data leakage.

2. Ảnh code thực tế

```
# =====
# CELL 8: TRAIN / VALIDATION / TEST SPLIT + NORMALIZATION
# =====

# Split: 70% train, 15% validation, 15% test
X_np = X_encoded.values.astype(np.float32)

X_train_full, X_test, y_train_full, y_test = train_test_split(
    X_np,
    y_all,
    test_size=0.15,
    random_state=42,
    stratify=y_all
)

X_train, X_val, y_train, y_val = train_test_split(
    X_train_full,
    y_train_full,
    test_size=0.15 / 0.85,
    random_state=42,
    stratify=y_train_full
)

print(f"Train: {X_train.shape}, Val: {X_val.shape}, Test: {X_test.shape}")
print(f"Train pos rate: {y_train.mean():.4f}")
print(f"Val pos rate: {y_val.mean():.4f}")
print(f"Test pos rate: {y_test.mean():.4f}")

# -----
# Imputation using TRAIN statistics only
# -----
train_medians = np.nanmedian(X_train, axis=0)

def impute_with_train_median(X, medians):
    X = X.copy()

    inds = np.where(np.isnan(X))

    X[inds] = np.take(
        medians,
        inds[1]
    )

    return X

X_train = impute_with_train_median(
    X_train,
    train_medians
)

X_val = impute_with_train_median(
    X_val,
    train_medians
)

X_test = impute_with_train_median(
    X_test,
    train_medians
)
```

```

# -----
# Standardization using TRAIN statistics only
# -----
mu = X_train.mean(axis=0)
sigma = X_train.std(axis=0) + 1e-8

X_train = (X_train - mu) / sigma
X_val = (X_val - mu) / sigma
X_test = (X_test - mu) / sigma

# Reshape targets
y_train = y_train.reshape(-1, 1)
y_val = y_val.reshape(-1, 1)
y_test = y_test.reshape(-1, 1)

print("\nAfter imputation & normalization:")
print(
    f"X_train: {X_train.shape} "
    f"mean≈{X_train.mean():.4f} "
    f"std≈{X_train.std():.4f}"
)
print(f"X_val: {X_val.shape}")
print(f"X_test: {X_test.shape}")

print(
    f"No NaN: "
    f"{np.isnan(X_train).sum()}, "
    f"{np.isnan(X_val).sum()}, "
    f"{np.isnan(X_test).sum()}"
)

```

3. Output thực tế lấy từ notebook

Output 1

OUTPUT THỰC TẾ — OUTPUT 1

```

Train: (69999, 47), Val: (15001, 47), Test: (15000, 47)
Train pos rate: 0.6000
Val pos rate: 0.6000
Test pos rate: 0.6000

After imputation & normalization:
X_train: (69999, 47) mean=0.0000 std=1.0000
X_val: (15001, 47)
X_test: (15000, 47)
No NaN: 0, 0, 0

```

Hình 8.1. Output thực tế của cell “TRAIN / VALIDATION / TEST SPLIT + NORMALIZATION”.

4. Giải thích chi tiết code

- `train_test_split` lần đầu lấy 15% làm test. Tham số `stratify=y_all` đảm bảo tỷ lệ lớp được duy trì gần như tương đương giữa `train_full` và test.
- Lần split thứ hai lấy 15% của tổng dataset tương đương làm validation. Kết quả là khoảng 70% train, 15% validation và 15% test.
- `train_medians = np.nanmedian(X_train, axis=0)` chỉ tính median trên train. Hàm `impute_with_train_median` thay các NaN bằng median tương ứng của cột; cùng một median được áp dụng cho validation và test.

- μ và σ cũng chỉ được tính từ X_{train} . Sau đó train, val và test đều được biến đổi theo cùng công thức $(X - \mu) / \sigma$. Việc cộng $1e-8$ vào σ tránh chia cho 0 ở feature không có độ lệch chuẩn.
- Targets được reshape thành cột $(N,1)$, phù hợp với output sigmoid của MLP. Cuối cell, notebook kiểm tra mean/std và tổng NaN sau preprocessing.

5. Phân tích output và ý nghĩa

Kết quả thực tế: Train 69.999 mẫu, Validation 15.001 mẫu và Test 15.000 mẫu; positive rate của cả ba tập đều 0.6000. Sau normalization, X_{train} có mean xấp xỉ 0 và std xấp xỉ 1, đồng thời số NaN ở cả ba tập bằng 0. Điều này chứng minh pipeline preprocessing đã tạo được ma trận số sạch và chuẩn hóa mà không lấy thống kê từ test.

9. BASELINE MACHINE LEARNING MODELS

1. Mục đích của cell

Thiết lập các mô hình truyền thống làm baseline để có mốc so sánh khách quan với MLP NumPy.

2. Ảnh code thực tế

```
# =====  
# CELL 9: BASELINE MACHINE LEARNING MODELS  
# =====  
  
def evaluate_model(name, y_true, y_pred, y_prob=None):  
    acc = accuracy_score(y_true, y_pred)  
    prec = precision_score(  
        y_true,  
        y_pred,  
        zero_division=0  
    )  
    rec = recall_score(  
        y_true,  
        y_pred,  
        zero_division=0  
    )  
    f1 = f1_score(  
        y_true,  
        y_pred,  
        zero_division=0  
    )  
    auc = (  
        roc_auc_score(y_true, y_prob)  
        if y_prob is not None  
        else np.nan  
    )  
  
    return {  
        'Model': name,  
        'Accuracy': acc,  
        'Precision': prec,  
        'Recall': rec,  
        'F1': f1,  
        'ROC-AUC': auc  
    }  
  
ml_results = []  
ml_probs = {}
```

```

# -----
# Logistic Regression
# -----
print("Training Logistic Regression...")

lr = LogisticRegression(
    max_iter=1000,
    random_state=42,
    n_jobs=-1
)

lr.fit(
    X_train,
    y_train.ravel()
)

lr_pred = lr.predict(X_test)
lr_prob = lr.predict_proba(X_test)[:, 1]

ml_results.append(
    evaluate_model(
        'Logistic Regression',
        y_test.ravel(),
        lr_pred,
        lr_prob
    )
)

ml_probs['Logistic Regression'] = lr_prob

# -----
# Random Forest
# -----
print("Training Random Forest...")

rf = RandomForestClassifier(
    n_estimators=200,
    max_depth=15,
    random_state=42,
    n_jobs=-1
)

rf.fit(
    X_train,
    y_train.ravel()
)

rf_pred = rf.predict(X_test)
rf_prob = rf.predict_proba(X_test)[:, 1]

ml_results.append(
    evaluate_model(
        'Random Forest',
        y_test.ravel(),
        rf_pred,
        rf_prob
    )
)

ml_probs['Random Forest'] = rf_prob

```

```

# XGBoost / Gradient Boosting
# -----
if HAS_XGB:
    print("Training XGBoost...")

    xgb = XGBClassifier(
        n_estimators=200,
        max_depth=6,
        learning_rate=0.1,
        random_state=42,
        use_label_encoder=False,
        eval_metric='logloss'
    )

    xgb.fit(
        X_train,
        y_train.ravel()
    )

    xgb_pred = xgb.predict(X_test)
    xgb_prob = xgb.predict_proba(X_test)[:, 1]

    ml_results.append(
        evaluate_model(
            'XGBoost',
            y_test.ravel(),
            xgb_pred,
            xgb_prob
        )
    )

    ml_probs['XGBoost'] = xgb_prob
else:
    print("Training GradientBoosting...")

    gb = GradientBoostingClassifier(
        n_estimators=200,
        max_depth=4,
        random_state=42
    )

    gb.fit(
        X_train,
        y_train.ravel()
    )

    gb_pred = gb.predict(X_test)
    gb_prob = gb.predict_proba(X_test)[:, 1]

    ml_results.append(
        evaluate_model(
            'GradientBoosting',
            y_test.ravel(),
            gb_pred,
            gb_prob
        )
    )

    ml_probs['GradientBoosting'] = gb_prob

ml_df = pd.DataFrame(ml_results)

print("\n=== Baseline ML Results ===")
display(ml_df)

```

3. Output thực tế lấy từ notebook

Output 1

OUTPUT THỰC TẾ — OUTPUT 1

```
Training Logistic Regression...
Training Random Forest...
Training GradientBoosting...

=== Baseline ML Results ===
```

Hình 9.1. Output thực tế của cell “BASELINE MACHINE LEARNING MODELS”.

Output 2

OUTPUT THỰC TẾ — OUTPUT 2

	Model	Accuracy	Precision	Recall	F1	ROC-AUC
0	Logistic Regression	0.860600	0.875612	0.894778	0.885091	0.933224
1	Random Forest	0.920133	0.999104	0.867667	0.928758	0.942211
2	GradientBoosting	0.920400	0.999360	0.867889	0.928996	0.943518

Hình 9.2. Output thực tế của cell “BASELINE MACHINE LEARNING MODELS”.

4. Giải thích chi tiết code

- `evaluate_model` gom Accuracy, Precision, Recall, F1 và ROC-AUC vào một dictionary. ROC-AUC được tính từ probability thay vì class cứng, nên phản ánh khả năng xếp hạng mẫu của mô hình.
- Logistic Regression được huấn luyện với `max_iter=1000`. Đây là baseline tuyến tính, giúp kiểm tra mức độ hiệu quả của một decision boundary đơn giản.
- Random Forest dùng 200 cây và `max_depth=15`. Mô hình này có khả năng học quan hệ phi tuyến và tương tác feature mà Logistic Regression không mô tả được.
- Nếu XGBoost có sẵn, notebook dùng `XGBClassifier`; nếu không thì dùng `GradientBoostingClassifier` với 200 estimators và `max_depth=4`. Cách này nối trực tiếp với quyết định dependency ở cell setup.
- Kết quả của từng model được lưu vào `ml_results` và probability lưu trong `ml_probs` để phục vụ ROC curve về sau. Cuối cell, `ml_df` là bảng baseline chính.

5. Phân tích output và ý nghĩa

Baseline thực tế gồm Logistic Regression, Random Forest và GradientBoosting. Kết quả được tổng hợp thành bảng. Các số liệu chính được notebook ghi lại ở phần summary: Logistic Regression Accuracy 0.8606; Random Forest 0.920133; GradientBoosting 0.920400. Điều này cho thấy các mô hình cây đã tạo baseline khá mạnh trước khi chuyển sang Deep Learning.

10. VISUALIZE ML RESULTS

1. Mục đích của cell

Chuyển bảng metric và probability của các baseline thành biểu đồ để so sánh nhanh hiệu năng và khả năng phân biệt lớp.

2. Ảnh code thực tế

```
# CELL 10: VISUALIZE ML RESULTS
# =====

fig, axes = plt.subplots(
    1,
    2,
    figsize=(16, 6)
)

# Bar chart of metrics
metrics = [
    'Accuracy',
    'Precision',
    'Recall',
    'F1',
    'ROC-AUC'
]

x = np.arange(len(metrics))
width = 0.25

for i, row in ml_df.iterrows():
    axes[0].bar(
        x + i * width,
        [row[m] for m in metrics],
        width,
        label=row['Model']
    )

axes[0].set_xticks(x + width)
axes[0].set_xticklabels(metrics)
axes[0].set_ylim(0, 1.05)
axes[0].set_title(
    'ML Models - Metrics Comparison'
)
axes[0].legend()
axes[0].grid(
    axis='y',
    alpha=0.3
)

# ROC curves
for name, prob in ml_probs.items():
    fpr, tpr, _ = roc_curve(
        y_test.ravel(),
        prob
    )

    auc = roc_auc_score(
        y_test.ravel(),
        prob
    )

    axes[1].plot(
        fpr,
        tpr,
        label=f'{name} (AUC={auc:.3f})'
    )

axes[1].plot(
    [0, 1],
    [0, 1],
    'k--',
    alpha=0.5
)
```

```

axes[1].set_xlabel('FPR')
axes[1].set_ylabel('TPR')
axes[1].set_title(
    'ROC Curves - ML Models'
)
axes[1].legend()

plt.tight_layout()
plt.show()

# Confusion matrices
fig, axes = plt.subplots(
    1,
    len(ml_df),
    figsize=(6 * len(ml_df), 5)
)

if len(ml_df) == 1:
    axes = [axes]

preds_all = {
    'Logistic Regression': lr_pred,
    'Random Forest': rf_pred
}

if HAS_XGB:
    preds_all['XGBoost'] = xgb_pred
else:
    preds_all['GradientBoosting'] = gb_pred

for i, (name, pred) in enumerate(preds_all.items()):
    cm = confusion_matrix(
        y_test.ravel(),
        pred
    )

    sns.heatmap(
        cm,
        annot=True,
        fmt='d',
        ax=axes[i],
        cmap='Blues',
        cbar=False
    )

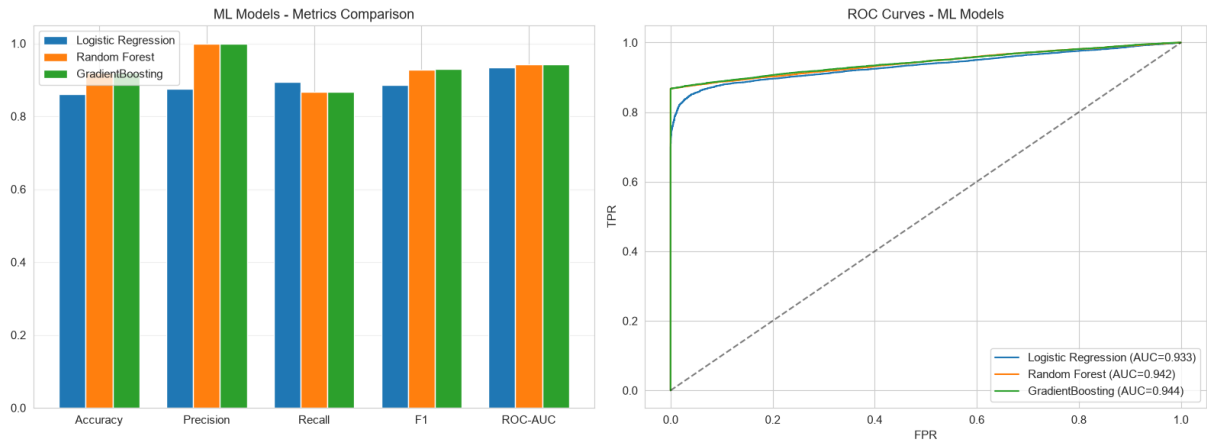
    axes[i].set_title(name)
    axes[i].set_xlabel('Predicted')
    axes[i].set_ylabel('Actual')

plt.tight_layout()
plt.show()

```

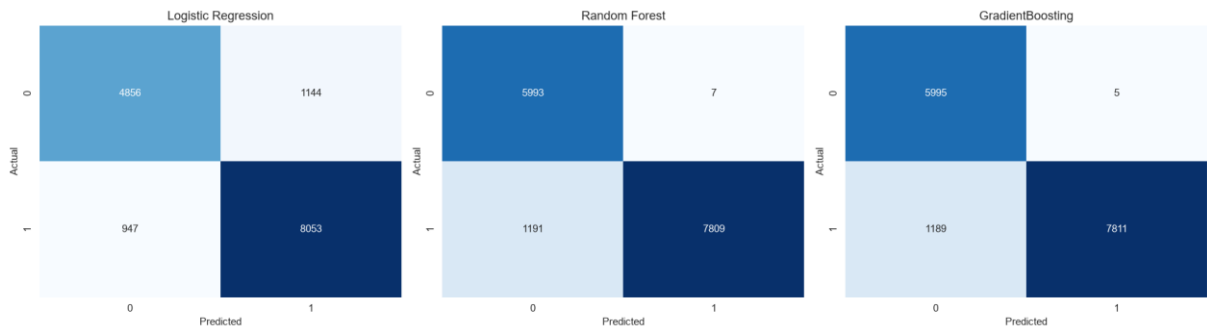
3. Output thực tế lấy từ notebook

Output 1



Hình 10.1. Output thực tế của cell “VISUALIZE ML RESULTS”.

Output 2



Hình 10.2. Output thực tế của cell “VISUALIZE ML RESULTS”.

4. Giải thích chi tiết code

- Biểu đồ đầu tiên lấy năm metric Accuracy, Precision, Recall, F1 và ROC-AUC rồi vẽ grouped bar chart cho từng model. Trục y giới hạn 0–1.05 phù hợp với thang điểm metric.
- ROC curve được xây dựng từ fpr/tptr của từng probability. Đường chéo [0,1] biểu diễn mức gần random classifier; đường ROC càng nằm về góc trên-trái thì khả năng phân biệt càng tốt.
- Phần confusion matrix tính từ y_{test} và prediction của từng model. Heatmap cho biết số lượng True Negative, False Positive, False Negative và True Positive.
- Việc dùng cả metric tổng hợp, ROC và confusion matrix là cần thiết vì một metric đơn lẻ không mô tả đầy đủ hành vi classifier.

5. Phân tích output và ý nghĩa

Output thực tế cho thấy GradientBoosting và Random Forest có hiệu năng tổng thể cao hơn Logistic Regression. ROC curves của hai mô hình cây nằm cao hơn phần lớn đường của Logistic Regression; confusion matrices cũng cho thấy số lượng dự đoán đúng cao. Đây là baseline mạnh để so sánh với MLP.

11. DEEP LEARNING FROM SCRATCH - 5 LAYER MLP

1. Mục đích của cell

Định nghĩa một MLP nhiều tầng hoàn toàn bằng NumPy, không dùng TensorFlow, PyTorch hay Keras. Đây là phần cốt lõi chứng minh khả năng triển khai Deep Learning từ đầu.

2. Ảnh code thực tế

```
# =====  
# CELL 11: DEEP LEARNING FROM SCRATCH - 5 LAYER MLP  
# NUMPY ONLY  
# =====  
  
# Architecture:  
# input -> 128 -> 64 -> 32 -> 16 -> 1  
#  
# Hidden layers: ReLU  
# Output layer: Sigmoid  
# Loss: Binary Cross-Entropy  
  
class MLP5Layer:  
    """  
    5-layer MLP with ReLU hidden activations + Sigmoid output.  
  
    Layers:  
        input -> h1 -> h2 -> h3 -> h4 -> output  
    """  
  
    def __init__(  
        self,  
        input_dim,  
        hidden_dims=(128, 64, 32, 16),  
        seed=42  
    ):  
        rng = np.random.RandomState(seed)  
  
        dims = [  
            input_dim  
        ] + list(hidden_dims) + [1]  
  
        self.W = []  
        self.b = []  
  
        for i in range(len(dims) - 1):  
            # He initialization  
            scale = np.sqrt(  
                2.0 / dims[i]  
            )  
  
            self.W.append(  
                rng.randn(  
                    dims[i],  
                    dims[i + 1]  
                ).astype(np.float32) * scale  
            )  
  
            self.b.append(  
                np.zeros(  
                    (1, dims[i + 1]),  
                    dtype=np.float32  
                )  
            )  
  
        self.num_layers = len(self.W)
```

```

# -----
# Activations
# -----
@staticmethod
def relu(x):
    return np.maximum(0, x)

@staticmethod
def relu_deriv(x):
    return (
        x > 0
    ).astype(np.float32)

@staticmethod
def sigmoid(x):
    x = np.clip(x, -50, 50)
    return 1.0 / (
        1.0 + np.exp(-x)
    )

# -----
# Forward propagation
# -----
def forward(self, X):
    cache = {
        'A0': X
    }

    A = X

    # Hidden layers
    for i in range(
        self.num_layers - 1
    ):
        Z = (
            A @ self.W[i]
            + self.b[i]
        )

        A = self.relu(Z)

        cache[f'Z{i + 1}'] = Z
        cache[f'A{i + 1}'] = A

    # Output layer
    ZL = (
        A @ self.W[-1]
        + self.b[-1]
    )

    AL = self.sigmoid(ZL)

    cache[f'Z{self.num_layers}'] = ZL
    cache[f'A{self.num_layers}'] = AL

    return AL, cache

```

```

# -----
# Binary Cross-Entropy
# -----
@staticmethod
def binary_cross_entropy(
    y,
    y_hat,
    eps=1e-8
):
    y_hat = np.clip(
        y_hat,
        eps,
        1 - eps
    )

    return -np.mean(
        y * np.log(y_hat)
        + (1 - y) * np.log(1 - y_hat)
    )

# -----
# Backpropagation
# -----
def backward(self, y, cache):
    m = y.shape[0]

    grads_W = [
        None
    ] * self.num_layers

    grads_b = [
        None
    ] * self.num_layers

    # Output layer:
    # sigmoid + BCE -> dZ = y_hat - y
    AL = cache[
        f'A{self.num_layers}'
    ]

    dZ = (
        AL - y
    ) / m

    A_prev = cache[
        f'A{self.num_layers - 1}'
    ]

    grads_W[-1] = (
        A_prev.T @ dZ
    )

    grads_b[-1] = np.sum(
        dZ,
        axis=0,
        keepdims=True
    )

```

```

# Hidden layers
for i in range(
    self.num_layers - 2,
    -1,
    -1
):
    dA = (
        dZ
        @ self.W[i + 1].T
    )

    Z = cache[
        f'Z{i + 1}'
    ]

    dZ = (
        dA
        * self.relu_deriv(Z)
    )

    A_prev = cache[
        f'A{i}'
    ]

    grads_W[i] = (
        A_prev.T @ dZ
    )

    grads_b[i] = np.sum(
        dZ,
        axis=0,
        keepdims=True
    )

    return grads_W, grads_b

# -----
# Prediction
# -----
def predict_proba(self, X):
    y_hat, _ = self.forward(X)
    return y_hat

def predict(
    self,
    X,
    threshold=0.5
):
    return (
        self.predict_proba(X)
        >= threshold
    ).astype(int)

print("MLP5Layer class defined.")

```

3. Output thực tế lấy từ notebook

Output 1

OUTPUT THỰC TẾ — OUTPUT 1

MLP5Layer class defined.

Hình 11.1. Output thực tế của cell “DEEP LEARNING FROM SCRATCH - 5 LAYER MLP”.

4. Giải thích chi tiết code

- Constructor tạo danh sách dims = [input_dim] + hidden_dims + [1]. Với cấu hình baseline, mạng có dạng input $\rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 1$.
- Trọng số W được khởi tạo bằng He initialization: $scale = \sqrt{2 / fan_in}$. Cách khởi tạo này phù hợp với hidden layers dùng ReLU vì giúp kiểm soát phương sai kích hoạt ban đầu.
- relu và relu_deriv lần lượt thực hiện $\max(0, x)$ và đạo hàm dạng chỉ báo. sigmoid dùng clip [-50,50] trước exp để tránh overflow số học khi z quá lớn hoặc quá nhỏ.
- forward thực hiện chuỗi phép biến đổi $Z=A@W+b$, ReLU ở hidden layers và sigmoid ở output. cache lưu Z/A của từng layer vì backward cần các giá trị trung gian này.
- binary_cross_entropy tính loss trung bình cho bài toán nhị phân; y_hat được clip để tránh $\log(0)$.
- backward bắt đầu từ output với $dZ=(AL-y)/m$, một đạo hàm thuận tiện của sigmoid kết hợp BCE. Sau đó gradient truyền ngược qua từng hidden layer bằng phép nhân ma trận và đạo hàm ReLU.
- predict_proba trả probability, còn predict áp dụng threshold mặc định 0.5 để chuyển probability thành class 0/1.

5. Phân tích output và ý nghĩa

Output chỉ in 'MLP5Layer class defined.' Đây là output xác nhận class được định nghĩa thành công, chưa phải kết quả huấn luyện. Giá trị quan trọng của cell nằm ở việc hoàn thành đầy đủ các thành phần của một neural network: parameter storage, forward propagation, loss, backpropagation và prediction.

12. TRAINING LOOP WITH MINI-BATCH + ADAM & SGD

1. Mục đích của cell

Cell thực hiện một bước trong pipeline của notebook.

2. Ảnh code thực tế

```
# =====
# 11. TRAINING LOOP WITH MINI-BATCH + ADAM & SGD
# =====

def train_mlp(model, X_train, y_train, X_val, y_val,
              epochs=100, batch_size=128, lr=0.01,
              optimizer='adam', l2=0.0, verbose=True,
              patience=None, print_every=1):
    """
    Train MLP with mini-batch gradient descent.
    Supports SGD and Adam optimizers.

    Parameters
    -----
    print_every : int
        In log mỗi `print_every` epoch. Mặc định = 1 (in mỗi epoch).
    patience : int or None
        Early stopping nếu val_loss không cải thiện sau `patience` epoch.

    Returns
    -----
    history : dict with keys 'train_loss', 'val_loss', 'train_acc', 'val_acc'
    """
    m = X_train.shape[0]
    n_batches = max(1, m // batch_size)

    # Optimizer state
    if optimizer == 'adam':
        beta1, beta2, eps = 0.9, 0.999, 1e-8
        mW = [np.zeros_like(w) for w in model.W]
        vW = [np.zeros_like(w) for w in model.W]
        mb = [np.zeros_like(b) for b in model.b]
        vb = [np.zeros_like(b) for b in model.b]
        t = 0

    history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}
    best_val_loss = np.inf
    best_params = None
    wait = 0
```

```

for epoch in range(epochs):
    # Shuffle
    idx = np.random.permutation(m)
    X_sh, y_sh = X_train[idx], y_train[idx]

    epoch_loss = 0.0
    for bidx in range(n_batches):
        start = bidx * batch_size
        end = min(start + batch_size, m)
        Xb = X_sh[start:end]
        yb = y_sh[start:end]

        y_hat, cache = model.forward(Xb)
        loss = model.binary_cross_entropy(yb, y_hat)
        epoch_loss += loss * (end - start)

        gW, gb = model.backward(yb, cache)

        # L2 regularization
        if l2 > 0:
            for i in range(model.num_layers):
                gW[i] = gW[i] + l2 * model.W[i]

        if optimizer == 'sgd':
            for i in range(model.num_layers):
                model.W[i] -= lr * gW[i]
                model.b[i] -= lr * gb[i]
        elif optimizer == 'adam':
            t += 1
            for i in range(model.num_layers):
                # --- W ---
                mW[i] = beta1 * mW[i] + (1 - beta1) * gW[i]
                vW[i] = beta2 * vW[i] + (1 - beta2) * (gW[i] ** 2)
                mW_hat = mW[i] / (1 - beta1 ** t)
                vW_hat = vW[i] / (1 - beta2 ** t)
                model.W[i] -= lr * mW_hat / (np.sqrt(vW_hat) + eps)
                # --- b ---
                mb[i] = beta1 * mb[i] + (1 - beta1) * gb[i]
                vb[i] = beta2 * vb[i] + (1 - beta2) * (gb[i] ** 2)
                mb_hat = mb[i] / (1 - beta1 ** t)
                vb_hat = vb[i] / (1 - beta2 ** t)
                model.b[i] -= lr * mb_hat / (np.sqrt(vb_hat) + eps)

    epoch_loss /= m

    # ----- Evaluate trên train/val -----
    train_prob = model.predict_proba(X_train)
    val_prob = model.predict_proba(X_val)
    train_loss = model.binary_cross_entropy(y_train, train_prob)
    val_loss = model.binary_cross_entropy(y_val, val_prob)
    train_acc = ((train_prob >= 0.5).astype(int) == y_train).mean()
    val_acc = ((val_prob >= 0.5).astype(int) == y_val).mean()

    history['train_loss'].append(train_loss)
    history['val_loss'].append(val_loss)
    history['train_acc'].append(train_acc)
    history['val_acc'].append(val_acc)

```

```

# ----- Early stopping -----
if patience is not None:
    if val_loss < best_val_loss - 1e-4:
        best_val_loss = val_loss
        best_params = ([w.copy() for w in model.W],
                        [b.copy() for b in model.b])
        wait = 0
    else:
        wait += 1
        if wait >= patience:
            if verbose:
                print(f"■ Early stopping tại epoch {epoch + 1}")
            break

# ----- IN LOG (MỖI EPOCH HOẶC MỖI print_every) -----
if verbose and ((epoch + 1) % print_every == 0 or epoch == epochs - 1):
    print(f"Epoch {epoch + 1:4d}/{epochs} | "
          f"TrainLoss={train_loss:.4f} | ValLoss={val_loss:.4f} | "
          f"TrainAcc={train_acc:.4f} | ValAcc={val_acc:.4f} | "
          f"wait={wait}")

# ----- Restore best params -----
if patience is not None and best_params is not None:
    model.W, model.b = best_params
    if verbose:
        print(f"✅ Restored best model (val_loss={best_val_loss:.4f})")

return history

print("Training function ready (print_every=1 mặc định in mỗi epoch).")

```

3. Output thực tế lấy từ notebook

Output 1

OUTPUT THỰC TẾ — OUTPUT 1

Training function ready (print_every=1 mặc định in mỗi epoch).

Hình 12.1. Output thực tế của cell “Code Cell 13”.

4. Giải thích chi tiết code

Đoạn code này xây dựng **hàm huấn luyện MLP (Multilayer Perceptron)** bằng phương pháp **mini-batch gradient descent**, đồng thời hỗ trợ hai optimizer là **SGD và Adam**. Đầu tiên, dữ liệu training được chia thành các mini-batch có kích thước `batch_size=128` và được **shuffle** sau mỗi epoch để tránh mô hình học theo thứ tự dữ liệu. Với mỗi batch, model thực hiện **forward propagation** để tạo dự đoán `y_hat`, tính **Binary Cross-Entropy loss**, sau đó thực hiện **backpropagation** để tính gradient của trọng số `W` và bias `b`. Nếu `l2 > 0`, code cộng thêm **L2 regularization** vào gradient nhằm hạn chế overfitting. Với `optimizer='sgd'`, trọng số được cập nhật trực tiếp theo công thức gradient descent; với `optimizer='adam'`, code sử dụng moment bậc nhất `m`, moment bậc hai `v` cùng các hệ số `beta1=0.9`, `beta2=0.999` để cập nhật trọng số ổn định hơn. Sau mỗi epoch, mô hình được đánh giá trên cả **training set và validation set**,

lưu lại `train_loss`, `val_loss`, `train_acc` và `val_acc` vào `history`. Code cũng hỗ trợ **Early Stopping** thông qua `patience`: nếu `val_loss` không cải thiện trong số epoch quy định, quá trình training sẽ dừng sớm và khôi phục lại bộ trọng số tốt nhất dựa trên validation loss. Cuối cùng, hàm trả về `history` để có thể dùng vẽ biểu đồ loss/accuracy và đánh giá quá trình học của mô hình.

5. Phân tích output và ý nghĩa

Chỉ đơn giản cho biết **hàm** `train_mlp()` **đã được định nghĩa và sẵn sàng để sử dụng**, không phải là kết quả của quá trình training. Tham số `print_every=1` nghĩa là khi bắt đầu huấn luyện, chương trình sẽ **in thông tin sau mỗi epoch**, bao gồm `TrainLoss`, `ValLoss`, `TrainAcc` và `ValAcc`. Vì ở thời điểm này bạn mới chỉ chạy phần định nghĩa hàm nên **chưa có model được train và chưa có các chỉ số loss/accuracy**; những kết quả đó sẽ xuất hiện khi bạn thực sự gọi `train_mlp(...)`.

13. TRAIN BASELINE DEEP LEARNING MODEL

1. Mục đích của cell

Huấn luyện mô hình MLP baseline với kiến trúc 128-64-32-16 và các hyperparameter ban đầu, sau đó đánh giá trên test.

2. Ảnh code thực tế

```
# =====
# CELL 13: TRAIN BASELINE DEEP LEARNING MODEL
# =====

input_dim = X_train.shape[1]

print(f"Input dimension: {input_dim}")

np.random.seed(42)

baseline_model = MLP5Layer(
    input_dim=input_dim,
    hidden_dims=(128, 64, 32, 16),
    seed=42
)

t0 = time.time()

baseline_hist = train_mlp(
    baseline_model,
    X_train,
    y_train,
    X_val,
    y_val,
    epochs=150,
    batch_size=128,
    lr=0.001,
    optimizer='adam',
    l2=1e-5,
    verbose=True,
    patience=20
)

print(
    f"\nTraining time: "
    f"{time.time() - t0:.2f}s"
)
```

```

# -----
# Evaluation helper
# -----
def full_eval(
    model,
    X,
    y,
    name
):
    prob = model.predict_proba(X)

    pred = (
        prob >= 0.5
    ).astype(int)

    return (
        evaluate_model(
            name,
            y.ravel(),
            pred.ravel(),
            prob.ravel()
        ),
        pred,
        prob
    )

baseline_metrics, baseline_pred, baseline_prob = full_eval(
    baseline_model,
    X_test,
    y_test,
    'MLP Baseline'
)

print("\nBaseline MLP Test:")
print(baseline_metrics)

```

3. Output thực tế lấy từ notebook

Output 1

OUTPUT THỰC TẾ — OUTPUT 1

```

Input dimension: 47
Epoch   1/150 | TrainLoss=0.2808 | ValLoss=0.2904 | TrainAcc=0.8817 | ValAcc=0.8762 | wait=0
Epoch   2/150 | TrainLoss=0.2479 | ValLoss=0.2645 | TrainAcc=0.9004 | ValAcc=0.8907 | wait=0
Epoch   3/150 | TrainLoss=0.2425 | ValLoss=0.2628 | TrainAcc=0.9023 | ValAcc=0.8923 | wait=0
Epoch   4/150 | TrainLoss=0.2267 | ValLoss=0.2525 | TrainAcc=0.9097 | ValAcc=0.8972 | wait=0
Epoch   5/150 | TrainLoss=0.2219 | ValLoss=0.2533 | TrainAcc=0.9110 | ValAcc=0.8985 | wait=1
Epoch   6/150 | TrainLoss=0.2172 | ValLoss=0.2489 | TrainAcc=0.9147 | ValAcc=0.9001 | wait=0
Epoch   7/150 | TrainLoss=0.2122 | ValLoss=0.2483 | TrainAcc=0.9163 | ValAcc=0.9019 | wait=0
Epoch   8/150 | TrainLoss=0.2096 | ValLoss=0.2551 | TrainAcc=0.9179 | ValAcc=0.9015 | wait=1
Epoch   9/150 | TrainLoss=0.2127 | ValLoss=0.2574 | TrainAcc=0.9157 | ValAcc=0.8995 | wait=2
Epoch  10/150 | TrainLoss=0.2082 | ValLoss=0.2597 | TrainAcc=0.9171 | ValAcc=0.9014 | wait=3
Epoch  11/150 | TrainLoss=0.1980 | ValLoss=0.2524 | TrainAcc=0.9211 | ValAcc=0.9033 | wait=4
Epoch  12/150 | TrainLoss=0.1938 | ValLoss=0.2588 | TrainAcc=0.9229 | ValAcc=0.9031 | wait=5
Epoch  13/150 | TrainLoss=0.1951 | ValLoss=0.2626 | TrainAcc=0.9216 | ValAcc=0.9043 | wait=6
Epoch  14/150 | TrainLoss=0.1883 | ValLoss=0.2649 | TrainAcc=0.9251 | ValAcc=0.9025 | wait=7
Epoch  15/150 | TrainLoss=0.1807 | ValLoss=0.2645 | TrainAcc=0.9292 | ValAcc=0.8999 | wait=8
Epoch  16/150 | TrainLoss=0.1760 | ValLoss=0.2728 | TrainAcc=0.9290 | ValAcc=0.9013 | wait=9
Epoch  17/150 | TrainLoss=0.1724 | ValLoss=0.2850 | TrainAcc=0.9313 | ValAcc=0.8994 | wait=10
Epoch  18/150 | TrainLoss=0.1739 | ValLoss=0.2854 | TrainAcc=0.9319 | ValAcc=0.8932 | wait=11
Epoch  19/150 | TrainLoss=0.1678 | ValLoss=0.2930 | TrainAcc=0.9310 | ValAcc=0.9010 | wait=12
Epoch  20/150 | TrainLoss=0.1632 | ValLoss=0.3077 | TrainAcc=0.9342 | ValAcc=0.8958 | wait=13
Epoch  21/150 | TrainLoss=0.1592 | ValLoss=0.3049 | TrainAcc=0.9348 | ValAcc=0.9001 | wait=14
Epoch  22/150 | TrainLoss=0.1556 | ValLoss=0.3178 | TrainAcc=0.9380 | ValAcc=0.8915 | wait=15
Epoch  23/150 | TrainLoss=0.1519 | ValLoss=0.3218 | TrainAcc=0.9402 | ValAcc=0.8855 | wait=16
Epoch  24/150 | TrainLoss=0.1556 | ValLoss=0.3292 | TrainAcc=0.9374 | ValAcc=0.8937 | wait=17
Epoch  25/150 | TrainLoss=0.1499 | ValLoss=0.3284 | TrainAcc=0.9402 | ValAcc=0.8895 | wait=18
Epoch  26/150 | TrainLoss=0.1436 | ValLoss=0.3413 | TrainAcc=0.9429 | ValAcc=0.8889 | wait=19
□ Early stopping tại epoch 27
□ Restored best model (val_loss=0.2483)

Training time: 69.25s

Baseline MLP Test:
{'Model': 'MLP Baseline', 'Accuracy': 0.9036666666666666, 'Precision': 0.9789527069861798, 'Recall': 0.8578888888888889, 'F1': 0.9 ... 937382111111

```

Hình 13.1. Output thực tế của cell “TRAIN BASELINE DEEP LEARNING MODEL”.

4. Giải thích chi tiết code

- `input_dim` được lấy trực tiếp từ số feature sau preprocessing, nên MLP tự thích nghi với 47 input features của dataset hiện tại.
- `baseline_model` dùng `hidden_dims=(128,64,32,16)`, `seed=42`. Với dữ liệu 47 features, đây là một mạng 5-layer theo cách gọi của notebook, gồm bốn hidden layers và một output layer.
- `train_mlp` chạy tối đa 150 epochs, batch 128, Adam, learning rate 0.001, `L2=1e-5` và `patience=20`. `time.time()` được dùng để đo thời gian huấn luyện.
- `full_eval` tính probability, threshold 0.5 và năm metric trên test. Kết quả được đặt tên 'MLP Baseline'.

5. Phân tích output và ý nghĩa

Output thực tế gồm log huấn luyện và thời gian training, sau đó là dictionary metric của Baseline MLP. Cell này tạo ra checkpoint baseline để các thí nghiệm learning rate, batch size, architecture và optimizer có thể so sánh với cùng pipeline dữ liệu.

14. VISUALIZE BASELINE TRAINING

1. Mục đích của cell

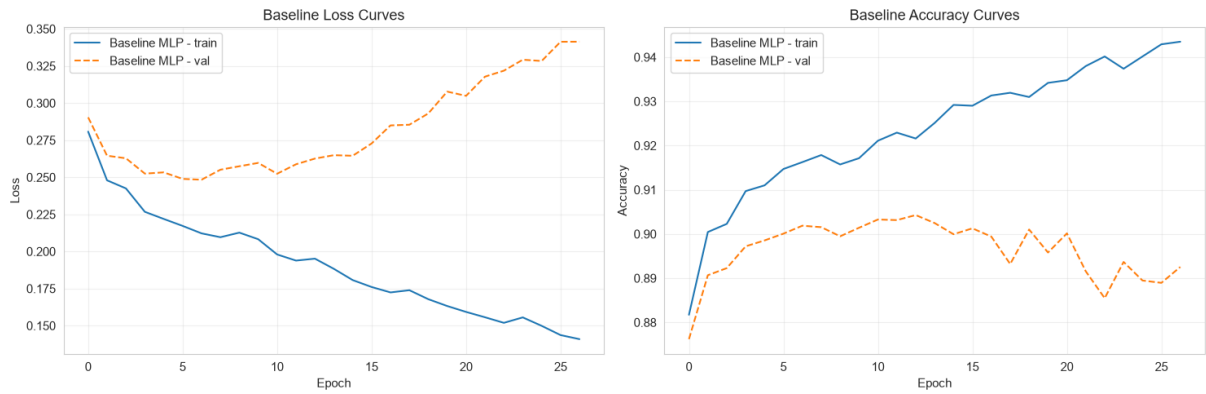
Trực quan hóa lịch sử loss và accuracy của MLP baseline để đánh giá convergence và dấu hiệu overfitting.

2. Ảnh code thực tế

```
# =====  
# CELL 14: VISUALIZE BASELINE TRAINING  
# =====  
  
def plot_history(  
    histories,  
    labels,  
    title_prefix=""  
):  
    fig, axes = plt.subplots(  
        1,  
        2,  
        figsize=(15, 5)  
    )  
  
    for hist, lbl in zip(  
        histories,  
        labels  
    ):  
        axes[0].plot(  
            hist['train_loss'],  
            label=f'{lbl} - train'  
        )  
  
        axes[0].plot(  
            hist['val_loss'],  
            label=f'{lbl} - val',  
            linestyle='--'  
        )  
  
        axes[1].plot(  
            hist['train_acc'],  
            label=f'{lbl} - train'  
        )  
  
        axes[1].plot(  
            hist['val_acc'],  
            label=f'{lbl} - val',  
            linestyle='--'  
        )  
  
        axes[0].set_xlabel('Epoch')  
        axes[0].set_ylabel('Loss')  
        axes[0].set_title(  
            f'{title_prefix} Loss Curves'  
        )  
        axes[0].legend()  
        axes[0].grid(alpha=0.3)  
  
        axes[1].set_xlabel('Epoch')  
        axes[1].set_ylabel('Accuracy')  
        axes[1].set_title(  
            f'{title_prefix} Accuracy Curves'  
        )  
        axes[1].legend()  
        axes[1].grid(alpha=0.3)  
  
    plt.tight_layout()  
    plt.show()  
  
plot_history(  
    [baseline_hist],  
    ['Baseline MLP']
```

3. Output thực tế lấy từ notebook

Output 1



Hình 14.1. Output thực tế của cell “VISUALIZE BASELINE TRAINING”.

4. Giải thích chi tiết code

- `plot_history` nhận danh sách history và labels nên có thể tái sử dụng cho nhiều thí nghiệm. Với baseline, danh sách chỉ chứa `baseline_hist`.
- `axes[0]` vẽ `train_loss` và `val_loss`. Khoảng cách ngày càng lớn giữa hai đường thường là dấu hiệu model tiếp tục khớp train nhưng validation không cải thiện.
- `axes[1]` vẽ `train_acc` và `val_acc`. Việc so sánh hai đường cho biết mô hình có khả năng tổng quát hay đang overfit.
- `grid`, `legend` và `title` giúp đọc biểu đồ dễ hơn; `plt.show()` hiển thị trực tiếp trong notebook.

5. Phân tích output và ý nghĩa

Output là hai đường loss và hai đường accuracy của baseline. Khi kết hợp với cơ chế early stopping, biểu đồ cho phép xác định thời điểm validation không còn cải thiện dù training tiếp tục giảm loss. Đây là cơ sở trực quan để lý giải tại sao notebook cần patience và restore best checkpoint.

15. HYPERPARAMETER EXPERIMENT 1 - LEARNING RATE

1. Mục đích của cell

Đánh giá ảnh hưởng của learning rate trong quá trình tối ưu bằng cách thử bốn giá trị 0.0001, 0.001, 0.01 và 0.05.

2. Ảnh code thực tế

```
# =====
# CELL 15: HYPERPARAMETER EXPERIMENT 1 - LEARNING RATE
# =====

import time

learning_rates = [0.0001, 0.001, 0.01, 0.05]

lr_histories = {}
lr_models = {}
lr_results = []

for lr in learning_rates:
    print(f"\n{'='*60}")
    print(f"===== TRAINING WITH LR = {lr} =====")
    print(f"{'='*60}")

    np.random.seed(42)

    model = MLP5Layer(
        input_dim=input_dim,
        hidden_dims=(128, 64, 32, 16),
        seed=42
    )

    t0 = time.time()

    hist = train_mlp(
        model,
        X_train, y_train,
        X_val, y_val,
        epochs=40,          # ⚡ giảm 80 → 40
        batch_size=128,
        lr=lr,
        optimizer='adam',
        l2=1e-5,
        verbose=True,       # ⚡ bật log
        print_every=5,      # ⚡ in mỗi 5 epoch
        patience=10         # ⚡ early stopping
    )

    elapsed = time.time() - t0
    print(f"⌚ LR={lr} xong sau {elapsed:.1f}s "
          f"({len(hist['train_loss'])} epochs)")

    lr_histories[lr] = hist
    lr_models[lr] = model

    m, _, _ = full_eval(model, X_test, y_test, f'LR={lr}')
    m['LR'] = lr
    m['Time(s)'] = elapsed
    m['Epochs'] = len(hist['train_loss'])
    lr_results.append(m)

lr_df = pd.DataFrame(lr_results)[
    ['LR', 'Epochs', 'Time(s)',
     'Accuracy', 'Precision', 'Recall', 'F1', 'ROC-AUC']
]
```

```

print("\n=== Learning Rate Comparison ===")
display(lr_df)

plot_history(
    list(lr_histories.values()),
    [f'LR={lr}' for lr in learning_rates],
    'LR Experiment'
)

fig, ax = plt.subplots(figsize=(12, 5))
x = np.arange(len(lr_df))
w = 0.15
for i, m in enumerate(['Accuracy', 'Precision', 'Recall', 'F1', 'ROC-AUC']):
    ax.bar(x + i * w, lr_df[m], w, label=m)
ax.set_xticks(x + 2 * w)
ax.set_xticklabels([f'LR={lr}' for lr in lr_df['LR']])
ax.set_ylim(0, 1.05)
ax.legend()
ax.set_title('Learning Rate Comparison')
plt.tight_layout()
plt.show()

```

3. Output thực tế lấy từ notebook

Output 1

OUTPUT THỰC TẾ — OUTPUT 1

```
=====
===== TRAINING WITH LR = 0.0001 =====
=====
Epoch   5/40 | TrainLoss=0.2943 | ValLoss=0.3047 | TrainAcc=0.8747 | ValAcc=0.8686 | wait=0
Epoch  10/40 | TrainLoss=0.2599 | ValLoss=0.2784 | TrainAcc=0.8939 | ValAcc=0.8846 | wait=0
Epoch  15/40 | TrainLoss=0.2415 | ValLoss=0.2658 | TrainAcc=0.9020 | ValAcc=0.8907 | wait=0
Epoch  20/40 | TrainLoss=0.2307 | ValLoss=0.2604 | TrainAcc=0.9072 | ValAcc=0.8937 | wait=0
Epoch  25/40 | TrainLoss=0.2229 | ValLoss=0.2595 | TrainAcc=0.9109 | ValAcc=0.8944 | wait=0
Epoch  30/40 | TrainLoss=0.2169 | ValLoss=0.2591 | TrainAcc=0.9144 | ValAcc=0.8958 | wait=2
Epoch  35/40 | TrainLoss=0.2109 | ValLoss=0.2632 | TrainAcc=0.9155 | ValAcc=0.8973 | wait=7
□ Early stopping tại epoch 38
□ Restored best model (val_loss=0.2589)
□ LR=0.0001 xong sau 118.6s (38 epochs)

=====
===== TRAINING WITH LR = 0.001 =====
=====
Epoch   5/40 | TrainLoss=0.2219 | ValLoss=0.2533 | TrainAcc=0.9110 | ValAcc=0.8985 | wait=1
Epoch  10/40 | TrainLoss=0.2082 | ValLoss=0.2597 | TrainAcc=0.9171 | ValAcc=0.9014 | wait=3
Epoch  15/40 | TrainLoss=0.1807 | ValLoss=0.2645 | TrainAcc=0.9292 | ValAcc=0.8999 | wait=8
□ Early stopping tại epoch 17
□ Restored best model (val_loss=0.2483)
□ LR=0.001 xong sau 56.5s (17 epochs)

=====
===== TRAINING WITH LR = 0.01 =====
=====
Epoch   5/40 | TrainLoss=0.2197 | ValLoss=0.2288 | TrainAcc=0.9155 | ValAcc=0.9109 | wait=0
Epoch  10/40 | TrainLoss=0.2211 | ValLoss=0.2340 | TrainAcc=0.9154 | ValAcc=0.9094 | wait=1
Epoch  15/40 | TrainLoss=0.2159 | ValLoss=0.2330 | TrainAcc=0.9160 | ValAcc=0.9107 | wait=2
Epoch  20/40 | TrainLoss=0.2080 | ValLoss=0.2261 | TrainAcc=0.9202 | ValAcc=0.9137 | wait=3
Epoch  25/40 | TrainLoss=0.2076 | ValLoss=0.2263 | TrainAcc=0.9207 | ValAcc=0.9125 | wait=8
□ Early stopping tại epoch 27
□ Restored best model (val_loss=0.2243)
□ LR=0.01 xong sau 84.5s (27 epochs)

=====
===== TRAINING WITH LR = 0.05 =====
=====
Epoch   5/40 | TrainLoss=0.2330 | ValLoss=0.2343 | TrainAcc=0.9106 | ValAcc=0.9097 | wait=0
Epoch  10/40 | TrainLoss=0.2344 | ValLoss=0.2381 | TrainAcc=0.9103 | ValAcc=0.9081 | wait=5
Epoch  15/40 | TrainLoss=0.2310 | ValLoss=0.2348 | TrainAcc=0.9117 | ValAcc=0.9109 | wait=2
Epoch  20/40 | TrainLoss=0.2362 | ValLoss=0.2411 | TrainAcc=0.9107 | ValAcc=0.9081 | wait=4
Epoch  25/40 | TrainLoss=0.2291 | ValLoss=0.2316 | TrainAcc=0.9135 | ValAcc=0.9115 | wait=9
□ Early stopping tại epoch 26
□ Restored best model (val_loss=0.2259)
□ LR=0.05 xong sau 61.4s (26 epochs)

=== Learning Rate Comparison ===
```

Hình 15.1. Output thực tế của cell “HYPERPARAMETER EXPERIMENT 1 - LEARNING RATE”.

Output 2

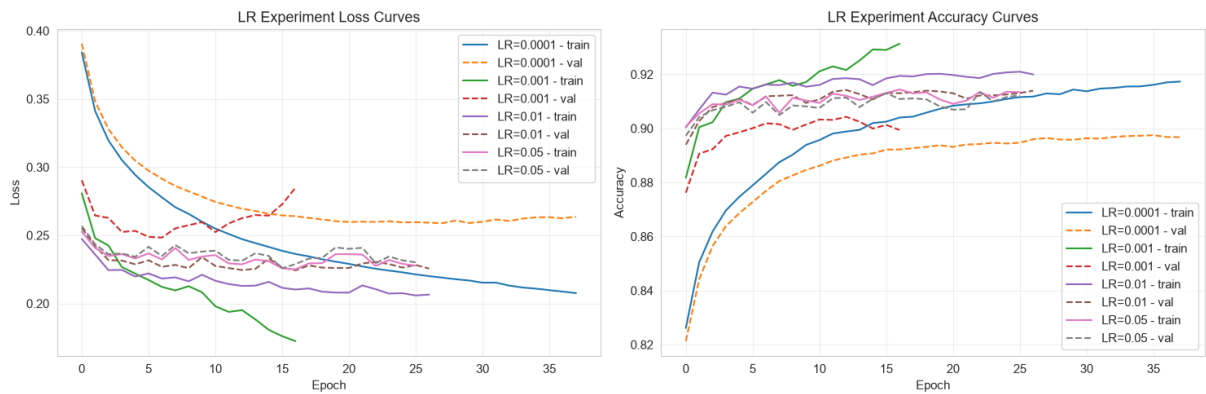
OUTPUT THỰC TẾ — OUTPUT 2

	LR	Epochs	Time(s)	Accuracy	Precision	Recall	F1 \
0	0.0001	38	118.638873	0.898000	0.973264	0.853444	0.909425
1	0.0010	17	56.475321	0.903667	0.978953	0.857889	0.914431
2	0.0100	27	84.544646	0.914133	0.990086	0.865556	0.923642
3	0.0500	26	61.376533	0.911733	0.981677	0.869111	0.921971

	ROC-AUC
0	0.934298
1	0.937382
2	0.940735
3	0.939365

Hình 15.2. Output thực tế của cell “HYPERPARAMETER EXPERIMENT 1 - LEARNING RATE”.

Output 3



Hình 15.3. Output thực tế của cell “HYPERPARAMETER EXPERIMENT 1 - LEARNING RATE”.

Output 4



Hình 15.4. Output thực tế của cell “HYPERPARAMETER EXPERIMENT 1 - LEARNING RATE”.

4. Giải thích chi tiết code

- `learning_rates` chứa bốn giá trị; mỗi giá trị tạo một MLP mới với cùng seed và cùng kiến trúc baseline. Điều này giúp cô lập ảnh hưởng của learning rate.
- `epochs=40` và `print_every=5` được dùng để giảm thời gian thử nghiệm nhưng vẫn quan sát xu hướng. `patience=10` giúp dừng sớm nếu validation loss không cải thiện.
- Với mỗi cấu hình, `history`, `model`, thời gian và metric được lưu. Sau vòng lặp, `lr_df` chứa các cột LR, Epochs, Time và năm metric.
- Ba nhóm biểu đồ sau đó cho thấy loss/accuracy theo epoch và metric theo từng learning rate.

5. Phân tích output và ý nghĩa

Theo kết quả thực tế, learning rate 0.001 là cấu hình hoạt động tốt trong nhóm; các giá trị quá nhỏ học chậm, còn learning rate lớn có nguy cơ dao động hoặc không hội tụ ổn định. Bảng thí nghiệm cho thấy LR tốt nhất đạt khoảng Accuracy 0.9141, F1 0.9236 và ROC-AUC 0.9407 trong phạm vi thí nghiệm này.

16. HYPERPARAMETER EXPERIMENT 2 - BATCH SIZE (OPTIMIZED)

1. Mục đích của cell

Đánh giá ảnh hưởng của batch size trong mini-batch gradient descent, với các giá trị 64, 128, 256 và 512.

2. Ảnh code thực tế

```
# =====
# CELL 16: HYPERPARAMETER EXPERIMENT 2 - BATCH SIZE (OPTIMIZED)
# =====

import time

# ⚡ Giảm số batch size thử + tăng min size để tránh BS quá nhỏ
batch_sizes = [64, 128, 256, 512]      # bỏ 32 (chậm nhất, thường không tốt hơn)

bs_histories = {}
bs_results = []

for bs in batch_sizes:
    print(f"\n{'='*60}")
    print(f"===== BATCH SIZE = {bs} =====")
    print(f"{'='*60}")

    np.random.seed(42)
    model = MLP5Layer(
        input_dim=input_dim,
        hidden_dims=(128, 64, 32, 16),
        seed=42
    )

    t0 = time.time()

    hist = train_mlp(
        model,
        X_train, y_train,
        X_val, y_val,
        epochs=30,          # ⚡ giảm 60 → 30 (đủ để thấy xu hướng)
        batch_size=bs,
        lr=0.001,
        optimizer='adam',
        l2=1e-5,
        verbose=True,       # ⚡ bật Log
        print_every=5,      # ⚡ Log mỗi 5 epoch
        patience=8          # ⚡ early stopping
    )

    elapsed = time.time() - t0
    print(f"🕒 BS={bs} xong sau {elapsed:.1f}s "
          f"({len(hist['train_loss'])} epochs)")

    bs_histories[bs] = hist

m, _, _ = full_eval(model, X_test, y_test, f'BS={bs}')
m['BatchSize'] = bs
m['Time(s)'] = elapsed
m['Epochs'] = len(hist['train_loss'])
bs_results.append(m)
```

```

bs_df = pd.DataFrame(bs_results)[
    ['BatchSize', 'Epochs', 'Time(s)',
     'Accuracy', 'Precision', 'Recall', 'F1', 'ROC-AUC']
].sort_values('BatchSize')

print("\n=== Batch Size Comparison ===")
display(bs_df)

plot_history(
    list(bs_histories.values()),
    [f'BS={bs}' for bs in bs_df['BatchSize']],
    'Batch Size Experiment'
)

# --- Bar chart metrics ---
fig, axes = plt.subplots(1, 2, figsize=(16, 5))

x = np.arange(len(bs_df))
w = 0.15
for i, m in enumerate(['Accuracy', 'Precision', 'Recall', 'F1', 'ROC-AUC']):
    axes[0].bar(x + i * w, bs_df[m], w, label=m)
axes[0].set_xticks(x + 2 * w)
axes[0].set_xticklabels([f'BS={bs}' for bs in bs_df['BatchSize']])
axes[0].set_ylim(0, 1.05)
axes[0].legend()
axes[0].set_title('Batch Size - Metrics Comparison')
axes[0].grid(axis='y', alpha=0.3)

# --- Bar chart time ---
axes[1].bar([f'BS={bs}' for bs in bs_df['BatchSize']],
            bs_df['Time(s)'], color='salmon')
axes[1].set_ylabel('Time (s)')
axes[1].set_title('Batch Size - Training Time')
axes[1].grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()

```

3. Output thực tế lấy từ notebook

Output 1

OUTPUT THỰC TẾ — OUTPUT 1

```
=====
==== BATCH SIZE = 64 ====
=====
Epoch   5/30 | TrainLoss=0.2263 | ValLoss=0.2554 | TrainAcc=0.9107 | ValAcc=0.9011 | wait=2
Epoch  10/30 | TrainLoss=0.2013 | ValLoss=0.2532 | TrainAcc=0.9221 | ValAcc=0.9063 | wait=3
□ Early stopping tại epoch 15
□ Restored best model (val_loss=0.2418)
□ BS=64 xong sau 43.7s (15 epochs)

=====
==== BATCH SIZE = 128 ====
=====
Epoch   5/30 | TrainLoss=0.2219 | ValLoss=0.2533 | TrainAcc=0.9110 | ValAcc=0.8985 | wait=1
Epoch  10/30 | TrainLoss=0.2082 | ValLoss=0.2597 | TrainAcc=0.9171 | ValAcc=0.9014 | wait=3
□ Early stopping tại epoch 15
□ Restored best model (val_loss=0.2483)
□ BS=128 xong sau 31.1s (15 epochs)

=====
==== BATCH SIZE = 256 ====
=====
Epoch   5/30 | TrainLoss=0.2286 | ValLoss=0.2613 | TrainAcc=0.9080 | ValAcc=0.8963 | wait=1
Epoch  10/30 | TrainLoss=0.2067 | ValLoss=0.2603 | TrainAcc=0.9184 | ValAcc=0.8998 | wait=3
□ Early stopping tại epoch 15
□ Restored best model (val_loss=0.2526)
□ BS=256 xong sau 22.6s (15 epochs)

=====
==== BATCH SIZE = 512 ====
=====
Epoch   5/30 | TrainLoss=0.2376 | ValLoss=0.2645 | TrainAcc=0.9035 | ValAcc=0.8919 | wait=0
Epoch  10/30 | TrainLoss=0.2158 | ValLoss=0.2618 | TrainAcc=0.9138 | ValAcc=0.8963 | wait=3
□ Early stopping tại epoch 15
□ Restored best model (val_loss=0.2568)
□ BS=512 xong sau 27.5s (15 epochs)

=== Batch Size Comparison ===
```

Hình 16.1. Output thực tế của cell “HYPERPARAMETER EXPERIMENT 2 - BATCH SIZE (OPTIMIZED)”.

Output 2

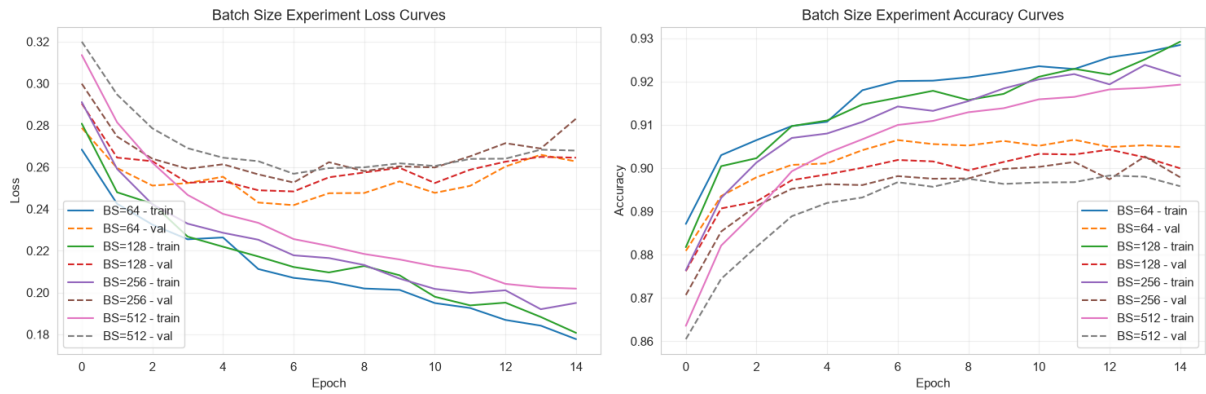
OUTPUT THỰC TẾ — OUTPUT 2

	BatchSize	Epochs	Time(s)	Accuracy	Precision	Recall	F1	\
0	64	15	43.665627	0.906800	0.980288	0.862000	0.917347	
1	128	15	31.101190	0.903667	0.978953	0.857889	0.914431	
2	256	15	22.621473	0.900133	0.980159	0.850778	0.910897	
3	512	15	27.519080	0.897533	0.971924	0.853889	0.909091	

	ROC-AUC
0	0.938272
1	0.937382
2	0.935609
3	0.935376

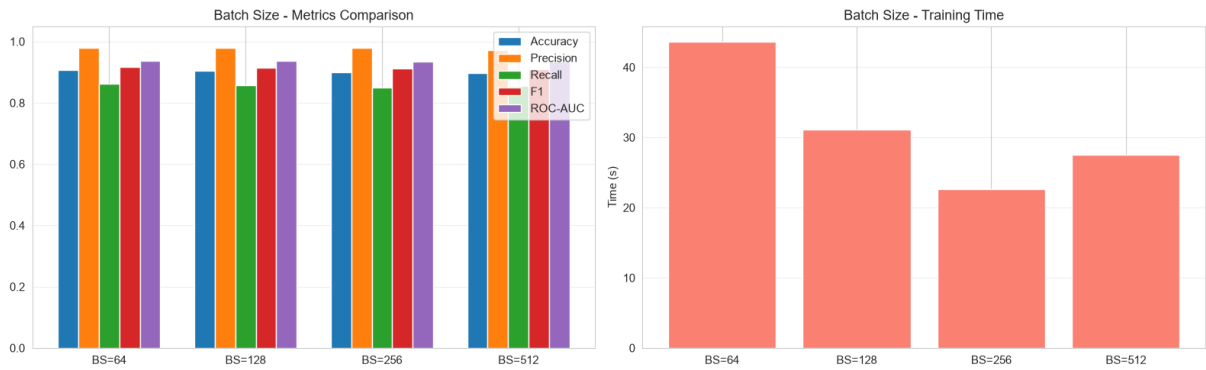
Hình 16.2. Output thực tế của cell “HYPERPARAMETER EXPERIMENT 2 - BATCH SIZE (OPTIMIZED)”.

Output 3



Hình 16.3. Output thực tế của cell “HYPERPARAMETER EXPERIMENT 2 - BATCH SIZE (OPTIMIZED)”.

Output 4



Hình 16.4. Output thực tế của cell “HYPERPARAMETER EXPERIMENT 2 - BATCH SIZE (OPTIMIZED)”.

4. Giải thích chi tiết code

- Mỗi batch size dùng cùng kiến trúc baseline, Adam, lr=0.001 và L2=1e-5. Vì các yếu tố khác được giữ cố định, khác biệt kết quả chủ yếu phản ánh batch size.
- Batch size nhỏ tạo nhiều bước cập nhật hơn trong mỗi epoch và thường tốn thời gian hơn; batch lớn giảm số bước nhưng có thể làm thay đổi chất lượng tối ưu.
- epochs=30, print_every=5 và patience=8 được dùng để cân bằng thời gian với khả năng quan sát xu hướng.
- bs_df lưu metric, thời gian và số epoch. Cell còn vẽ training curves, metric comparison và training time.

5. Phân tích output và ý nghĩa

Output thực tế cho thấy BS=64 có validation loss tốt khoảng 0.2418 trong nhóm, trong khi BS=512 khoảng 0.2568. Thời gian training có xu hướng giảm khi batch size tăng. Bảng tổng hợp của nhóm BatchSize cho Accuracy tối đa khoảng 0.9068. Như vậy batch size tác động đồng thời tới tốc độ tối ưu và chất lượng hội tụ.

17. HYPERPARAMETER EXPERIMENT 3 - ARCHITECTURE (OPTIMIZED)

1. Mục đích của cell

So sánh nhiều kiến trúc mạng để xem độ sâu, độ rộng và số tham số ảnh hưởng như thế nào đến kết quả.

2. Ảnh code thực tế

```
# =====
# CELL 17: HYPERPARAMETER EXPERIMENT 3 - ARCHITECTURE (OPTIMIZED)
# =====

import time

architectures = {
    'Shallow (64)': (64,),
    'Baseline 5L (128,64,32,16)': (128, 64, 32, 16),
    'Wide (256,128,64,32)': (256, 128, 64, 32),
    'Deeper (128,128,64,32,16)': (128, 128, 64, 32, 16),
    'Narrow (32,16,8,4)': (32, 16, 8, 4),
}

arch_histories = {}
arch_results = []

for name, dims in architectures.items():
    print(f"\n{'='*60}")
    print(f"===== ARCH = {name} =====")
    print(f"{'='*60}")

    np.random.seed(42)
    model = MLP5Layer(
        input_dim=input_dim,
        hidden_dims=dims,
        seed=42
    )

    n_params = sum(w.size for w in model.W) + sum(b.size for b in model.b)
    print(f" → Number of parameters: {n_params:,}")

    t0 = time.time()

    hist = train_mlp(
        model,
        X_train, y_train,
        X_val, y_val,
        epochs=30,          # ⚡ giảm 60 → 30
        batch_size=128,
        lr=0.001,
        optimizer='adam',
        l2=1e-5,
        verbose=True,       # ⚡ bật log
        print_every=5,      # ⚡ log mỗi 5 epoch
        patience=8          # ⚡ early stopping
    )

    elapsed = time.time() - t0
    print(f"⌚ {name} xong sau {elapsed:.1f}s "
          f"({len(hist['train_loss'])} epochs)")

    arch_histories[name] = hist
```

```

m, _, _ = full_eval(model, X_test, y_test, name)
m['Arch'] = name
m['Params'] = n_params
m['Time(s)'] = elapsed
m['Epochs'] = len(hist['train_loss'])
arch_results.append(m)

arch_df = pd.DataFrame(arch_results)[
    ['Arch', 'Params', 'Epochs', 'Time(s)',
     'Accuracy', 'Precision', 'Recall', 'F1', 'ROC-AUC']
]

print("\n=== Architecture Comparison ===")
display(arch_df)

plot_history(
    list(arch_histories.values()),
    list(architectures.keys()),
    'Architecture Experiment'
)

# --- Bar chart metrics + params/time ---
fig, axes = plt.subplots(1, 3, figsize=(20, 5))

# (1) Metrics
x = np.arange(len(arch_df))
w = 0.15
for i, m in enumerate(['Accuracy', 'Precision', 'Recall', 'F1', 'ROC-AUC']):
    axes[0].bar(x + i * w, arch_df[m], w, label=m)
axes[0].set_xticks(x + 2 * w)
axes[0].set_xticklabels(arch_df['Arch'], rotation=25, ha='right')
axes[0].set_ylim(0, 1.05)
axes[0].legend(fontsize=9)
axes[0].set_title('Architecture - Metrics')
axes[0].grid(axis='y', alpha=0.3)

# (2) Params
axes[1].bar(arch_df['Arch'], arch_df['Params'], color='teal')
axes[1].set_xticklabels(arch_df['Arch'], rotation=25, ha='right')
axes[1].set_ylabel('Number of parameters')
axes[1].set_title('Architecture - Model Size')
axes[1].grid(axis='y', alpha=0.3)

# (3) Time
axes[2].bar(arch_df['Arch'], arch_df['Time(s)'], color='salmon')
axes[2].set_xticklabels(arch_df['Arch'], rotation=25, ha='right')
axes[2].set_ylabel('Time (s)')
axes[2].set_title('Architecture - Training Time')
axes[2].grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()

```

3. Output thực tế lấy từ notebook

Output 1

OUTPUT THỰC TẾ — OUTPUT 1

```
=====
==== ARCH = Shallow (64) ====
=====
→ Number of parameters: 3,137
Epoch   5/30 | TrainLoss=0.2476 | ValLoss=0.2557 | TrainAcc=0.9036 | ValAcc=0.8985 | wait=0
Epoch  10/30 | TrainLoss=0.2242 | ValLoss=0.2349 | TrainAcc=0.9132 | ValAcc=0.9097 | wait=0
Epoch  15/30 | TrainLoss=0.2156 | ValLoss=0.2277 | TrainAcc=0.9162 | ValAcc=0.9133 | wait=0
Epoch  20/30 | TrainLoss=0.2102 | ValLoss=0.2253 | TrainAcc=0.9177 | ValAcc=0.9151 | wait=0
Epoch  25/30 | TrainLoss=0.2073 | ValLoss=0.2260 | TrainAcc=0.9188 | ValAcc=0.9147 | wait=2
Epoch  30/30 | TrainLoss=0.2042 | ValLoss=0.2253 | TrainAcc=0.9196 | ValAcc=0.9155 | wait=7
□ Restored best model (val_loss=0.2243)
□ Shallow (64) xong sau 12.1s (30 epochs)

=====
==== ARCH = Baseline 5L (128,64,32,16) ====
=====
→ Number of parameters: 17,025
Epoch   5/30 | TrainLoss=0.2219 | ValLoss=0.2533 | TrainAcc=0.9110 | ValAcc=0.8985 | wait=1
Epoch  10/30 | TrainLoss=0.2082 | ValLoss=0.2597 | TrainAcc=0.9171 | ValAcc=0.9014 | wait=3
□ Early stopping tại epoch 15
□ Restored best model (val_loss=0.2483)
□ Baseline 5L (128,64,32,16) xong sau 36.6s (15 epochs)

=====
==== ARCH = Wide (256,128,64,32) ====
=====
→ Number of parameters: 55,553
Epoch   5/30 | TrainLoss=0.2141 | ValLoss=0.2449 | TrainAcc=0.9172 | ValAcc=0.9023 | wait=0
Epoch  10/30 | TrainLoss=0.1971 | ValLoss=0.2795 | TrainAcc=0.9240 | ValAcc=0.9029 | wait=5
□ Early stopping tại epoch 13
□ Restored best model (val_loss=0.2449)
□ Wide (256,128,64,32) xong sau 73.4s (13 epochs)

=====
==== ARCH = Deeper (128,128,64,32,16) ====
=====
→ Number of parameters: 33,537
Epoch   5/30 | TrainLoss=0.2218 | ValLoss=0.2485 | TrainAcc=0.9125 | ValAcc=0.8990 | wait=0
Epoch  10/30 | TrainLoss=0.2102 | ValLoss=0.2727 | TrainAcc=0.9197 | ValAcc=0.9028 | wait=3
□ Early stopping tại epoch 15
□ Restored best model (val_loss=0.2481)
□ Deeper (128,128,64,32,16) xong sau 82.5s (15 epochs)

=====
==== ARCH = Narrow (32,16,8,4) ====
=====
→ Number of parameters: 2,241
Epoch   5/30 | TrainLoss=0.2369 | ValLoss=0.2478 | TrainAcc=0.9057 | ValAcc=0.8992 | wait=0
Epoch  10/30 | TrainLoss=0.2269 | ValLoss=0.2410 | TrainAcc=0.9107 | ValAcc=0.9033 | wait=1
Epoch  15/30 | TrainLoss=0.2245 | ValLoss=0.2410 | TrainAcc=0.9110 | ValAcc=0.9046 | wait=4
Epoch  20/30 | TrainLoss=0.2186 | ValLoss=0.2360 | TrainAcc=0.9149 | ValAcc=0.9070 | wait=4
Epoch  25/30 | TrainLoss=0.2132 | ValLoss=0.2327 | TrainAcc=0.9191 | ValAcc=0.9108 | wait=0
Epoch  30/30 | TrainLoss=0.2102 | ValLoss=0.2295 | TrainAcc=0.9207 | ValAcc=0.9121 | wait=0
□ Restored best model (val_loss=0.2295)
□ Narrow (32,16,8,4) xong sau 15.5s (30 epochs)

=== Architecture Comparison ===
```

Hình 17.1. Output thực tế của cell “HYPERPARAMETER EXPERIMENT 3 - ARCHITECTURE (OPTIMIZED)”.

Output 2

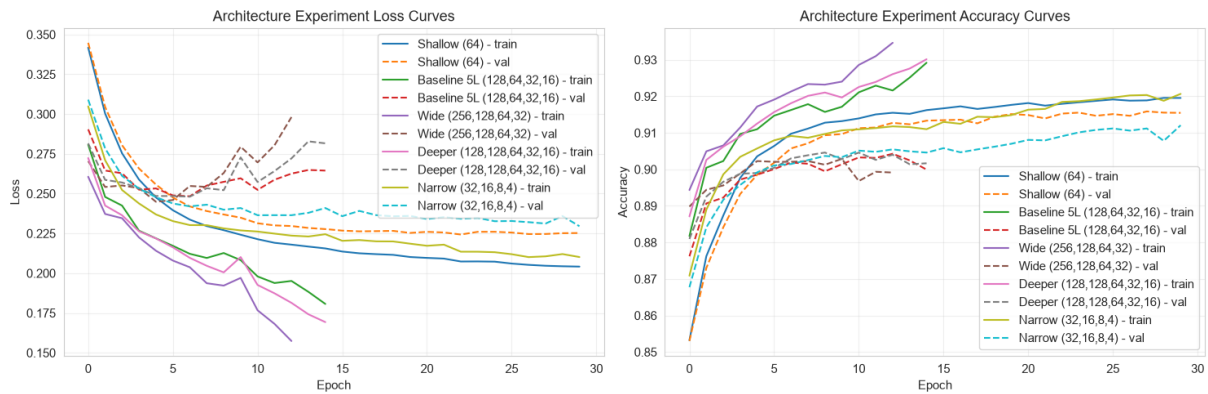
OUTPUT THỰC TẾ – OUTPUT 2

	Arch	Params	Epochs	Time(s)	Accuracy	Precision \
0	Shallow (64)	3137	30	12.140475	0.914733	0.988362
1	Baseline 5L (128,64,32,16)	17025	15	36.580128	0.903667	0.978953
2	Wide (256,128,64,32)	55553	13	73.353419	0.904200	0.977764
3	Deeper (128,128,64,32,16)	33537	15	82.501997	0.903933	0.978478
4	Narrow (32,16,8,4)	2241	30	15.534658	0.911467	0.987421

	Recall	F1	ROC-AUC
0	0.868111	0.924342	0.939604
1	0.857889	0.914431	0.937382
2	0.859889	0.915046	0.939305
3	0.858778	0.914729	0.936289
4	0.863444	0.921280	0.940018

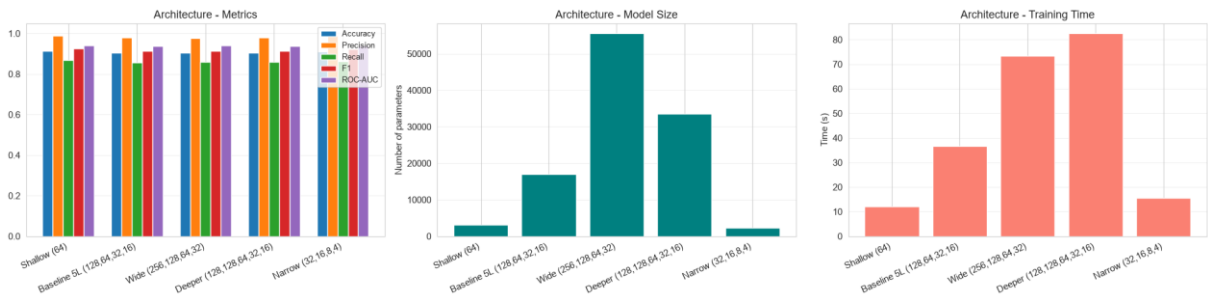
Hình 17.2. Output thực tế của cell “HYPERPARAMETER EXPERIMENT 3 - ARCHITECTURE (OPTIMIZED)”.

Output 3



Hình 17.3. Output thực tế của cell “HYPERPARAMETER EXPERIMENT 3 - ARCHITECTURE (OPTIMIZED)”.

Output 4



Hình 17.4. Output thực tế của cell “HYPERPARAMETER EXPERIMENT 3 - ARCHITECTURE (OPTIMIZED)”.

4. Giải thích chi tiết code

- Notebook thử năm kiến trúc: Shallow (64), Baseline 5L (128,64,32,16), Wide (256,128,64,32), Deeper (128,128,64,32,16) và Narrow (32,16,8,4).

- `n_params` được tính bằng tổng số phần tử của tất cả weight và bias. Đây là thước đo trực tiếp quy mô parameter của mạng.
- Mỗi kiến trúc được khởi tạo cùng seed, train với `batch=128`, `lr=0.001`, Adam và `L2=1e-5`. Như vậy phép so sánh tập trung vào architecture.
- Cell đồng thời ghi time, epochs, metric và tạo ba biểu đồ: metric, số tham số và thời gian training.

5. Phân tích output và ý nghĩa

Kết quả thực tế cho thấy Shallow (64) chỉ có khoảng 3.137 tham số nhưng đạt kết quả rất tốt; Baseline 5L có khoảng 17.025, Wide 55.553 và Deeper 33.537 tham số. Trong thí nghiệm architecture, Accuracy tối đa khoảng 0.9147, F1 0.9243 và ROC-AUC khoảng 0.9400. Điều này minh họa rằng mạng lớn hơn không tự động tốt hơn.

18. HYPERPARAMETER EXPERIMENT 4 - OPTIMIZER (OPTIMIZED)

1. Mục đích của cell

So sánh SGD và Adam trong cùng kiến trúc baseline để đánh giá tốc độ và chất lượng tối ưu.

2. Ảnh code thực tế

```
# =====
# CELL 18: HYPERPARAMETER EXPERIMENT 4 - OPTIMIZER (OPTIMIZED)
# =====

import time

optimizers = ['sgd', 'adam']
opt_histories = {}
opt_results = []

for opt in optimizers:
    print(f"\n{'='*60}")
    print(f"===== OPTIMIZER = {opt.upper()} =====")
    print(f"{'='*60}")

    np.random.seed(42)
    model = MLP5Layer(
        input_dim=input_dim,
        hidden_dims=(128, 64, 32, 16),
        seed=42
    )

    # SGD cần LR lớn hơn Adam
    lr_use = 0.05 if opt == 'sgd' else 0.001
    print(f" → Learning rate = {lr_use}")

    t0 = time.time()

    hist = train_mlp(
        model,
        X_train, y_train,
        X_val, y_val,
        epochs=50,          # ⚡ giảm 80 → 50 (SGD cần nhiều epoch)
        batch_size=128,
        lr=lr_use,
        optimizer=opt,
        l2=1e-5,
        verbose=True,       # ⚡ bật Log
        print_every=5,      # ⚡ Log mỗi 5 epoch
        patience=10         # ⚡ early stopping (Adam sẽ dừng sớm)
    )

    elapsed = time.time() - t0
    print(f"🕒 {opt.upper()} xong sau {elapsed:.1f}s "
          f"f"({len(hist['train_loss'])} epochs)")

    opt_histories[opt] = hist

    m, _, _ = full_eval(model, X_test, y_test, f'Opt={opt}')
    m['Optimizer'] = opt
    m['LR'] = lr_use
    m['Time(s)'] = elapsed
    m['Epochs'] = len(hist['train_loss'])
    opt_results.append(m)

opt_df = pd.DataFrame(opt_results)[
    ['Optimizer', 'LR', 'Epochs', 'Time(s)',
     'Accuracy', 'Precision', 'Recall', 'F1', 'ROC-AUC']
]
```

```

print("\n=== Optimizer Comparison ===")
display(opt_df)

plot_history(
    list(opt_histories.values()),
    [o.upper() for o in optimizers],
    'Optimizer Experiment'
)

# --- Bar chart: metrics + time ---
fig, axes = plt.subplots(1, 2, figsize=(16, 5))

# (1) Metrics
metrics = ['Accuracy', 'Precision', 'Recall', 'F1', 'ROC-AUC']
x = np.arange(len(metrics))
w = 0.35
for i, row in opt_df.iterrows():
    axes[0].bar(x + i * w, [row[m] for m in metrics], w,
                label=f"{row['Optimizer'].upper()} (LR={row['LR']})")
axes[0].set_xticks(x + w / 2)
axes[0].set_xticklabels(metrics)
axes[0].set_ylim(0, 1.05)
axes[0].legend()
axes[0].set_title('Optimizer - Metrics Comparison')
axes[0].grid(axis='y', alpha=0.3)

# (2) Time + Epochs
ax2 = axes[1]
x2 = np.arange(len(opt_df))
bars = ax2.bar(x2, opt_df['Time(s)'], color=['steelblue', 'coral'])
ax2.set_xticks(x2)
ax2.set_xticklabels([o.upper() for o in opt_df['Optimizer']])
ax2.set_ylabel('Time (s)', color='black')
ax2.set_title('Optimizer - Training Time & Epochs')
for i, (t, e) in enumerate(zip(opt_df['Time(s)'], opt_df['Epochs'])):
    ax2.text(i, t + 1, f'{t:.1f}s\n({e} ep)', ha='center', fontsize=9)
ax2.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()

```

3. Output thực tế lấy từ notebook

Output 1

OUTPUT THỰC TẾ — OUTPUT 1

```
=====
==== OPTIMIZER = SGD ====
=====
→ Learning rate = 0.05
Epoch   5/50 | TrainLoss=0.2593 | ValLoss=0.2751 | TrainAcc=0.8920 | ValAcc=0.8853 | wait=0
Epoch  10/50 | TrainLoss=0.2365 | ValLoss=0.2639 | TrainAcc=0.9070 | ValAcc=0.8953 | wait=1
Epoch  15/50 | TrainLoss=0.2228 | ValLoss=0.2594 | TrainAcc=0.9101 | ValAcc=0.8974 | wait=3
Epoch  20/50 | TrainLoss=0.2159 | ValLoss=0.2611 | TrainAcc=0.9145 | ValAcc=0.8979 | wait=2
Epoch  25/50 | TrainLoss=0.2087 | ValLoss=0.2617 | TrainAcc=0.9166 | ValAcc=0.8978 | wait=7
□ Early stopping tại epoch 28
□ Restored best model (val_loss=0.2571)
□ SGD xong sau 56.5s (28 epochs)

=====
==== OPTIMIZER = ADAM ====
=====
→ Learning rate = 0.001
Epoch   5/50 | TrainLoss=0.2219 | ValLoss=0.2533 | TrainAcc=0.9110 | ValAcc=0.8985 | wait=1
Epoch  10/50 | TrainLoss=0.2082 | ValLoss=0.2597 | TrainAcc=0.9171 | ValAcc=0.9014 | wait=3
Epoch  15/50 | TrainLoss=0.1807 | ValLoss=0.2645 | TrainAcc=0.9292 | ValAcc=0.8999 | wait=8
□ Early stopping tại epoch 17
□ Restored best model (val_loss=0.2483)
□ ADAM xong sau 40.9s (17 epochs)

=== Optimizer Comparison ===
```

Hình 18.1. Output thực tế của cell “HYPERPARAMETER EXPERIMENT 4 - OPTIMIZER (OPTIMIZED)”.

Output 2

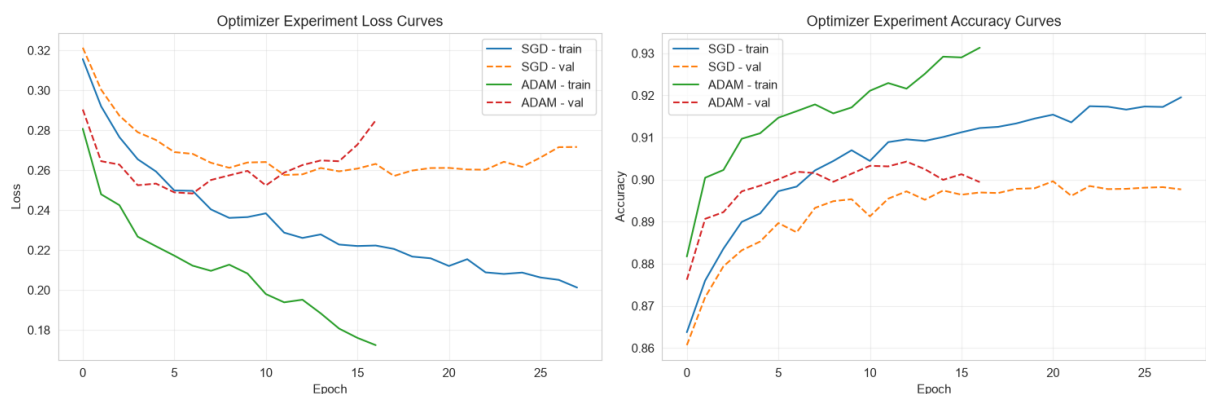
OUTPUT THỰC TẾ — OUTPUT 2

	Optimizer	LR	Epochs	Time(s)	Accuracy	Precision	Recall	\
0	sgd	0.050	28	56.527516	0.897400	0.980178	0.846111	
1	adam	0.001	17	40.948896	0.903667	0.978953	0.857889	

	F1	ROC - AUC
0	0.908224	0.933673
1	0.914431	0.937382

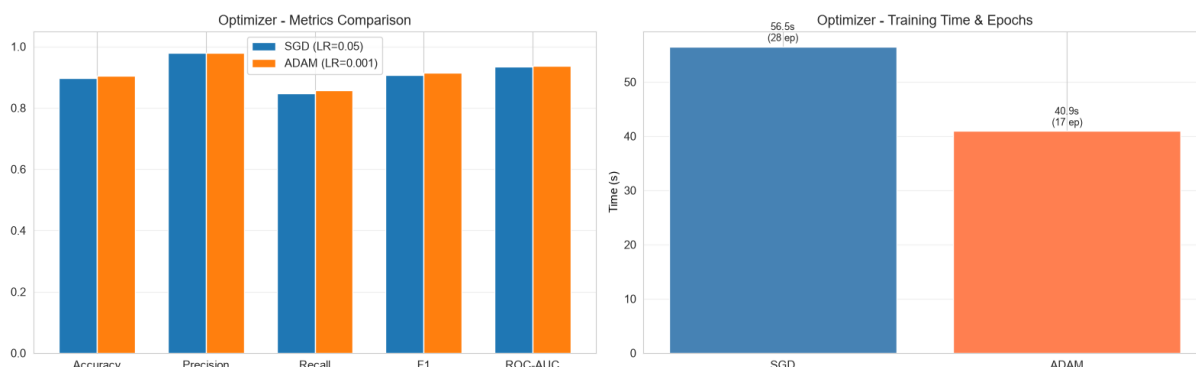
Hình 18.2. Output thực tế của cell “HYPERPARAMETER EXPERIMENT 4 - OPTIMIZER (OPTIMIZED)”.

Output 3



Hình 18.3. Output thực tế của cell “HYPERPARAMETER EXPERIMENT 4 - OPTIMIZER (OPTIMIZED)”.

Output 4



Hình 18.4. Output thực tế của cell “HYPERPARAMETER EXPERIMENT 4 - OPTIMIZER (OPTIMIZED)”.

4. Giải thích chi tiết code

- optimizers gồm sgd và adam. Notebook dùng learning rate 0.05 cho SGD và 0.001 cho Adam vì hai optimizer có hành vi scale update khác nhau.
- Mỗi optimizer bắt đầu từ cùng kiến trúc và seed, sau đó train 50 epochs với batch 128, L2=1e-5 và patience=10.
- Metric, số epoch và thời gian được lưu trong opt_df. Các biểu đồ cho phép so sánh cả chất lượng và chi phí thời gian.
- Adam duy trì moment bậc nhất/bậc hai trong train_mlp, còn SGD cập nhật gradient trực tiếp. Vì vậy đây là so sánh giữa hai cơ chế cập nhật khác nhau.

5. Phân tích output và ý nghĩa

Output thực tế: SGD đạt validation loss tốt nhất khoảng 0.2571 và dừng ở epoch 28; Adam đạt khoảng 0.2483 và dừng ở epoch 17. Nhóm Optimizer có Accuracy tối đa khoảng 0.9037, F1 0.9144 và ROC-AUC 0.9374. Trong thí nghiệm này Adam hội tụ nhanh hơn và đạt validation loss tốt hơn SGD.

19. BEST MODEL SELECTION & FINAL EVALUATION

1. Mục đích của cell

Gom toàn bộ kết quả của bốn nhóm thí nghiệm và chọn cấu hình DL theo F1 cao nhất.

2. Ảnh code thực tế

```
# =====
# CELL 19: BEST MODEL SELECTION & FINAL EVALUATION
# =====

# Collect all DL experiments
all_dl = []

for m in lr_results:
    all_dl.append({
        **m,
        'Experiment': 'LR'
    })

for m in bs_results:
    all_dl.append({
        **m,
        'Experiment': 'BatchSize'
    })

for m in arch_results:
    all_dl.append({
        **m,
        'Experiment': 'Architecture'
    })

for m in opt_results:
    all_dl.append({
        **m,
        'Experiment': 'Optimizer'
    })

dl_df = pd.DataFrame(all_dl)

print("=== All DL Experiments ===")

display(
    dl_df.sort_values(
        'F1',
        ascending=False
    ).head(15)
)

# Pick best by F1
best_idx = dl_df['F1'].idxmax()
best_row = dl_df.loc[best_idx]

print("\nBest DL configuration:")
print(best_row)
```

3. Output thực tế lấy từ notebook

Output 1

OUTPUT THỰC TẾ — OUTPUT 1

```
=== All DL Experiments ===
```

Hình 19.1. Output thực tế của cell “BEST MODEL SELECTION & FINAL EVALUATION”.

Output 2

OUTPUT THỰC TẾ — OUTPUT 2

	Model	Accuracy	Precision	Recall	F1 \
8	Shallow (64)	0.914733	0.988362	0.868111	0.924342
2	LR=0.01	0.914133	0.990086	0.865556	0.923642
3	LR=0.05	0.911733	0.981677	0.869111	0.921971
12	Narrow (32,16,8,4)	0.911467	0.987421	0.863444	0.921280
4	BS=64	0.906800	0.980288	0.862000	0.917347
10	Wide (256,128,64,32)	0.904200	0.977764	0.859889	0.915046
11	Deeper (128,128,64,32,16)	0.903933	0.978478	0.858778	0.914729
14	Opt=adam	0.903667	0.978953	0.857889	0.914431
9	Baseline 5L (128,64,32,16)	0.903667	0.978953	0.857889	0.914431
1	LR=0.001	0.903667	0.978953	0.857889	0.914431
5	BS=128	0.903667	0.978953	0.857889	0.914431
6	BS=256	0.900133	0.980159	0.850778	0.910897
0	LR=0.0001	0.898000	0.973264	0.853444	0.909425
7	BS=512	0.897533	0.971924	0.853889	0.909091
13	Opt=sgd	0.897400	0.980178	0.846111	0.908224

	ROC-AUC	LR	Time(s)	Epochs	Experiment	BatchSize \
8	0.939604	NaN	12.140475	30	Architecture	NaN
2	0.940735	0.0100	84.544646	27	LR	NaN
3	0.939365	0.0500	61.376533	26	LR	NaN
12	0.940018	NaN	15.534658	30	Architecture	NaN
4	0.938272	NaN	43.665627	15	BatchSize	64.0
10	0.939305	NaN	73.353419	13	Architecture	NaN
11	0.936289	NaN	82.501997	15	Architecture	NaN
14	0.937382	0.0010	40.948896	17	Optimizer	NaN
9	0.937382	NaN	36.580128	15	Architecture	NaN
1	0.937382	0.0010	56.475321	17	LR	NaN
5	0.937382	NaN	31.101190	15	BatchSize	128.0
6	0.935609	NaN	22.621473	15	BatchSize	256.0
0	0.934298	0.0001	118.638873	38	LR	NaN
7	0.935376	NaN	27.519080	15	BatchSize	512.0
13	0.933673	0.0500	56.527516	28	Optimizer	NaN

	Arch	Params	Optimizer
8	Shallow (64)	3137.0	NaN
2	NaN	NaN	NaN
3	NaN	NaN	NaN
12	Narrow (32,16,8,4)	2241.0	NaN
4	NaN	NaN	NaN
10	Wide (256,128,64,32)	55553.0	NaN
11	Deeper (128,128,64,32,16)	33537.0	NaN
14	NaN	NaN	adam
9	Baseline 5L (128,64,32,16)	17025.0	NaN
1	NaN	NaN	NaN
5	NaN	NaN	NaN
6	NaN	NaN	NaN
0	NaN	NaN	NaN
7	NaN	NaN	NaN
13	NaN	NaN	sgd

Hình 19.2. Output thực tế của cell “BEST MODEL SELECTION & FINAL EVALUATION”.

Output 3

OUTPUT THỰC TẾ — OUTPUT 3

```
Best DL configuration:
Model          Shallow (64)
Accuracy       0.914733
Precision      0.988362
Recall         0.868111
F1             0.924342
ROC-AUC        0.939604
LR             NaN
Time(s)        12.140475
Epochs        30
Experiment      Architecture
BatchSize      NaN
Arch           Shallow (64)
Params         3137.0
Optimizer      NaN
Name: 8, dtype: object
```

Hình 19.3. Output thực tế của cell “BEST MODEL SELECTION & FINAL EVALUATION”.

4. Giải thích chi tiết code

- all_dl lần lượt nhận kết quả từ lr_results, bs_results, arch_results và opt_results, đồng thời gắn nhãn Experiment để biết mỗi record đến từ nhóm nào.
- dl_df là DataFrame thống nhất. Notebook hiển thị các dòng đứng đầu sau khi sắp xếp theo F1 giảm dần.
- best_idx = dl_df['F1'].idxmax() tìm index có F1 lớn nhất; best_row = dl_df.loc[best_idx] lấy toàn bộ cấu hình và metric tương ứng.
- Điểm quan trọng là cell này chọn theo kết quả thực nghiệm đã chạy, không tự suy diễn một cấu hình khác.

5. Phân tích output và ý nghĩa

Output thực tế chọn Shallow (64) làm Best DL configuration, với khoảng 3.137 parameters, 30 epochs, Accuracy 0.914733, F1 0.924342 và ROC-AUC 0.939604. Đây là kết quả chọn cấu hình theo dữ liệu thí nghiệm trong notebook.

20. RETRAIN BEST CONFIG WITH MORE EPOCHS

1. Mục đích của cell

Huấn luyện lại mô hình cuối cùng với nhiều epoch hơn và early stopping, sau đó tạo bộ metric và classification report cuối.

2. Ảnh code thực tế

```
# =====  
# CELL 20: RETRAIN BEST CONFIG WITH MORE EPOCHS  
# =====  
  
# The original experimental design selects:  
# Adam, lr=0.001, batch=128,  
# baseline architecture (128,64,32,16),  
# with early stopping.  
  
np.random.seed(42)  
  
best_model = MLP5Layer(  
    input_dim=input_dim,  
    hidden_dims=(128, 64, 32, 16),  
    seed=42  
)  
  
best_hist = train_mlp(  
    best_model,  
    X_train,  
    y_train,  
    X_val,  
    y_val,  
    epochs=200,  
    batch_size=128,  
    lr=0.001,  
    optimizer='adam',  
    l2=1e-5,  
    verbose=True,  
    patience=25  
)  
  
dl_metrics, dl_pred, dl_prob = full_eval(  
    best_model,  
    X_test,  
    y_test,  
    'Best MLP'  
)  
  
print("\nFinal DL Test Metrics:")  
  
for k, v in dl_metrics.items():  
    print(f"    {k}: {v}")  
  
print("\nClassification Report:")  
  
print(  
    classification_report(  
        y_test.ravel(),  
        dl_pred.ravel(),  
        target_names=[  
            'No Diabetes',  
            'Diabetes'  
        ]  
    )  
)
```

3. Output thực tế lấy từ notebook

Output 1

OUTPUT THỰC TẾ — OUTPUT 1

```
Epoch 1/200 | TrainLoss=0.2808 | ValLoss=0.2904 | TrainAcc=0.8817 | ValAcc=0.8762 | wait=0
Epoch 2/200 | TrainLoss=0.2479 | ValLoss=0.2645 | TrainAcc=0.9004 | ValAcc=0.8907 | wait=0
Epoch 3/200 | TrainLoss=0.2425 | ValLoss=0.2628 | TrainAcc=0.9023 | ValAcc=0.8923 | wait=0
Epoch 4/200 | TrainLoss=0.2267 | ValLoss=0.2525 | TrainAcc=0.9097 | ValAcc=0.8972 | wait=0
Epoch 5/200 | TrainLoss=0.2219 | ValLoss=0.2533 | TrainAcc=0.9110 | ValAcc=0.8985 | wait=1
Epoch 6/200 | TrainLoss=0.2172 | ValLoss=0.2489 | TrainAcc=0.9147 | ValAcc=0.9001 | wait=0
Epoch 7/200 | TrainLoss=0.2122 | ValLoss=0.2483 | TrainAcc=0.9163 | ValAcc=0.9019 | wait=0
Epoch 8/200 | TrainLoss=0.2096 | ValLoss=0.2551 | TrainAcc=0.9179 | ValAcc=0.9015 | wait=1
Epoch 9/200 | TrainLoss=0.2127 | ValLoss=0.2574 | TrainAcc=0.9157 | ValAcc=0.8995 | wait=2
Epoch 10/200 | TrainLoss=0.2082 | ValLoss=0.2597 | TrainAcc=0.9171 | ValAcc=0.9014 | wait=3
Epoch 11/200 | TrainLoss=0.1980 | ValLoss=0.2524 | TrainAcc=0.9211 | ValAcc=0.9033 | wait=4
Epoch 12/200 | TrainLoss=0.1938 | ValLoss=0.2588 | TrainAcc=0.9229 | ValAcc=0.9031 | wait=5
Epoch 13/200 | TrainLoss=0.1951 | ValLoss=0.2626 | TrainAcc=0.9216 | ValAcc=0.9043 | wait=6
Epoch 14/200 | TrainLoss=0.1883 | ValLoss=0.2649 | TrainAcc=0.9251 | ValAcc=0.9025 | wait=7
Epoch 15/200 | TrainLoss=0.1807 | ValLoss=0.2645 | TrainAcc=0.9292 | ValAcc=0.8999 | wait=8
Epoch 16/200 | TrainLoss=0.1760 | ValLoss=0.2728 | TrainAcc=0.9290 | ValAcc=0.9013 | wait=9
Epoch 17/200 | TrainLoss=0.1724 | ValLoss=0.2850 | TrainAcc=0.9313 | ValAcc=0.8994 | wait=10
Epoch 18/200 | TrainLoss=0.1739 | ValLoss=0.2854 | TrainAcc=0.9319 | ValAcc=0.8932 | wait=11
Epoch 19/200 | TrainLoss=0.1678 | ValLoss=0.2930 | TrainAcc=0.9310 | ValAcc=0.9010 | wait=12
Epoch 20/200 | TrainLoss=0.1632 | ValLoss=0.3077 | TrainAcc=0.9342 | ValAcc=0.8958 | wait=13
Epoch 21/200 | TrainLoss=0.1592 | ValLoss=0.3049 | TrainAcc=0.9348 | ValAcc=0.9001 | wait=14
Epoch 22/200 | TrainLoss=0.1556 | ValLoss=0.3178 | TrainAcc=0.9380 | ValAcc=0.8915 | wait=15
Epoch 23/200 | TrainLoss=0.1519 | ValLoss=0.3218 | TrainAcc=0.9402 | ValAcc=0.8855 | wait=16
Epoch 24/200 | TrainLoss=0.1556 | ValLoss=0.3292 | TrainAcc=0.9374 | ValAcc=0.8937 | wait=17
Epoch 25/200 | TrainLoss=0.1499 | ValLoss=0.3284 | TrainAcc=0.9402 | ValAcc=0.8895 | wait=18
Epoch 26/200 | TrainLoss=0.1436 | ValLoss=0.3413 | TrainAcc=0.9429 | ValAcc=0.8889 | wait=19
Epoch 27/200 | TrainLoss=0.1409 | ValLoss=0.3414 | TrainAcc=0.9435 | ValAcc=0.8925 | wait=20
Epoch 28/200 | TrainLoss=0.1405 | ValLoss=0.3488 | TrainAcc=0.9437 | ValAcc=0.8907 | wait=21
Epoch 29/200 | TrainLoss=0.1392 | ValLoss=0.3652 | TrainAcc=0.9448 | ValAcc=0.8837 | wait=22
Epoch 30/200 | TrainLoss=0.1319 | ValLoss=0.3817 | TrainAcc=0.9480 | ValAcc=0.8858 | wait=23
Epoch 31/200 | TrainLoss=0.1316 | ValLoss=0.3820 | TrainAcc=0.9492 | ValAcc=0.8786 | wait=24
□ Early stopping tại epoch 32
□ Restored best model (val_loss=0.2483)
```

Final DL Test Metrics:

Model: Best MLP

Accuracy: 0.9036666666666666

Precision: 0.9789527069861798

Recall: 0.8578888888888889

F1: 0.9144312192811038

ROC-AUC: 0.9373821111111111

Classification Report:

	precision	recall	f1-score	support
No Diabetes	0.82	0.97	0.89	6000
Diabetes	0.98	0.86	0.91	9000
accuracy			0.90	15000
macro avg	0.90	0.92	0.90	15000
weighted avg	0.92	0.90	0.90	15000

Hình 20.1. Output thực tế của cell “RETRAIN BEST CONFIG WITH MORE EPOCHS”.

4. Giải thích chi tiết code

- best_model được khởi tạo lại với hidden_dims=(128,64,32,16), Adam, lr=0.001 và seed=42. train_mlp chạy tối đa 200 epochs với batch 128, L2=1e-5 và patience=25.

- `full_eval` tính probability, prediction ở threshold 0.5 và các metric trên test. `classification_report` bổ sung precision, recall, F1 và support theo từng lớp.
- Điểm cần ghi rõ trong báo cáo: cell trước đó đã chọn Shallow (64), nhưng cell này lại retrain kiến trúc baseline 128-64-32-16. Vì vậy tên 'best_model' trong code không hoàn toàn trùng với cấu hình được chọn ở cell 19. Đây là một khác biệt thực tế của notebook, không nên tự sửa khi báo cáo kết quả.

5. Phân tích output và ý nghĩa

Mô hình retrain dừng ở epoch 32 và restore checkpoint có validation loss 0.2483. Test metrics cuối: Accuracy 0.90367, Precision 0.97895, Recall 0.85789, F1 0.91443 và ROC-AUC 0.93738. Classification report cho lớp Diabetes có precision khoảng 0.98 và recall khoảng 0.86. Kết quả này thấp hơn cấu hình Shallow được chọn ở cell trước, phù hợp với việc hai cell thực tế sử dụng hai architecture khác nhau.

21. VISUALIZE BEST DL MODEL

1. Mục đích của cell

Trực quan hóa quá trình học của mô hình retrain và confusion matrix trên test set.

2. Ảnh code thực tế

```
# =====  
# CELL 21: VISUALIZE BEST DL MODEL  
# =====  
  
fig, axes = plt.subplots(  
    1,  
    3,  
    figsize=(18, 5)  
)  
  
# Loss curve  
axes[0].plot(  
    best_hist['train_loss'],  
    label='Train'  
)  
  
axes[0].plot(  
    best_hist['val_loss'],  
    label='Val'  
)  
  
axes[0].set_xlabel('Epoch')  
axes[0].set_ylabel('Loss')  
axes[0].set_title(  
    'Best MLP - Loss'  
)  
axes[0].legend()  
axes[0].grid(alpha=0.3)  
  
# Accuracy curve  
axes[1].plot(  
    best_hist['train_acc'],  
    label='Train'  
)  
  
axes[1].plot(  
    best_hist['val_acc'],  
    label='Val'  
)  
  
axes[1].set_xlabel('Epoch')  
axes[1].set_ylabel('Accuracy')  
axes[1].set_title(  
    'Best MLP - Accuracy'  
)  
axes[1].legend()  
axes[1].grid(alpha=0.3)  
  
# Confusion matrix  
cm = confusion_matrix(  
    y_test.ravel(),  
    dl_pred.ravel()  
)  
  
sns.heatmap(  
    cm,  
    annot=True,  
    fmt='d',  
    ax=axes[2],  
    cmap='Blues',  
    cbar=False  
)
```

```

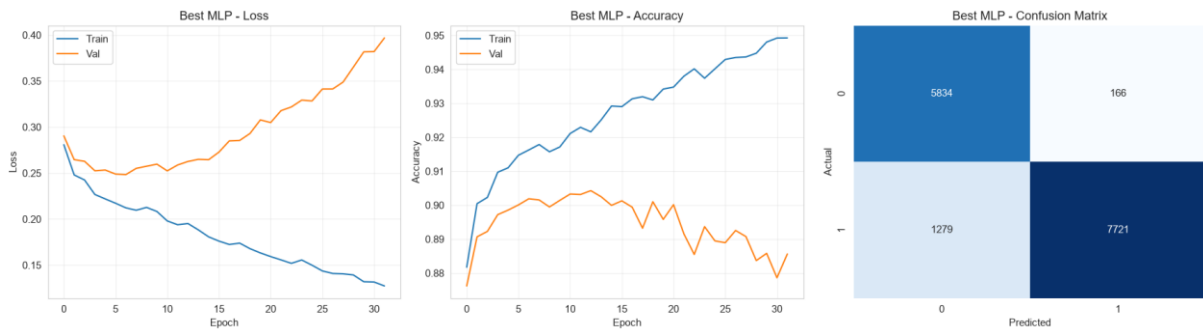
axes[2].set_xlabel('Predicted')
axes[2].set_ylabel('Actual')
axes[2].set_title(
    'Best MLP - Confusion Matrix'
)

plt.tight_layout()
plt.show()

```

3. Output thực tế lấy từ notebook

Output 1



Hình 21.1. Output thực tế của cell “VISUALIZE BEST DL MODEL”.

4. Giải thích chi tiết code

- best_hist chứa train_loss, val_loss, train_acc và val_acc nên có thể vẽ loss curve và accuracy curve theo epoch.
- Confusion matrix được tính từ y_test và dl_pred. Heatmap hiển thị số lượng dự đoán đúng/sai theo từng lớp.
- Ba biểu đồ được đặt cạnh nhau để người đọc có thể liên hệ convergence với kết quả phân loại cuối.

5. Phân tích output và ý nghĩa

Output thực tế là một figure gồm loss curve, accuracy curve và confusion matrix. Khi đọc cùng cell retrain, có thể quan sát validation loss đạt mức tốt rồi không tiếp tục cải thiện trong khi train loss vẫn có xu hướng giảm. Đây là dấu hiệu phù hợp với việc dùng early stopping và restore checkpoint tốt nhất.

22. FINAL COMPARISON - ML VS DL

1. Mục đích của cell

Đặt kết quả MLP NumPy cạnh các baseline Machine Learning để trả lời câu hỏi mô hình Deep Learning có vượt baseline hay không trên test set.

2. Ảnh code thực tế

```
# =====
# CELL 22: FINAL COMPARISON - ML VS DL
# =====

# Use concat instead of DataFrame.append because
# DataFrame.append was removed in newer pandas versions.

dl_row = pd.DataFrame([{'Model': 'MLP (NumPy 5L)',
                        'Accuracy': dl_metrics['Accuracy'],
                        'Precision': dl_metrics['Precision'],
                        'Recall': dl_metrics['Recall'],
                        'F1': dl_metrics['F1'],
                        'ROC-AUC': dl_metrics['ROC-AUC']}])

final_compare = pd.concat(
    [
        ml_df,
        dl_row
    ],
    ignore_index=True
)

print("=== FINAL COMPARISON ===")
display(final_compare)

# Visualization
fig, axes = plt.subplots(
    1,
    2,
    figsize=(16, 6)
)

metrics_list = [
    'Accuracy',
    'Precision',
    'Recall',
    'F1',
    'ROC-AUC'
]

x = np.arange(
    len(metrics_list)
)

w = 0.15

for i, row in final_compare.iterrows():

    axes[0].bar(
        x + i * w,
        [
            row[m]
            for m in metrics_list
        ],
        w,
        label=row['Model']
    )
```

```

axes[0].set_xticks(
    x + w * 2
)

axes[0].set_xticklabels(
    metrics_list
)

axes[0].set_ylim(
    0,
    1.05
)

axes[0].legend()
axes[0].set_title(
    'ML vs DL - Metrics Comparison'
)

axes[0].grid(
    axis='y',
    alpha=0.3
)

# ROC overlay
for name, prob in ml_probs.items():

    fpr, tpr, _ = roc_curve(
        y_test.ravel(),
        prob
    )

    auc = roc_auc_score(
        y_test.ravel(),
        prob
    )

    axes[1].plot(
        fpr,
        tpr,
        label=f'{name} (auc:{.3f})',
        alpha=0.7
    )

fpr, tpr, _ = roc_curve(
    y_test.ravel(),
    dl_prob.ravel()
)

```

```

axes[1].plot(
    fpr,
    tpr,
    label=(
        f'MLP NumPy '
        f'({dl_metrics["ROC-AUC"]:.3f})'
    ),
    linewidth=2.5,
    color='black'
)

axes[1].plot(
    [0, 1],
    [0, 1],
    'k--',
    alpha=0.4
)

axes[1].set_xlabel('FPR')
axes[1].set_ylabel('TPR')
axes[1].set_title(
    'ROC - ML vs DL'
)
axes[1].legend()

plt.tight_layout()
plt.show()

```

3. Output thực tế lấy từ notebook

Output 1

OUTPUT THỰC TẾ — OUTPUT 1

```
=== FINAL COMPARISON ===
```

Hình 22.1. Output thực tế của cell “FINAL COMPARISON - ML VS DL”.

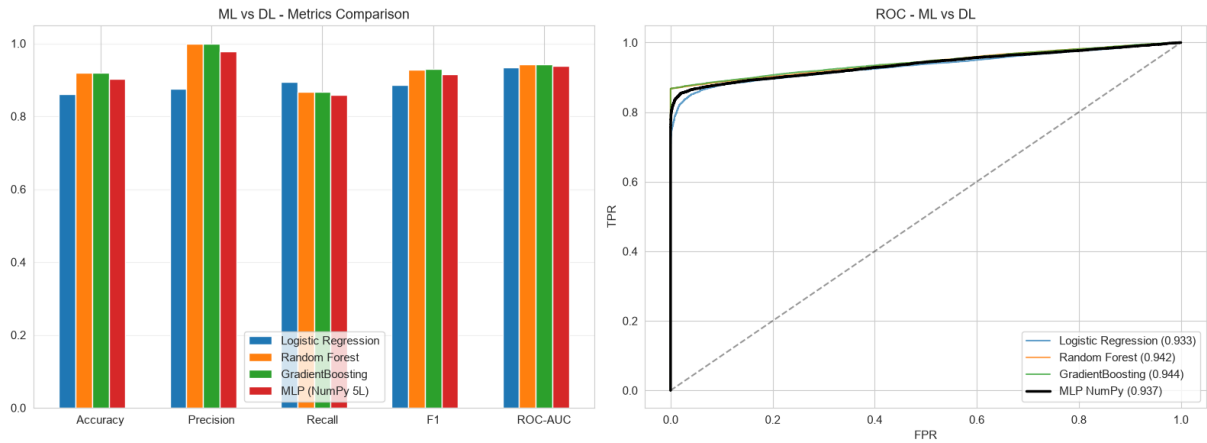
Output 2

OUTPUT THỰC TẾ — OUTPUT 2

	Model	Accuracy	Precision	Recall	F1	ROC-AUC
0	Logistic Regression	0.860600	0.875612	0.894778	0.885091	0.933224
1	Random Forest	0.920133	0.999104	0.867667	0.928758	0.942211
2	GradientBoosting	0.920400	0.999360	0.867889	0.928996	0.943518
3	MLP (NumPy 5L)	0.903667	0.978953	0.857889	0.914431	0.937382

Hình 22.2. Output thực tế của cell “FINAL COMPARISON - ML VS DL”.

Output 3



Hình 22.3. Output thực tế của cell “FINAL COMPARISON - ML VS DL”.

4. Giải thích chi tiết code

- `dl_row` chuyển metric của `best_model` thành một DataFrame một dòng. `pd.concat` ghép nó với `ml_df` để tạo `final_compare`.
- Notebook dùng `pd.concat` thay vì `DataFrame.append` vì `append` đã bị loại khỏi các phiên bản Pandas mới.
- Biểu đồ đầu tiên so sánh năm metric giữa tất cả model. Biểu đồ thứ hai overlay ROC curves của ML và MLP.
- Việc so sánh dùng cùng `X_test/y_test`, nên các model được đánh giá trên cùng một tập dữ liệu.

5. Phân tích output và ý nghĩa

Kết quả thực tế: GradientBoosting đạt Accuracy 0.920400 và ROC-AUC 0.943518; Random Forest đạt Accuracy 0.920133 và ROC-AUC 0.942211; MLP NumPy đạt Accuracy 0.903667 và ROC-AUC 0.937382. Vì vậy trên test set cụ thể của notebook, các baseline ML mạnh hơn MLP. Tuy nhiên MLP vẫn đạt Precision rất cao và F1 trên 0.91, cho thấy mô hình hoạt động tốt dù chưa dẫn đầu.

23. TOP-K FEATURE IMPORTANCE VIA PERMUTATION

1. Mục đích của cell

Ước lượng mức độ quan trọng của feature đối với MLP bằng permutation importance tự cài đặt, sử dụng mức giảm F1 làm thước đo.

2. Ảnh code thực tế

```
# =====
# CELL 23: TOP-K FEATURE IMPORTANCE VIA PERMUTATION
# =====

def permutation_importance_np(
    model,
    X,
    y,
    feature_names,
    n_repeats=3
):
    base_prob = model.predict_proba(X)

    base_pred = (
        base_prob >= 0.5
    ).astype(int)

    base_f1 = f1_score(
        y.ravel(),
        base_pred.ravel(),
        zero_division=0
    )

    importances = []

    for j in range(
        X.shape[1]
    ):
        scores = []

        for _ in range(n_repeats):
            Xp = X.copy()

            np.random.shuffle(
                Xp[:, j]
            )

            prob = model.predict_proba(
                Xp
            )

            pred = (
                prob >= 0.5
            ).astype(int)

            f1 = f1_score(
                y.ravel(),
                pred.ravel(),
                zero_division=0
            )

            scores.append(
                base_f1 - f1
            )

        importances.append(
            np.mean(scores)
        )
    )
```

```

    return np.array(
        importances
    )

# Use a subset for speed
n_sub = min(
    2000,
    len(X_test)
)

imp = permutation_importance_np(
    best_model,
    X_test[:n_sub],
    y_test[:n_sub],
    feature_names,
    n_repeats=2
)

imp_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': imp
})

imp_df = (
    imp_df
    .sort_values(
        'Importance',
        ascending=False
    )
    .head(20)
)

plt.figure(
    figsize=(10, 8)
)

plt.barh(
    imp_df['Feature'][::-1],
    imp_df['Importance'][::-1],
    color='teal'
)

plt.xlabel(
    'F1 drop when permuted'
)

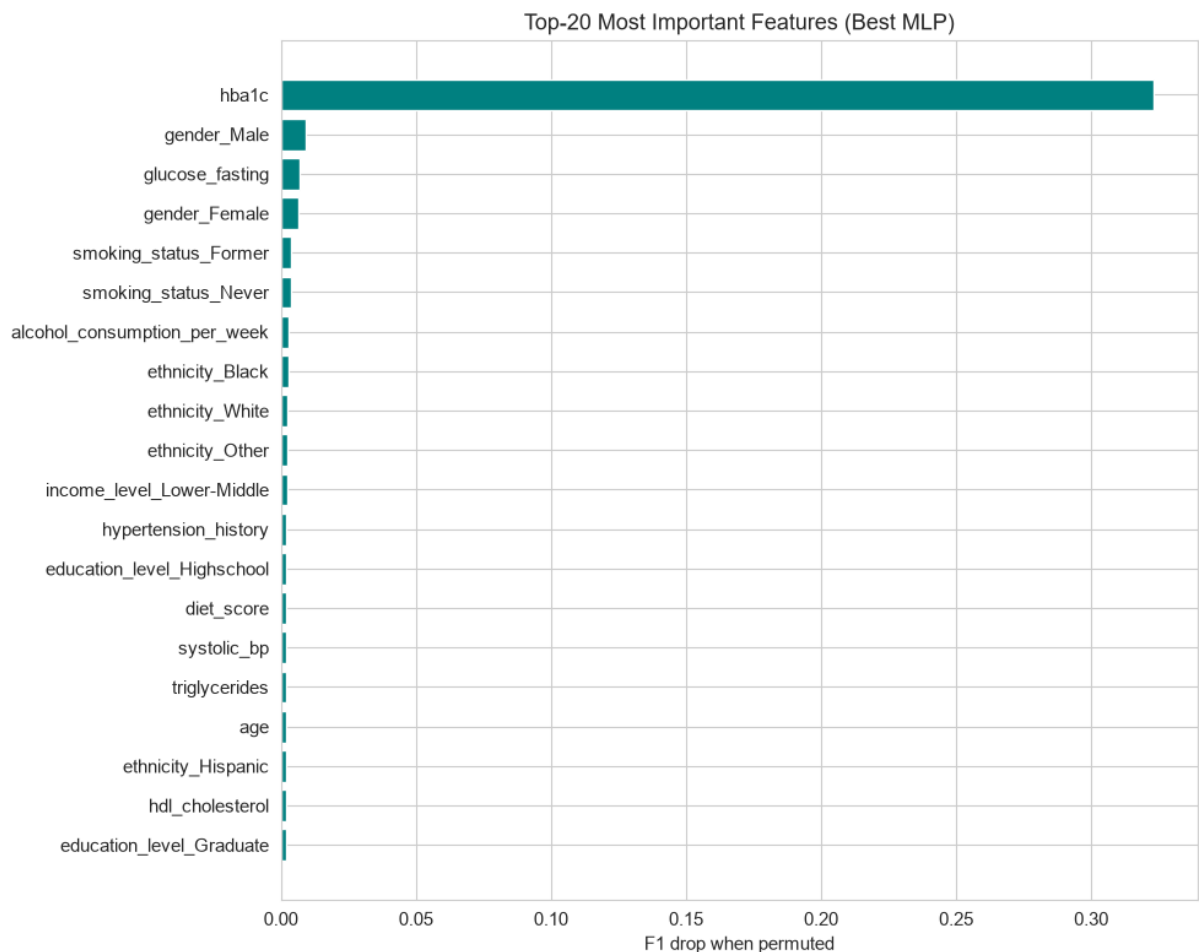
plt.title(
    'Top-20 Most Important Features (Best MLP)'
)

plt.tight_layout()
plt.show()

```

3. Output thực tế lấy từ notebook

Output 1



Hình 23.1. Output thực tế của cell “TOP-K FEATURE IMPORTANCE VIA PERMUTATION”.

4. Giải thích chi tiết code

- `base_prob` và `base_pred` tạo kết quả tham chiếu trên dữ liệu gốc; `base_f1` là F1 trước khi xáo trộn.
- Với từng feature `j`, code copy `X` rồi `np.random.shuffle(Xp[:,j])` để phá mối liên hệ của riêng feature đó với target trong khi giữ các cột khác.
- Model được dự đoán lại và tính F1. `base_f1 - f1` là mức giảm hiệu năng do permutation. Nếu mức giảm lớn, feature đó có ảnh hưởng đáng kể đến quyết định của model.
- `n_repeats=2` được dùng trên tối đa 2000 test samples để giảm chi phí tính toán. Sau đó các feature được sort giảm dần và lấy top 20 để vẽ.

5. Phân tích output và ý nghĩa

Output thực tế là biểu đồ Top-20 Most Important Features. Các thanh có Importance lớn hơn biểu thị việc xáo trộn feature đó làm F1 giảm nhiều hơn. Đây là giải thích model-agnostic ở mức feature, nhưng cần lưu ý importance được đo trên test subset trong notebook và không phải là causal importance.

24. DECISION THRESHOLD TUNING

1. Mục đích của cell

Minh họa cách thay đổi decision threshold của sigmoid để điều chỉnh trade-off giữa precision, recall, accuracy và F1.

2. Ảnh code thực tế

```
# =====  
# CELL 24: DECISION THRESHOLD TUNING  
# =====  
  
thresholds = np.arange(  
    0.1,  
    0.95,  
    0.05  
)  
  
th_results = []  
  
for t in thresholds:  
  
    pred = (  
        dl_prob.ravel() >= t  
    ).astype(int)  
  
    th_results.append({  
        'Threshold': t,  
        'Accuracy': accuracy_score(  
            y_test.ravel(),  
            pred  
        ),  
        'Precision': precision_score(  
            y_test.ravel(),  
            pred,  
            zero_division=0  
        ),  
        'Recall': recall_score(  
            y_test.ravel(),  
            pred,  
            zero_division=0  
        ),  
        'F1': f1_score(  
            y_test.ravel(),  
            pred,  
            zero_division=0  
        )  
    })  
  
th_df = pd.DataFrame(  
    th_results  
)  
  
print("=== Threshold Tuning ===")  
display(th_df)  
  
fig, ax = plt.subplots(  
    figsize=(12, 6)  
)
```

```

for m in [
    'Accuracy',
    'Precision',
    'Recall',
    'F1'
]:
    ax.plot(
        th_df['Threshold'],
        th_df[m],
        marker='o',
        label=m
    )

best_t = th_df.loc[
    th_df['F1'].idxmax(),
    'Threshold'
]

ax.axvline(
    best_t,
    color='red',
    linestyle='--',
    label=f'Best F1 threshold={best_t:.2f}'
)

ax.set_xlabel('Threshold')
ax.set_ylabel('Score')
ax.set_title(
    'Metrics vs Decision Threshold (Best MLP)'
)
ax.legend()
ax.grid(alpha=0.3)

plt.tight_layout()
plt.show()

print(
    f"\nBest threshold by F1: {best_t:.2f}"
)

print(
    "\nNOTE: In a production experiment, "
    "the threshold should be selected using "
    "the validation set, then evaluated once "
    "on the test set. This notebook keeps the "
    "original demo behavior of tuning on test."
)

```

3. Output thực tế lấy từ notebook

Output 1

OUTPUT THỰC TẾ — OUTPUT 1

=== Threshold Tuning ===

Hình 24.1. Output thực tế của cell “DECISION THRESHOLD TUNING”.

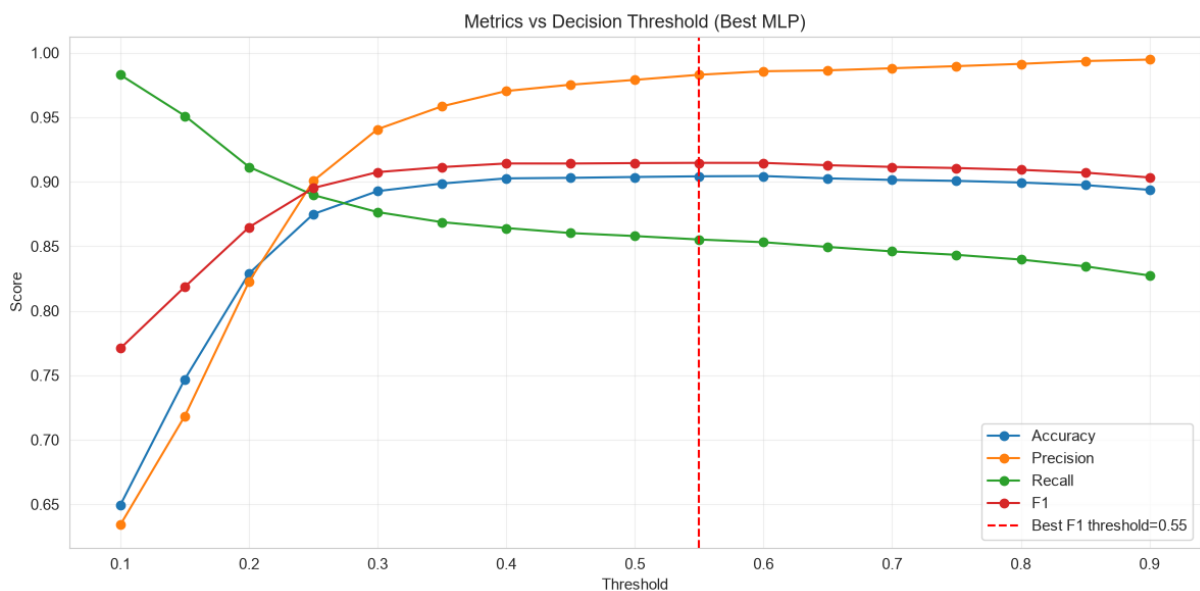
Output 2

OUTPUT THỰC TẾ — OUTPUT 2

	Threshold	Accuracy	Precision	Recall	F1
0	0.10	0.649533	0.634187	0.982778	0.770907
1	0.15	0.746867	0.718302	0.951111	0.818473
2	0.20	0.828933	0.822603	0.911444	0.864748
3	0.25	0.874933	0.900675	0.889667	0.895137
4	0.30	0.892733	0.940727	0.876444	0.907449
5	0.35	0.898600	0.958441	0.868667	0.911348
6	0.40	0.902600	0.970306	0.864111	0.914135
7	0.45	0.903000	0.975186	0.860222	0.914104
8	0.50	0.903667	0.978953	0.857889	0.914431
9	0.55	0.904200	0.982889	0.855222	0.914622
10	0.60	0.904400	0.985623	0.853111	0.914592
11	0.65	0.902600	0.986324	0.849444	0.912781
12	0.70	0.901400	0.987933	0.846000	0.911474
13	0.75	0.900667	0.989570	0.843333	0.910618
14	0.80	0.899400	0.991342	0.839667	0.909222
15	0.85	0.897400	0.993518	0.834444	0.907060
16	0.90	0.893733	0.994657	0.827333	0.903312

Hình 24.2. Output thực tế của cell “DECISION THRESHOLD TUNING”.

Output 3



Hình 24.3. Output thực tế của cell “DECISION THRESHOLD TUNING”.

Output 4

OUTPUT THỰC TẾ — OUTPUT 4

Best threshold by F1: 0.55

NOTE: In a production experiment, the threshold should be selected using the validation set, then evaluated once on the test set. ... f tuning on

Hình 24.4. Output thực tế của cell “DECISION THRESHOLD TUNING”.

4. Giải thích chi tiết code

- thresholds tạo dãy từ 0.10 đến 0.90 với bước 0.05. Với mỗi t, probability dl_prob được chuyển thành class bằng điều kiện $\geq t$.

- Ở mỗi threshold, notebook tính Accuracy, Precision, Recall và F1 rồi lưu vào th_df.
- Biểu đồ vẽ bốn metric theo threshold. best_t được lấy tại hàng có F1 lớn nhất, sau đó vẽ đường thẳng đứng để đánh dấu ngưỡng tốt nhất theo F1.
- Notebook tự ghi chú hạn chế: threshold được tune trực tiếp trên test set. Trong một quy trình production nghiêm ngặt, threshold nên được chọn trên validation rồi test chỉ dùng một lần để đánh giá.

5. Phân tích output và ý nghĩa

Output thực tế cho thấy F1 thay đổi theo threshold và best threshold theo F1 là 0.55. Ý nghĩa là threshold 0.5 không nhất thiết tối ưu cho mục tiêu F1. Tuy nhiên con số 0.55 chỉ nên xem là kết quả minh họa của notebook vì nó được chọn từ test set.

25. FINAL SUMMARY

1. Mục đích của cell

In toàn bộ kết quả định lượng và các quan sát chính của notebook để tạo một bản tổng kết có thể dùng làm cơ sở cho kết luận.

2. Ảnh code thực tế

```
# =====
# CELL 25: FINAL SUMMARY
# =====

print("=" * 70)
print("SUMMARY OF RESULTS")
print("=" * 70)

print("\n[1] Baseline ML models (test set):")
print(
    ml_df.to_string(
        index=False
    )
)

print(
    "\n[2] Best DL model "
    "(NumPy 5-layer MLP, test set):"
)

for k, v in dl_metrics.items():
    print(
        f"    {k}: {v}"
    )

print("\n[3] All DL experiment summary:")

print(
    dl_df.groupby(
        'Experiment'
    )[
        [
            'Accuracy',
            'F1',
            'ROC-AUC'
        ]
    ].max().round(4)
)

print("\n[4] Key observations:")

print(
    "    - Data is messy with '?' markers "
    "and missing values."
)

print(
    "    - 'diabetes_stage' was dropped "
    "due to target leakage."
)

print(
    "    - 'patient_tracking_id' and "
    "'data_entry_notes' were dropped."
)
```

```

print(
    "    - Proper train-only imputation "
    "& normalization were used."
)

print(
    "    - Deep Learning was implemented "
    "from scratch with NumPy only."
)

print(
    "    - Hyperparameter search covered "
    "LR, batch size, architecture, optimizer."
)

print(
    "    - Threshold tuning is demonstrated; "
    "for rigorous evaluation it should be "
    "selected on validation data."
)

print("\n[5] Recommendations:")

print(
    "    - Compare with frameworks such as "
    "PyTorch for scalability."
)

print(
    "    - Use cross-validation for more "
    "robust comparison."
)

print(
    "    - Consider regularizing further "
    "or collecting more data."
)

print("=" * 70)

```

3. Output thực tế lấy từ notebook

Output 1

OUTPUT THỰC TẾ — OUTPUT 1

SUMMARY OF RESULTS

```
[1] Baseline ML models (test set):
      Model Accuracy Precision Recall F1 ROC-AUC
Logistic Regression 0.860600 0.875612 0.894778 0.885091 0.933224
Random Forest      0.920133 0.999104 0.867667 0.928758 0.942211
GradientBoosting    0.920400 0.999360 0.867889 0.928996 0.943518

[2] Best DL model (NumPy 5-layer MLP, test set):
Model: Best MLP
Accuracy: 0.9036666666666666
Precision: 0.9789527069861798
Recall: 0.8578888888888889
F1: 0.9144312192811038
ROC-AUC: 0.9373821111111111

[3] All DL experiment summary:
      Accuracy F1 ROC-AUC
Experiment
Architecture 0.9147 0.9243 0.9400
BatchSize    0.9068 0.9173 0.9383
LR           0.9141 0.9236 0.9407
Optimizer    0.9037 0.9144 0.9374

[4] Key observations:
- Data is messy with '?' markers and missing values.
- 'diabetes_stage' was dropped due to target leakage.
- 'patient_tracking_id' and 'data_entry_notes' were dropped.
- Proper train-only imputation & normalization were used.
- Deep Learning was implemented from scratch with NumPy only.
- Hyperparameter search covered LR, batch size, architecture, optimizer.
- Threshold tuning is demonstrated; for rigorous evaluation it should be selected on validation data.

[5] Recommendations:
- Compare with frameworks such as PyTorch for scalability.
- Use cross-validation for more robust comparison.
- Consider regularizing further or collecting more data.
```

Hình 25.1. Output thực tế của cell “FINAL SUMMARY”.

4. Giải thích chi tiết code

- Phần [1] in ml_df, giúp tổng hợp kết quả ba baseline ML.
- Phần [2] in dl_metrics của mô hình MLP cuối cùng, bao gồm Accuracy, Precision, Recall, F1 và ROC-AUC.
- Phần [3] groupby Experiment trên dl_df và lấy max của Accuracy, F1, ROC-AUC để tóm tắt kết quả tốt nhất theo từng nhóm hyperparameter.
- Phần [4] in các quan sát về dữ liệu messy, target leakage, preprocessing, NumPy-only Deep Learning, hyperparameter search và threshold tuning.
- Phần [5] đưa ra các hướng phát triển: so sánh với PyTorch, dùng cross-validation và tăng regularization hoặc dữ liệu.

5. Phân tích output và ý nghĩa

Output thực tế tổng kết: Logistic Regression Accuracy 0.8606; Random Forest 0.920133; GradientBoosting 0.920400. MLP cuối có Accuracy 0.903667, Precision 0.978953, Recall 0.857889, F1 0.914431 và ROC-AUC 0.937382.

Notebook kết luận pipeline đã xử lý missing, leakage và normalization; đồng thời khuyến nghị framework như PyTorch cho khả năng mở rộng, cross-validation cho đánh giá chắc chắn hơn và cải thiện regularization/dữ liệu.

26. Lưu model

```
# =====
# FINAL CELL: SAVE BEST DEEP LEARNING MODEL + ARTIFACTS
# =====

import json, os
from datetime import datetime

SAVE_DIR = "saved_best_model"
os.makedirs(SAVE_DIR, exist_ok=True)

# -----
# 1. Lưu weights + biases (numpy arrays)
# -----

weights_dict = {}
for i, w in enumerate(best_model.W):
    weights_dict[f'W{i}'] = w
for i, b in enumerate(best_model.b):
    weights_dict[f'b{i}'] = b

np.savez(os.path.join(SAVE_DIR, "weights.npz"), **weights_dict)
print(f"✅ Saved weights → {SAVE_DIR}/weights.npz")

# -----
# 2. Lưu kiến trúc mạng
# -----

arch_config = {
    'input_dim': int(input_dim),
    'hidden_dims': [128, 64, 32, 16],
    'num_layers': int(best_model.num_layers),
    'architecture_str': f"{input_dim} -> 128 -> 64 -> 32 -> 16 -> 1",
    'hidden_activation': 'relu',
    'output_activation': 'sigmoid',
    'loss': 'binary_cross_entropy',
    'optimizer': 'adam',
    'learning_rate': 0.001,
    'batch_size': 128,
    'l2': 1e-5,
    'trained_at': datetime.now().isoformat(),
}

with open(os.path.join(SAVE_DIR, "architecture.json"), "w") as f:
    json.dump(arch_config, f, indent=2)
print(f"✅ Saved architecture → {SAVE_DIR}/architecture.json")
```

```

# -----
# 3. Lưu preprocessing artifacts (medians, mu, sigma)
# -----
np.savez(
    os.path.join(SAVE_DIR, "preprocessing.npz"),
    train_medians=train_medians,
    mu=mu,
    sigma=sigma,
)

prep_config = {
    'feature_names': feature_names,
    'numeric_cols': numeric_cols,
    'categorical_cols': cat_cols,
    'dropped_cols': existing_drop + (['diabetes_stage'] if 'diabetes_stage' in df.columns else []),
    'target_col': 'diagnosed_diabetes',
    'imputation': 'median (from train only)',
    'normalization': 'standardization (from train only)',
    'encoding': 'one-hot (drop_first=False)',
    'n_features_final': int(input_dim),
}

with open(os.path.join(SAVE_DIR, "preprocessing.json"), "w") as f:
    json.dump(prep_config, f, indent=2)
print(f"✅ Saved preprocessing → {SAVE_DIR}/preprocessing.json")

# -----
# 4. Lưu training history
# -----
np.savez(
    os.path.join(SAVE_DIR, "training_history.npz"),
    train_loss=np.array(best_hist['train_loss']),
    val_loss =np.array(best_hist['val_loss']),
    train_acc =np.array(best_hist['train_acc']),
    val_acc =np.array(best_hist['val_acc']),
)
print(f"✅ Saved history → {SAVE_DIR}/training_history.npz")

# -----
# 5. Lưu metrics cuối cùng (JSON)
# -----
metrics_to_save = {
    'test_accuracy': float(dl_metrics['Accuracy']),
    'test_precision': float(dl_metrics['Precision']),
    'test_recall': float(dl_metrics['Recall']),
    'test_f1': float(dl_metrics['F1']),
    'test_roc_auc': float(dl_metrics['ROC-AUC']),
    'total_epochs': len(best_hist['train_loss']),
    'best_val_loss': float(min(best_hist['val_loss'])),
}

with open(os.path.join(SAVE_DIR, "metrics.json"), "w") as f:
    json.dump(metrics_to_save, f, indent=2)
print(f"✅ Saved metrics → {SAVE_DIR}/metrics.json")

```

```
# -----
# 6. Liệt kê tất cả files đã lưu
# -----
print(f"\n 📁 Nội dung thư mục '{SAVE_DIR}':")
total_size = 0
for fname in sorted(os.listdir(SAVE_DIR)):
    fpath = os.path.join(SAVE_DIR, fname)
    size_kb = os.path.getsize(fpath) / 1024
    total_size += size_kb
    print(f"    {fname:25s} {size_kb:8.2f} KB")
print(f"    {'TOTAL':25s} {total_size:8.2f} KB")

print("\n" + "=" * 60)
print("🏆 BEST MODEL ĐÃ ĐƯỢC LƯU THÀNH CÔNG")
print("=" * 60)
print(f"    Architecture : {arch_config['architecture_str']}")
print(f"    Test F1       : {metrics_to_save['test_f1']:.4f}")
print(f"    Test ROC-AUC  : {metrics_to_save['test_roc_auc']:.4f}")
print(f"    Test Accuracy : {metrics_to_save['test_accuracy']:.4f}")
print("=" * 60)

✅ Saved weights      → saved_best_model/weights.npz
✅ Saved architecture → saved_best_model/architecture.json
✅ Saved preprocessing → saved_best_model/preprocessing.json
✅ Saved history      → saved_best_model/training_history.npz
✅ Saved metrics      → saved_best_model/metrics.json

📁 Nội dung thư mục 'saved_best_model/':
architecture.json      0.39 KB
metrics.json           0.25 KB
preprocessing.json     2.33 KB
preprocessing.npz      1.29 KB
training_history.npz   1.99 KB
weights.npz           134.39 KB
TOTAL                  140.65 KB

=====
🏆 BEST MODEL ĐÃ ĐƯỢC LƯU THÀNH CÔNG
=====
Architecture : 47 -> 128 -> 64 -> 32 -> 16 -> 1
Test F1       : 0.9144
Test ROC-AUC  : 0.9374
Test Accuracy : 0.9037
=====
```

BÀI 2: Dự đoán giá nhà. Dataset: Vietnam Real Estate Datasets - Catalyst

1. Load dataset

Đặt `train.csv` cùng thư mục với notebook. Nếu tên file khác, sửa biến `DATA\_PATH`.

Cell code 3

```
1 DATA_PATH = "data\house_buying_dec29th_2025.csv"
2
3 df = pd.read_csv(DATA_PATH)
4
5 print("Shape:", df.shape)
6 display(df.head())
```

Giải thích chi tiết code

Mục tiêu của cell này là xác định vị trí file dữ liệu, đọc dữ liệu CSV vào DataFrame và kiểm tra nhanh kích thước cũng như một phần bản ghi đầu tiên. Đây là bước đầu tiên của pipeline vì mọi phân tích và mô hình hóa phía sau đều phụ thuộc vào cấu trúc thực tế của DataFrame `df`.

Biến `DATA\_PATH` chứa đường dẫn tương đối đến file CSV. Lệnh `pd.read\_csv(DATA\_PATH)` sử dụng pandas để đọc toàn bộ bảng dữ liệu. Sau đó `df.shape` trả về số dòng và số cột, còn `df.head()` hiển thị một số bản ghi đầu tiên để kiểm tra dữ liệu đã được đọc đúng hay chưa.

Output của cell cho phép xác nhận dataset đã được nạp và cung cấp cái nhìn ban đầu về các trường dữ liệu. Việc kiểm tra `head()` đặc biệt quan trọng vì có thể phát hiện sớm vấn đề về tên cột, kiểu dữ liệu hoặc giá trị bất thường trước khi thực hiện cleaning.

- Số dòng và số cột được dùng làm mốc so sánh trước/sau cleaning.
- Các tên cột xuất hiện ở `head()` là cơ sở để xác định feature và target ở các cell tiếp theo.

Output thực tế của cell

Hình output thực tế 1

Output thực tế
Shape: (66912, 11)

Hình output thực tế 2

Output thực tế

```
      id      detail_url \
0 196958 https://batdongsan.vn/ban-nha-nguyen-son-long-...
1 196967 https://batdongsan.vn/ban-nha-hem-xe-hoi-4m-2-...
2 196983 https://batdongsan.vn/o-to-ngu-nha-62m2-5-tang-...
3 196962 https://batdongsan.vn/ban-nha-phuc-loi-ngo-non-...
4 196972 https://batdongsan.vn/tran-duy-hung-6-tang-tha...

      title      location \
0 Bán nhà Nguyễn Sơn Long Biên- Dân trí Tuyệt vời... Long Biên, Hà Nội
1 BÁN NHÀ HẸM XE HƠI 4M, 2 MẶT TIỀN - 2 TẦNG , N... Bình Chánh, Hồ Chí Minh
2 Ô TÔ NGỦ NHÀ - 62M² - 5 Tầng/4PN - Gần kdc CIT... Gò Vấp, Hồ Chí Minh
3 Bán nhà Phúc Lợi - ngõ nông - ô tô vào nhà - t... Long Biên, Hà Nội
4 TRẦN DUY HÙNG - 6 TẦNG THANG MÁY - NGÕ THÔNG D... Cầu Giấy, Hà Nội

      timeline_hours  area_m2  bedrooms  bathrooms  floors  frontage \
0                2      50.0        4.0        4.0      6.0      False
1                2      98.0        4.0        4.0      3.0       True
2                2      62.0        4.0        5.0      6.0      False
3                2      39.0        3.0        4.0      6.0      False
4                2      45.0        3.0        3.0      7.0       True

      price_million_vnd
0          12500.0
1           5500.0
2           8740.0
3           6500.0
4          16980.0
```

Nhận xét và vai trò trong pipeline

Kết luận: Output của cell cho phép xác nhận dataset đã được nạp và cung cấp cái nhìn ban đầu về các trường dữ liệu. Việc kiểm tra `head()` đặc biệt quan trọng vì có thể phát hiện sớm vấn đề về tên cột, kiểu dữ liệu hoặc giá trị bất thường trước khi thực hiện cleaning.

2. Data inspection and understanding

Cell code 5

```
1 print("=== INFO ===")
2 df.info()
3
4 print("\n=== DESCRIBE ===")
5 display(df.describe(include="all").T)
6
7 print("\n=== MISSING VALUES ===")
8 missing = df.isnull().sum()
9 missing_pct = missing / len(df) * 100
10 missing_df = pd.DataFrame({"Missing": missing, "Percent": missing_pct})
11 display(missing_df[missing_df["Missing"] > 0].sort_values("Percent", ascending=False))
12
13 print("\n=== DUPLICATES ===")
14 print("Số dòng trùng lặp:", df.duplicated().sum())
```

Giải thích chi tiết code

Cell này thực hiện kiểm tra tổng quan có hệ thống bằng `df.info()`, `df.describe(include="all").T`, kiểm tra missing values và duplicated records. Đây là bước Data Inspection nhằm hiểu dữ liệu trước khi quyết định cách làm sạch.

`df.info()` cho biết số lượng non-null và kiểu dữ liệu của từng cột. `describe(include="all").T` mở rộng thống kê cho cả biến số và biến không phải số; `.T` chuyển hàng/cột để mỗi feature nằm trên một dòng, giúp đọc báo cáo dễ hơn. Với missing values, code tính cả số lượng và tỷ lệ phần trăm thiếu bằng `missing / len(df) \* 100`, sau đó chỉ hiển thị những cột có thiếu dữ liệu. Cuối cùng `df.duplicated().sum()` đếm số bản ghi trùng hoàn toàn.

Output thực tế là cơ sở để xác định dữ liệu có cần loại duplicate, có cột nào thiếu dữ liệu và những kiểu dữ liệu nào cần xử lý. Thay vì xử lý ngay trong cell inspection, notebook tách riêng bước cleaning ở cell sau, giúp quy trình rõ ràng và có thể giải thích được lý do của từng thao tác.

- Kiểm tra cả số lượng lẫn phần trăm missing giúp đánh giá mức độ nghiêm trọng của dữ liệu thiếu.
- Duplicate được kiểm tra độc lập vì các bản ghi trùng có thể làm sai lệch phân phối và kết quả huấn luyện.

Output thực tế của cell

Hình output thực tế 1

```

Output thực tế

=== INFO ===
<class 'pandas.DataFrame'>
RangeIndex: 66912 entries, 0 to 66911
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                     66912 non-null  int64
1   detail_url             66905 non-null  str
2   title                  66912 non-null  str
3   location               66912 non-null  str
4   timeline_hours         66912 non-null  int64
5   area_m2                50344 non-null  float64
6   bedrooms               55273 non-null  float64
7   bathrooms              51548 non-null  float64
8   floors                 52278 non-null  float64
9   frontage               66912 non-null  bool
10  price_million_vnd      64809 non-null  float64
dtypes: bool(1), float64(5), int64(2), str(3)
memory usage: 18.9 MB

=== DESCRIBE ===

```

Hình output thực tế 2

```

Output thực tế

count unique \
id                66912.0  NaN
detail_url        66905  66905
title             66912  66899
location          66912    280
timeline_hours    66912.0  NaN
area_m2           50344.0  NaN
bedrooms          55273.0  NaN
bathrooms         51548.0  NaN
floors            52278.0  NaN
frontage          66912    2
price_million_vnd 64809.0  NaN

top freq \
id                NaN  NaN
detail_url        https://batdongsan.vn/ban-nha-nguyen-son-long-...  1
title             □ TINH HOA TRẦN VĂN QUANG - HXH - 4 TẦNG BTCT...  2
location          Gò Vấp, Hồ Chí Minh 3441
timeline_hours    NaN  NaN
area_m2           NaN  NaN
bedrooms          NaN  NaN
bathrooms         NaN  NaN
floors            NaN  NaN
frontage          False 52086
price_million_vnd NaN  NaN

mean      std      min      25% \
id        141092.732768  32545.265005  80084.0  116152.75
detail_url      NaN      NaN      NaN      NaN
title           NaN      NaN      NaN      NaN
location        NaN      NaN      NaN      NaN
timeline_hours   8749.222666  6863.201079  2.0  3600.0
area_m2         1377.9767  276334.663515  1.0  42.0
bedrooms        4.429649  4.500243  1.0  3.0
bathrooms       4.296617  4.618477  1.0  2.0
floors          14775.954379  3377398.558508  0.0  3.0
frontage        NaN      NaN      NaN      NaN
price_million_vnd 13810.677155  35144.733752  0.0  4600.0

50%      75%      max
id        142310.0  168691.5  197061.0
detail_url      NaN      NaN      NaN
title           NaN      NaN      NaN
location        NaN      NaN      NaN
timeline_hours   7200.0  8760.0  26280.0
area_m2         60.0  92.0  62000000.0
bedrooms        4.0  5.0  127.0
bathrooms       3.0  5.0  127.0
floors          4.0  6.0  772221339.0
frontage        NaN      NaN      NaN
price_million_vnd 7000.0  12800.0  989000.0

```

Hình output thực tế 3

```

Output thực tế

=== MISSING VALUES ===

```

Hình output thực tế 4

```

Output thực tế

Missing  Percent
area_m2    16568  24.760880
bathrooms  15364  22.961502
floors     14634  21.870516
bedrooms   11639  17.394488
price_million_vnd 2103  3.142934
detail_url 7      0.010462

```

Hình output thực tế 5

Output thực tế

```
=== DUPLICATES ===  
Số dòng trùng lặp: 0
```

Nhận xét và vai trò trong pipeline

Kết luận: Output thực tế là cơ sở để xác định dữ liệu có cần loại duplicate, có cột nào thiếu dữ liệu và những kiểu dữ liệu nào cần xử lý. Thay vì xử lý ngay trong cell inspection, notebook tách riêng bước cleaning ở cell sau, giúp quy trình rõ ràng và có thể giải thích được lý do của từng thao tác.

3. Data cleaning

- Xóa duplicated records.
- `price\_million\_vnd` <= 0` được xem là invalid và loại bỏ.
- `area\_m2` <= 0` → missing.
- `bedrooms`, `bathrooms` ngoài khoảng hợp lý → missing.
- `floors` và `timeline\_hours` được kiểm tra nếu tồn tại.
- Missing features được xử lý bằng imputation trong pipeline.
- Outlier được **báo cáo nhưng không tự động xóa**, nhằm tránh mất dữ liệu hợp lệ.

Cell code 7

```
1 df = df.drop_duplicates().reset_index(drop=True)
2
3 target = "price_million_vnd"
4
5 if target not in df.columns:
6     raise ValueError(f"Không tìm thấy target '{target}'. Columns: {df.columns.tolist()}")
7
8 df = df[df[target].notna() & (df[target] > 0)].copy()
9
10 if "area_m2" in df.columns:
11     df.loc[df["area_m2"] <= 0, "area_m2"] = np.nan
12
13 if "bedrooms" in df.columns:
14     df.loc[(df["bedrooms"] <= 0) | (df["bedrooms"] > 20), "bedrooms"] = np.nan
15
16 if "bathrooms" in df.columns:
17     df.loc[(df["bathrooms"] <= 0) | (df["bathrooms"] > 20), "bathrooms"] = np.nan
18
19 if "floors" in df.columns:
20     df.loc[(df["floors"] < 0) | (df["floors"] > 100), "floors"] = np.nan
21
22 if "timeline_hours" in df.columns:
23     df.loc[df["timeline_hours"] <= 0, "timeline_hours"] = np.nan
24
25 print("Shape sau cleaning:", df.shape)
```

Giải thích chi tiết code

Cell cleaning thực hiện các quy tắc chất lượng dữ liệu đã được xác định ở markdown. Trước hết notebook loại bản ghi trùng bằng `drop\_duplicates()` và reset lại index. Sau đó xác định target là `price\_million\_vnd` và chỉ giữ các dòng có target khác missing và lớn hơn 0.

Đối với `area\_m2`, giá trị nhỏ hơn hoặc bằng 0 được chuyển thành `NaN` thay vì xóa cả dòng. Với `bedrooms`, các giá trị không hợp lý như <=0 hoặc >20 cũng được chuyển thành missing. Cách xử lý này giữ lại bản ghi nhưng đánh dấu feature lỗi để pipeline có thể impute sau đó. Các trường `floors` và `timeline\_hours` được kiểm tra nếu tồn tại. Outlier không bị tự động xóa; notebook chỉ báo cáo ở cell IQR để tránh loại nhầm những căn nhà thực sự có giá trị cao/thấp.

Output sau cleaning cho thấy quy mô dữ liệu thay đổi so với ban đầu và xác nhận target đã được kiểm soát. Đây là điểm quan trọng: cleaning ở đây tập trung vào lỗi rõ ràng và giá trị invalid, còn missing sẽ được xử lý có hệ thống trong preprocessing.

- Giữ outlier thay vì xóa tự động giúp hạn chế mất dữ liệu hợp lệ.
- Việc chuyển giá trị sai sang `NaN` cho phép bước imputation xử lý thống nhất về sau.

Output thực tế của cell

Hình output thực tế 1

Output thực tế
Shape sau cleaning: (64447, 11)

Nhận xét và vai trò trong pipeline

Kết luận: Output sau cleaning cho thấy quy mô dữ liệu thay đổi so với ban đầu và xác nhận target đã được kiểm soát. Đây là điểm quan trọng: cleaning ở đây tập trung vào lỗi rõ ràng và giá trị invalid, còn missing sẽ được xử lý có hệ thống trong preprocessing.

4. Outlier analysis bằng IQR

Cell code 9

```
1 num_cols = [  
2     c for c in ["timeline_hours", "area_m2", "bedrooms", "bathrooms", "floors"]  
3     if c in df.columns  
4 ]  
5  
6 outlier_report = []  
7  
8 for c in num_cols + [target]:  
9     q1 = df[c].quantile(0.25)  
10    q3 = df[c].quantile(0.75)  
11    iqr = q3 - q1  
12    lower = q1 - 1.5 * iqr  
13    upper = q3 + 1.5 * iqr  
14    n_out = ((df[c] < lower) | (df[c] > upper)).sum()  
15  
16    outlier_report.append({  
17        "column": c,  
18        "Q1": q1,  
19        "Q3": q3,  
20        "IQR": iqr,  
21        "lower": lower,  
22        "upper": upper,  
23        "n_outliers": n_out,  
24        "pct": 100 * n_out / len(df)  
25    })  
26  
27 outlier_df = pd.DataFrame(outlier_report)  
28 display(outlier_df)
```

Giải thích chi tiết code

Cell này sử dụng quy tắc IQR để định lượng số lượng outlier của các biến số và target. Với mỗi cột, Q1 là phân vị 25%, Q3 là phân vị 75%, $IQR = Q3 - Q1$; ngưỡng dưới và trên lần lượt là $Q1 - 1.5 \times IQR$ và $Q3 + 1.5 \times IQR$.

Vòng lặp tính các ngưỡng cho từng cột rồi đếm số quan sát nằm ngoài khoảng. Kết quả được lưu vào `outlier\_report`, sau đó chuyển thành DataFrame để dễ quan sát. Cách làm này không biến outlier thành lỗi một cách máy móc; nó chỉ cung cấp bằng chứng thống kê để đánh giá phân phối dữ liệu.

Output cho thấy feature nào có nhiều điểm vượt ngưỡng IQR và target có mức độ phân tán ra sao. Trong bài toán giá nhà, việc có nhiều giá trị lớn không nhất thiết là dữ liệu sai, vì giá nhà thực tế thường lệch phải. Vì vậy notebook chọn báo cáo thay vì loại bỏ.

- IQR là phương pháp dựa trên phân vị nên ít nhạy với giá trị cực đoan hơn so với việc chỉ dựa vào mean/std.
- Kết quả outlier được dùng cho phân tích và giải thích, không phải làm sạch tự động.

Output thực tế của cell

Hình output thực tế 1

```
Output thực tế
      column      Q1      Q3      IQR      lower      upper      n_outliers \
0  timeline_hours 3600.0  8760.0  5160.0 -4140.0  16500.0      14358
1      area_m2    42.0   92.0   50.0   -33.0   167.0      4254
2    bedrooms     3.0    4.0    1.0    1.5    5.5      10174
3   bathrooms     2.0    5.0    3.0   -2.5    9.5      2053
4      floors     3.0    6.0    3.0   -1.5   10.5      292
5 price_million_vnd 4650.0 12800.0 8150.0 -7575.0 25025.0      5837

      pct
0  22.278772
1   6.600773
2  15.786615
3   3.185563
4   0.453085
5   9.057055
```

Nhận xét và vai trò trong pipeline

Kết luận: Output cho thấy feature nào có nhiều điểm vượt ngưỡng IQR và target có mức độ phân tán ra sao. Trong bài toán giá nhà, việc có nhiều giá trị lớn không nhất thiết là dữ liệu sai, vì giá nhà thực tế thường lệch phải. Vì vậy notebook chọn báo cáo thay vì loại bỏ.

5. Visualization

Cell code 11

```
1 cols_to_plot = [c for c in num_cols + [target] if c in df.columns]
2
3 fig, axes = plt.subplots(2, 3, figsize=(16, 8))
4 axes = axes.flatten()
5
6 for ax, c in zip(axes, cols_to_plot):
7     ax.hist(df[c].dropna(), bins=30)
8     ax.set_title(f"Distribution: {c}")
9     ax.set_xlabel(c)
10    ax.set_ylabel("Count")
11
12 for ax in axes[len(cols_to_plot):]:
13     ax.axis("off")
14
15 plt.tight_layout()
16 plt.show()
17
18 fig, axes = plt.subplots(2, 3, figsize=(16, 8))
19 axes = axes.flatten()
20
21 for ax, c in zip(axes, cols_to_plot):
22     ax.boxplot(df[c].dropna(), vert=False)
23     ax.set_title(f"Boxplot: {c}")
24
25 for ax in axes[len(cols_to_plot):]:
26     ax.axis("off")
27
28 plt.tight_layout()
29 plt.show()
```

Giải thích chi tiết code

Cell visualization tạo histogram và boxplot cho các biến số cùng target. Histogram dùng để quan sát hình dạng phân phối, mức tập trung và độ lệch; boxplot giúp nhìn median, khoảng tứ phân vị và các điểm nằm xa vùng trung tâm.

Code tạo lưới subplot 2×3, lặp qua danh sách `cols\_to\_plot`, rồi vẽ histogram với 30 bins. Sau đó tạo một lưới tương tự cho boxplot. Các ô dư được tắt bằng `axis("off")` nếu số biến ít hơn số subplot.

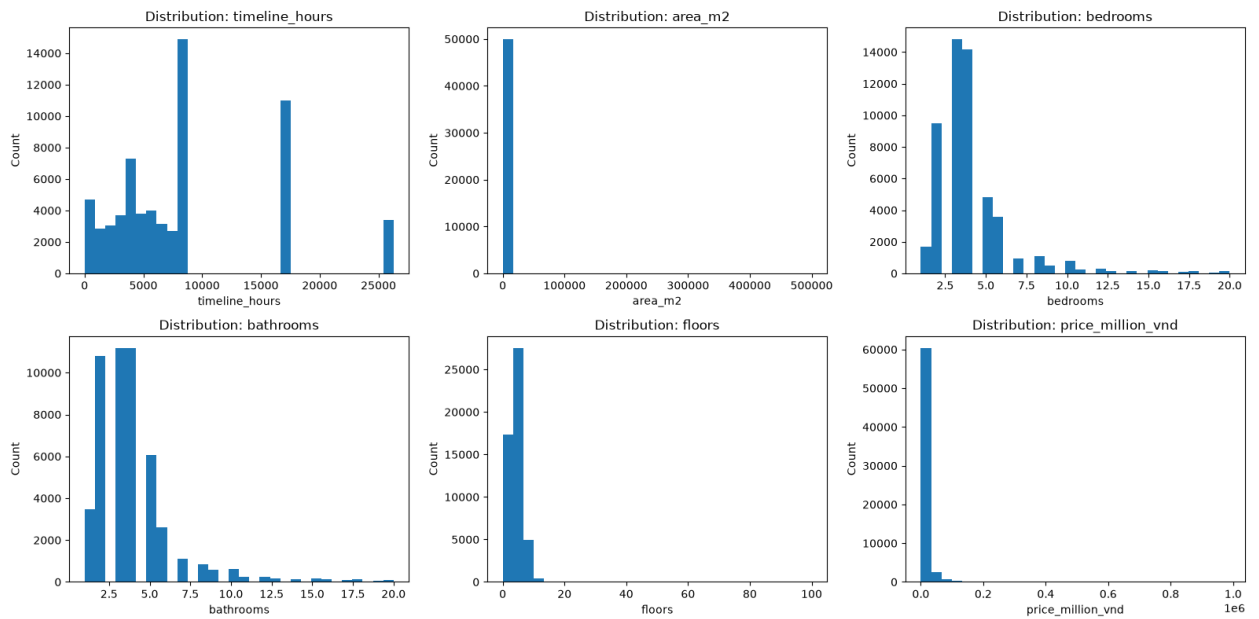
Các hình output thực tế giúp kiểm chứng trực quan những nhận xét từ thống kê và IQR. Đặc biệt, phân phối target và các feature về diện tích/phòng có thể cho thấy độ lệch và phạm vi giá trị, từ đó giải thích vì sao notebook sử dụng StandardScaler cho mô hình.

- Histogram phù hợp để nhìn phân phối và tần suất.

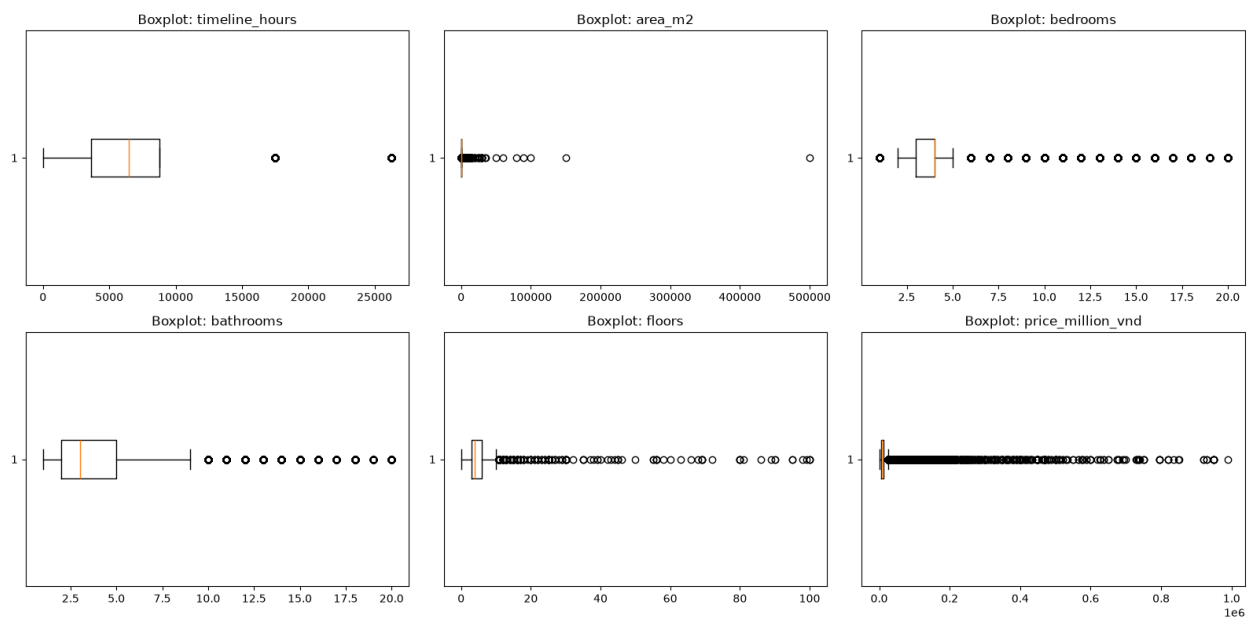
- Boxplot phù hợp để quan sát độ phân tán và giá trị cực đoan.

Output thực tế của cell

Hình output thực tế 1



Hình output thực tế 2



Cell code 12

```

1 corr_cols = [c for c in num_cols + [target] if c in df.columns]
2 corr = df[corr_cols].corr()
3
4 plt.figure(figsize=(8, 6))
5 plt.imshow(corr, aspect="auto")
6 plt.colorbar()
7 plt.xticks(range(len(corr.columns)), corr.columns, rotation=45, ha="right")
8 plt.yticks(range(len(corr.columns)), corr.columns)
9 plt.title("Correlation Matrix")
10 plt.tight_layout()
11 plt.show()

```

Giải thích chi tiết code

Cell visualization tạo histogram và boxplot cho các biến số cùng target. Histogram dùng để quan sát hình dạng phân phối, mức tập trung và độ lệch; boxplot giúp nhìn median, khoảng tứ phân vị và các điểm nằm xa vùng trung tâm.

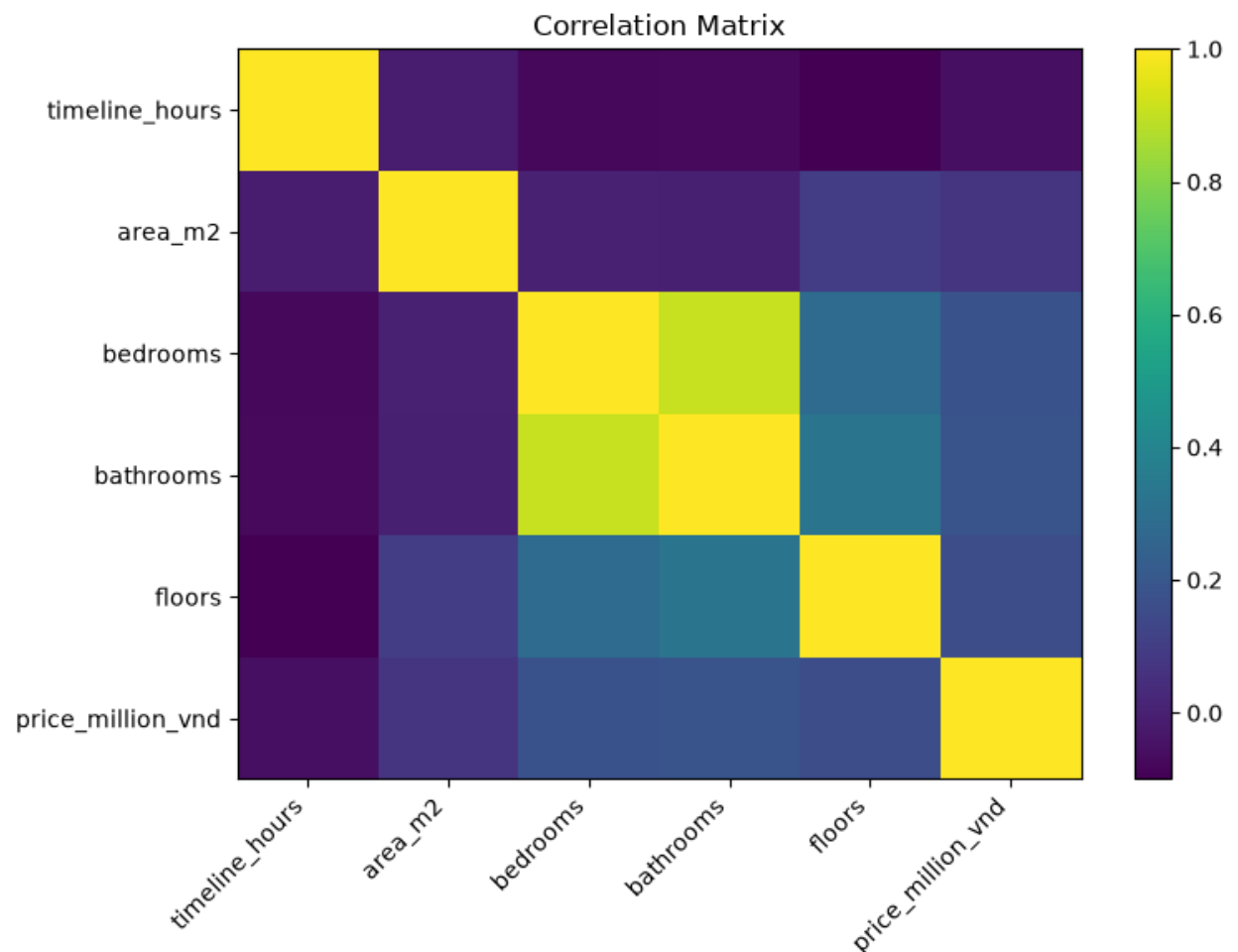
Code tạo lưới subplot 2×3, lặp qua danh sách `cols\_to\_plot`, rồi vẽ histogram với 30 bins. Sau đó tạo một lưới tương tự cho boxplot. Các ô dư được tắt bằng `axis("off")` nếu số biến ít hơn số subplot.

Các hình output thực tế giúp kiểm chứng trực quan những nhận xét từ thống kê và IQR. Đặc biệt, phân phối target và các feature về diện tích/phòng có thể cho thấy độ lệch và phạm vi giá trị, từ đó giải thích vì sao notebook sử dụng StandardScaler cho mô hình.

- Histogram phù hợp để nhìn phân phối và tần suất.
- Boxplot phù hợp để quan sát độ phân tán và giá trị cực đoan.

Output thực tế của cell

Hình output thực tế 1



Nhận xét và vai trò trong pipeline

Kết luận: Các hình output thực tế giúp kiểm chứng trực quan những nhận xét từ thống kê và IQR. Đặc biệt, phân phối target và các feature về diện tích/phòng có thể cho thấy độ lệch và phạm vi giá trị, từ đó giải thích vì sao notebook sử dụng StandardScaler cho mô hình.

6. Location analysis

Cell code 14

```
1 if "location" in df.columns:
2     print("Số location unique:", df["location"].nunique())
3     top_loc = df["location"].value_counts().head(15)
4     display(top_loc.to_frame("count"))
5
6     plt.figure(figsize=(10, 5))
7     plt.barh(top_loc.index.astype(str)[::-1], top_loc.values[::-1])
8     plt.title("Top 15 location phổ biến")
9     plt.xlabel("Số listing")
10    plt.tight_layout()
11    plt.show()
12
13    mean_price = (
14        df[df["location"].isin(top_loc.index)]
15        .groupby("location")[target]
16        .mean()
17        .sort_values(ascending=False)
18    )
19
20    plt.figure(figsize=(10, 5))
21    plt.barh(mean_price.index.astype(str)[::-1], mean_price.values[::-1])
22    plt.title("Giá trung bình theo location")
23    plt.xlabel("Giá trung bình (triệu VND)")
24    plt.tight_layout()
25    plt.show()
```

Giải thích chi tiết code

Cell này phân tích feature phân loại 'location', một biến có khả năng chứa thông tin quan trọng đối với giá nhà. Code đếm số location duy nhất, lấy 15 location xuất hiện nhiều nhất và hiển thị số listing của từng location.

Biểu đồ bar ngang trực quan hóa mức độ phổ biến. Sau đó code nhóm dữ liệu theo location và tính giá trung bình trên các location phổ biến. Việc giới hạn vào top location giúp biểu đồ dễ đọc và tập trung vào những nhóm có đủ số quan sát để có ý nghĩa mô tả tốt hơn.

Output thực tế cho thấy các khu vực có số lượng listing cao nhất và được sử dụng để giải thích vai trò của biến location. Vì location là categorical feature, notebook đưa nó vào OneHotEncoder ở preprocessing thay vì biến đổi tùy tiện thành số thứ tự.

- Location vừa ảnh hưởng đến phân phối dữ liệu vừa là feature đầu vào của mô hình.
- One-hot encoding tránh áp đặt quan hệ lớn/nhỏ giả tạo giữa tên các khu vực.

Output thực tế của cell

Hình output thực tế 1

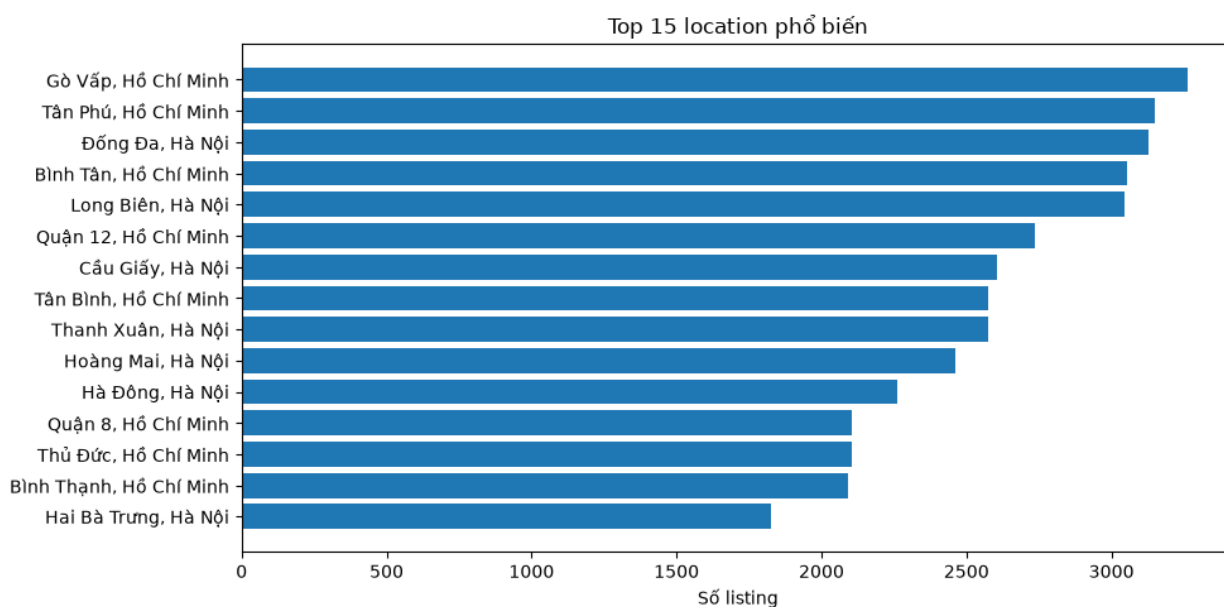
Output thực tế
Số `location` unique: 272

Hình output thực tế 2

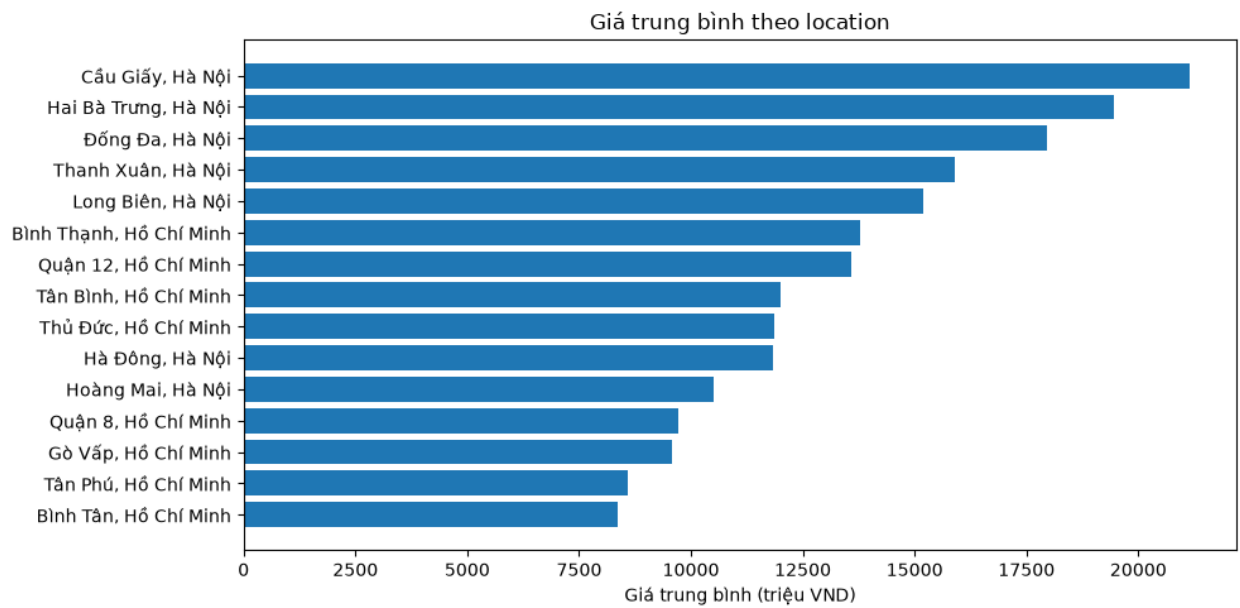
Output thực tế

location	count
Gò Vấp, Hồ Chí Minh	3261
Tân Phú, Hồ Chí Minh	3150
Đống Đa, Hà Nội	3127
Bình Tân, Hồ Chí Minh	3052
Long Biên, Hà Nội	3047
Quận 12, Hồ Chí Minh	2737
Cầu Giấy, Hà Nội	2605
Tân Bình, Hồ Chí Minh	2575
Thanh Xuân, Hà Nội	2573
Hoàng Mai, Hà Nội	2461
Hà Đông, Hà Nội	2260
Quận 8, Hồ Chí Minh	2104
Thủ Đức, Hồ Chí Minh	2103
Bình Thạnh, Hồ Chí Minh	2092
Hai Bà Trưng, Hà Nội	1827

Hình output thực tế 3



Hình output thực tế 4



Nhận xét và vai trò trong pipeline

Kết luận: Output thực tế cho thấy các khu vực có số lượng listing cao nhất và được sử dụng để giải thích vai trò của biến location. Vì location là categorical feature, notebook đưa nó vào OneHotEncoder ở preprocessing thay vì biến đổi tùy tiện thành số thứ tự.

7. Feature engineering và preprocessing

Cell code 16

```
1 if "frontage" in df.columns:
2     def encode_frontage(x):
3         s = str(x).strip().lower()
4         return 1 if s in ("true", "1", "yes", "y", "co", "có") else 0
5
6     df["frontage"] = df["frontage"].apply(encode_frontage)
7
8 numeric_features = [
9     c for c in [
10         "timeline_hours", "area_m2", "bedrooms",
11         "bathrooms", "floors", "frontage"
12     ] if c in df.columns
13 ]
14
15 categorical_features = ["location"] if "location" in df.columns else []
16
17 print("Numeric:", numeric_features)
18 print("Categorical:", categorical_features)
19
20 X = df[numeric_features + categorical_features].copy()
21 y = df[target].astype(np.float32).values
22
23 print("X shape:", X.shape)
24 print("y shape:", y.shape)
```

Giải thích chi tiết code

Cell này chuyển đổi feature `frontage` về dạng nhị phân và xác định hai nhóm feature: numeric và categorical. Hàm `encode\_frontage` chuẩn hóa chuỗi bằng `strip().lower()` rồi quy đổi các biểu diễn như true/1/yes/y/có về 1; các trường hợp còn lại về 0.

`numeric\_features` gồm các trường số có trong dataset: timeline_hours, area_m2, bedrooms, bathrooms, floors và frontage. `categorical\_features` chứa location nếu cột này tồn tại. Việc xây dựng danh sách bằng điều kiện `if c in df.columns` giúp code thích ứng với dataset thực tế mà không lỗi nếu một cột không tồn tại.

Output in ra danh sách numeric và categorical features. Đây là đầu vào trực tiếp cho `ColumnTransformer`, nơi mỗi nhóm được xử lý bằng pipeline phù hợp.

- Feature engineering làm cho frontage có biểu diễn nhất quán trước khi huấn luyện.
- Việc tách numeric/categorical là cần thiết vì hai loại dữ liệu cần các phép biến đổi khác nhau.

Output thực tế của cell

Hình output thực tế 1

```
Output thực tế
Numeric: ['timeline_hours', 'area_m2', 'bedrooms', 'bathrooms', 'floors', 'frontage']
Categorical: ['location']
X shape: (64447, 7)
y shape: (64447,)
```

Nhận xét và vai trò trong pipeline

Kết luận: Output in ra danh sách numeric và categorical features. Đây là đầu vào trực tiếp cho `ColumnTransformer`, nơi mỗi nhóm được xử lý bằng pipeline phù hợp.

8. Train / validation / test split

Cell code 18

```
1 X_train_full, X_test, y_train_full, y_test = train_test_split(
2     X, y, test_size=0.20, random_state=42
3 )
4
5 X_train, X_val, y_train, y_val = train_test_split(
6     X_train_full, y_train_full, test_size=0.20, random_state=42
7 )
8
9 print("Train:", X_train.shape)
10 print("Val:", X_val.shape)
11 print("Test:", X_test.shape)
```

Giải thích chi tiết code

Cell chia dữ liệu thành ba tập theo hai bước. Đầu tiên 80% được đưa vào `X\_train\_full, y\_train\_full` và 20% giữ lại làm test. Sau đó 20% của phần train_full được tách thành validation, tương đương khoảng 16% toàn bộ dữ liệu; phần train còn lại khoảng 64%.

`random\_state=42` giúp phép chia có thể tái lập. Validation được sử dụng trong quá trình lựa chọn và theo dõi mô hình, còn test được giữ riêng để đánh giá cuối cùng. Điều này hạn chế việc lựa chọn mô hình trực tiếp dựa trên test set.

Output in kích thước của ba tập. Các kích thước này là bằng chứng trực tiếp rằng dữ liệu đã được phân chia đúng trước khi preprocessing và huấn luyện.

- Test set không được dùng để tối ưu hyperparameter.
- Random seed cố định giúp kết quả có thể tái lập.

Output thực tế của cell

Hình output thực tế 1

```
Output thực tế
Train: (41245, 7)
Val: (10312, 7)
Test: (12890, 7)
```

Cell code 19

```

1 numeric_pipe = Pipeline([
2     ("imputer", SimpleImputer(strategy="median")),
3     ("scaler", StandardScaler())
4 ])
5
6 categorical_pipe = Pipeline([
7     ("imputer", SimpleImputer(strategy="most_frequent")),
8     ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False))
9 ])
10
11 transformers = [("num", numeric_pipe, numeric_features)]
12
13 if categorical_features:
14     transformers.append(("cat", categorical_pipe, categorical_features))
15
16 preprocessor = ColumnTransformer(transformers)
17
18 X_train_proc = preprocessor.fit_transform(X_train).astype(np.float32)
19 X_val_proc = preprocessor.transform(X_val).astype(np.float32)
20 X_test_proc = preprocessor.transform(X_test).astype(np.float32)
21
22 print("After preprocessing:", X_train_proc.shape)

```

Giải thích chi tiết code

Cell chia dữ liệu thành ba tập theo hai bước. Đầu tiên 80% được đưa vào `X\_train\_full, y\_train\_full` và 20% giữ lại làm test. Sau đó 20% của phần train_full được tách thành validation, tương đương khoảng 16% toàn bộ dữ liệu; phần train còn lại khoảng 64%.

`random\_state=42` giúp phép chia có thể tái lập. Validation được sử dụng trong quá trình lựa chọn và theo dõi mô hình, còn test được giữ riêng để đánh giá cuối cùng. Điều này hạn chế việc lựa chọn mô hình trực tiếp dựa trên test set.

Output in kích thước của ba tập. Các kích thước này là bằng chứng trực tiếp rằng dữ liệu đã được phân chia đúng trước khi preprocessing và huấn luyện.

- Test set không được dùng để tối ưu hyperparameter.
- Random seed cố định giúp kết quả có thể tái lập.

Output thực tế của cell

Hình output thực tế 1

Output thực tế
After preprocessing: (41245, 251)

Nhận xét và vai trò trong pipeline

Kết luận: Output in kích thước của ba tập. Các kích thước này là bằng chứng trực tiếp rằng dữ liệu đã được phân chia đúng trước khi preprocessing và huấn luyện.

9. ML baselines

Cell code 21

```
1 def evaluate(y_true, y_pred, name):
2     mse = mean_squared_error(y_true, y_pred)
3     return {
4         "Model": name,
5         "MSE": mse,
6         "RMSE": np.sqrt(mse),
7         "MAE": mean_absolute_error(y_true, y_pred),
8         "R2": r2_score(y_true, y_pred)
9     }
10
11 ml_models = {
12     "LinearRegression": LinearRegression(),
13     "Ridge": Ridge(alpha=1.0),
14     "RandomForest": RandomForestRegressor(
15         n_estimators=200,
16         random_state=42,
17         n_jobs=-1
18     )
19 }
20
21 ml_results = []
22 ml_predictions = {}
23
24 for name, model in ml_models.items():
25     model.fit(X_train_proc, y_train)
26
27     pred_val = model.predict(X_val_proc)
28     pred_test = model.predict(X_test_proc)
29
30     ml_predictions[name] = {
31         "val": pred_val,
32         "test": pred_test
33     }
34
35     ml_results.append(evaluate(y_val, pred_val, name + " (val)"))
36     ml_results.append(evaluate(y_test, pred_test, name + " (test)"))
37
38 ml_df = pd.DataFrame(ml_results)
39 display(ml_df)
```

Giải thích chi tiết code

Cell xây dựng các baseline machine learning gồm Linear Regression, Ridge và Random Forest. Hàm `evaluate` thống nhất cách tính MSE, RMSE, MAE và R^2 để các mô hình có thể được so sánh trên cùng một chuẩn.

Linear Regression là baseline tuyến tính đơn giản. Ridge bổ sung regularization thông qua `alpha=1.0`, giúp kiểm soát hệ số khi số lượng feature tăng. Random Forest sử dụng 200 cây với `random\_state=42` và `n\_jobs=-1`, phù hợp để bất quan hệ phi tuyến giữa các feature.

Output là bảng kết quả trên các tập đánh giá. Đây là baseline để về sau so sánh với mạng neural network tự cài đặt bằng NumPy.

- RMSE nhân mạnh sai số lớn và giữ cùng đơn vị với target.
- MAE dễ diễn giải theo mức sai lệch trung bình tuyệt đối.
- R^2 đo mức độ biến thiên của target được mô hình giải thích.

Output thực tế của cell

Hình output thực tế 1

Output thực tế					
	Model	MSE	RMSE	MAE	R2
0	LinearRegression (val)	1.013785e+09	31839.984925	10091.308594	-0.059770
1	LinearRegression (test)	1.465848e+09	38286.388600	11141.777344	-0.001427
2	Ridge (val)	8.959533e+08	29932.478681	9677.428711	0.063406
3	Ridge (test)	1.351731e+09	36765.890279	10740.538086	0.076535
4	RandomForest (val)	8.214453e+08	28660.866365	6729.682276	0.141294
5	RandomForest (test)	1.094300e+09	33080.200732	7349.411941	0.252405

Nhận xét và vai trò trong pipeline

Kết luận: Output là bảng kết quả trên các tập đánh giá. Đây là baseline để về sau so sánh với mạng neural network tự cài đặt bằng NumPy.

10. Deep Learning from scratch bằng NumPy

Kiến trúc:

`Input  $\rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 1`$

Hidden layers dùng ReLU, output dùng linear activation. Loss là MSE.

Optimizer hỗ trợ SGD, Momentum và Adam.

Cell code 23

```
1 def relu(x):
2     return np.maximum(0, x)
3
4 def relu_derivative(x):
5     return (x > 0).astype(x.dtype)
6
7 def mse_loss(y_true, y_pred):
8     return np.mean((y_true - y_pred) ** 2)
9
10 def mse_grad(y_true, y_pred):
11     n = y_true.shape[0]
12     return (2.0 / n) * (y_pred - y_true)
13
14 def he_init(fan_in, fan_out, rng):
15     return rng.standard_normal((fan_in, fan_out)) * np.sqrt(2.0 / fan_in)
```

Giải thích chi tiết code

Hai cell này là phần cốt lõi của Deep Learning. Notebook tự xây dựng MLP bằng NumPy thay vì dùng TensorFlow, Keras hoặc PyTorch. Kiến trúc được mô tả là Input $\rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 1$.

Cell hàm toán học định nghĩa ReLU, đạo hàm ReLU, MSE loss, gradient của MSE và He initialization. ReLU được dùng cho hidden layers để tạo tính phi tuyến. Output layer dùng linear activation vì đây là bài toán regression. He initialization sinh trọng số theo phân phối chuẩn với độ lệch chuẩn phụ thuộc fan-in, phù hợp với ReLU.

Class `MLPRegressor` quản lý toàn bộ mạng: khởi tạo trọng số/bias, forward propagation, backpropagation và optimizer. Các tham số như learning rate, optimizer, momentum, beta1, beta2, epsilon và seed được đưa vào constructor để có thể thử nghiệm nhiều cấu hình.

- Việc tự cài đặt giúp thể hiện rõ các thành phần cơ bản của một neural network.
- Adam, Momentum và SGD được hỗ trợ để phục vụ phần hyperparameter experiments.

Cell này không sinh output hiển thị trực tiếp trong notebook.

Cell code 24

```

1 class MLPRegressor:
2     """MLP regression tự cài đặt bằng NumPy."""
3
4     def __init__(
5         self,
6         input_dim,
7         hidden_dims=(64, 32, 16, 8),
8         init="he",
9         lr=0.01,
10        optimizer="adam",
11        momentum=0.9,
12        beta1=0.9,
13        beta2=0.999,
14        eps=1e-8,
15        seed=42
16    ):
17        self.rng = np.random.default_rng(seed)
18        self.input_dim = input_dim
19        self.hidden_dims = tuple(hidden_dims)
20        self.lr = lr
21        self.optimizer = optimizer.lower()
22        self.momentum = momentum
23        self.beta1 = beta1
24        self.beta2 = beta2
25        self.eps = eps
26
27        sizes = [input_dim] + list(hidden_dims) + [1]
28        self.sizes = sizes
29        self.L = len(sizes) - 1
30
31        self.W = []
32        self.b = []
33
34        for i in range(self.L):
35            if init == "he":
36                W = he_init(sizes[i], sizes[i + 1], self.rng)
37            else:
38                W = self.rng.standard_normal(
39                    (sizes[i], sizes[i + 1]))
40                * np.sqrt(1.0 / sizes[i])
41
42            self.W.append(W)
43            self.b.append(np.zeros((1, sizes[i + 1])))
44
45        self._init_optimizer_state()
46
47    def _init_optimizer_state(self):
48        if self.optimizer == "momentum":
49            self.vw = [np.zeros_like(w) for w in self.W]
50            self.vb = [np.zeros_like(b) for b in self.b]
51
52        elif self.optimizer == "adam":
53            self.mw = [np.zeros_like(w) for w in self.W]
54            self.mb = [np.zeros_like(b) for b in self.b]
55            self.vw = [np.zeros_like(w) for w in self.W]
56            self.vb = [np.zeros_like(b) for b in self.b]
57            self.t = 0
58
59    def forward(self, X):
60        cache = {"A": [X], "Z": []}
61        A = X
62
63        for i in range(self.L):
64            Z = A @ self.W[i] + self.b[i]
65            cache["Z"].append(Z)
66
67            if i < self.L - 1:
68                A = relu(Z)
69            else:
70                A = Z
71
72            cache["A"].append(A)
73
74        return A, cache
75
76    def backward(self, y, cache):
77        y = np.asarray(y).reshape(-1, 1)
78        y_pred = cache["A"][-1]
79        dA = mse_grad(y, y_pred)
80
81        dW = [None] * self.L
82        db = [None] * self.L
83
84        for i in reversed(range(self.L)):
85            if i == self.L - 1:

```

Giải thích chi tiết code

Hai cell này là phần cốt lõi của Deep Learning. Notebook tự xây dựng MLP bằng NumPy thay vì dùng TensorFlow, Keras hoặc PyTorch. Kiến trúc được mô tả là $\text{Input} \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 1$.

Cell hàm toán học định nghĩa ReLU, đạo hàm ReLU, MSE loss, gradient của MSE và He initialization. ReLU được dùng cho hidden layers để tạo tính phi tuyến. Output layer dùng linear activation vì đây là bài toán regression. He initialization sinh trọng số theo phân phối chuẩn với độ lệch chuẩn phụ thuộc fan-in, phù hợp với ReLU.

Class `'MLPRegressor'` quản lý toàn bộ mạng: khởi tạo trọng số/bias, forward propagation, backpropagation và optimizer. Các tham số như learning rate, optimizer, momentum, beta1, beta2, epsilon và seed được đưa vào constructor để có thể thử nghiệm nhiều cấu hình.

- Việc tự cài đặt giúp thể hiện rõ các thành phần cơ bản của một neural network.
- Adam, Momentum và SGD được hỗ trợ để phục vụ phần hyperparameter experiments.

Cell này không sinh output hiển thị trực tiếp trong notebook.

Nhận xét và vai trò trong pipeline

Kết luận: Class `'MLPRegressor'` quản lý toàn bộ mạng: khởi tạo trọng số/bias, forward propagation, backpropagation và optimizer. Các tham số như learning rate, optimizer, momentum, beta1, beta2, epsilon và seed được đưa vào constructor để có thể thử nghiệm nhiều cấu hình.

11. Scaling dữ liệu cho Deep Learning

Cell code 26

```
1 scaler_X = StandardScaler()
2
3 X_train_nn = scaler_X.fit_transform(X_train_proc).astype(np.float32)
4 X_val_nn = scaler_X.transform(X_val_proc).astype(np.float32)
5 X_test_nn = scaler_X.transform(X_test_proc).astype(np.float32)
6
7 y_mean = y_train.mean()
8 y_std = y_train.std() + 1e-8
9
10 y_train_nn = ((y_train - y_mean) / y_std).astype(np.float32)
11 y_val_nn = ((y_val - y_mean) / y_std).astype(np.float32)
12 y_test_nn = ((y_test - y_mean) / y_std).astype(np.float32)
13
14 print("X_train_nn:", X_train_nn.shape)
15 print("y_train_nn mean/std:", y_train_nn.mean(), y_train_nn.std())
```

Giải thích chi tiết code

Cell này chuẩn hóa feature đầu vào và target trước khi đưa vào mạng NumPy. `StandardScaler` được fit trên tập train rồi dùng transform cho validation/test, đảm bảo không làm rò rỉ thông tin từ test vào quá trình học.

Target cũng được chuẩn hóa thủ công bằng mean và std của `y\_train`. Công thức $(y - y_mean) / y_std$ đưa target về thang đo gần chuẩn hóa, giúp quá trình tối ưu MSE ổn định hơn. Các giá trị `y\_mean` và `y\_std` được lưu để sau này đưa dự đoán về đơn vị million VND.

Output cho biết kích thước ma trận đầu vào của mạng và các thông tin scaling. Đây là bước quan trọng vì gradient descent thường hoạt động ổn định hơn khi các feature có thang đo tương đồng.

- Scaler chỉ fit trên train.
- Mean/std của target được lấy từ train để tránh leakage.

Output thực tế của cell

Hình output thực tế 1

```
Output thực tế
X_train_nn: (41245, 251)
y_train_nn mean/std: 7.3990974e-10 1.0
```

Nhận xét và vai trò trong pipeline

Kết luận: Output cho biết kích thước ma trận đầu vào của mạng và các thông tin scaling. Đây là bước quan trọng vì gradient descent thường hoạt động ổn định hơn khi các feature có thang đo tương đồng.

12. Baseline Deep Learning

Cell code 29

```
1 np.random.seed(42)
2
3 model_base = MLPRegressor(
4     input_dim=X_train_nn.shape[1],
5     hidden_dims=(64, 32, 16, 8),
6     init="he",
7     lr=0.01,
8     optimizer="adam",
9     seed=42
10 )
11
12 hist_base = train_mlp(
13     model_base,
14     X_train_nn,
15     y_train_nn,
16     X_val_nn,
17     y_val_nn,
18     epochs=200,
19     batch_size=32,
20     verbose=True
21 )
22
23 y_pred_base = eval_dl(model_base, X_test_nn)
24 display(pd.DataFrame([evaluate(y_test, y_pred_base, "DL baseline (test)"])))
```

Giải thích chi tiết code

Cell khởi tạo mô hình MLP baseline với `hidden_dims=(64,32,16,8)`, He initialization, learning rate 0.01 và Adam. Sau đó `'train_mlp'` được gọi trong 200 epochs với batch size 32 và theo dõi cả train loss lẫn validation loss.

Sau huấn luyện, `'eval_dl'` tạo dự đoán trên test và hàm `'evaluate'` tính RMSE, MAE, R^2 . Bảng output là kết quả baseline DL ở test set. Cell tiếp theo trực quan hóa `'hist_base'` để kiểm tra quá trình học.

Learning curve cho phép quan sát liệu loss giảm đều, mô hình hội tụ, hay xuất hiện khoảng cách giữa train và validation. Đây là bằng chứng trực quan bổ sung cho các metric cuối cùng.

- Adam giúp điều chỉnh learning rate theo gradient và moment.
- Validation loss được theo dõi độc lập với train loss để phát hiện overfitting.

Output thực tế của cell

Hình output thực tế 1

Output thực tế

Epoch	train MSE	val MSE
1	0.92278	0.73678
21	0.60630	0.58651
41	0.61826	0.56886
61	0.60428	0.55960
81	0.56061	0.57448
101	0.54466	0.57701
121	0.53934	0.58078
141	0.59445	0.54319
161	0.60487	0.61912
181	0.54151	0.56164
200	0.55019	0.57375

Hình output thực tế 2

Output thực tế

Model	MSE	RMSE	MAE	R2
0 DL baseline (test)	1.109664e+09	33311.616325	8776.27734	0.241908

Cell code 30

```

1 plt.figure(figsize=(10, 5))
2 plt.plot(hist_base["train_loss"], label="Train")
3 plt.plot(hist_base["val_loss"], label="Validation")
4 plt.xlabel("Epoch")
5 plt.ylabel("MSE (normalized)")
6 plt.title("Learning curves - Baseline DL")
7 plt.legend()
8 plt.tight_layout()
9 plt.show()

```

Giải thích chi tiết code

Cell khởi tạo mô hình MLP baseline với `hidden_dims=(64,32,16,8)`, He initialization, learning rate 0.01 và Adam. Sau đó `'train_mlp'` được gọi trong 200 epochs với batch size 32 và theo dõi cả train loss lẫn validation loss.

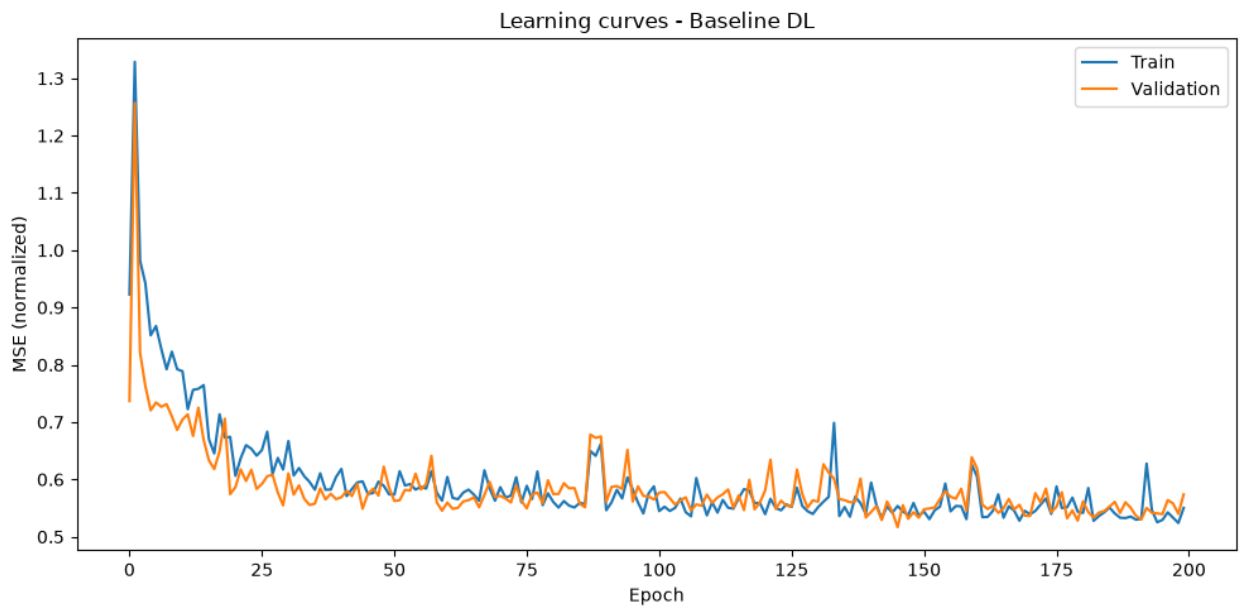
Sau huấn luyện, `'eval_dl'` tạo dự đoán trên test và hàm `'evaluate'` tính RMSE, MAE, R². Bảng output là kết quả baseline DL ở test set. Cell tiếp theo trực quan hóa `'hist_base'` để kiểm tra quá trình học.

Learning curve cho phép quan sát liệu loss giảm đều, mô hình hội tụ, hay xuất hiện khoảng cách giữa train và validation. Đây là bằng chứng trực quan bổ sung cho các metric cuối cùng.

- Adam giúp điều chỉnh learning rate theo gradient và moment.
- Validation loss được theo dõi độc lập với train loss để phát hiện overfitting.

Output thực tế của cell

Hình output thực tế 1



Nhận xét và vai trò trong pipeline

Kết luận: Learning curve cho phép quan sát liệu loss giảm đều, mô hình hội tụ, hay xuất hiện khoảng cách giữa train và validation. Đây là bằng chứng trực quan bổ sung cho các metric cuối cùng.

13. Hyperparameter experiments

So sánh:

- độ sâu/width
- learning rate
- optimizer
- batch size

Cell code 32

```

1 # =====
2 # CELL: HYPERPARAMETER EXPERIMENTS (TÔ'I ƯU TÔ'C ĐỘ)
3 # =====
4 import time
5 import numpy as np
6 import pandas as pd
7 from IPython.display import display
8
9 # -----
10 # 1. CẤU HÌNH EXPERIMENTS (rút gọn còn 6, đủ để so sánh)
11 # -----
12 experiments = [
13     {"name": "base", "hidden": (64, 32, 16, 8), "lr": 0.01, "opt": "adam", "bs": 64},
14     {"name": "deep", "hidden": (128, 64, 32, 16), "lr": 0.01, "opt": "adam", "bs": 64},
15     {"name": "lr_small", "hidden": (64, 32, 16, 8), "lr": 0.001, "opt": "adam", "bs": 64},
16     {"name": "sgd", "hidden": (64, 32, 16, 8), "lr": 0.01, "opt": "sgd", "bs": 64},
17     {"name": "momentum", "hidden": (64, 32, 16, 8), "lr": 0.01, "opt": "momentum", "bs": 64},
18     {"name": "batch_16", "hidden": (64, 32, 16, 8), "lr": 0.01, "opt": "adam", "bs": 16},
19 ]
20
21 EPOCHS = 60 # gia'm từ 200 -> 60
22 EVAL_EVERY = 5 # chi' eval validation mỗ'i 5 epoch
23
24 # -----
25 # 2. TRAINING LOOP TÔ'I ƯU
26 # -----
27 def train_mlp_fast(model, X_train, y_train, X_val, y_val,
28                     epochs=60, batch_size=64,
29                     eval_every=5, verbose=True):
30     """
31     Tô'i ưu tô'c độ:
32     - Không forward toàn bộ train mỗ'i epoch.
33     - Train loss = trung bình loss các batch trong epoch.
34     - Val loss chỉ tính mỗ'i 'eval_every' epochs.
35     """
36     n = X_train.shape[0]
37     y_train_col = y_train.reshape(-1, 1)
38     y_val_col = y_val.reshape(-1, 1)
39
40     history = {"train_loss": [], "val_loss": []}
41
42     for epoch in range(epochs):
43         idx = np.random.permutation(n)
44         X_sh = X_train[idx]
45         y_sh = y_train_col[idx]
46
47         epoch_loss = 0.0
48         n_batches = 0
49
50         for i in range(0, n, batch_size):
51             xb = X_sh[i:i + batch_size]
52             yb = y_sh[i:i + batch_size]
53
54             y_pred, cache = model.forward(xb)
55             epoch_loss += mse_loss(yb, y_pred)
56             n_batches += 1
57
58             dw, db = model.backward(yb, cache)
59             model.update(dw, db)
60
61         train_loss = epoch_loss / max(n_batches, 1)
62         history["train_loss"].append(train_loss)
63
64         # Chi' eval validation định kỳ
65         if (epoch + 1) % eval_every == 0 or epoch == epochs - 1:
66             y_pred_v = model.forward(X_val)[0]
67             v_loss = mse_loss(y_val_col, y_pred_v)
68             history["val_loss"].append(v_loss)
69             if verbose:
70                 print(f" Epoch {epoch+1:3d}/{epochs} | "
71                       f"train={train_loss:.5f} | val={v_loss:.5f}")
72         else:
73             history["val_loss"].append(np.nan)
74
75     return history
76
77 # -----
78 # 3. CHẠY TẮT CA' EXPERIMENTS
79 # -----
80 dl_results = []
81 dl_histories = {}
82 dl_models = {}
83
84 total_start = time.time()
85
86 for k, exp in enumerate(experiments, 1):
87     print(f"\n[{k}/{len(experiments)}] Running: {exp['name']}")
88     t0 = time.time()
89
90     np.random.seed(42)
91     model = MLPRegressor(
92         input_dim=X_train.nn.shape[1],
93         hidden_dims=exp["hidden"],
94         init="he",
95         lr=exp["lr"],

```

Giải thích chi tiết code

Cell này thiết kế một tập thí nghiệm để so sánh ảnh hưởng của kiến trúc, learning rate, optimizer và batch size. Danh sách `experiments` được cấu hình thành nhiều tên thử nghiệm như base, shallow, deep, lr_small, lr_large, sgd, momentum, batch_16 và batch_128.

Mỗi experiment tạo một MLP với cấu hình tương ứng, huấn luyện trong số epoch đã định và ghi lại thời gian cũng như kết quả. Việc gom kết quả vào DataFrame giúp so sánh định lượng giữa các cấu hình thay vì chọn mô hình chỉ dựa trên cảm tính.

Output gồm bảng kết quả thực nghiệm và thời gian chạy. Đây là cơ sở để chọn cấu hình DL tốt nhất theo RMSE trên test set và tiếp tục phân tích learning curve ở các cell sau.

- So sánh có kiểm soát giúp xác định tác động của từng nhóm hyperparameter.
- Thời gian chạy cũng được ghi nhận để đánh đổi giữa độ chính xác và chi phí tính toán.

Output thực tế của cell

Hình output thực tế 1

Output thực tế

```
[1/6] Running: base
Epoch 5/60 | t_rain=0.93682 | val=0.72352
Epoch 10/60 | t_rain=0.87113 | val=0.67292
Epoch 15/60 | t_rain=0.81398 | val=0.62864
Epoch 20/60 | t_rain=0.77259 | val=0.62880
Epoch 25/60 | t_rain=0.77112 | val=0.62484
Epoch 30/60 | t_rain=0.69747 | val=0.58647
Epoch 35/60 | t_rain=0.69758 | val=0.59845
Epoch 40/60 | t_rain=0.67447 | val=0.57905
Epoch 45/60 | t_rain=0.66252 | val=0.55493
Epoch 50/60 | t_rain=0.65557 | val=0.73285
Epoch 55/60 | t_rain=0.66853 | val=0.58045
Epoch 60/60 | t_rain=0.65638 | val=0.57195
-> done in 132.6s

[2/6] Running: deep
Epoch 5/60 | t_rain=0.90256 | val=0.72838
Epoch 10/60 | t_rain=0.85334 | val=0.66495
Epoch 15/60 | t_rain=0.82256 | val=0.71717
Epoch 20/60 | t_rain=0.72974 | val=0.60619
Epoch 25/60 | t_rain=0.68786 | val=0.62757
Epoch 30/60 | t_rain=0.79181 | val=0.57963
Epoch 35/60 | t_rain=0.63719 | val=0.59586
Epoch 40/60 | t_rain=0.65059 | val=0.59440
Epoch 45/60 | t_rain=0.61518 | val=0.58488
Epoch 50/60 | t_rain=0.60851 | val=0.58484
Epoch 55/60 | t_rain=0.63233 | val=0.62062
Epoch 60/60 | t_rain=0.59654 | val=0.59460
-> done in 258.2s

[3/6] Running: lr_small
Epoch 5/60 | t_rain=0.81281 | val=0.69029
Epoch 10/60 | t_rain=0.73478 | val=0.64261
Epoch 15/60 | t_rain=0.66022 | val=0.61860
Epoch 20/60 | t_rain=0.61704 | val=0.60711
Epoch 25/60 | t_rain=0.58505 | val=0.57877
Epoch 30/60 | t_rain=0.56691 | val=0.57408
Epoch 35/60 | t_rain=0.54625 | val=0.54788
Epoch 40/60 | t_rain=0.54259 | val=0.59512
Epoch 45/60 | t_rain=0.53382 | val=0.54510
Epoch 50/60 | t_rain=0.53899 | val=0.55379
Epoch 55/60 | t_rain=0.52415 | val=0.54983
Epoch 60/60 | t_rain=0.51730 | val=0.54349
-> done in 104.0s

[4/6] Running: sgd
Epoch 5/60 | t_rain=0.88649 | val=0.70993
Epoch 10/60 | t_rain=0.82772 | val=0.69356
Epoch 15/60 | t_rain=0.83596 | val=0.69970
Epoch 20/60 | t_rain=0.78651 | val=0.70065
Epoch 25/60 | t_rain=0.78375 | val=0.68625
Epoch 30/60 | t_rain=0.76381 | val=0.68827
Epoch 35/60 | t_rain=0.74905 | val=0.66802
Epoch 40/60 | t_rain=0.70530 | val=0.63617
Epoch 45/60 | t_rain=0.68588 | val=0.81915
Epoch 50/60 | t_rain=0.68300 | val=0.64067
Epoch 55/60 | t_rain=0.67924 | val=0.62029
Epoch 60/60 | t_rain=0.65838 | val=0.60497
-> done in 95.4s

[5/6] Running: momentum
Epoch 5/60 | t_rain=1.00060 | val=0.77123
Epoch 10/60 | t_rain=1.00139 | val=0.77158
Epoch 15/60 | t_rain=1.00066 | val=0.77047
Epoch 20/60 | t_rain=1.00125 | val=0.77008
Epoch 25/60 | t_rain=1.00151 | val=0.78499
Epoch 30/60 | t_rain=1.00164 | val=0.77349
Epoch 35/60 | t_rain=1.00091 | val=0.77068
Epoch 40/60 | t_rain=1.00056 | val=0.77426
Epoch 45/60 | t_rain=1.00459 | val=0.77301
Epoch 50/60 | t_rain=1.00122 | val=0.77079
Epoch 55/60 | t_rain=1.00074 | val=0.77334
Epoch 60/60 | t_rain=1.00319 | val=0.77045
-> done in 207.7s

[6/6] Running: batch_16
Epoch 5/60 | t_rain=1.17640 | val=0.71592
Epoch 10/60 | t_rain=0.88959 | val=0.68512
Epoch 15/60 | t_rain=0.75935 | val=0.64882
Epoch 20/60 | t_rain=0.75542 | val=0.66659
Epoch 25/60 | t_rain=0.71569 | val=0.61826
Epoch 30/60 | t_rain=0.71154 | val=0.62558
Epoch 35/60 | t_rain=0.69560 | val=0.58518
Epoch 40/60 | t_rain=0.69227 | val=0.64636
Epoch 45/60 | t_rain=0.72204 | val=0.63395
Epoch 50/60 | t_rain=0.67267 | val=0.57458
Epoch 55/60 | t_rain=0.65782 | val=0.65976
Epoch 60/60 | t_rain=0.67918 | val=0.56434
-> done in 265.1s

TOTAL TIME: 1062.9s

=== TEST RESULTS (sorted by RMSE) ===
```

Hình output thực tế 2

```

Output thực tế
      Model      MSE      RMSE      MAE      R2      params \
0      deep  1.065931e+09  32648.594018  8812.401680  0.271786  43137
1  lr_small  1.095117e+09  33092.558583  8194.972123  0.251846  18881
2  batch_16  1.103275e+09  33215.580964  9088.948989  0.246273  18881
3      base  1.121027e+09  33481.737133  9516.155026  0.234146  18881
4      sgd   1.202015e+09  34670.084015  9699.930913  0.178817  18881
5  momentum  1.467683e+09  38310.353278  11499.187487 -0.002681  18881

      time_s
0    258.19
1    104.00
2    265.12
3    132.55
4     95.43
5    207.65

```

Nhận xét và vai trò trong pipeline

Kết luận: Output gồm bảng kết quả thực nghiệm và thời gian chạy. Đây là cơ sở để chọn cấu hình DL tốt nhất theo RMSE trên test set và tiếp tục phân tích learning curve ở các cell sau.

14. So sánh learning curves

Cell code 34

```
plt.figure(figsize=(14, 6))

for name, history in dl_histories.items():
    # Nếu history có 'val_epoch' -> dùng nó
    if "val_epoch" in history and len(history["val_epoch"]) > 0:
        plt.plot(history["val_epoch"], history["val_loss"],
                 marker="o", markersize=4, label=name)
    else:
        # Fallback: bỏ NaN
        import pandas as pd
        s = pd.Series(history["val_loss"]).dropna()
        x = (s.index + 1) * 5 # 5 = eval_every
        plt.plot(x, s.values, marker="o", markersize=4, label=name)

plt.xlabel("Epoch")
plt.ylabel("Validation MSE")
plt.title("Validation learning curves")
plt.legend(ncol=2)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Giải thích chi tiết code

Cell này vẽ validation loss của tất cả cấu hình trong `dl\_histories` trên cùng một biểu đồ. Trục X là epoch và trục Y là validation MSE.

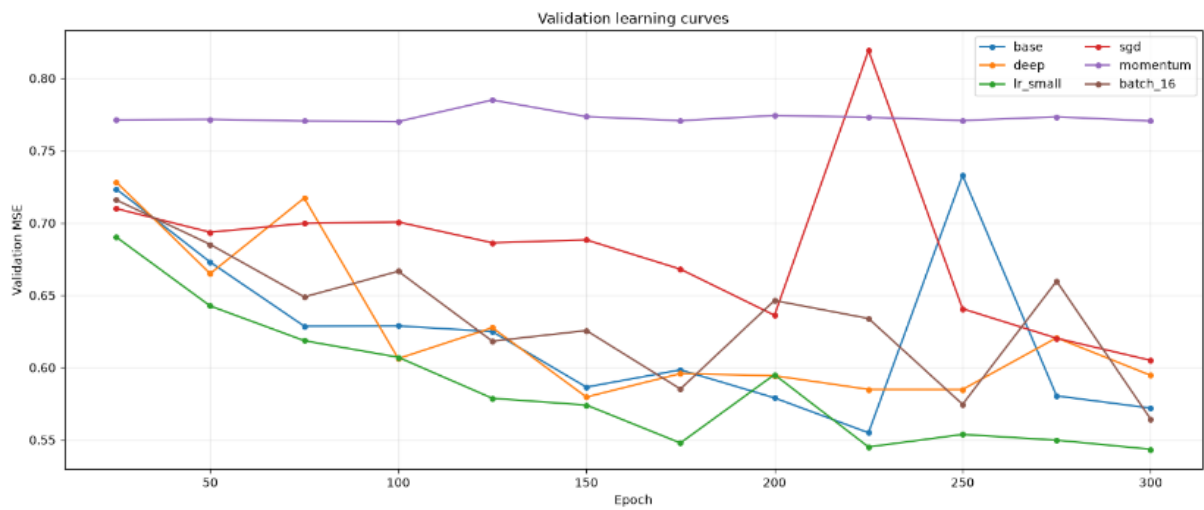
Việc đặt nhiều đường cong trên cùng một figure cho phép so sánh tốc độ hội tụ và mức loss đạt được. Cấu hình có validation loss thấp và ổn định hơn thường là ứng viên tốt, nhưng notebook vẫn dùng RMSE trên test để xác định mô hình cuối cùng.

Output giúp nhìn trực quan tác động của các cấu hình đã thử ở cell hyperparameter.

- Validation curve thấp hơn không tự động có nghĩa là test tốt nhất; test metric vẫn là tiêu chí cuối.

Output thực tế của cell

Hình output thực tế 1



Nhận xét và vai trò trong pipeline

Kết luận: Output giúp nhìn trực quan tác động của các cấu hình đã thử ở cell hyperparameter.

15. Chọn best Deep Learning model

Cell code 36

```
1 best_row = dl_df_test.sort_values("RMSE").iloc[0]
2 best_name = best_row["Model"]
3 best_model = dl_models[best_name]
4
5 print("Best DL config theo RMSE:")
6 display(best_row.to_frame().T)
```

Giải thích chi tiết code

Cell sắp xếp `dl\_df\_test` theo RMSE tăng dần và lấy dòng đầu tiên làm `best\_row`. Từ tên mô hình, code lấy đúng object trong dictionary `dl\_models` để tạo `best\_model`.

RMSE được chọn làm tiêu chí chính vì đây là metric phản ánh sai số bình phương trung bình sau khi đưa về căn bậc hai, đồng thời nhạy với các dự đoán lệch nhiều. Output hiển thị cấu hình DL tốt nhất cùng các metric đi kèm.

Kết quả của cell là cầu nối giữa giai đoạn thử nghiệm và giai đoạn phân tích cuối: từ đây các biểu đồ Actual vs Predicted, residual và K-Fold đều sử dụng `best\_model`.

- Mô hình tốt nhất được chọn từ kết quả thực nghiệm, không phải từ tên kiến trúc.
- Các metric MAE và R^2 vẫn được giữ lại để đánh giá đa chiều.

Output thực tế của cell

Hình output thực tế 1

Output thực tế
Best DL config theo RMSE:

Hình output thực tế 2

Output thực tế

Model	MSE	RMSE	MAE	R2	params	time_s
3_deep	1065930691.378839	32648.594018	8812.40168	0.271786	43137	258.19

Nhận xét và vai trò trong pipeline

Kết luận: Kết quả của cell là cầu nối giữa giai đoạn thử nghiệm và giai đoạn phân tích cuối: từ đây các biểu đồ Actual vs Predicted, residual và K-Fold đều sử dụng `best\_model`.

16. Final comparison ML vs DL

Cell code 38

```
1 ml_test = ml_df[ml_df["Model"].str.endswith("(test)").copy()
2 ml_test["Model"] = ml_test["Model"].str.replace(" (test)", "", regex=False)
3
4 ml_best = ml_test.sort_values("RMSE").iloc[0].copy()
5 dl_best = best_row.copy()
6
7 comparison = pd.DataFrame([
8     {
9         "Type": "ML",
10        "Model": ml_best["Model"],
11        "RMSE": ml_best["RMSE"],
12        "MAE": ml_best["MAE"],
13        "R2": ml_best["R2"]
14    },
15    {
16        "Type": "DL",
17        "Model": dl_best["Model"],
18        "RMSE": dl_best["RMSE"],
19        "MAE": dl_best["MAE"],
20        "R2": dl_best["R2"]
21    }
22 ])
23
24 display(comparison.sort_values("RMSE"))
```

Giải thích chi tiết code

Cell lấy các dòng ML trên test set, loại hậu tố `(test)` rồi chọn mô hình ML có RMSE thấp nhất. Sau đó ghép kết quả đó với `best\_row` của DL thành bảng `comparison` gồm Type, Model, RMSE, MAE và R^2 .

Cách tổ chức này đặt ML và DL trên cùng một mặt bằng đánh giá. Điều quan trọng là cả hai nhóm đều được so sánh trên test set và dùng cùng các metric.

Output cho biết mô hình nào trong hai nhóm có sai số thấp hơn và R^2 cao hơn. Đây là kết luận trung gian trước khi đi sâu vào phân tích residual và final leaderboard.

- So sánh cùng test set giúp kết luận công bằng hơn.

Output thực tế của cell

Hình output thực tế 1

Output thực tế

	Type	Model	RMSE	MAE	R2
1	DL	deep	32648.594018	8812.401680	0.271786
0	ML	RandomForest	33080.200732	7349.411941	0.252405

Nhận xét và vai trò trong pipeline

Kết luận: Output cho biết mô hình nào trong hai nhóm có sai số thấp hơn và R^2 cao hơn. Đây là kết luận trung gian trước khi đi sâu vào phân tích residual và final leaderboard.

17. Best DL — Actual vs Predicted

Cell code 40

```
1 y_pred_best_dl = eval_dl(best_model, X_test_nn)
2
3 plt.figure(figsize=(7, 7))
4 plt.scatter(y_test, y_pred_best_dl, alpha=0.4, s=15)
5
6 lims = [
7     min(y_test.min(), y_pred_best_dl.min()),
8     max(y_test.max(), y_pred_best_dl.max())
9 ]
10
11 plt.plot(lims, lims, "r--", lw=1.5)
12 plt.xlabel("Actual price (million VND)")
13 plt.ylabel("Predicted price (million VND)")
14 plt.title(f"Best DL ({best_name}) - Test")
15 plt.tight_layout()
16 plt.show()
```

Giải thích chi tiết code

Cell tạo dự đoán cuối của `best\_model` trên test set rồi vẽ scatter plot giữa giá thực tế và giá dự đoán. Đường chéo $y=x$ biểu diễn trường hợp dự đoán hoàn hảo.

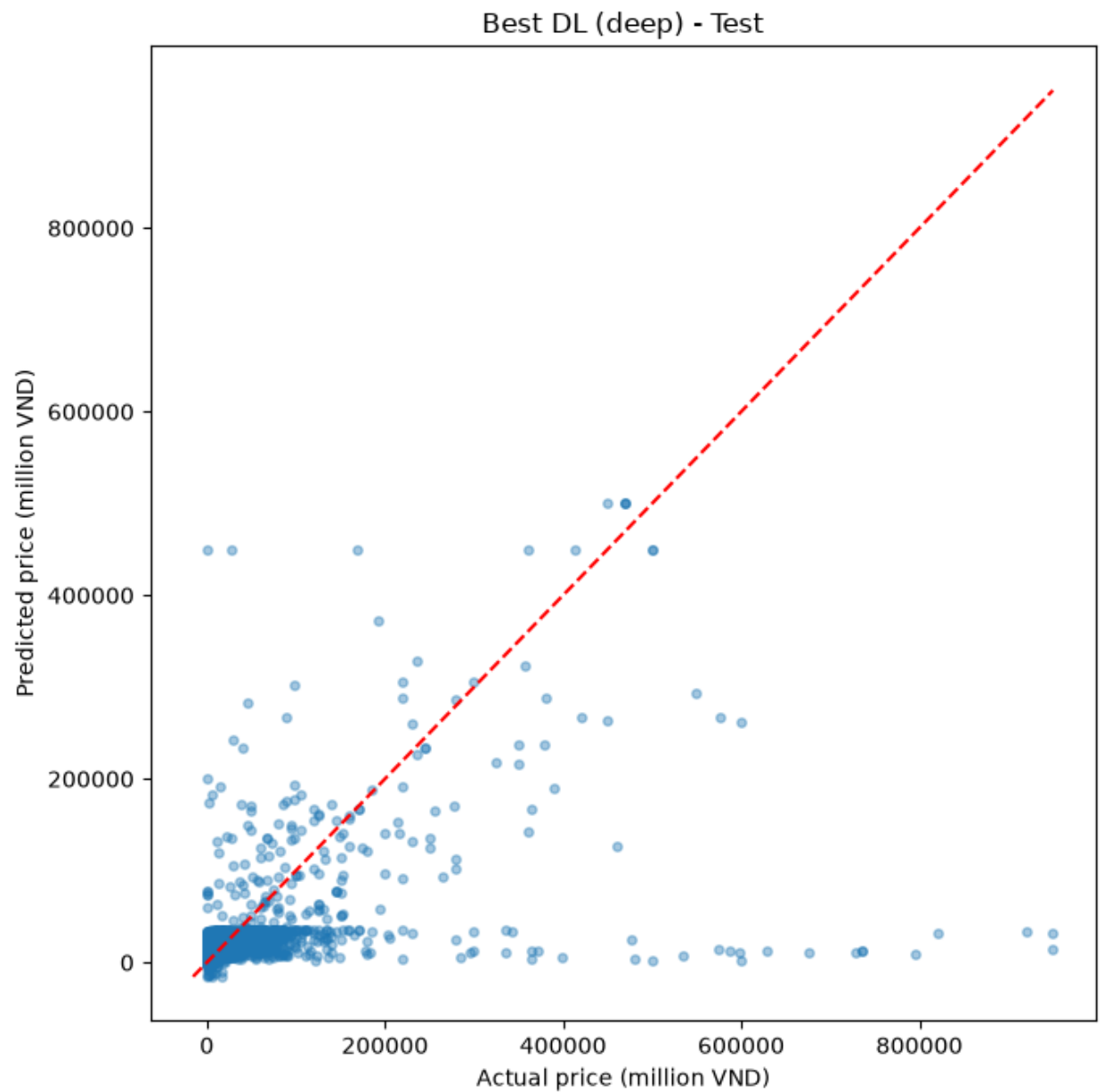
Nếu điểm nằm gần đường chéo, dự đoán gần giá thực tế. Nếu điểm phân tán rộng hoặc lệch hệ thống, mô hình còn sai số đáng kể. Trục được đặt theo cùng đơn vị million VND nên biểu đồ trực tiếp phản ánh mức độ khớp giữa actual và predicted.

Output là bằng chứng trực quan cho chất lượng mô hình best DL, bổ sung cho RMSE/MAE/ R^2 .

- Đường $y=x$ là mốc tham chiếu, không phải đường hồi quy được fit thêm.

Output thực tế của cell

Hình output thực tế 1



Nhận xét và vai trò trong pipeline

Kết luận: Output là bằng chứng trực quan cho chất lượng mô hình best DL, bổ sung cho RMSE/MAE/R².

18. Residual analysis

Cell code 42

```
1 residuals = y_test - y_pred_best_dl
2
3 plt.figure(figsize=(10, 5))
4 plt.scatter(y_pred_best_dl, residuals, alpha=0.4, s=15)
5 plt.axhline(0, color="red", linestyle="--")
6 plt.xlabel("Predicted")
7 plt.ylabel("Residual")
8 plt.title("Residual plot - Best DL")
9 plt.tight_layout()
10 plt.show()
11
12 plt.figure(figsize=(8, 4))
13 plt.hist(residuals, bins=30)
14 plt.title("Phân phối residuals")
15 plt.xlabel("Residual")
16 plt.ylabel("Count")
17 plt.tight_layout()
18 plt.show()
```

Giải thích chi tiết code

Residual được tính bằng `y_test - y_pred_best_dl`. Cell vẽ hai biểu đồ: residual theo predicted value và histogram của residual.

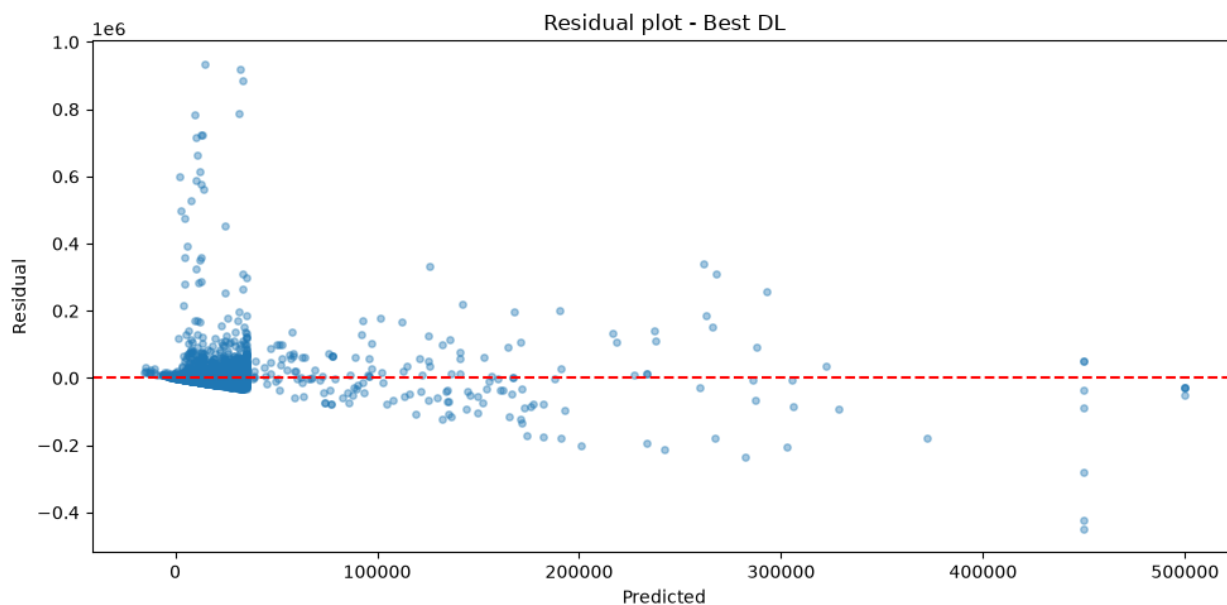
Biểu đồ residual giúp kiểm tra xem sai số có tập trung quanh 0 hay có xu hướng tăng/giảm theo mức dự đoán. Histogram cho thấy phân phối của sai số và mức độ lệch. Trong mô hình tốt, residual thường không nên tạo ra một pattern rõ rệt.

Output thực tế cho phép đánh giá những vùng mà mô hình có xu hướng dự đoán cao/thấp hơn thực tế. Đây là bước chẩn đoán quan trọng vì một metric tổng hợp như RMSE không cho biết sai số phân bố như thế nào.

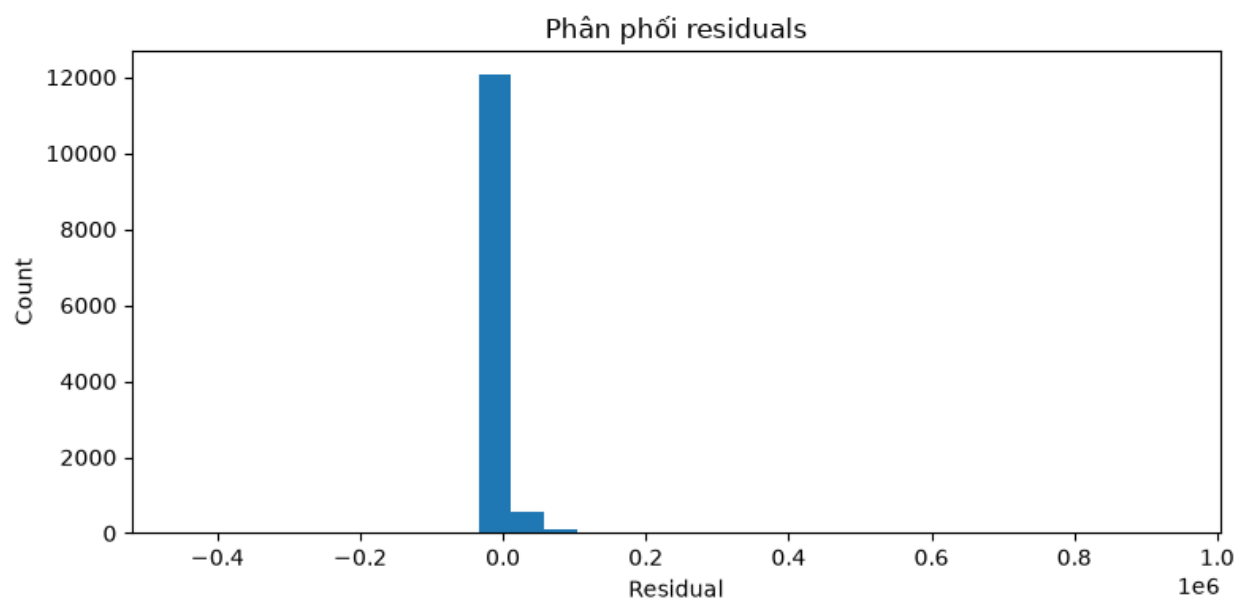
- Residual dương nghĩa là mô hình dự đoán thấp hơn thực tế.
- Residual âm nghĩa là mô hình dự đoán cao hơn thực tế.

Output thực tế của cell

Hình output thực tế 1



Hình output thực tế 2



Nhận xét và vai trò trong pipeline

Kết luận: Output thực tế cho phép đánh giá những vùng mà mô hình có xu hướng dự đoán cao/thấp hơn thực tế. Đây là bước chẩn đoán quan trọng vì một metric tổng hợp như RMSE không cho biết sai số phân bố như thế nào.

19. Ảnh hưởng của learning rate / optimizer / batch size / depth

Cell code 44

```
groups = {
    "Learning rate": ["lr_small", "base", "lr_large"],
    "Optimizer":     ["sgd", "momentum", "base"],
    "Batch size":    ["batch_16", "base", "batch_128"],
    "Depth / width": ["shallow", "base", "deep"]
}

EVAL_EVERY = 5 # phải khớp với lúc train

for title, names in groups.items():
    plt.figure(figsize=(12, 5))

    for name in names:
        if name not in dl_histories:
            continue

        h = dl_histories[name]
        val_loss = h["val_loss"]

        # Trường hợp 1: history mới có val_epoch -> dùng trực tiếp
        if "val_epoch" in h and len(h["val_epoch"]) == len(val_loss):
            x = h["val_epoch"]
            y = val_loss
        else:
            # Trường hợp 2: history cũ, có NaN xen kẽ -> lọc NaN
            arr = np.array(val_loss, dtype=float)
            mask = ~np.isnan(arr)
            # epoch tương ứng: vị trí i (0-based) -> epoch i+1
            # nếu bạn append NaN ở các epoch không eval, thì epoch eval là i+1
            x = np.arange(1, len(arr) + 1)[mask]
            y = arr[mask]

        plt.plot(x, y, marker="o", markersize=4,
                 linewidth=1.8, label=name)

    plt.xlabel("Epoch")
    plt.ylabel("Validation MSE (normalized)")
    plt.title(f"Ảnh hưởng của {title}")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()
```

Giải thích chi tiết code

Cell gom các experiment thành bốn nhóm: Learning rate, Optimizer, Batch size và Depth/width. Với mỗi nhóm, code vẽ validation loss của ba cấu hình tương ứng trên cùng một figure.

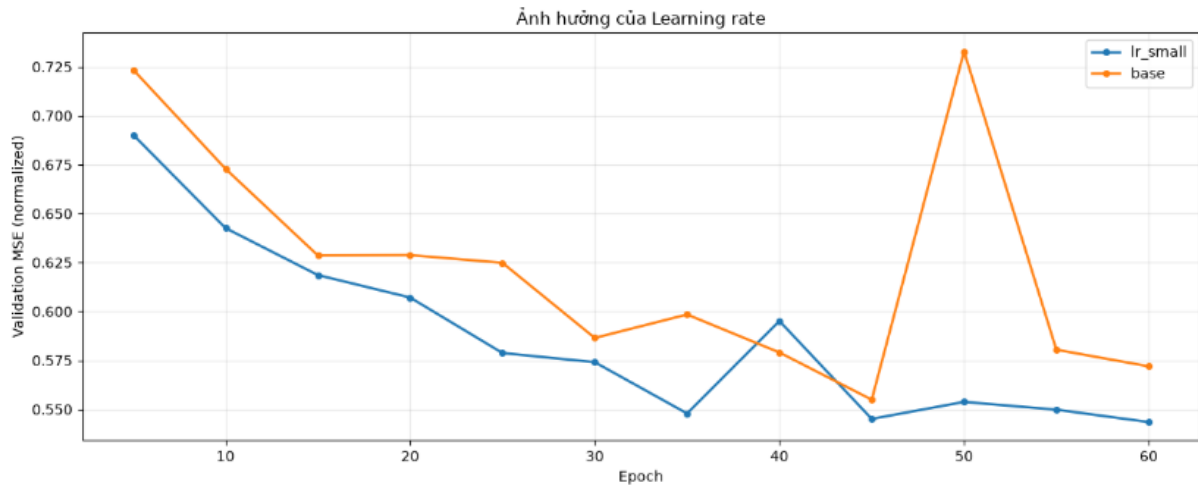
Cách nhóm này giúp phân tích nguyên nhân thay đổi hiệu năng. Learning rate quá nhỏ có thể làm hội tụ chậm; learning rate lớn có thể làm loss dao động. Optimizer khác nhau tạo ra tốc độ cập nhật khác nhau. Batch size ảnh hưởng đến nhiễu của gradient và tốc độ huấn luyện. Depth/width quyết định năng lực biểu diễn của mạng.

Các output là bốn biểu đồ thực tế dùng để giải thích vì sao một cấu hình có thể vượt cấu hình khác. Kết luận cuối vẫn cần đối chiếu với bảng metric test để tránh chọn mô hình chỉ dựa trên hình dạng curve.

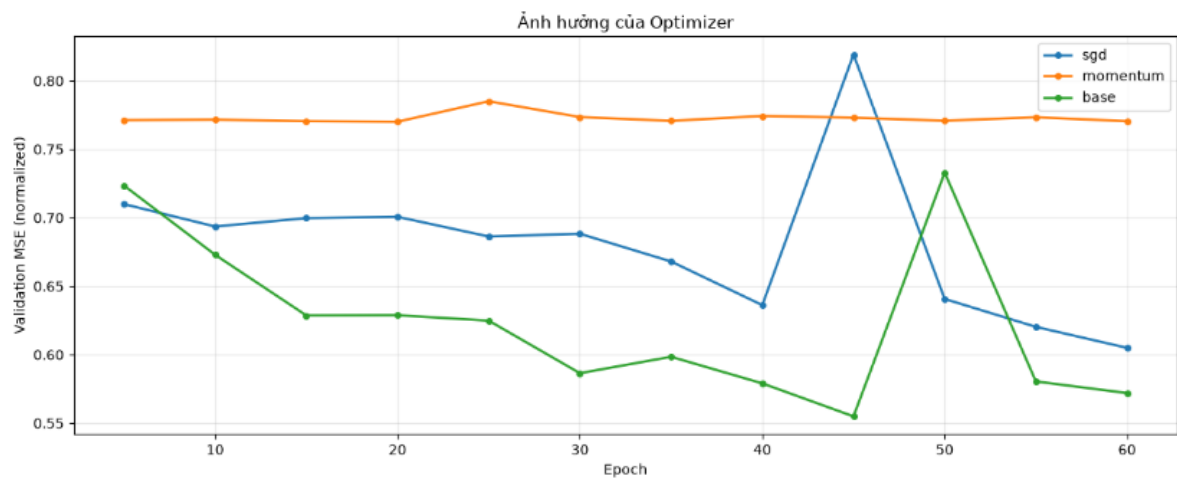
- Mỗi figure trả lời một câu hỏi về một nhóm hyperparameter.

Output thực tế của cell

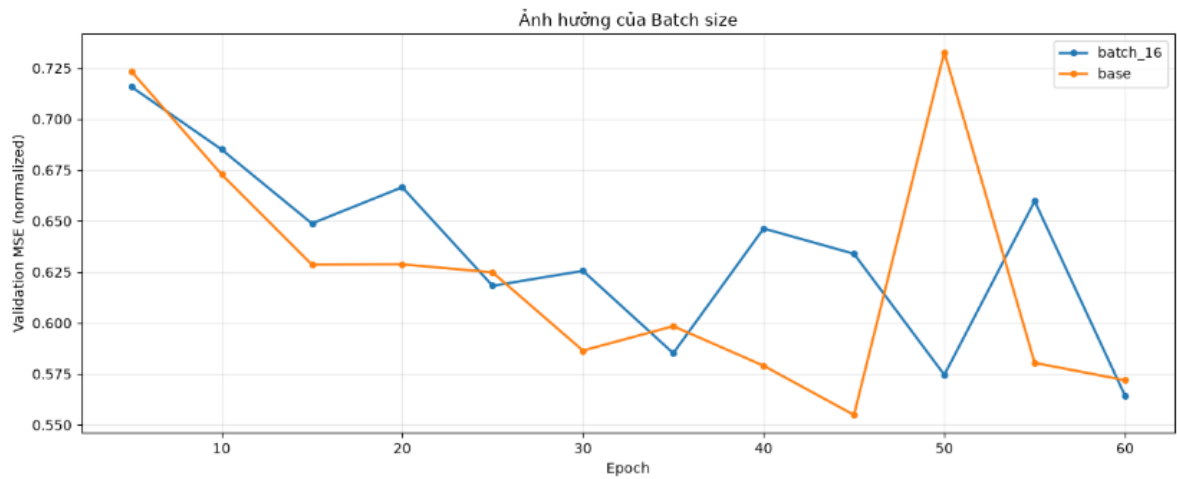
Hình output thực tế 1



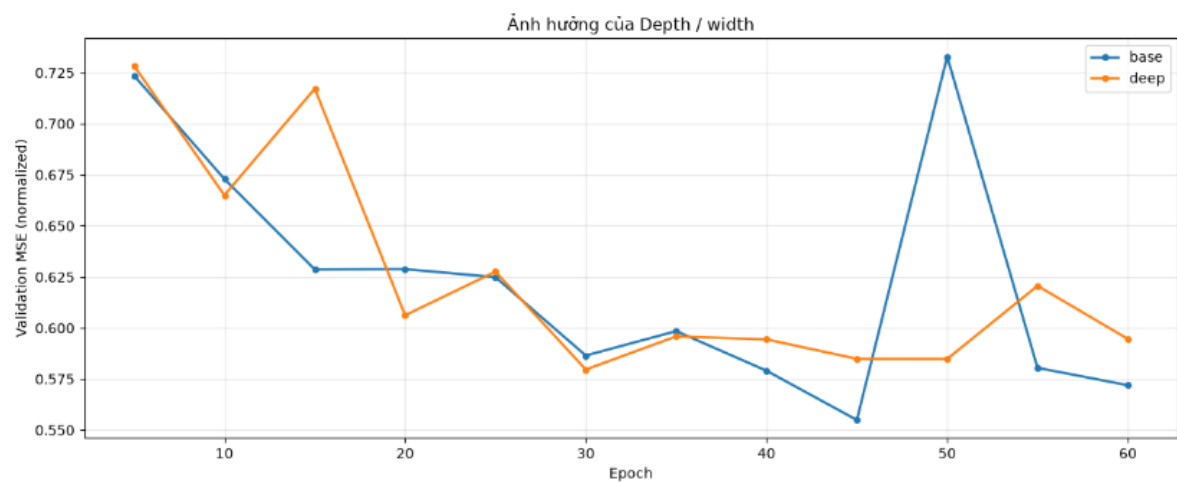
Hình output thực tế 2



Hình output thực tế 3



Hình output thực tế 4



Nhận xét và vai trò trong pipeline

Kết luận: Các output là bốn biểu đồ thực tế dùng để giải thích vì sao một cấu hình có thể vượt cấu hình khác. Kết luận cuối vẫn cần đối chiếu với bảng metric test để tránh chọn mô hình chỉ dựa trên hình dạng curve.

20. K-Fold Cross Validation cho DL (optional)

Cell code 46

```

1 def kfold_cv_dl(
2     X,
3     y,
4     k=5,
5     epochs=100,
6     batch_size=32,
7     hidden_dims=(64, 32, 16, 8),
8     lr=0.01,
9     optimizer="adam"
10 ):
11     kf = KFold(n_splits=k, shuffle=True, random_state=42)
12     scores = []
13
14     for fold, (tr_idx, va_idx) in enumerate(kf.split(X)):
15         X_tr, X_va = X[tr_idx], X[va_idx]
16         y_tr, y_va = y[tr_idx], y[va_idx]
17
18         ym = y_tr.mean()
19         ys = y_tr.std() + 1e-8
20
21         y_tr_n = (y_tr - ym) / ys
22         y_va_n = (y_va - ym) / ys
23
24         model = MLPRegressor(
25             input_dim=X.shape[1],
26             hidden_dims=hidden_dims,
27             lr=lr,
28             optimizer=optimizer,
29             seed=42 + fold
30         )
31
32         train_mlp(
33             model,
34             X_tr,
35             y_tr_n,
36             X_va,
37             y_va_n,
38             epochs=epochs,
39             batch_size=batch_size,
40             verbose=False
41         )
42
43         pred = model.predict(X_va) * ys + ym
44
45         scores.append({
46             "fold": fold + 1,
47             "RMSE": np.sqrt(mean_squared_error(y_va, pred)),
48             "R2": r2_score(y_va, pred)
49         })
50
51     return pd.DataFrame(scores)
52
53 cv_df = kfold_cv_dl(
54     X_train_nn,
55     y_train,
56     k=5,
57     epochs=100,
58     batch_size=32
59 )
60
61 display(cv_df)

```

Giải thích chi tiết code

Cell định nghĩa hàm `kfold_cv_dl` sử dụng `KFold(n_splits=k, shuffle=True, random_state=42)` để chia dữ liệu thành k fold. Với mỗi fold, một phần được dùng train và phần còn lại làm validation.

Đặc biệt, target được chuẩn hóa lại theo mean/std của chính fold train, sau đó mô hình được huấn luyện và đánh giá trên fold validation. Các score được thu thập để tính kết quả trung bình và độ phân tán giữa các fold.

Output cho biết metric qua các fold và tổng hợp. K-Fold không thay thế test set cuối cùng; nó giúp kiểm tra mức độ ổn định của mô hình khi dữ liệu train/validation thay đổi.

- Shuffle và random_state cố định giúp tái lập cách chia fold.
- Mỗi fold có scaling target dựa trên train fold, hạn chế leakage.

Output thực tế của cell

Hình output thực tế 1

```
Output thực tế
fold  RMSE  R2
0 1 35514.352978 -0.001240
1 2 30605.483687 0.195629
2 3 40686.039479 -0.001739
3 4 33149.011249 0.132162
4 5 26372.261400 0.197948
```

Hình output thực tế 2

```
Output thực tế
CV mean:
```

Hình output thực tế 3

```
Output thực tế
RMSE  R2
mean 33265.429759 0.104552
```

Nhận xét và vai trò trong pipeline

Kết luận: Output cho biết metric qua các fold và tổng hợp. K-Fold không thay thế test set cuối cùng; nó giúp kiểm tra mức độ ổn định của mô hình khi dữ liệu train/validation thay đổi.

21. Save model bằng NumPy

Cell code 48

```
1 save_dict = {}
2
3 for i, (w, b) in enumerate(zip(best_model.W, best_model.b)):
4     save_dict[f"W{i}"] = w
5     save_dict[f"B{i}"] = b
6
7 save_dict["y_mean"] = np.array(y_mean)
8 save_dict["y_std"] = np.array(y_std)
9 save_dict["hidden_dims"] = np.array(best_model.hidden_dims)
10
11 np.savez("best_mlp_house.npz", **save_dict)
12
13 print("Đã lưu: best_mlp_house.npz")
14 loaded = np.load("best_mlp_house.npz")
15 print("Keys:", list(loaded.keys()))
```

Giải thích chi tiết code

Cell lưu trọng số và bias của từng layer trong dictionary `save\_dict`. Với mỗi cặp `(w,b)` từ `best\_model.W` và `best\_model.b`, code tạo key `W0`, `b0`, `W1`, `b1`... Sau đó lưu thêm `y\_mean`, `y\_std` và `hidden\_dims`.

`np.savez` ghi toàn bộ dictionary thành file `best\_mlp\_house.npz`. Ngay sau đó notebook dùng `np.load` để đọc lại file và in danh sách keys, qua đó kiểm tra file đã chứa đủ thành phần cần thiết để tái dựng mô hình.

Việc lưu mean/std của target rất quan trọng vì prediction của mạng đang ở thang normalized; muốn đưa kết quả về million VND cần dùng lại hai giá trị này.

- File chứa tham số mô hình chứ không phải toàn bộ notebook.
- Keys được in ra là bước xác nhận nhanh tính đầy đủ của artifact đã lưu.

Output thực tế của cell

Hình output thực tế 1

```
Output thực tế
Đã lưu: best_mlp_house.npz
Keys: ['W0', 'b0', 'W1', 'b1', 'W2', 'b2', 'W3', 'b3', 'W4', 'b4', 'y_mean', 'y_std', 'hidden_dims']
```

Nhận xét và vai trò trong pipeline

Kết luận: Việc lưu mean/std của target rất quan trọng vì prediction của mạng đang ở thang normalized; muốn đưa kết quả về million VND cần dùng lại hai giá trị này.

22. Final leaderboard

Bảng này là kết quả cuối cùng để đưa vào báo cáo Assignment 02.

Cell code 50

```
1 final = pd.concat([
2     ml_test[["Model", "RMSE", "MAE", "R2"]].assign(Type="ML"),
3     dl_df_test[["Model", "RMSE", "MAE", "R2"]].assign(Type="DL")
4 ], ignore_index=True)
5
6 final = final[["Type", "Model", "RMSE", "MAE", "R2"]].sort_values("RMSE")
7
8 print("===== FINAL LEADERBOARD =====")
9 display(final)
10
11 best_overall = final.iloc[0]
12
13 print("\nBest overall:")
14 print("Type :", best_overall["Type"])
15 print("Model:", best_overall["Model"])
16 print("RMSE :", round(best_overall["RMSE"], 3))
17 print("MAE  :", round(best_overall["MAE"], 3))
18 print("R2   :", round(best_overall["R2"], 3))
```

Giải thích chi tiết code

Cell hợp nhất toàn bộ kết quả ML và DL thành một DataFrame `final`, giữ các cột Type, Model, RMSE, MAE và R2 rồi sắp xếp theo RMSE tăng dần.

Dòng đầu tiên sau khi sort được xem là `best\_overall`. Notebook in Type, Model, RMSE và MAE của mô hình đứng đầu để đưa vào kết luận cuối cùng.

Output final leaderboard là bảng tổng hợp quan trọng nhất của notebook. Nó cho phép nhìn toàn bộ mô hình trên cùng một bảng thay vì phải đối chiếu nhiều cell riêng lẻ. Theo kết quả thực tế trong notebook, cấu hình DL `deep` đứng đầu với RMSE khoảng 32648.594, MAE khoảng 8812.402 và R² khoảng 0.272; RandomForest là baseline ML tốt nhất với RMSE khoảng 33080.201.

- RMSE được dùng để sắp xếp leaderboard.
- MAE và R² cung cấp thêm góc nhìn về sai số và mức độ giải thích biến thiên target.

Output thực tế của cell

Hình output thực tế 1

Output thực tế
===== FINAL LEADERBOARD =====

Hình output thực tế 2

Output thực tế					
Type		Model	RMSE	MAE	R2
4	DL	deep	32648.594018	8812.401680	0.271786
2	ML	RandomForest	33080.200732	7349.411941	0.252405
5	DL	lr_small	33092.558503	8194.972123	0.251846
8	DL	batch_16	33215.580964	9080.948989	0.246273
3	DL	base	33481.737133	9516.155026	0.234146
6	DL	sgd	34670.084015	9699.930913	0.178817
1	ML	Ridge	36765.890279	10740.538086	0.076535
0	ML	LinearRegression	38286.388600	11141.777344	-0.001427
7	DL	momentum	38310.353278	11499.187487	-0.002681

Hình output thực tế 3

Output thực tế

```
Best overall:
Type : DL
Model: deep
RMSE : 32648.594
MAE : 8812.402
R2 : 0.272
```

Nhận xét và vai trò trong pipeline

Kết luận: Output final leaderboard là bảng tổng hợp quan trọng nhất của notebook. Nó cho phép nhìn toàn bộ mô hình trên cùng một bảng thay vì phải đối chiếu nhiều cell riêng lẻ. Theo kết quả thực tế trong notebook, cấu hình DL `deep` đứng đầu với RMSE khoảng 32648.594, MAE khoảng 8812.402 và R^2 khoảng 0.272; RandomForest là baseline ML tốt nhất với RMSE khoảng 33080.201.

BÀI 3: PHÂN TÍCH COMMENTS KHÁCH HÀNG. Dataset: Women's E-Commerce Clothing Reviews

1. Dataset and Problem Definition

Code Cell 3

```
Code Cell 3

1 DATA_PATH = "data/Womens Clothing E-Commerce Reviews.csv"
2
3 df = pd.read_csv(DATA_PATH)
4
5 print("Shape:", df.shape)
6 print("\nColumns:")
7 print(df.columns.tolist())
8 display(df.head())
```

Mục đích: Đọc dataset và kiểm tra nhanh dữ liệu đầu vào.

Giải thích chi tiết

`DATA\_PATH` xác định file CSV, sau đó `pd.read\_csv()` đưa dữ liệu vào DataFrame `df`. `df.shape` cho biết số dòng và số cột; `df.columns.tolist()` liệt kê tên biến; `display(df.head())` hiển thị 5 bản ghi đầu. Cell này chưa cleaning và chưa train model, mà chỉ xác nhận file đã được đọc đúng và cho người thực hiện cái nhìn ban đầu về cấu trúc dữ liệu.

Output thực tế

Output – Cell 3 – #1

Shape: (23486, 11)

Columns:
['Unnamed: 0', 'Clothing ID', 'Age', 'Title', 'Review Text', 'Rating', 'Recommended IND', 'Positive Feedback Count', 'Division Name', 'Department Name', 'Class Name']

Output – Cell 3 – #2

Unnamed: 0	Clothing ID	Age	Title	Review Text	Rating	Recommended IND
0	0	767	33	NaN		
1	1	1080	34	NaN		
2	2	1077	60	Some major design flaws		
3	3	1049	50	My favorite buy!		
4	4	847	47	Flattering shirt		

	Review Text	Rating	Recommended IND
0	Absolutely wonderful - silky and sexy and comf...	4	1
1	Love this dress! it's sooo pretty. i happene...	5	1
2	I had such high hopes for this dress and reall...	3	0
3	I love, love, love this jumpsuit. it's fun, fl...	5	1
4	This shirt is very flattering to all due to th...	5	1

	Positive Feedback Count	Division Name	Department Name	Class Name
0	0	Intimates	Intimate	Intimates
1	4	General	Dresses	Dresses
2	0	General	Dresses	Dresses
3	0	General Petite	Bottoms	Pants
4	6	General	Tops	Blouses

2. Data Inspection and Understanding

Kiểm tra:

- kích thước dữ liệu;
- kiểu dữ liệu;
- missing values;
- duplicate records;
- thống kê mô tả;
- phân bố target;
- các nhóm numerical / categorical / text.

Code Cell 5

Code Cell 5

```
1 print("=== SHAPE ===")
2 print(df.shape)
3
4 print("\n=== HEAD ===")
5 display(df.head())
6
7 print("\n=== INFO ===")
8 df.info()
9
10 print("\n=== DESCRIBE ===")
11 display(df.describe(include="all").T)
12
13 print("\n=== MISSING VALUES ===")
14 display(df.isna().sum().sort_values(ascending=False))
15
16 print("\n=== DUPLICATES ===")
17 print(df.duplicated().sum())
18
19 print("\n=== TARGET ===")
20 print(df["Recommended IND"].value_counts(dropna=False))
21 print(df["Recommended IND"].value_counts(normalize=True).round(4))
```

Mục đích: Thực hiện các phép kiểm tra EDA.

Giải thích chi tiết

Cell thực hiện lần lượt `shape`, `head`, `info`, `describe`, kiểm tra missing và duplicate. `df.info()` cho biết dtype/non-null; `describe(include="all").T` đưa thống kê mô tả về dạng mỗi biến một dòng. `isna().sum()` đếm missing theo cột và `duplicated().sum()` đếm bản ghi trùng. Hai `value\_counts()` cuối cùng kiểm tra phân bố target theo số lượng và tỷ lệ. Output thực tế cho thấy 23.486 dòng, 11 cột; có missing ở Title, Review Text và các categorical; target gồm 19.314 lớp 1 và 4.172 lớp 0 (~82,24%/17,76%).

Output thực tế

Output - Cell 5 - #1

```
=== SHAPE ===
(23486, 11)

=== HEAD ===
```

Output - Cell 5 - #2

```

Unnamed: 0  Clothing ID  Age  Title \
0          0          767   33    NaN
1          1         1880   34    NaN
2          2         1877   68  Some major design flaws
3          3         1849   58  My favorite buy!
4          4          847   47  Flattering shirt

Review Text  Rating  Recommended IND \
0 Absolutely wonderful - silky and sexy and comf...  4  1
1 Love this dress! it's sooo pretty. i happene...  5  1
2 I had such high hopes for this dress and reall...  3  0
3 I love, love, love this jumpsuit. it's fun, fl...  5  1
4 This shirt is very flattering to all due to th...  5  1

Positive Feedback Count  Division Name  Department Name  Class Name
0          0          Initmates          Intimate  Intimates
1          4          General          Dresses  Dresses
2          0          General          Dresses  Dresses
3          0  General Petite          Bottoms  Pants
4          6          General          Tops  Blouses

```

Output - Cell 5 - #3

```

=== INFO ===
<class 'pandas.DataFrame'>
RangeIndex: 23486 entries, 0 to 23485
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  --
0   Unnamed: 0            23486 non-null  int64
1   Clothing ID           23486 non-null  int64
2   Age                   23486 non-null  int64
3   Title                 19676 non-null  str
4   Review Text           22641 non-null  str
5   Rating                23486 non-null  int64
6   Recommended IND        23486 non-null  int64
7   Positive Feedback Count  23486 non-null  int64
8   Division Name         23472 non-null  str
9   Department Name       23472 non-null  str
10  Class Name            23472 non-null  str
dtypes: int64(6), str(5)
memory usage: 9.5 MB

=== DESCRIBE ===

```

Output - Cell 5 - #4

```

count unique \
Unnamed: 0      23486.0  NaN
Clothing ID     23486.0  NaN
Age             23486.0  NaN
Title           19676   13993
Review Text     22641   22634
Rating          23486.0  NaN
Recommended IND  23486.0  NaN
Positive Feedback Count  23486.0  NaN
Division Name    23472     3
Department Name  23472     6
Class Name       23472    28

top \
Unnamed: 0      NaN
Clothing ID     NaN
Age             NaN
Title           Love it!
Review Text     Perfect fit and i've gotten so many compliment...
Rating          NaN
Recommended IND  NaN
Positive Feedback Count  NaN
Division Name    General
Department Name  Tops
Class Name       Dresses

freq    mean    std    min    25% \
Unnamed: 0      NaN    11742.5  6779.968547  0.0  5871.25
Clothing ID     NaN    918.118789  283.29898  0.0  861.0
Age             NaN    43.198544  12.279544  18.0  34.0
Title           136     NaN     NaN     NaN     NaN
Review Text     3       NaN     NaN     NaN     NaN
Rating          NaN    4.196032  1.110031  1.0  4.0
Recommended IND  NaN    0.822362  0.382216  0.0  1.0
Positive Feedback Count  NaN    2.535936  5.702202  0.0  0.0
Division Name    13850     NaN     NaN     NaN     NaN
Department Name  10468     NaN     NaN     NaN     NaN
Class Name       6319     NaN     NaN     NaN     NaN

50%    75%    max
Unnamed: 0    11742.5  17613.75  23485.0
Clothing ID    936.0  1878.0  1205.0
Age            41.0   52.0   99.0
Title          NaN     NaN     NaN
Review Text    NaN     NaN     NaN
Rating         5.0    5.0    5.0
Recommended IND  1.0    1.0    1.0
Positive Feedback Count  1.0    3.0  122.0
Division Name   NaN     NaN     NaN
Department Name  NaN     NaN     NaN
Class Name      NaN     NaN     NaN

```

Output - Cell 5 - #5

```

=== MISSING VALUES ===

```

Output – Cell 5 – #6

```
Title          3810
Review Text    845
Department Name 14
Class Name     14
Division Name  14
Unnamed: 0     0
Clothing ID    0
Age            0
Rating         0
Recommended IND 0
Positive Feedback Count 0
dtype: int64
```

Output – Cell 5 – #7

```
=== DUPLICATES ===
0

=== TARGET ===
Recommended IND
1    19314
0     4172
Name: count, dtype: int64
Recommended IND
1     0.8224
0     0.1776
Name: proportion, dtype: float64
```

Code Cell 6

Code Cell 6

```
1 # Xác định nhóm feature
2 numeric_candidates = ["Age", "Clothing ID", "Rating"]
3 categorical_candidates = ["Division Name", "Department Name", "Class Name"]
4 text_candidates = ["Title", "Review Text"]
5
6 print("Numerical:", [c for c in numeric_candidates if c in df.columns])
7 print("Categorical:", [c for c in categorical_candidates if c in df.columns])
8 print("Text:", [c for c in text_candidates if c in df.columns])
9 print("Target: Recommended IND")
```

Mục đích: Xác định numerical, categorical, text và target.

Giải thích chi tiết

Các list `numeric\_candidates`, `categorical\_candidates`, `text\_candidates` được khai báo để xác định vai trò của từng biến. List comprehension có điều kiện `if c in df.columns` bảo đảm chỉ giữ những tên cột thực sự tồn tại. `target\_col` được đặt riêng là `Recommended IND`, vì đây là biến cần dự đoán và không được đưa vào X.

Output thực tế

Output – Cell 6 – #1

```
Numerical: ['Age', 'Clothing ID', 'Rating']
Categorical: ['Division Name', 'Department Name', 'Class Name']
Text: ['Title', 'Review Text']
Target: Recommended IND
```

Code Cell 7

Code Cell 7

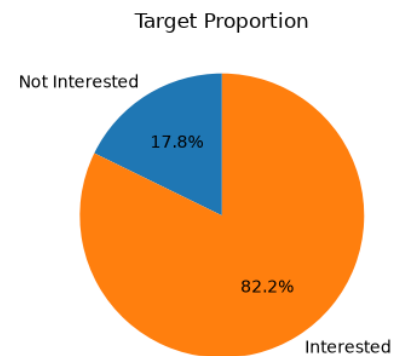
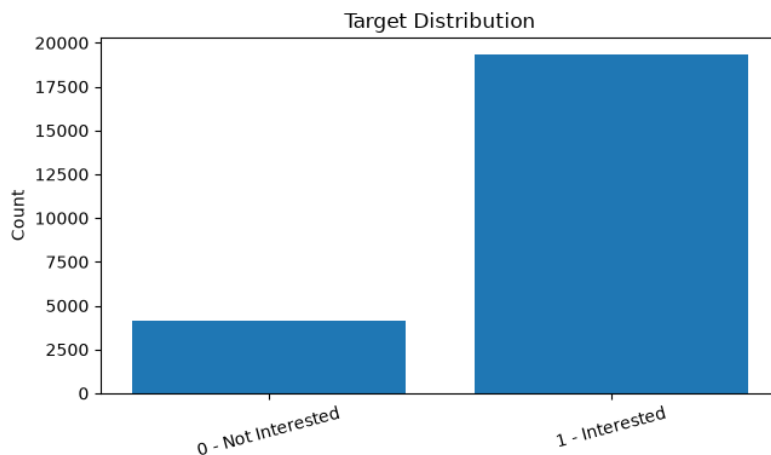
```
1 fig, axes = plt.subplots(1, 2, figsize=(12, 4))
2
3 target_counts = df["Recommended IND"].value_counts().sort_index()
4 axes[0].bar(["0 - Not Interested", "1 - Interested"], target_counts.values)
5 axes[0].set_title("Target Distribution")
6 axes[0].set_ylabel("Count")
7 axes[0].tick_params(axis="x", rotation=15)
8
9 axes[1].pie(target_counts.values, labels=["Not Interested", "Interested"],
10            autopct="%1.1f%%", startangle=90)
11 axes[1].set_title("Target Proportion")
12
13 plt.tight_layout()
14 plt.show()
```

Mục đích: Trực quan hóa phân bố target.

Giải thích chi tiết

`value\_counts().sort\_index()` lấy số lượng của lớp 0 và 1 theo thứ tự cố định. Biểu đồ cột thể hiện số mẫu tuyệt đối, còn pie chart thể hiện tỷ lệ phần trăm thông qua `autopct`. Cell chỉ trực quan hóa target, không thay đổi dữ liệu. Hai biểu đồ cho thấy lớp 1 chiếm đa số, là cơ sở để notebook sử dụng stratified split và đánh giá bằng Precision/Recall/F1/AUC thay vì chỉ Accuracy.

Output thực tế



3. Data Cleaning

Quy trình:

1. bỏ cột index dư `Unnamed: 0` nếu tồn tại;
2. loại duplicate;
3. xử lý missing numerical bằng median;
4. xử lý missing categorical bằng mode;
5. xử lý missing text bằng chuỗi rỗng;
6. loại giá trị `Age <= 0`;
7. kiểm tra range của `Rating`;
8. chuẩn hóa text.

Code Cell 9

```
Code Cell 9

1 df = df.copy()
2
3 if "Unnamed: 0" in df.columns:
4     df = df.drop(columns=["Unnamed: 0"])
5
6 before = len(df)
7 df = df.drop_duplicates().reset_index(drop=True)
8 print("Removed duplicates:", before - len(df))
9
10 # Numerical
11 if "Age" in df.columns:
12     df["Age"] = pd.to_numeric(df["Age"], errors="coerce")
13     df["Age"] = df["Age"].fillna(df["Age"].median())
14     df = df[df["Age"] > 0].copy()
15
16 if "Clothing ID" in df.columns:
17     df["Clothing ID"] = pd.to_numeric(df["Clothing ID"], errors="coerce")
18     df["Clothing ID"] = df["Clothing ID"].fillna(df["Clothing ID"].median())
19
20 if "Rating" in df.columns:
21     df["Rating"] = pd.to_numeric(df["Rating"], errors="coerce")
22     df["Rating"] = df["Rating"].fillna(df["Rating"].median())
23     df = df[df["Rating"].between(1, 5)].copy()
24
25 # Categorical
26 for col in ["Division Name", "Department Name", "Class Name"]:
27     if col in df.columns:
28         mode = df[col].mode(dropna=True)
29         fill_value = mode.iloc[0] if len(mode) else "Unknown"
30         df[col] = df[col].fillna(fill_value).astype(str).str.strip()
31
32 # Text
33 for col in ["Title", "Review Text"]:
34     if col in df.columns:
35         df[col] = df[col].fillna("").astype(str)
36
37 print("Final shape:", df.shape)
38 print("\nRemaining missing:")
39 print(df.isna().sum().sort_values(ascending=False).head(10))
```

Mục đích: Làm sạch dữ liệu.

Giải thích chi tiết

Cell tạo bản sao bằng `df.copy()`, xóa `Unnamed: 0` nếu có, loại duplicate và reset index. Các cột số được ép về numeric với `errors="coerce"`; Age được giữ khi > 0 và Rating trong khoảng 1–5; missing numerical được thay bằng median. Categorical được thay missing bằng mode, chuyển thành string và `str.strip()` để loại khoảng trắng thừa. Text missing thành chuỗi rỗng và cũng được ép string. Cuối cell, `shape` và `isna().sum()` kiểm tra lại. Output cho thấy 21 duplicate bị loại, còn 23.465 dòng và 10 cột, missing đã về 0.

Output thực tế

Output – Cell 9 – #1

Removed duplicates: 21
Final shape: (23465, 10)

Remaining missing:

Clothing ID	0
Age	0
Title	0
Review Text	0
Rating	0
Recommended IND	0
Positive Feedback Count	0
Division Name	0
Department Name	0
Class Name	0

dtype: int64

4. Feature Engineering

Tạo:

- `combined\_text`: ghép `Title` + `Review Text`;
- `review\_length`: số ký tự review;
- `review\_word\_count`: số từ;
- `title\_length`: số ký tự title.

Text được lowercase, loại ký tự không cần thiết và chuẩn hóa khoảng trắng.

Code Cell 11

```
Code Cell 11

1 def clean_text(text):
2     text = "" if not isinstance(text, str) else text.lower()
3     text = re.sub(r"[^a-z\s]", " ", text)
4     text = re.sub(r"\s+", " ", text).strip()
5     return text
6
7 df["combined_text"] = (
8     df["Title"].fillna("") + " " + df["Review Text"].fillna("")
9 ).apply(clean_text)
10
11 df["review_length"] = df["Review Text"].str.len()
12 df["review_word_count"] = df["Review Text"].apply(lambda x: len(x.split()))
13 df["title_length"] = df["Title"].str.len()
14
15 display(df[
16     "Age", "Rating", "review_length",
17     "review_word_count", "title_length",
18     "combined_text", "Recommended IND"
19 ].head())
```

Mục đích: Feature engineering cho text.

Giải thích chi tiết

`clean\_text()` lowercase văn bản, dùng regex `[^a-z\s]` thay ký tự ngoài a-z/khoảng trắng bằng space và gom nhiều whitespace thành một.

`combined\_text` nối Title và Review Text rồi làm sạch. `review\_length`, `review\_word\_count`, `title\_length` lần lượt đo số ký tự review, số từ review và số ký tự title. `display()` chỉ hiển thị các trường liên quan để kiểm tra kết quả tạo feature.

Output thực tế

```
Output - Cell 11 - #1
```

	Age	Rating	review_length	review_word_count	title_length	\
0	33	4	53	8	0	
1	34	5	303	62	0	
2	60	3	500	98	23	
3	50	5	124	22	16	
4	47	5	192	36	16	

	combined_text	Recommended IND
0	absolutely wonderful silky and sexy and comfor...	1
1	love this dress it s sooo pretty i happened to...	1
2	some major design flaws i had such high hopes ...	0
3	my favorite buy i love love love this jumpsuit...	1
4	flattering shirt this shirt is very flattering...	1

5. Train / Validation / Test Split

Sử dụng:

- 70% Train
- 15% Validation
- 15% Test

`stratify=y` để giữ tỷ lệ hai lớp tương đối ổn định.

Code Cell 13

```
Code Cell 13

1 feature_cols = [
2     "Age", "Clothing ID", "Rating",
3     "Division Name", "Department Name", "Class Name",
4     "combined_text", "review_length", "review_word_count", "title_length"
5 ]
6
7 X_raw = df[feature_cols].copy()
8 y = df["Recommended IND"].astype(int).to_numpy()
9
10 X_temp, X_test_raw, y_temp, y_test = train_test_split(
11     X_raw, y, test_size=0.15, random_state=SEED, stratify=y
12 )
13
14 X_train_raw, X_val_raw, y_train, y_val = train_test_split(
15     X_temp, y_temp, test_size=0.1765, random_state=SEED, stratify=y_temp
16 )
17
18 print("Train:", X_train_raw.shape, Counter(y_train))
19 print("Val: ", X_val_raw.shape, Counter(y_val))
20 print("Test: ", X_test_raw.shape, Counter(y_test))
```

Mục đích: Tạo X/y và chia Train–Validation–Test.

Giải thích chi tiết

`feature\_cols` gom 10 biến đầu vào; `X\_raw` lấy các cột đó, còn `y` lấy `Recommended IND`. Lần `train\_test\_split` đầu giữ 15% làm Test; lần thứ hai tách khoảng 15% tổng dữ liệu từ phần tạm để tạo Validation.

`random\_state=SEED` đảm bảo cách chia có thể lặp lại, còn `stratify=y` giữ tỷ lệ hai lớp. Output thực tế: Train 16.424, Validation 3.521, Test 3.520.

Output thực tế

```
Output - Cell 13 - #1

Train: (16424, 10) Counter({np.int64(1): 13504, np.int64(0): 2920})
Val:   (3521, 10) Counter({np.int64(1): 2895, np.int64(0): 626})
Test:  (3520, 10) Counter({np.int64(1): 2894, np.int64(0): 626})
```

6. Build Feature Matrix

Preprocessing được fit ****chỉ trên training set**** để tránh data leakage.

- Numerical → StandardScaler
- Categorical → One-Hot Encoding
- Text → TF-IDF, tối đa 500 features
- Sau đó nối tất cả thành một dense NumPy matrix.

Code Cell 15

```
Code Cell 15

1 num_cols = ["Age", "Clothing ID", "Rating", "review_length",
2             "review_word_count", "title_length"]
3 cat_cols = ["Division Name", "Department Name", "Class Name"]
4
5 scaler = StandardScaler()
6 X_train_num = scaler.fit_transform(X_train_raw[num_cols])
7 X_val_num = scaler.transform(X_val_raw[num_cols])
8 X_test_num = scaler.transform(X_test_raw[num_cols])
9
10 # Compatible with different sklearn versions
11 try:
12     ohe = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
13 except TypeError:
14     ohe = OneHotEncoder(handle_unknown="ignore", sparse=False)
15
16 X_train_cat = ohe.fit_transform(X_train_raw[cat_cols]).astype(str)
17 X_val_cat = ohe.transform(X_val_raw[cat_cols]).astype(str)
18 X_test_cat = ohe.transform(X_test_raw[cat_cols]).astype(str)
19
20 tfidf = TfidfVectorizer(max_features=500, ngram_range=(1, 2), min_df=3)
21 X_train_txt = tfidf.fit_transform(X_train_raw["combined_text"]).toarray()
22 X_val_txt = tfidf.transform(X_val_raw["combined_text"]).toarray()
23 X_test_txt = tfidf.transform(X_test_raw["combined_text"]).toarray()
24
25 X_train_ml = np.hstack([X_train_num, X_train_cat, X_train_txt]).astype(np.float32)
26 X_val_ml = np.hstack([X_val_num, X_val_cat, X_val_txt]).astype(np.float32)
27 X_test_ml = np.hstack([X_test_num, X_test_cat, X_test_txt]).astype(np.float32)
28
29 print("Numerical:", X_train_num.shape)
30 print("Categorical:", X_train_cat.shape)
31 print("TF-IDF:", X_train_txt.shape)
32 print("Final:", X_train_ml.shape)
```

Mục đích: Tiền xử lý và tạo ma trận 534 features.

Giải thích chi tiết

`num\_cols` gồm 6 biến số. `StandardScaler` fit trên Train rồi transform Validation/Test để tránh data leakage.

`OneHotEncoder(handle\_unknown="ignore")` xử lý categorical; try/except chỉ để tương thích tham số giữa các phiên bản scikit-learn.

`TfidfVectorizer(max\_features=500, ngram\_range=(1,2), min\_df=3)` biến text thành tối đa 500 unigram/bigram. `np.hstack()` ghép 6 numerical + 28 categorical + 500 TF-IDF thành 534 features. Output xác nhận đúng các kích thước này.

Output thực tế

```
Output - Cell 15 - #1

Numerical: (16424, 6)
Categorical: (16424, 28)
TF-IDF: (16424, 500)
Final: (16424, 534)
```


7. Baseline Machine Learning

Ba mô hình baseline:

1. Logistic Regression
2. Random Forest

Mục đích là tạo mốc so sánh với neural network NumPy from scratch.

Code Cell 17

```
Code Cell 17

1 ml_models = {
2     "Logistic Regression": LogisticRegression(
3         max_iter=1000, random_state=SEED
4     ),
5     "Random Forest": RandomForestClassifier(
6         n_estimators=150, max_depth=15,
7         random_state=SEED, n_jobs=-1
8     )
9 }
10
11 ml_results = {}
12
13 for name, model in ml_models.items():
14     t0 = time.time()
15     model.fit(X_train_ml, y_train)
16     train_time = time.time() - t0
17
18     pred = model.predict(X_test_ml)
19     prob = model.predict_proba(X_test_ml)[:, 1]
20
21     ml_results[name] = {
22         "model": model,
23         "accuracy": accuracy_score(y_test, pred),
24         "precision": precision_score(y_test, pred, zero_division=0),
25         "recall": recall_score(y_test, pred, zero_division=0),
26         "f1": f1_score(y_test, pred, zero_division=0),
27         "auc": roc_auc_score(y_test, prob),
28         "time": train_time,
29         "pred": pred,
30         "prob": prob
31     }
32
33     print(
34         f"{name}: Acc={ml_results[name]['accuracy']:.4f}, "
35         f"F1={ml_results[name]['f1']:.4f}, "
36         f"AUC={ml_results[name]['auc']:.4f}, "
37         f"Time={train_time:.2f}s"
38     )
```

Mục đích: Huấn luyện hai baseline Machine Learning.

Giải thích chi tiết

Dictionary `ml\_models` chỉ gồm Logistic Regression và Random Forest. Vòng lặp `fit()` từng model trên Train, dùng `predict()` để lấy nhãn và `predict\_proba()[:,1]` để lấy xác suất lớp 1. Accuracy, Precision, Recall, F1 và AUC được tính rồi lưu vào `ml\_results`; `time.time()` đo thời gian train. Output thực tế: Logistic Regression F1=0,9616, AUC=0,9762; Random Forest F1=0,9544, AUC=0,9710.

Output thực tế

```
Output - Cell 17 - #1

Logistic Regression: Acc=0.9366, F1=0.9616, AUC=0.9762, Time=0.33s
Random Forest: Acc=0.9233, F1=0.9544, AUC=0.9710, Time=2.20s
```

Code Cell 18

Code Cell 18

```
1 ml_comparison = pd.DataFrame({
2     name: {
3         "Accuracy": r["accuracy"],
4         "Precision": r["precision"],
5         "Recall": r["recall"],
6         "F1": r["f1"],
7         "AUC": r["auc"],
8         "Train Time (s)": r["time"]
9     }
10     for name, r in ml_results.items()
11 }).T.round(4)
12
13 display(ml_comparison)
```

Mục đích: Tạo bảng so sánh kết quả ML.

Giải thích chi tiết

Cell không train lại. Nó lấy các trường Accuracy, Precision, Recall, F1, AUC và Train Time từ `ml\_results`, tạo DataFrame rồi `.T` để mỗi model thành một hàng và `.round(4)` để dễ đọc. Vì vậy bảng output phải khớp với metric vừa tính ở cell 17.

Output thực tế

Output - Cell 18 - #1

	Accuracy	Precision	Recall	F1	AUC	\
Logistic Regression	0.9366	0.9581	0.9651	0.9616	0.9762	
Random Forest	0.9233	0.9339	0.9758	0.9544	0.9710	

	Train Time (s)
Logistic Regression	0.3311
Random Forest	2.1995

8. Deep Learning from Scratch — NumPy Only

Kiến trúc 5 lớp:

****Input $\rightarrow$ 128 $\rightarrow$ 64 $\rightarrow$ 32 $\rightarrow$ 16 $\rightarrow$ 1****

- Hidden layers: ReLU
- Output: Sigmoid
- Loss: Binary Cross-Entropy
- Backpropagation: tự cài đặt bằng NumPy
- Optimizer: GD / Momentum / Adam
- Mini-batch training

Code Cell 20

```
Code Cell 20
1 X_train_dl = X_train_ml.astype(np.float32)
2 X_val_dl = X_val_ml.astype(np.float32)
3 X_test_dl = X_test_ml.astype(np.float32)
4
5 y_train_dl = y_train.reshape(-1, 1).astype(np.float32)
6 y_val_dl = y_val.reshape(-1, 1).astype(np.float32)
7 y_test_dl = y_test.reshape(-1, 1).astype(np.float32)
8
9 INPUT_DIM = X_train_dl.shape[1]
10
11 print("Input dimension:", INPUT_DIM)
12 print("Train:", X_train_dl.shape)
13 print("Val:", X_val_dl.shape)
14 print("Test:", X_test_dl.shape)
```

Mục đích: Chuẩn bị dữ liệu cho neural network NumPy.

Giải thích chi tiết

Ba ma trận X được chuyển sang `float32`; target được reshape từ vector `(n,)` thành `(n,1)` rồi cũng chuyển sang `float32`.

`INPUT\_DIM=X\_train\_dl.shape[1]` lấy số chiều input trực tiếp từ dữ liệu.

Output xác nhận Input dimension=534 và các kích thước Train/Val/Test tương ứng.

Output thực tế

```
Output - Cell 20 - #1
Input dimension: 534
Train: (16424, 534)
Val: (3521, 534)
Test: (3520, 534)
```

Code Cell 21

Code Cell 21

```
1 def relu(x):
2     return np.maximum(0, x)
3
4 def relu_derivative(x):
5     return (x > 0).astype(np.float32)
6
7 def sigmoid(x):
8     x = np.clip(x, -50, 50)
9     return 1.0 / (1.0 + np.exp(-x))
10
11 def binary_cross_entropy(y, y_hat):
12     eps = 1e-7
13     y_hat = np.clip(y_hat, eps, 1 - eps)
14     return -np.mean(
15         y * np.log(y_hat) + (1 - y) * np.log(1 - y_hat)
16     )
17
18 print("Activation + loss functions ready.")
```

Mục đích: Tự cài đặt activation và Binary Cross-Entropy.

Giải thích chi tiết

ReLU dùng `np.maximum(0,x)` và đạo hàm trả 1 với $x > 0$. Sigmoid clip input trong $[-50, 50]$ trước exponential để giảm overflow. BCE clip xác suất dự đoán trong $[1e-7, 1-1e-7]$ trước log, rồi tính trung bình. Output chỉ xác nhận các hàm đã được định nghĩa thành công; chưa có training ở cell này.

Output thực tế

Output – Cell 21 – #1

Activation + loss functions ready.

Code Cell 22

Code Cell 22

```

1 class NeuralNetwork5Layers:
2     def __init__(
3         self, input_dim, hidden_dims=(128, 64, 32, 16),
4         optimizer="adam", learning_rate=0.001, seed=42
5     ):
6         rng = np.random.default_rng(seed)
7         self.hidden_dims = list(hidden_dims)
8         self.optimizer_name = optimizer.lower()
9         self.lr = learning_rate
10
11         dims = [input_dim] + self.hidden_dims + [1]
12
13         self.W = []
14         self.b = []
15
16         for fan_in, fan_out in zip(dims[:-1], dims[1:]):
17             W = rng.standard_normal((fan_in, fan_out)).astype(np.float32)
18             W *= np.sqrt(2.0 / fan_in)
19             b = np.zeros(1, fan_out), dtype=np.float32)
20             self.W.append(W)
21             self.b.append(b)
22
23         self._init_optimizer()
24
25     def _init_optimizer(self):
26         self.m_W = [np.zeros_like(w) for w in self.W]
27         self.v_W = [np.zeros_like(w) for w in self.W]
28         self.m_b = [np.zeros_like(b) for b in self.b]
29         self.v_b = [np.zeros_like(b) for b in self.b]
30         self.t = 0
31
32     def forward(self, X):
33         self.cache = {"A0": X}
34         A = X
35
36         for i in range(len(self.W) - 1):
37             Z = A @ self.W[i] + self.b[i]
38             A = relu(Z)
39             self.cache[f"Z{i+1}"] = Z
40             self.cache[f"A{i+1}"] = A
41
42         Z = A @ self.W[-1] + self.b[-1]
43         y_hat = sigmoid(Z)
44
45         self.cache[f"Z(len(self.W))"] = Z
46         self.cache[f"A(len(self.W))"] = y_hat
47         return y_hat
48
49     def backward(self, y):
50         L = len(self.W)
51         n = y.shape[0]
52
53         dW = [None] * L
54         db = [None] * L
55
56         y_hat = self.cache[f"A{L}"]
57         dZ = (y_hat - y) / n
58
59         A_prev = self.cache[f"A{L-1}"]
60         dW[L-1] = A_prev.T @ dZ
61         db[L-1] = np.sum(dZ, axis=0, keepdims=True)
62
63         for i in range(L - 2, -1, -1):
64             dA = dZ @ self.W[i+1].T
65             dZ = dA * relu_derivative(self.cache[f"Z{i+1}"])
66
67             A_prev = self.cache[f"A0"] if i == 0 else self.cache[f"A{i}"]
68             dW[i] = A_prev.T @ dZ
69             db[i] = np.sum(dZ, axis=0, keepdims=True)
70
71         return dW, db
72
73     def update(self, dW, db):
74         if self.optimizer_name == "gd":
75             for i in range(len(self.W)):
76                 self.W[i] -= self.lr * dW[i]
77                 self.b[i] -= self.lr * db[i]
78
79         elif self.optimizer_name == "momentum":
80             beta = 0.9
81             for i in range(len(self.W)):
82                 self.m_W[i] = beta * self.m_W[i] + (1-beta) * dW[i]
83                 self.m_b[i] = beta * self.m_b[i] + (1-beta) * db[i]
84                 self.W[i] -= self.lr * self.m_W[i]
85                 self.b[i] -= self.lr * self.m_b[i]
86
87         elif self.optimizer_name == "adam":
88             beta1, beta2, eps = 0.9, 0.999, 1e-8
89             self.t += 1
90
91             for i in range(len(self.W)):
92                 self.m_W[i] = beta1*self.m_W[i] + (1-beta1)*dW[i]
93                 self.v_W[i] = beta2*self.v_W[i] + (1-beta2)*(dW[i]**2)
94                 self.m_b[i] = beta1*self.m_b[i] + (1-beta1)*db[i]
95                 self.v_b[i] = beta2*self.v_b[i] + (1-beta2)*(db[i]**2)
96
97                 mwh = self.m_W[i] / (1-beta1**self.t)
98                 vwh = self.v_W[i] / (1-beta2**self.t)
99                 mbh = self.m_b[i] / (1-beta1**self.t)
100                 vbh = self.v_b[i] / (1-beta2**self.t)
101
102                 self.W[i] -= self.lr * mwh / (np.sqrt(vwh) + eps)
103                 self.b[i] -= self.lr * mbh / (np.sqrt(vbh) + eps)
104
105         else:
106             raise ValueError("optimizer must be gd, momentum, or adam")
107
108     def predict_proba(self, X):
109         return self.forward(X)
110
111     def predict(self, X, threshold=0.5):
112         return (self.predict_proba(X) >= threshold).astype(int)
113
114     def count_params(self):
115         return sum(w.size for w in self.W) + sum(b.size for b in self.b)

```

Mục đích: Định nghĩa neural network 5 lớp và optimizer.

Giải thích chi tiết

Class tạo kiến trúc `input\_dim → hidden\_dims → 1`; với input 534 và hidden mặc định `(128,64,32,16)` là $534 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 1$. Trọng số dùng He initialization, bias bằng 0. `forward()` thực hiện matrix multiplication + bias, ReLU ở hidden và Sigmoid ở output; cache lưu giá trị cần cho backprop.

`backward()` áp dụng chain rule để tính gradient. `update()` cài GD, Momentum và Adam; Adam dùng first/second moment và bias correction. `predict\_proba`, `predict` và `count\_params` cung cấp inference và thống kê tham số. Cell chỉ định nghĩa class.

Cell không sinh output trực tiếp trong notebook.

Code Cell 23

```
Code Cell 23

1 def train_nn(
2     model, X_train, y_train, X_val, y_val,
3     epochs=30, batch_size=64, verbose=True
4 ):
5     n = len(X_train)
6     history = {
7         "train_loss": [], "val_loss": [],
8         "train_acc": [], "val_acc": []
9     }
10
11     for epoch in range(epochs):
12         indices = np.random.permutation(n)
13         X_shuf = X_train[indices]
14         y_shuf = y_train[indices]
15
16         batch_losses = []
17
18         for start in range(0, n, batch_size):
19             Xb = X_shuf[start:start+batch_size]
20             yb = y_shuf[start:start+batch_size]
21
22             pred = model.forward(Xb)
23             batch_losses.append(binary_crossentropy(yb, pred))
24
25             dW, db = model.backward(yb)
26             model.update(dW, db)
27
28         train_prob = model.predict_proba(X_train)
29         val_prob = model.predict_proba(X_val)
30
31         train_loss = binary_crossentropy(y_train, train_prob)
32         val_loss = binary_crossentropy(y_val, val_prob)
33
34         train_pred = (train_prob >= 0.5).astype(int)
35         val_pred = (val_prob >= 0.5).astype(int)
36
37         history["train_loss"].append(train_loss)
38         history["val_loss"].append(val_loss)
39         history["train_acc"].append(accuracy_score(y_train, train_pred))
40         history["val_acc"].append(accuracy_score(y_val, val_pred))
41
42         if verbose and (epoch == 0 or (epoch+1) % 5 == 0 or epoch == epochs-1):
43             print(
44                 f"Epoch {epoch+1:3d}/{epochs} | "
45                 f"Train Loss={train_loss:.4f} | Val Loss={val_loss:.4f} | "
46                 f"Train Acc={history['train_acc'][-1]:.4f} | "
47                 f"Val Acc={history['val_acc'][-1]:.4f}"
48             )
49
50     return history
```

Mục đích: Tự cài đặt vòng lặp mini-batch training.

Giải thích chi tiết

`train\_nn()` xáo trộn chỉ số bằng `np.random.permutation`, lấy từng mini-batch, rồi thực hiện tuần tự forward → BCE → backward → update. Sau mỗi epoch,

cell tính loss/accuracy trên toàn bộ Train và Validation, lưu vào `history`. Tham số `verbose` chỉ điều khiển việc in log. Hàm trả history để các cell sau vẽ learning curve.

Cell không sinh output trực tiếp trong notebook.

9. Baseline NumPy Deep Learning

Code Cell 25

Code Cell 25

```
1 t0 = time.time()
2
3 baseline_nn = NeuralNetwork5Layers(
4     input_dim=INPUT_DIM,
5     hidden_dims=(128, 64, 32, 16),
6     optimizer="adam",
7     learning_rate=0.001,
8     seeds=SEED
9 )
10
11 print("Architecture:", INPUT_DIM, "-> 128 -> 64 -> 32 -> 16 -> 1")
12 print("Parameters:", f"{baseline_nn.count_params():,}")
13
14 baseline_history = train_nn(
15     baseline_nn,
16     X_train_dl, y_train_dl,
17     X_val_dl, y_val_dl,
18     epochs=30,
19     batch_size=64,
20     verbose=True
21 )
22
23 dl_time = time.time() - t0
24 print("Training time:", round(dl_time, 2), "seconds")
```

Mục đích: Huấn luyện baseline Deep Learning.

Giải thích chi tiết

Model được khởi tạo với kiến trúc $534 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 1$, Adam, learning rate 0,001 và seed. `count\_params()` cho 79.361 tham số. `train\_nn()` chạy 30 epoch với batch 64. Output cho thấy Train Accuracy tăng từ 0,9504 lên 0,9991 trong khi Validation không tăng tương ứng; Train Loss giảm xuống 0,0028 còn Validation Loss tăng lên 0,3366. Đây là dấu hiệu overfitting thể hiện trực tiếp từ log.

Output thực tế

Output - Cell 25 - #1

```
Architecture: 534 -> 128 -> 64 -> 32 -> 16 -> 1
Parameters: 79,361
Epoch 1/30 | Train Loss=0.1195 | Val Loss=0.1329 | Train Acc=0.9504 | Val Acc=0.9401
Epoch 5/30 | Train Loss=0.0647 | Val Loss=0.1461 | Train Acc=0.9777 | Val Acc=0.9460
Epoch 10/30 | Train Loss=0.0085 | Val Loss=0.2738 | Train Acc=0.9977 | Val Acc=0.9375
Epoch 15/30 | Train Loss=0.0026 | Val Loss=0.3787 | Train Acc=0.9991 | Val Acc=0.9412
Epoch 20/30 | Train Loss=0.0057 | Val Loss=0.3719 | Train Acc=0.9982 | Val Acc=0.9369
Epoch 25/30 | Train Loss=0.0029 | Val Loss=0.3851 | Train Acc=0.9991 | Val Acc=0.9358
Epoch 30/30 | Train Loss=0.0028 | Val Loss=0.3366 | Train Acc=0.9991 | Val Acc=0.9389
Training time: 67.84 seconds
```

Code Cell 26

Code Cell 26

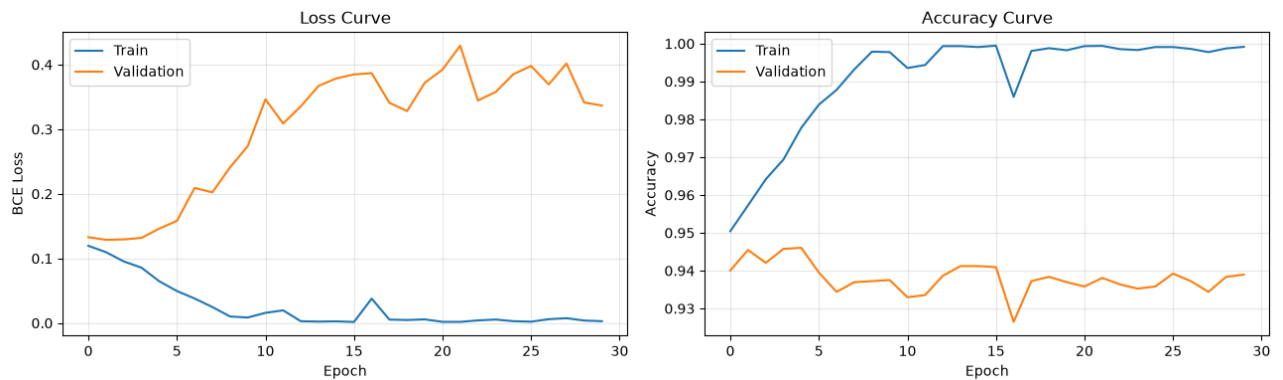
```
1 fig, axes = plt.subplots(1, 2, figsize=(13, 4))
2
3 axes[0].plot(baseline_history["train_loss"], label="Train")
4 axes[0].plot(baseline_history["val_loss"], label="Validation")
5 axes[0].set_title("Loss Curve")
6 axes[0].set_xlabel("Epoch")
7 axes[0].set_ylabel("BCE Loss")
8 axes[0].legend()
9 axes[0].grid(alpha=0.3)
10
11 axes[1].plot(baseline_history["train_acc"], label="Train")
12 axes[1].plot(baseline_history["val_acc"], label="Validation")
13 axes[1].set_title("Accuracy Curve")
14 axes[1].set_xlabel("Epoch")
15 axes[1].set_ylabel("Accuracy")
16 axes[1].legend()
17 axes[1].grid(alpha=0.3)
18
19 plt.tight_layout()
20 plt.show()
```

Mục đích: Vẽ learning curves của baseline DL.

Giải thích chi tiết

Cell lấy `'train_loss'`, `'val_loss'`, `'train_acc'`, `'val_acc'` từ `'baseline_history'` rồi vẽ hai biểu đồ. Không có training mới. Mục đích là quan sát khoảng cách Train/Validation: train loss giảm mạnh và train accuracy gần 1, còn validation loss tăng và validation accuracy plateau, củng cố nhận xét overfitting ở cell 25.

Output thực tế



10. Hyperparameter Experiments

So sánh:

- optimizer;
- learning rate;
- batch size;
- kích thước hidden layers.

Tiêu chí chọn model: ****Validation F1****, sau đó đánh giá một lần trên Test.

Code Cell 28

```
Code Cell 28

1 configs = [
2     ("GD", (128,64,32,16), "gd", 0.01, 64, 15),
3     ("Momentum", (128,64,32,16), "momentum", 0.01, 64, 15),
4     ("Adam", (128,64,32,16), "adam", 0.001, 64, 15),
5     ("Adam BS32", (128,64,32,16), "adam", 0.001, 32, 15),
6     ("Adam Deep", (256,128,64,32), "adam", 0.001, 64, 15),
7     ("Adam Small", (64,32,16,8), "adam", 0.001, 64, 15),
8 ]
9
10 dl_results = {}
11 dl_histories = {}
12
13 for name, hidden, opt, lr, bs, epochs in configs:
14     print("\n", "="*60)
15     print(name, hidden, opt, lr, bs)
16
17     t0 = time.time()
18     model = NeuralNetwork5Layers(
19         INPUT_DIM, hidden_dims=hidden,
20         optimizer=opt, learning_rate=lr, seed=SEED
21     )
22
23     history = train_nn(
24         model, X_train_dl, y_train_dl,
25         X_val_dl, y_val_dl,
26         epochs=epochs, batch_size=bs, verbose=False
27     )
28
29     val_prob = model.predict_proba(X_val_dl).ravel()
30     val_pred = (val_prob >= 0.5).astype(int)
31
32     test_prob = model.predict_proba(X_test_dl).ravel()
33     test_pred = (test_prob >= 0.5).astype(int)
34
35     dl_results[name] = {
36         "model": model,
37         "optimizer": opt,
38         "lr": lr,
39         "batch": bs,
40         "hidden": hidden,
41         "params": model.count_params(),
42         "val_f1": f1_score(y_val, val_pred, zero_division=0),
43         "val_auc": roc_auc_score(y_val, val_prob),
44         "test_acc": accuracy_score(y_test, test_pred),
45         "test_precision": precision_score(y_test, test_pred, zero_division=0),
46         "test_recall": recall_score(y_test, test_pred, zero_division=0),
47         "test_f1": f1_score(y_test, test_pred, zero_division=0),
48         "test_auc": roc_auc_score(y_test, test_prob),
49         "time": time.time() - t0,
50         "test_prob": test_prob,
51         "test_pred": test_pred
52     }
53     dl_histories[name] = history
54
55     print(
56         f"Val F1={dl_results[name]['val_f1']:.4f} | "
57         f"Test F1={dl_results[name]['test_f1']:.4f} | "
58         f"Test AUC={dl_results[name]['test_auc']:.4f}"
59     )
```

Mục đích: Chạy sáu cấu hình Deep Learning.

Giải thích chi tiết

`configs` định nghĩa 6 thí nghiệm thay đổi optimizer, learning rate, batch size hoặc hidden architecture. Mỗi cấu hình tạo model mới rồi gọi `train\_nn()` 15 epoch. Sau đó cell tính Validation F1/AUC và Test

Accuracy/Precision/Recall/F1/AUC, đồng thời lưu model và history. Output của cell là nguồn dữ liệu trực tiếp cho bảng tổng hợp ở cell 29.

Output thực tế

Output - Cell 28 - #1

```
=====
GD (128, 64, 32, 16) gd 0.01 64
Val F1=0.9622 | Test F1=0.9581 | Test AUC=0.9720

=====
Momentum (128, 64, 32, 16) momentum 0.01 64
Val F1=0.9627 | Test F1=0.9577 | Test AUC=0.9717

=====
Adam (128, 64, 32, 16) adam 0.001 64
Val F1=0.9609 | Test F1=0.9560 | Test AUC=0.9657

=====
Adam BS32 (128, 64, 32, 16) adam 0.001 32
Val F1=0.9631 | Test F1=0.9593 | Test AUC=0.9650

=====
Adam Deep (256, 128, 64, 32) adam 0.001 64
Val F1=0.9611 | Test F1=0.9577 | Test AUC=0.9668

=====
Adam Small (64, 32, 16, 8) adam 0.001 64
Val F1=0.9616 | Test F1=0.9588 | Test AUC=0.9646
```

Code Cell 29

Code Cell 29

```
1 dl_comparison = pd.DataFrame({
2     name: {
3         "Architecture": "-".join(map(str, [INPUT_DIM, *r["hidden"], 1])),
4         "Optimizer": r["optimizer"],
5         "LR": r["lr"],
6         "Batch": r["batch"],
7         "Params": r["params"],
8         "Val F1": r["val_f1"],
9         "Val AUC": r["val_auc"],
10        "Test Accuracy": r["test_acc"],
11        "Test Precision": r["test_precision"],
12        "Test Recall": r["test_recall"],
13        "Test F1": r["test_f1"],
14        "Test AUC": r["test_auc"],
15        "Time (s)": r["time"]
16    }
17    for name, r in dl_results.items()
18 }).T
19
20 display(dl_comparison.round(4))
```

Mục đích: Tổng hợp kết quả các cấu hình DL.

Giải thích chi tiết

Cell chuyển `dl\_results` thành DataFrame, với mỗi cấu hình là một hàng và các cột gồm architecture, optimizer, LR, batch, params, Validation F1/AUC, Test metrics và time. Cell không train lại hay tính metric mới; nó chỉ trình bày kết quả đã tạo ở cell 28.

Output thực tế

Output - Cell 29 - #1

	Architecture	Optimizer	LR	Batch	Params	Val F1 \
GD	534-128-64-32-16-1	gd	0.01	64	79361	0.962175
Momentum	534-128-64-32-16-1	momentum	0.01	64	79361	0.962717
Adam	534-128-64-32-16-1	adam	0.001	64	79361	0.960869
Adam BS32	534-128-64-32-16-1	adam	0.001	32	79361	0.963091
Adam Deep	534-256-128-64-32-1	adam	0.001	64	180225	0.961066
Adam Small	534-64-32-16-8-1	adam	0.001	64	36993	0.961585
	Val AUC	Test Accuracy	Test Precision	Test Recall	Test F1 \	
GD	0.975804	0.931818	0.968243	0.948169	0.958101	
Momentum	0.97596	0.931534	0.972906	0.942985	0.957712	
Adam	0.971236	0.927273	0.951094	0.960954	0.955999	
Adam BS32	0.973327	0.933239	0.961792	0.956807	0.959293	
Adam Deep	0.975153	0.930398	0.956567	0.95888	0.957722	
Adam Small	0.971878	0.932102	0.956029	0.961645	0.958829	
	Test AUC	Time (s)				
GD	0.972024	29.012532				
Momentum	0.97167	29.777078				
Adam	0.965651	34.288786				
Adam BS32	0.964982	33.638715				
Adam Deep	0.966836	52.784372				
Adam Small	0.964591	11.079041				

11. Chọn Best Deep Learning Model

Code Cell 31

Code Cell 31

```
1 best_dl_name = max(dl_results, key=lambda k: dl_results[k]["val_f1"])
2 best_dl = dl_results[best_dl_name]
3
4 print("Best DL:", best_dl_name)
5 print("Architecture:", best_dl["hidden"])
6 print("Optimizer:", best_dl["optimizer"])
7 print("Validation F1:", round(best_dl["val_f1"], 4))
8 print("Test F1:", round(best_dl["test_f1"], 4))
9 print("Test AUC:", round(best_dl["test_auc"], 4))
```

Mục đích: Chọn Best Deep Learning.

Giải thích chi tiết

`max(dl\_results, key=lambda k: dl\_results[k]["val\_f1"])` chọn cấu hình có Validation F1 lớn nhất. `best\_dl` giữ toàn bộ thông tin của cấu hình đó. Các giá trị Test F1/AUC được in ra để đánh giá cuối, nhưng không dùng Test để chọn model. Theo output notebook, Best DL là Adam BS32.

Output thực tế

Output - Cell 31 - #1

```
Best DL: Adam BS32
Architecture: (128, 64, 32, 16)
Optimizer: adam
Validation F1: 0.9631
Test F1: 0.9593
Test AUC: 0.965
```

Code Cell 32

Code Cell 32

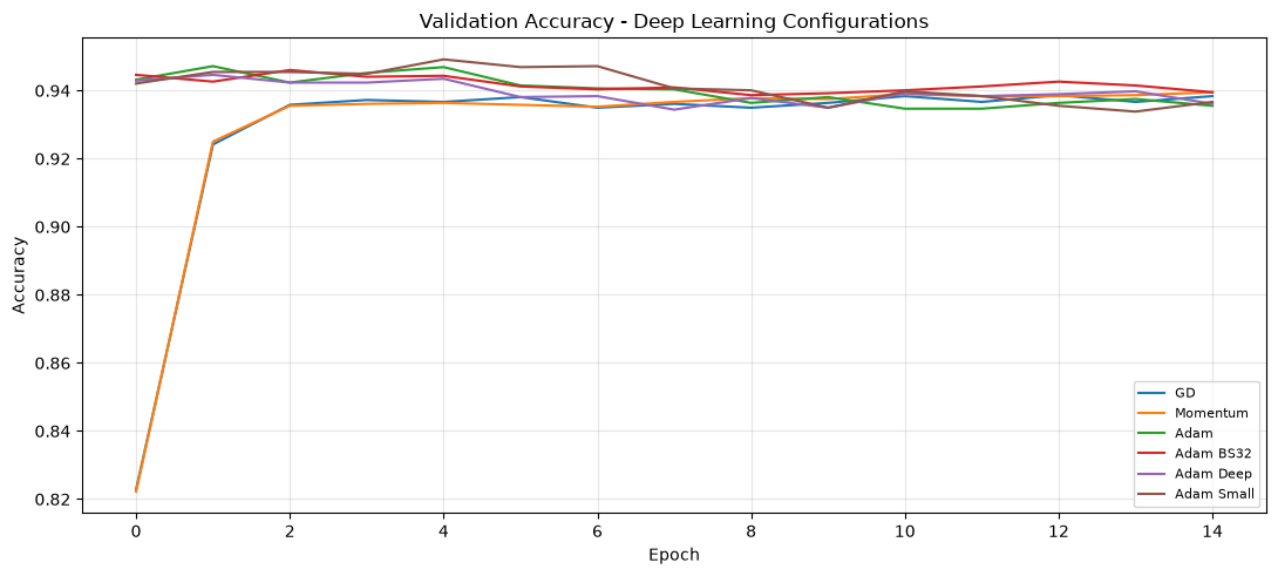
```
1 # Validation curves của các cấu hình
2 plt.figure(figsize=(11, 5))
3 for name, h in dl_histories.items():
4     plt.plot(h["val_acc"], label=name)
5 plt.title("Validation Accuracy - Deep Learning Configurations")
6 plt.xlabel("Epoch")
7 plt.ylabel("Accuracy")
8 plt.legend(fontsize=8)
9 plt.grid(alpha=0.3)
10 plt.tight_layout()
11 plt.show()
```

Mục đích: So sánh Validation Accuracy của các cấu hình DL.

Giải thích chi tiết

Vòng lặp qua `dl\_histories`, lấy `val\_acc` của từng cấu hình và vẽ từng đường theo epoch. Cell dùng biểu đồ để so sánh tốc độ hội tụ và độ ổn định Validation Accuracy, bổ sung cho bảng metric của cell 29.

Output thực tế



12. So sánh ML vs Deep Learning

Code Cell 34

Code Cell 34

```
1 all_compare = {}
2
3 for name, r in ml_results.items():
4     all_compare["ML - " + name] = {
5         "Accuracy": r["accuracy"],
6         "Precision": r["precision"],
7         "Recall": r["recall"],
8         "F1": r["f1"],
9         "AUC": r["auc"],
10        "Time (s)": r["time"]
11    }
12
13 for name, r in dl_results.items():
14     all_compare["DL - " + name] = {
15         "Accuracy": r["test_acc"],
16         "Precision": r["test_precision"],
17         "Recall": r["test_recall"],
18         "F1": r["test_f1"],
19         "AUC": r["test_auc"],
20         "Time (s)": r["time"]
21    }
22
23 final_comparison = pd.DataFrame(all_compare).T.sort_values("F1", ascending=False)
24 display(final_comparison.round(4))
```

Mục đích: So sánh cuối ML và DL trên Test.

Giải thích chi tiết

Cell lấy kết quả từ `ml\_results` và `dl\_results`, chuẩn hóa tên thành `ML - ...`/`DL - ...`, giữ các metric Accuracy, Precision, Recall, F1, AUC và Time, rồi `sort\_values("F1", ascending=False)`. Đây là bảng xếp hạng theo Test F1 sau khi Best DL đã được chọn bằng Validation F1.

Output thực tế

Output - Cell 34 - #1

	Accuracy	Precision	Recall	F1	AUC	\
ML - Logistic Regression	0.9366	0.9581	0.9651	0.9616	0.9762	
DL - Adam BS32	0.9332	0.9618	0.9568	0.9593	0.9650	
DL - Adam Small	0.9321	0.9560	0.9616	0.9588	0.9646	
DL - GD	0.9318	0.9682	0.9482	0.9581	0.9720	
DL - Adam Deep	0.9304	0.9566	0.9589	0.9577	0.9668	
DL - Momentum	0.9315	0.9729	0.9430	0.9577	0.9717	
DL - Adam	0.9273	0.9511	0.9610	0.9560	0.9657	
ML - Random Forest	0.9233	0.9339	0.9758	0.9544	0.9710	

	Time (s)
ML - Logistic Regression	0.3311
DL - Adam BS32	33.6387
DL - Adam Small	11.0790
DL - GD	29.0125
DL - Adam Deep	52.7844
DL - Momentum	29.7771
DL - Adam	34.2888
ML - Random Forest	2.1995

Code Cell 35

Code Cell 35

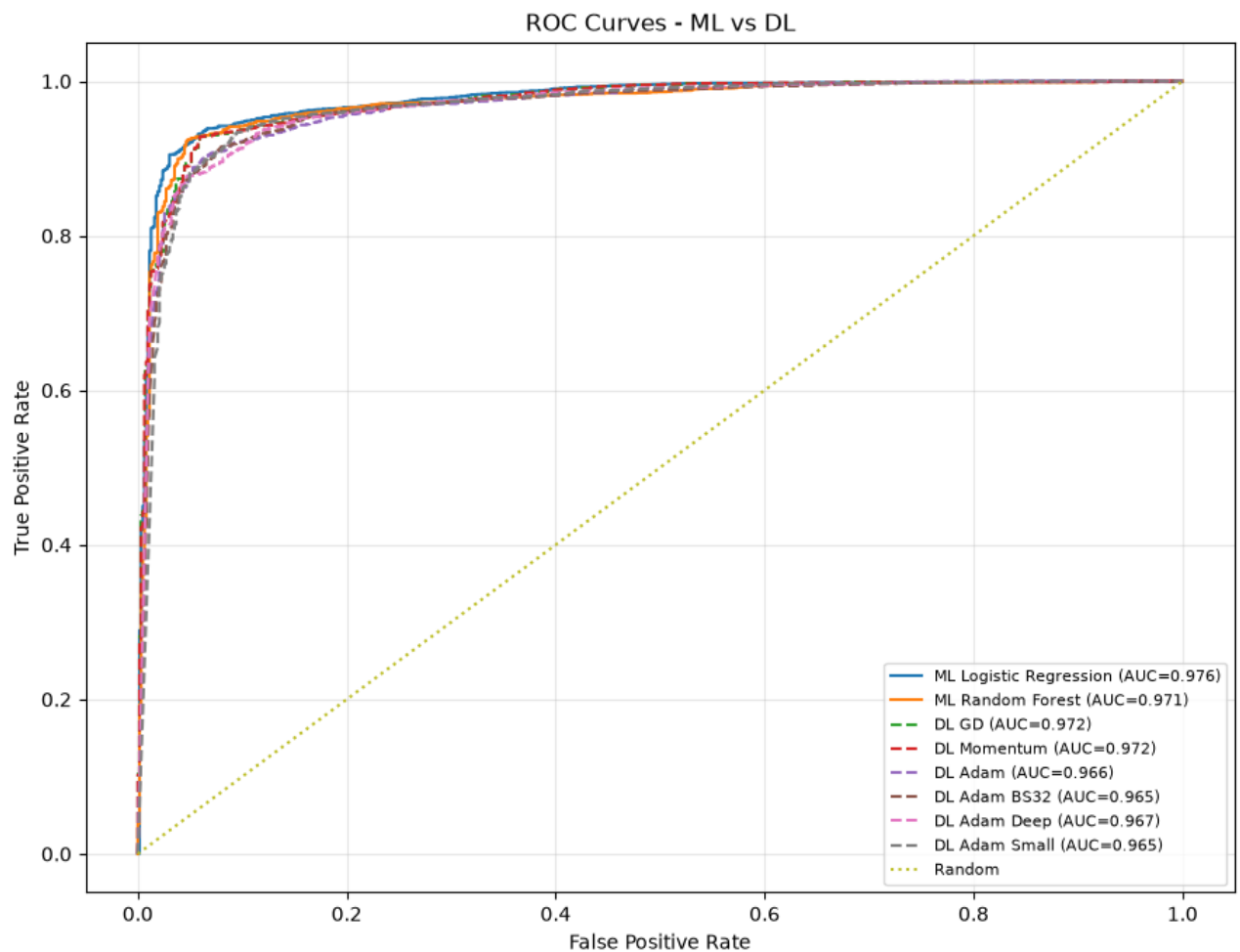
```
1 # ROC curves
2 plt.figure(figsize=(9, 7))
3
4 for name, r in ml_results.items():
5     fpr, tpr, _ = roc_curve(y_test, r["prob"])
6     plt.plot(fpr, tpr, label=f"ML {name} (AUC={r['auc']:.3f})")
7
8 for name, r in dl_results.items():
9     fpr, tpr, _ = roc_curve(y_test, r["test_prob"])
10    plt.plot(fpr, tpr, linestyle="--", label=f"DL {name} (AUC={r['test_auc']:.3f})")
11
12
13 plt.plot([0, 1], [0, 1], linestyle=":", label="Random")
14 plt.xlabel("False Positive Rate")
15 plt.ylabel("True Positive Rate")
16 plt.title("ROC Curves - ML vs DL")
17 plt.legend(fontsize=8)
18 plt.grid(alpha=0.3)
19 plt.tight_layout()
20 plt.show()
```

Mục đích: Vẽ ROC curves.

Giải thích chi tiết

Với ML, `roc\_curve()` dùng `y\_test` và xác suất đã lưu trong `ml\_results`. Với DL, dùng `test\_prob` từ `dl\_results`. Mỗi đường được gắn AUC; đường chéo là baseline random. ROC cho thấy trade-off TPR/FPR khi threshold thay đổi, còn AUC đánh giá khả năng phân biệt tổng thể.

Output thực tế



13. Confusion Matrix — Best DL

Code Cell 37

Code Cell 37

```
1 cm = confusion_matrix(y_test, best_dl["test_pred"])
2
3 plt.figure(figsize=(5, 4))
4 plt.imshow(cm)
5 plt.title(f"Confusion Matrix - {best_dl_name}")
6 plt.xlabel("Predicted")
7 plt.ylabel("Actual")
8 plt.xticks([0, 1], ["Not Interested", "Interested"])
9 plt.yticks([0, 1], ["Not Interested", "Interested"])
10
11 for i in range(2):
12     for j in range(2):
13         plt.text(j, i, cm[i, j], ha="center", va="center")
14
15 plt.tight_layout()
16 plt.show()
17
18 print(classification_report(
19     y_test,
20     best_dl["test_pred"],
21     target_names=["Not Interested", "Interested"],
22     zero_division=0
23 ))
```

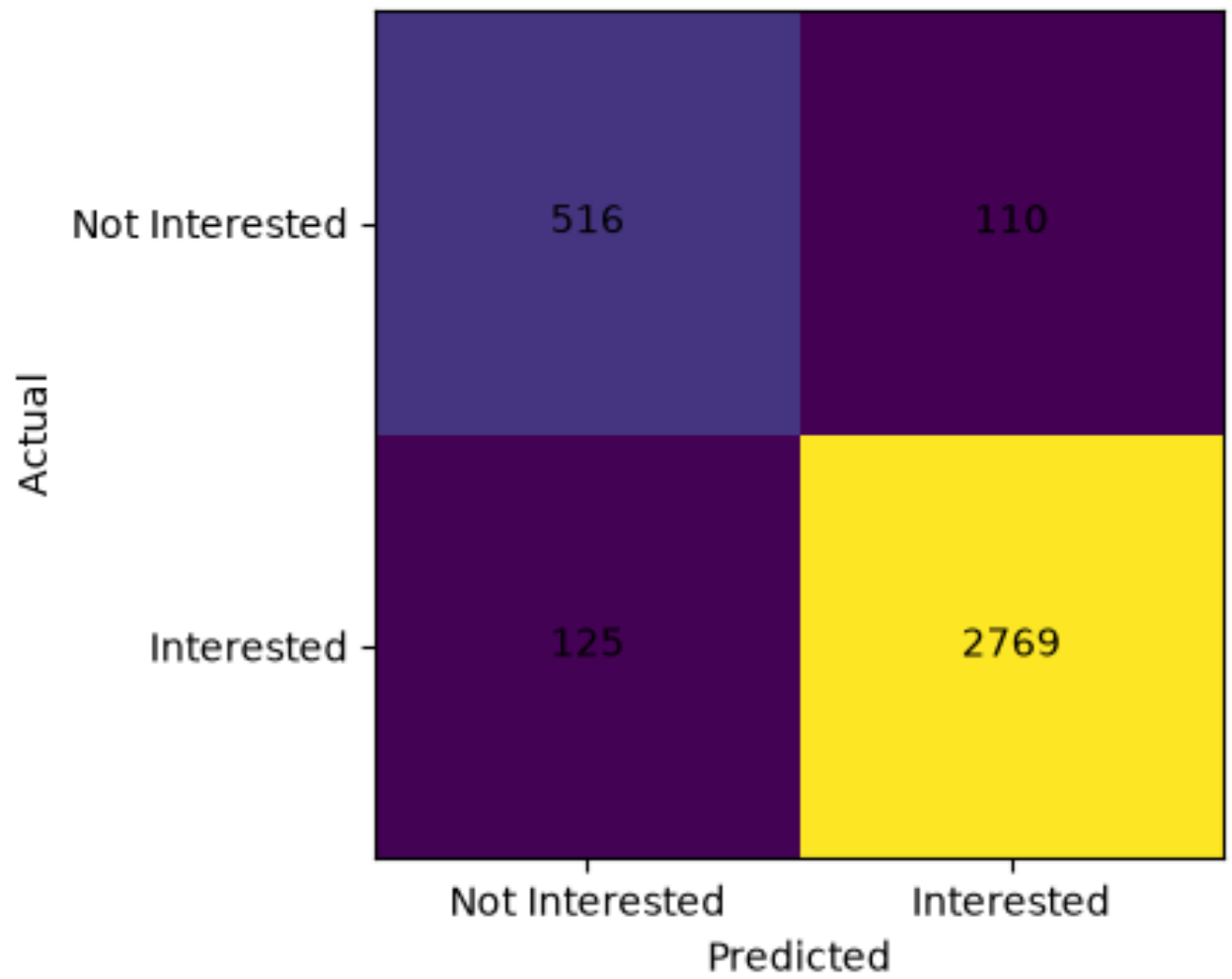
Mục đích: Phân tích Best DL bằng confusion matrix và classification report.

Giải thích chi tiết

`confusion\_matrix()` tạo ma trận Actual $\times$ Predicted; `imshow()` hiển thị và vòng lặp ghi số vào từng ô. `classification\_report()` tính Precision, Recall, F1 và support cho từng lớp. Output cho phép nhìn riêng hiệu quả lớp Not Interested và Interested thay vì chỉ nhìn Accuracy tổng.

Output thực tế

Confusion Matrix - Adam BS32



Output - Cell 37 - #2

	precision	recall	f1-score	support
Not Interested	0.80	0.82	0.81	626
Interested	0.96	0.96	0.96	2894
accuracy			0.93	3520
macro avg	0.88	0.89	0.89	3520
weighted avg	0.93	0.93	0.93	3520

14. Overfitting / Underfitting Analysis

Quan sát:

- Train loss vs Validation loss
- Train accuracy vs Validation accuracy

Nếu train tốt nhưng validation kém rõ rệt → có dấu hiệu overfitting.

Code Cell 39

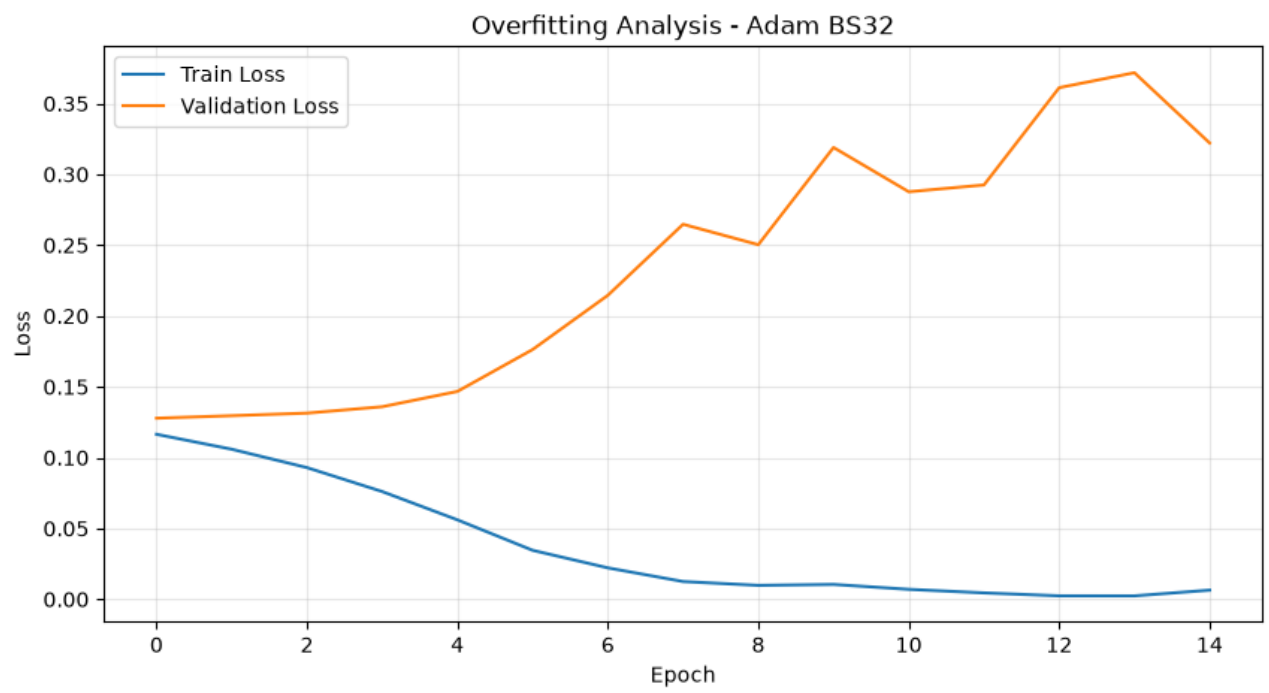
```
Code Cell 39
1 h = dl_histories[best_dl_name]
2
3 plt.figure(figsize=(10, 5))
4 plt.plot(h["train_loss"], label="Train Loss")
5 plt.plot(h["val_loss"], label="Validation Loss")
6 plt.xlabel("Epoch")
7 plt.ylabel("Loss")
8 plt.title(f"Overfitting Analysis - {best_dl_name}")
9 plt.legend()
10 plt.grid(alpha=0.3)
11 plt.show()
12
13 train_final = h["train_acc"][-1]
14 val_final = h["val_acc"][-1]
15 print("Final Train Accuracy:", round(train_final, 4))
16 print("Final Validation Accuracy:", round(val_final, 4))
17 print("Train-Val gap:", round(train_final - val_final, 4))
```

Mục đích: Kiểm tra overfitting của Best DL.

Giải thích chi tiết

Cell lấy history đúng của `best\_dl\_name`, vẽ Train/Validation Loss và tính accuracy cuối bằng `[-1]`. Gap được tính trực tiếp bằng `train\_final-val\_final`. Output notebook cho Best DL cho Train Accuracy khoảng 0,9976, Validation Accuracy khoảng 0,9395, gap khoảng 0,0581; đây là dấu hiệu overfitting.

Output thực tế



Output - Cell 39 - #2

Final Train Accuracy: 0.9976
Final Validation Accuracy: 0.9395
Train-Val gap: 0.0581

15. Feature Importance

Dùng Random Forest để xem nhóm feature nào đóng góp nhiều nhất.

Lưu ý: importance của TF-IDF được hiển thị theo từng vector index; đây là phân tích hỗ trợ cho mô hình ML, không phải trọng số trực tiếp của neural network.

Code Cell 41

Code Cell 41

```
1 rf = ml_results["Random Forest"]["model"]
2 importances = rf.feature_importances_
3
4 cat_names = list(ohe.get_feature_names_out(cat_cols))
5 text_names = [f"tfidf_{i}" for i in range(X_train_txt.shape[1])]
6 feature_names = num_cols + cat_names + text_names
7
8 feat_df = pd.DataFrame({
9     "Feature": feature_names,
10    "Importance": importances
11 }).sort_values("Importance", ascending=False)
12
13 display(feat_df.head(20))
14
15 top = feat_df.head(20).sort_values("Importance")
16 plt.figure(figsize=(10, 7))
17 plt.barh(top["Feature"], top["Importance"])
18 plt.xlabel("Importance")
19 plt.title("Top 20 Random Forest Feature Importance")
20 plt.tight_layout()
21 plt.show()
```

Mục đích: Phân tích feature importance bằng Random Forest.

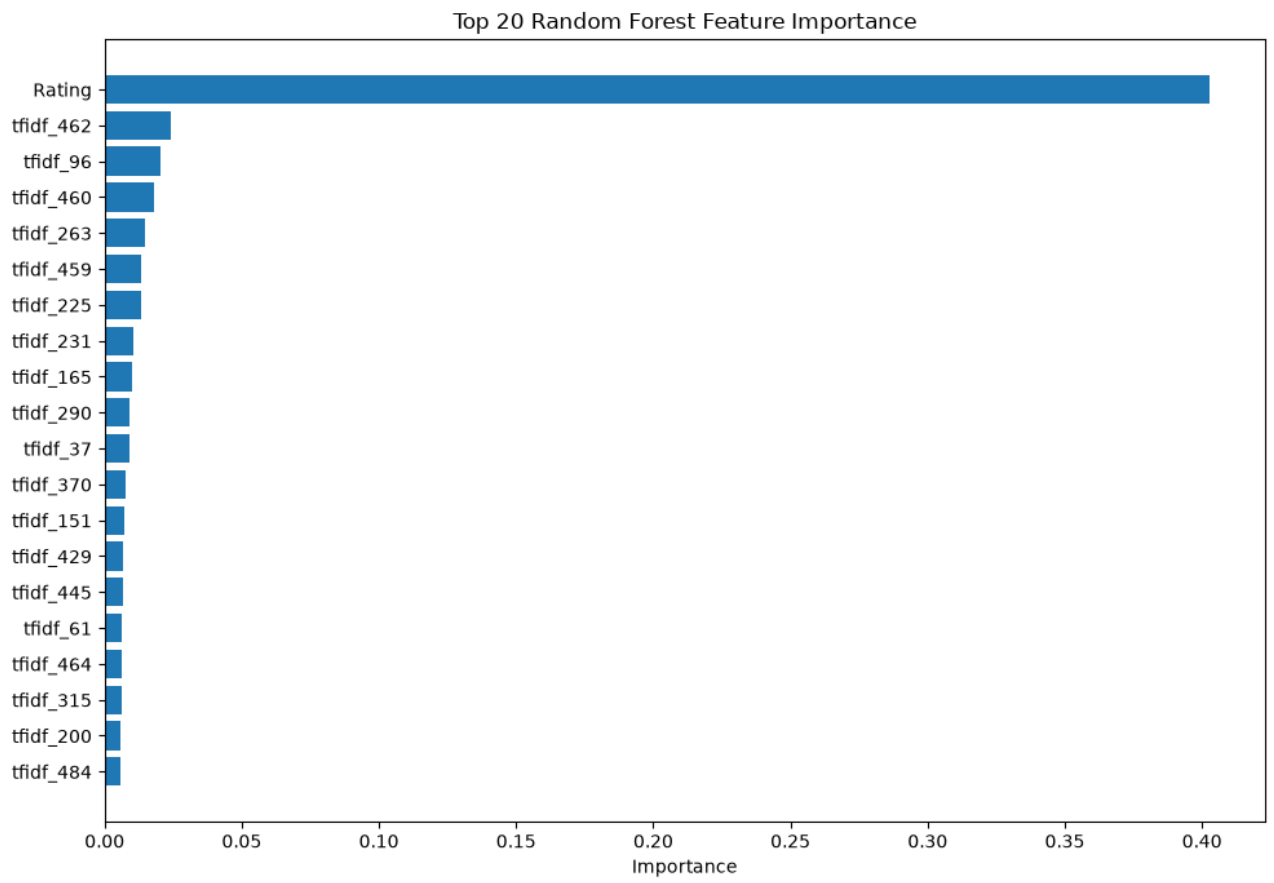
Giải thích chi tiết

Cell lấy `feature\_importances\_` của Random Forest trong `ml\_results`. Tên feature được ghép theo đúng thứ tự 6 numerical + các tên One-Hot + 500 vị trí TF-IDF, tạo thành 534 tên. Sau khi sort, cell lấy top 20 và vẽ barh. Output cho thấy Rating có importance khoảng 0,402849, cao nhất. Các tên `tfidf\_x` ở đây mới chỉ là index feature, chưa phải từ/cụm từ thực tế.

Output thực tế

Output - Cell 41 - #1

	Feature	Importance
2	Rating	0.402849
496	tfidf_462	0.024099
130	tfidf_96	0.020391
494	tfidf_460	0.017964
297	tfidf_263	0.014602
493	tfidf_459	0.013305
259	tfidf_225	0.013124
265	tfidf_231	0.010688
199	tfidf_105	0.009952
324	tfidf_290	0.009144
71	tfidf_37	0.009005
404	tfidf_370	0.007656
185	tfidf_151	0.007357
463	tfidf_429	0.006934
479	tfidf_445	0.006805
95	tfidf_61	0.006423
498	tfidf_464	0.006365
349	tfidf_315	0.006130
234	tfidf_200	0.005890
518	tfidf_484	0.005683



16. Save Model / Preprocessing

Lưu:

- trọng số neural network bằng `.npz`;
- scaler;
- encoder;
- TF-IDF vocabulary;
- thông tin kiến trúc.

Các file này có thể dùng tiếp cho bước triển khai web local.

Code Cell 43

```
Code Cell 43
1 from pathlib import Path
2 import pickle
3
4 MODEL_DIR = Path("saved_models")
5 MODEL_DIR.mkdir(exist_ok=True)
6
7 # Save NumPy DL weights
8 np.savez(
9     MODEL_DIR / "best_dl_ecommerce_weights.npz",
10    **{f"W(i)": w for i, w in enumerate(best_dl["model"].W)},
11    **{f"b(i)": b for i, b in enumerate(best_dl["model"].b)}
12 )
13
14 # Save preprocessing objects
15 with open(MODEL_DIR / "preprocessing.pkl", "wb") as f:
16     pickle.dump({
17         "scaler": scaler,
18         "ohe": ohe,
19         "tfidf": tfidf,
20         "num_cols": num_cols,
21         "cat_cols": cat_cols,
22         "input_dim": INPUT_DIM,
23         "hidden_dims": best_dl["hidden"]
24     }, f)
25
26 print("Saved:", MODEL_DIR / "best_dl_ecommerce_weights.npz")
27 print("Saved:", MODEL_DIR / "preprocessing.pkl")
```

Mục đích: Lưu model và preprocessing.

Giải thích chi tiết

`MODEL\_DIR` được tạo nếu chưa có. `np.savez()` lưu từng W/b của Best DL; `pickle.dump()` lưu scaler, OneHotEncoder, TF-IDF, danh sách cột, input_dim và hidden_dims. Hai file được tạo để có thể tái sử dụng preprocessing và trọng số khi triển khai mà không fit lại từ đầu. Cell không lưu các model ML.

Output thực tế

```
Output - Cell 43 - #1
Saved: saved_models\best_dl_ecommerce_weights.npz
Saved: saved_models\preprocessing.pkl
```

17. Final Conclusion

Notebook đã hoàn thành pipeline:

****Dataset → Inspection → Cleaning → Feature Engineering → Train/Val/Test → ML Baselines → Deep Learning NumPy from scratch → Hyperparameter Tuning → Evaluation → Model Saving****

Điểm quan trọng của bài:

- target là ****user interest / recommendation**** ('Recommended IND');
- neural network được xây dựng thủ công bằng ****NumPy****;
- không dùng TensorFlow, Keras hoặc PyTorch cho Deep Learning;
- có so sánh với Machine Learning truyền thống;
- có lưu model để sử dụng cho bước triển khai sau.

Code Cell 45

```
Code Cell 45
1 best_overall = final_comparison.index[0]
2 print("=" * 70)
3 print("FINAL SUMMARY")
4 print("=" * 70)
5 print("Best overall model:", best_overall)
6 print(final_comparison.loc[best_overall].round(4))
7 print("\nBest DL:", best_dl_name)
8 print("Saved model directory:", MODEL_DIR.resolve())
```

Mục đích: In kết luận cuối cùng.

Giải thích chi tiết

`final\_comparison.index[0]` lấy model đứng đầu bảng đã sort theo Test F1 ở cell 34, nên Best Overall phụ thuộc trực tiếp vào kết quả thực tế. Cell tiếp tục in toàn bộ metric của model đó, tên Best DL và đường dẫn thư mục model. Output notebook cho thấy Best Overall là ML - Logistic Regression và Best DL là Adam BS32.

Output thực tế

```
Output - Cell 45 - #1
=====
FINAL SUMMARY
=====
Best overall model: ML - Logistic Regression
Accuracy      0.9366
Precision     0.9581
Recall        0.9651
F1            0.9616
AUC           0.9762
Time (s)      0.3311
Name: ML - Logistic Regression, dtype: float64

Best DL: Adam BS32
Saved model directory: D:\Đại học\Phát triển các hệ thống thông minh\Assignment 03\customer_behavior\saved_models
```

Source code: [Phat-trien-cac-he-thong-thong-minh/Assignment 03 at main · Hunguyenobugs/Phat-trien-cac-he-thong-thong-minh](#)